



Moneris Gateway API - Integration Guide - PHP – PHP

Version: 1.6.9

Copyright © Moneris Solutions, 2025

All rights reserved. No part of this publication may be reproduced, stored in retrieval systems, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of Moneris Solutions Corporation.

Security and Compliance

Your solution may be required to demonstrate compliance with the card associations' PCI/CISP/PABP requirements. For more information on how to make your application PCI-DSS compliant, contact the Moneris Sales Center and visit <https://developer.moneris.com> to download the PCI_DSS Implementation Guide.

All Merchants and Service Providers that store, process, or transmit cardholder data must comply with PCI DSS and the Card Association Compliance Programs. However, certification requirements vary by business and are contingent upon your "Merchant Level" or "Service Provider Level".

The card association has some data security standards that define specific requirements for all organizations that store, process, or transmit cardholder data. As a Moneris client or partner using this method of integration, your solution must demonstrate compliance to the Payment Card Industry Data Security Standard (PCI DSS) and/or the Payment Application Data Security Standard (PA DSS). These standards are designed to help the cardholders and merchants in such ways as they ensure credit card numbers are encrypted when transmitted/stored in a database and that merchants have strong access control measures.

Non-compliant solutions may prevent merchant boarding with Moneris. A non-compliant merchant can also be subject to fines, fees, assessments or termination of processing services.

For further information on PCI DSS & PA DSS requirements, visit <http://www.pcisecuritystandards.org>.

Confidentiality

You have a responsibility to protect cardholder and merchant related confidential account information. Under no circumstances should ANY confidential information be sent via email while attempting to diagnose integration or production issues. When sending sample files or code for analysis by Moneris staff, all references to valid card numbers, merchant accounts and transaction tokens should be removed and or obscured. Under no circumstances should live cardholder accounts be used in the test environment.

Changes in v1.6.9

- Added Surcharge Lookup to the Basic Transaction set. Surcharge Lookup enables merchants to confirm card eligibility before applying surcharges to transactions.
- Added Vault Surcharge Lookup to the Vault Transaction set. Similar to the basic version, this lookup accepts a Moneris Vault `data_key` for confirming eligibility of the underlying card.
- Added `surcharge_info` object to the transactions Purchase, Pre-Authorization, Vault Purchase, Vault Pre-Authorization, 3DS Purchase, 3DS Pre-Authorization, 3DS & Vault Purchase, and 3DS & Vault Pre-Authorization
- Added `surcharge_amount` as a child to the `surcharge_info` object.
- Added additional Response Codes for the Moneris Surcharge Eligibility service to "Response Codes" on page 1

Changes in v1.6.8

- Added Vault Tokenize Credit Card request to the Vault transaction set. This transaction allows for tokenizing a card used in a previous financial transaction without resubmitting card data
- Added `return_issuer_id` to the Definition of Request Fields for Vault.
- Added new values to `request_type` in 3DS requests.
- Updated Visa Secure (3DS) support for the field `RIndicator` to support 01, 02, 06, 07, and 11 for Payment Transactions and 03, 04, 05, and 10 for Non-Payment Transactions

Changes in v1.6.7

- Removed **Interac Online** chapter and related appendix topics for certification with Interac Online. Support for Interac Online has terminated effective October 31st, 2024.

Changes in v1.6.6

- Added **Increment Preauthorization** in Basic transactions
- Added field `is_incremental` to the Basic Pre-Authorization and Vault Pre-Authorization transactions

Changes in v1.6.5

- Added new request object and fields to support Visa Account Name Verification as an option- within the basic Card Verification transaction. See `account_name_verification` and its sub-fields `first_name`, `middle_name`, and `last_name`
- Added response field `AccountNameVerificationResultCode` to the core response field definitions as part of the new Visa Account Name Verification

Changes in v1.6.4

- Added `BrowserIP` to MPI 3DS Authentication Request - Browser Channel
- Added `WorkPhone`, `home_phone` and `mobile_phone` to MPI 3DS Authentication Request - Browser Channel
- Added 3 new transaction types `GooglePayTokenTempAdd`, `GooglePayTokenPurchase`, and `GooglePayTokenPreauth`. These transactions allow for sending an encrypted `GooglePay` payload and receiving a Moneris temporary token in exchange for processing 3DS authentication.
- Added new request objects and fields to support `GooglePayToken` transactions such as `PaymentToken` and its subfields `signature`, `protocol version`, and `signed message`
- Added new response field `GooglePayPaymentMethod`

Changes in v1.6.3

- Added Merchant Advice Code in Purchase, Pre-Authorization, CAVV Purchase; CAVV Pre-Authorization responses samples

- Updated DS trans ID Type and Limits, and Description in B.1 Definition of Response Fields – 3-D Secure
- Removed the RequestType field from MPI 3DS Authentication Request - 3RI with recurring
- Updated the note for the fields RecurringFrequency and RecurringExpiry
- Added Set Methods in MPI 3DS Authentication Request
- Removed status check in MPI 3DS Authentication Request
- Removed ThreeDSCompletionInd in MPI 3DS Authentication Request - 3RI with recurring/no recurring
- Moved PriorAuthenticationInfo field to optional section for MPI 3DS Prior Authentication Info for MPI 3DS Authentication Request - 3RI with recurring
- Updated the note related to the Field Email
- Added Merchant Advice Code field in the Appendix B - Definitions of Response Fields

Changes in v1.6.2

- Updated the 3-D version to 2.2
- Added the 3RI flow chart Transaction Flow for 3-D Secure - 3RI channel
- Updated the field Email moving from optional to required
- Added a note to the field Email
- Added the status “D” in the TransStatus Code
- Updated transaction status reason Decline Codes
- Removed the Visa Checkout section. Visa Checkout has been decommissioned in June 1st 2023
- Updated the note related to the Field DSTransId
- Updated the note related to the Field ThreeDSSTransId
- Added a comment to Visa Secure

- Updated the note related to the Field RiIndicator
- Added note to ds_trans_id on only submitting it in financial transactions if using a 3rd party 3DS Secure service
- Added new 3DS fields to 3DS Authentication to support 3RI such as MessageCategory, DeviceChannel, RiIndicator
- Added new 3DS fields to 3DS Authentication to support 3RI Decoupled Authentication such as DecoupledRequestIndicator, DecoupledRequestMaxTime, DecoupledRequestAsyncUrl
- Added new 3DS object prior_request_auth_data to 3DS Authentication to support 3RI, including its fields PriorAuthenticationInfo , prior_request_auth_method, prior_request_auth_ref, prior_request_auth_timestamp
- Added additional fields to 3DS responses ThreeDSVersion and AuthenticationType
- Added additional topics on 3DS Authentication for the 3RI scenarios with and without recurring features
- Retitled the existing 3DS Authentication scenario to specify it is intended for the browser channel only
- Added topic on 3RI channel authentication flow and retitled previous 3DS flow to specify it is intended for browser channel only
- Added topic on Handling 3RI Decoupled Authentication flow to explain the asynchronous response handling
- Added topic on Server To Server endpoints to cover the separate URL for 3DS Authentication

Changes in v1.6.1

- Added note to ds_trans_id on only submitting it in financial transactions if using a 3rd party 3DS Secure service

Changes in v1.6.0

- Added new `foreign_indicator` field to Basic Transaction set: purchase and preauth
- Added new `foreign_indicator` field to 3-D Secure Transaction set: `cavv_purchase` and `cavvPreauth`
- Added new `foreign_indicator` values in Appendix A.2 Definition of Request Fields - Core Fields
- Updated sample code with new `foreign_indicator` field
- Updated AVS Response Codes table

Changes in v1.5.0

- Added new definition for MPI 3DS Authentication Request transaction type
- Added new value V for request field **payment indicator** to reflect support for merchants who bill in recurring variable payments
- Amended allowable values for **e-commerce indicator** to reflect new **payment indicator** value and amended corresponding topic in Credential on File section
- Added note to the limit for request field **cardholder name** to indicate that accented characters are not allowable
- Added new section and topics about Installments by Visa
- Added topic about Vault and Installments in the Vault section

Changes in v1.4.4

- Added new topic to Vault section listing Vault transactions that support temporary tokens
- Convenience Fee response field **convenience fee status** limit has been corrected to 3-character alphanumeric
- Information added to About Convenience Fee and response field definitions to indicate convenience fee transactions do not support MCP or digital wallets

- Transaction type descriptions have been added or amended in Vault Add Token, Vault Delete, Vault Is Corporate Card, Vault Add Credit Card,

Changes in v1.4.3

- Added new MCP transaction topics MCP Purchase with 3-D Secure, MCP Purchase with 3-D Secure and Vault, MCP Pre-Authorization with 3-D Secure, and MCP Pre-Authorization with 3-D Secure
- Added transaction type topics for Purchase with 3-D Secure and Vault, Pre-Authorization with 3-D Secure and Vault, and Purchase with 3-D Secure and Recurring Billing in 3-D Secure 2.0 section
- Added new request field **DS transaction ID**
- Made billing-related request fields mandatory for 3DS Authentication Request transaction

Changes in v1.4.2

- Corrected the limits for the request field **start date**
- Corrected presentation of set methods for the request fields **browser language**, **browser java enabled**, **browser screen height**, **browser screen width**,

Changes in v1.4.1

- Added information about the request fields 3DS version and 3DS server indicator in the transaction topics for Purchase with 3-D Secure and Pre-Authorization with 3-D Secure
- corrected the variable name for the request field 3DS server indicator

Changes in v1.4.0

- 3-D Secure 2.0 section has replaced MPI section (which related to 3-D Secure 1.0)
- Added new fields for 3-D Secure 2.0 to the Purchase with 3-D Secure and Pre-Authorization with 3-D Secure transactions
- References related to 3-D Secure2.0 have been added, including CAVV Result Codes for Visa, Mastercard and American Express card brands, and 3-D Secure 2.0 TransStatus Codes
- Added Definition of Response Fields for 3-D Secure
- Added Definition of Request Fields – 3-D Secure 2.0 and removed the previous Definition of Request Fields – MPI

Changes in v1.3.1

- Amended response code 959 under "Other Response Codes" to 599

Changes in v1.3.0

- Removed wording about testing only with Visa cards in About Credential on File topic
- Removed differentiation between supported card brands in regards to AVS and CVD in Card Verification
- Added information about American Express support in Card Verification with AVS and CVD and Card Verification with Vault
- Removed the Re-Authorization transaction type and information related to it
- Removed the Mag Swipe transaction set section
- Added Response Codes reference
- Added new supported currencies to MCP Currency Codes
- Added transaction type definitions to Visa Checkout topics
- Added information about dynamic descriptor behaviour in Pre-Authorization transactions to all Pre-Authorization and Pre-Authorization Completion topics

- Added missing dynamic descriptor field information to Purchase with 3-D Secure transaction topic
- Added information about payment indicator and e-commerce indicator field values, including new topic Payment Indicator and E-Commerce Indicator Values
- Reorganized information for Convenience Fee, including new topics Supported Transactions for Convenience Fee and Convenience Fee Info Object
- Renamed Convenience Fee transaction type topics to emphasize that Purchase is the base transactions and Convenience Fee is an additional feature
- Reorganized Definition of Request Fields by feature sets; Mag Swipe request fields removed
- New request field definition topics for connection fields, core fields, Vault, MPI, and Convenience Fee

Previous version changes

Changes in v1.2.12

- Added warnings about 3-D Secure implementations using frames to MPI section and 3-D Secure related transaction type topics

Changes in v1.2.11

- Added information about Google Pay™ and removed references to Android Pay
- Separated out transaction process flows into two separate topics for Apple Pay and Google Pay™, Apple Pay Transaction Process Overview and Google Pay™ Transaction Process Overview

Changes in v1.2.10

- Added missing request fields in MCP Purchase with Vault

Changes in v1.2.9

- Corrected sample code for some financial transactions (to remove old method of MCP processing)

Changes in v1.2.8

- Added section about Multi-Currency Pricing (MCP) transactions
- Added new conceptual topics in Credential on File:
 - Merchant- vs. Cardholder-Initiated COF Transactions
 - COF With Previously Stored Credentials
- Discover test card number has been changed to 6011000992927602
- Amended electronic commerce indicator description in Definition of Response Fields to remove deprecated allowable values (8 and 9)

Changes in v1.2.7

Missing limits for request variables amount, completion amount and transaction amount were replaced

Changes in v1.2.6

Electronic commerce Indicator (crypt_type) request variable's value of '7' amended to reflect that it also represents an American ExpressSafeKey non-authenticated transaction

Changes in v1.2.5

Purchase transaction amended to include Customer ID variable

Changes in v1.2.4

Changes limits in Amount, Transaction Amount, Completion Amount request variables to reflect 10 decimals.

Changes in v1.2.3

This version adds information about passing Offlinx™ data for the Card Match pixel tag via Unified API transactions.

Table of Contents

Security and Compliance	2
Confidentiality	2
Changes in v1.6.9	3
Getting Help	18
1 About This Documentation	19
1.1 Purpose	19
1.2 Who Is This Guide For?	19
1.3 Adding cURL CA Root Certificate to PHP API	19
2 Basic Transaction Set	21
2.1 Purchase	22
2.2 Pre-Authorization	26
2.3 Incremental Pre-Authorization	31
2.4 Pre-Authorization Completion	33
2.5 Force Post	37
2.6 Purchase Correction	40
2.7 Refund	43
2.8 Independent Refund	46
2.9 Card Verification with AVS and CVD	49
2.10 Surcharge Lookup	52
2.11 Batch Close	54
2.12 Open Totals	56
3 Credential on File	58
3.1 About Credential on File	58
3.2 Credential on File Info Object and Variables	58
3.3 Credential on File Transaction Types	59
3.4 Merchant- vs. Cardholder-Initiated COF Transactions	60
3.5 Initial Transactions in Credential on File	60
3.6 COF With Previously Stored Credentials	61
3.7 Payment Indicator and E-Commerce Indicator Values	61
3.8 Vault Tokenize Credit Card and Credential on File	62
3.9 Credential on File and Converting Temporary Tokens	62
3.10 Card Verification and Credential on File Transactions	62
3.10.1 When to Use Card Verification With COF	63
3.10.2 Credential on File and Vault Add Token	63
3.10.3 Credential on File and Vault Update Credit Card	63
3.10.4 Credential on File and Vault Add Credit Card	64
3.10.5 Credential on File and Recurring Billing	64
4 Vault	66
4.1 About the Vault Transaction Set	66
4.2 Vault Transaction Types	66
4.2.1 Administrative Vault Transaction types	66
4.2.2 Financial Vault Transaction types	68
4.3 Vault Transactions That Support Temporary Tokens	68
4.4 Vault and Installments	69
4.5 Vault Administrative Transactions	69
4.5.1 Vault Add Credit Card – ResAddCC	69
4.5.1.1 Vault Data Key	73
4.5.1.2 Vault Encrypted Add Credit Card – EncResAddCC	73
4.5.2 Vault Temporary Token Add – ResTempAdd	76
4.5.3 Vault Update Credit Card – ResUpdateCC	79
4.5.3.1 Vault Encrypted Update CC – EncResUpdateCC	83

4.5.4 Vault Delete – ResDelete	86
4.5.5 Vault Lookup Full – ResLookupFull	87
4.5.6 Vault Lookup Masked – ResLookupMasked	89
4.5.7 Vault Get Expiring – ResGetExpiring	90
4.5.8 Vault Is Corporate Card - ResIsCorporateCard	92
4.5.9 Vault Add Token – ResAddToken	94
4.5.10 Vault Tokenize Credit Card – ResTokenizeCC	97
4.6 Vault Financial Transactions	102
4.6.1 Customer ID Changes	102
4.6.2 Purchase with Vault – ResPurchaseCC	102
4.6.3 Pre-Authorization with Vault – ResPreauthCC	108
4.6.4 Vault Independent Refund – ResIndRefundCC	114
4.6.5 Force Post with Vault – ResForcePostCC	118
4.6.6 Card Verification with Vault – ResCardVerificationCC	121
4.6.7 Surcharge Lookup with Vault	125
4.7 Hosted Tokenization	127
5 Level 2/3 Transactions	128
5.1 About Level 2/3 Transactions	128
5.2 Level 2/3 Visa Transactions	128
5.2.1 Level 2/3 Transaction Types for Visa	128
5.2.2 Level 2/3 Transaction Flow for Visa	130
5.2.3 VS Completion	130
5.2.4 VS Purchase Correction	135
5.2.5 VS Force Post	137
5.2.6 VS Refund	143
5.2.7 VS Independent Refund	148
5.2.8 VS Corpais	153
5.2.8.1 VS Purcha – Corporate Card Common Data	155
5.2.8.2 VS Purchl – Line Item Details	158
5.2.8.3 Sample Code for VS Corpais	161
5.3 Level 2/3 Mastercard Transactions	162
5.3.1 Level 2/3 Transaction Types for Mastercard	163
5.3.2 Level 2/3 Transaction Flow for Mastercard	164
5.3.3 MC Completion	165
5.3.4 MC Force Post	167
5.3.5 MC Purchase Correction	169
5.3.6 MC Refund	171
5.3.7 MC Independent Refund	173
5.3.8 MC Corpais - Corporate Card Common Data with Line Item Details	175
5.3.8.1 MC Corpac - Corporate Card Common Data	177
5.3.8.2 MC Corpal - Line Item Details	185
5.3.8.3 Tax Array Object - MC Corpais	189
5.3.8.4 Sample Code for MC Corpais	191
5.4 Level 2/3 American Express Transactions	193
5.4.1 Level 2/3 Transaction Types for Amex	193
5.4.2 Level 2/3 Transaction Flow for Amex	196
5.4.3 Level 2/3 Data Objects in Amex	197
5.4.3.1 About the Level 2/3 Data Objects for Amex	197
5.4.3.2 Definition of the AxLevel23 Object	197
Table 1 Object	198
Table 1 - Setting the N1Loop Object	199
Table 1 - Setting the AxRef Object	201
Table 2 Object	202
Table 2 - Setting the Axlt1Loop Object	203
Table 2 - Setting the Axlt106s Object	206

Table 2 - Setting the AxTxI Object	207
Table 3 Object	211
Table 3 - Setting the AxTxI Object	212
5.4.4 AX Completion	215
5.4.5 AX Force Post	219
5.4.6 AX Purchase Correction	223
5.4.7 AX Refund	224
5.4.8 AX Independent Refund	228
6 3-D Secure 2.2	232
6.1 About 3-D Secure 2.2	232
6.1.1 3-D Secure Implementations	233
6.1.2 Out of Scope/Not Supported Check	234
6.1.3 Version Compatibility	234
6.1.4 Upgrading from 3-D Secure 2.0 to 3-D Secure 2.2 Check	234
6.2 Building Your 3-D Secure 2.2 Integration	235
6.2.1 Activating 3-D Secure Functionality	235
6.2.2 Transaction Flow for 3-D Secure - Browser channel	235
6.2.3 Transaction Flow for 3-D Secure - 3RI channel	237
6.2.3.1 Decoupled Authentication	238
6.3 Implementing Card Lookup Request	238
6.3.1 Card Lookup Request – mpiCardLookup	238
6.4 Handling the 3DS Method for Device Fingerprinting	240
6.5 Implementing MPI 3DS Authentication Request	241
6.5.1 MPI 3DS Authentication Request - Browser Channel	241
6.5.2 MPI 3DS Authentication Request - 3RI with recurring	252
6.5.3 MPI 3DS Authentication Request - 3RI, non-recurring	262
6.6 Handling the Challenge Flow	271
6.6.1 Cavv Lookup Request – mpiCavvLookup	271
6.7 Handling the Decoupled Authentication Flow	272
6.8 Performing the Authorization	276
6.8.1 Purchase with 3-D Secure – cavv_purchase	276
6.8.2 Pre-Authorization with 3-D Secure – cavv_preauth	281
6.8.3 Purchase with Vault and 3-D Secure	288
6.8.4 Pre-Authorization with Vault & 3-D Secure	292
6.8.5 Purchase with 3-D Secure and Recurring Billing	297
6.9 Testing Your 3-D Secure 2.2 Integration	299
6.10 Moving to Production With 3-D Secure 2.2	299
6.11 3-D Secure 2.2 TransStatus Codes	300
6.12 3-D Secure 2.2 Commons TransStatusReason Decline Codes	300
6.13 CAVV Result Codes	302
6.13.1 Visa CAVV Result Codes	302
6.13.2 Mastercard CAVV Result Codes	303
6.13.3 American Express CAVV Result Codes	304
7 Multi-Currency Pricing (MCP)	306
7.1 About Multi-Currency Pricing (MCP)	306
7.2 Methods of Processing MCP Transactions	307
7.3 MCP Get Rate	307
7.4 MCP Purchase	309
7.5 MCP Purchase with 3-D Secure	313
7.6 MCP Purchase with 3-D Secure and Vault	319
7.7 MCP Pre-Authorization	324
7.8 MCP Pre-Authorization with 3-D Secure	328
7.9 MCP Pre-Authorization with 3-D Secure and Vault	333
7.10 MCP Pre-Authorization Completion	338
7.11 MCP Purchase Correction	342
7.12 MCP Refund	344

7.13	MCP Independent Refund	348
7.14	MCP Purchase with Vault	352
7.15	MCP Pre-Authorization with Vault	356
7.16	MCP Independent Refund with Vault	360
7.17	MCP Currency Codes	364
7.18	MCP Error Codes	370
8	Installments by Visa	371
8.1	About Installments by Visa	371
8.2	Installments by Visa Transaction Types	371
8.3	Sending Transactions with Installments by Visa	372
8.4	Installment Plan Lookup	372
8.5	Vault Installment Plan Lookup	375
8.6	Installment Info Object	378
9	e-Fraud Tools	380
9.1	Address Verification Service	382
9.1.1	About Address Verification Service (AVS)	382
9.1.2	AVS Info Object	383
9.1.3	AVS Response Codes	384
9.1.4	AVS Sample Code	386
9.2	Card Validation Digits (CVD)	387
9.2.1	About Card Validation Digits (CVD)	387
9.2.2	Transactions Where CVD Is Required	387
9.2.3	CVD Information Object	388
9.2.4	CVD Result Codes	389
9.2.5	Sample Purchase with CVD Info Object	390
9.3	Transaction Risk Management Tool	391
9.3.1	About the Transaction Risk Management Tool	391
9.3.2	Introduction to Queries	391
9.3.3	Session Query	392
9.3.3.1	Session Query Transaction Flow	398
9.3.4	Attribute Query	399
9.3.4.1	Attribute Query Transaction Flow	403
9.3.5	Handling Response Information	403
9.3.5.1	TRMT Response Fields	404
9.3.5.2	Understanding the Risk Score	407
9.3.5.3	Understanding the Rule Codes, Rule Names and Rule Messages	408
9.3.5.4	Examples of Risk Response	417
	Session Query	417
	Attribute Query	417
9.3.6	Inserting the Profiling Tags Into Your Website	418
9.4	Encorporating All Available Fraud Tools	420
9.4.1	Implementation Options for TRMT	420
9.4.2	Implementation Checklist	421
9.4.3	Making a Decision	422
10	Apple Pay and Google Pay™ Integrations	423
10.1	About Apple Pay and Google Pay™ Integration	423
10.2	Apple Pay Transaction Process Overview	423
10.3	Google Pay™ Transaction Process Overview	425
10.4	About API Integration of Apple Pay and Google Pay™	430
10.4.1	Transaction Types Used for Apple Pay and Google Pay™	430
10.5	Cavv Purchase – Apple Pay and Google Pay™	431
10.6	Cavv Pre-Authorization – Apple Pay & Google Pay™	435
10.7	GooglePay Temporary Token Add – GooglePayTokenTempAdd	440
10.8	Google Pay™ Token Preauth	444
10.9	Google Pay™ Token Purchase	449
11	Offlinx™	455

11.1	What Is a Pixel Tag?	455
11.2	Offlinx™ and API Transactions	455
12	Convenience Fee	457
12.1	About Convenience Fee	457
12.2	Convenience Fee Information Object	457
12.3	Purchase with Convenience Fee	458
12.4	Purchase with Customer Info and Convenience Fee	461
12.5	Purchase with 3-D Secure and Convenience Fee	466
13	Recurring Billing	471
13.1	About Recurring Billing	471
13.2	Purchase with Recurring Billing	471
13.3	Recurring Billing Update	474
13.4	Recurring Billing Response Fields and Codes	478
13.5	Credential on File and Recurring Billing	479
14	Customer Information	481
14.1	Using the Customer Information Object	481
14.1.1	Customer Info Object – Miscellaneous Properties	482
14.1.2	Customer Info Object – Billing/Shipping Information	482
14.1.2.1	Set Methods for Billing and Shipping Info	483
14.1.2.2	Using Hash Tables for Billing and Shipping Info	485
14.1.3	Customer Info Object – Item Information	485
14.1.3.1	Set Methods for Item Information	486
14.1.3.2	Using Hash Tables for Item Information	486
14.2	Customer Information Sample Code	486
15	Status Check	491
15.1	About Status Check	491
15.2	Using Status Check Response Fields	491
15.3	Sample Purchase with Status Check	492
16	Testing a Solution	493
16.1	About the Merchant Resource Center	493
16.2	Logging In to the QA Merchant Resource Center	493
16.3	Test Credentials for Merchant Resource Center	494
16.4	Getting a Unique Test Store ID and API Token	495
16.5	Processing a Transaction	497
16.5.1	Overview	497
16.5.2	HttpPostRequest Object	498
16.5.3	Receipt Object	500
16.6	Testing MPI Solutions	500
16.7	Test Card Numbers	502
16.7.1	Test Card Numbers for Level 2/3	502
16.7.2	Test Cards for Visa Checkout	503
16.8	Simulator Host	503
17	Moving to Production	505
17.1	Activating a Production Store Account	505
17.2	Configuring a Store for Production	505
17.3	Receipt Requirements	506
17.3.1	Certification Requirements	507
Appendix A	Definition of Request Fields	508
A.1	Definition of Request Fields – Connection Fields	508
A.2	Definition of Request Fields – Core Fields	509
A.3	Definition of Request Fields – Credential on File	514
A.4	Definition of Request Fields – GooglePay Token Temp Add	516
A.5	Definition of Request Fields – Vault	517
A.6	Definition of Request Fields for Level 2/3 - Visa	518
A.7	Definition of Request Fields for Level 2/3 - Mastercard	528

A.8 Definition of Request Fields for Level 2/3 - Amex	538
A.9 Definition of Request Fields – 3-D Secure 2.2	548
A.10 Definition of Request Fields – MCP	556
A.11 Definition of Request Fields – Offlinx™	557
A.12 Definition of Request Fields – Convenience Fee	558
A.13 Definition of Request Fields – Recurring	558
A.14 Definition of Request Fields – Installments by Visa	559
A.15 Definition of Request Fields – Account Name Verification Object	560
Appendix B Definitions of Response Fields	561
B.1 Definition of Response Fields – 3-D Secure	577
B.2 Definition of Response Fields – MCP	581
B.3 Definition of Response Fields –Installments by Visa	583
Appendix C Response Codes	590
Appendix D Error Messages	605
Copyright Notice	607
Trademarks	607

Getting Help

Moneris has help for you at every stage of the integration process.

Getting Started	During Development	Production
Contact our Client Integration Specialists: clientintegrations@moneris.com	If you are already working with an integration specialist and need technical development assistance, contact our eProducts Technical Consultants: 1-866-319-7450 api@moneris.com	If your application is already live and you need production support, contact Moneris Customer Service: onlinepayments@moneris.com 1-866-319-7450 Available 24/7

For additional support resources, you can also make use of our community forums at

<http://community.moneris.com/product-forums/>

1 About This Documentation

1.1 Purpose

This document describes the transaction information for using the Moneris PHP API for sending credit card transactions. In particular, it describes the format for sending transactions and the corresponding responses you will receive.

1.2 Who Is This Guide For?

The Moneris Gateway API - Integration Guide - PHP is intended for developers integrating with the Moneris Gateway.

This guide assumes that the system you are trying to integrate meets the requirements outlined below and that you have some familiarity with the PHP programming language.

System Requirements

- PHP
- Port 443 open for bi-directional communication
- Web server with a SSL certificate
- cURL - PHP interface – see Adding cURL CA Root Certificate to PHP API

1.3 Adding cURL CA Root Certificate to PHP API

cURL CA Root Certificate File:

The default installation of PHP/cURL does not include the cURL CA root certificate file. In order for the Moneris Gateway PHP API to connect to the Moneris Gateway during transaction processing, the 'mpg-classes.php' file that's included with the PHP API package needs to be modified to include a path to the CA root certificate file.

To add the cURL CA root certificate file to the PHP API package, do the following:

1. If cURL was not installed separately from your PHP installation, libcurl is included in your PHP installation. You need to download the 'cacert.pem' file from <http://curl.haxx.se/docs/caextract.html> and save it to the necessary directory.

2. Once downloaded, rename the file to 'curl-ca-bundle.crt' (e.g., 'C:\path\to\curl-ca-bundle.crt'). If cURL was installed separately from PHP, you may need to determine the path to the cURL CA root certificate bundle on your system (e.g., 'C:\path\to\curl-ca-bundle.crt').
3. Insert the code below into the 'mpgclasses.php' file as part of the cURL option setting, at approximately line 73 below the line 'curl_setopt(\$ch, CURLOPT_SSL_VERIFYPEER, TRUE);'

```
curl_setopt($ch, CURLOPT_CAINFO, 'C:\path\to\curl-ca-bundle.crt');
```

For more information regarding the `CURLOPT_SSL_VERIFYPEER` option, please refer to your PHP documentation.

2 Basic Transaction Set

- 2.1 Purchase
- 2.2 Pre-Authorization
- 2.4 Pre-Authorization Completion
- 2.5 Force Post
- 2.6 Purchase Correction
- 2.7 Refund
- 2.8 Independent Refund
- 2.9 Card Verification with AVS and CVD
- 2.11 Batch Close

2.1 Purchase

Verifies funds on the customer's card, removes the funds and prepares them for deposit into the merchant's account.

```
$txnArray = array('type'=>'purchase', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Purchase transaction request fields – Required

Variable Name	Type and Limits	
order ID	String 50-character alphanumeric	'order_id'=>\$order_id
amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
credit card number	String 20-character alphanumeric	'pan'=>\$pan

Variable Name	Type and Limits	
expiry date	<i>String</i> 4-character alphanumeric (YYMM format)	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Purchase transaction request fields – Optional

Variable Name	Type and Limits	
customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo (\$mpgCvdInfo);
NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only—merchants must not store CVD information.		
Surcharge Information	<i>Object</i> N/A	\$mpgTxn->setSurchargeInfo (\$surchargeInfo);
NOTE: This object requires use of the Surcharge Lookup transaction prior to confirm card eligibility.		
Convenience Fee Information	<i>Object</i> N/A	\$mpgConvFee = new mpgConvFeeInfo (\$convFeeTemplate);
NOTE: This variable does not apply to Credential on File transactions.		

Variable Name	Type and Limits	
dynamic descriptor	<i>String</i> 20-character alphanumeric	'dynamic_descriptor'=>\$dynamic_descriptor
foreign indicator	<i>Boolean</i> true or false	'foreign_indicator'=>\$foreign_indicator
Surcharge Information	<i>Object</i> N/A	purchase \$mpgTxn->setSurchargeInfo(\$surchargeInfo); Contains fields related to Surcharge feature.
Installment Info	<i>Object</i> N/A	\$mpgTxn->setInstallmentInfo(\$installmentInfo);
For fields in this object, see 8.6 Installment Info Object		
NOTE: Do not send the Installment Info object on any transaction that is not intended to offer Installments by Visa functionality; doing so may cause the transaction to fail.		

Sample Purchase

```
<?php
require "../mpgClasses.php";

/***** Request Variables *****/
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';

/***** Transactional Variables *****/
$type='purchase';
$order_id='Test'.date("dmy-G:i:s");
$amount='6000.00';
$pan='4622943127023886';
$expdate='2212';
$crypt='7';
$dynamic_descriptor='123';
$status_check = 'false';
$foreign_indicator='true';

// TrId and TokenCryptogram are optional, refer documentation for more details.
$tr_id = '50189815682';
$token_cryptogram = 'APmbM/41le0uAAH+s6xMAADFA==';

/***** Transactional Associative Array *****/
```



```

$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expdate,
'crypt_type'=>$crypt
// 'dynamic_descriptor'=>$dynamic_descriptor
//, 'wallet_indicator' => '' //Refer to documentation for details
//, 'cm_id' => '8nAK8712sGaAkls56' //set only for usage with Offlinx - Unique max 50
alphanumeric characters transaction id generated by merchant
//, 'tr_id' => $tr_id
'foreign_indicator'=>$foreign_indicator
//, 'token_cryptogram' => $token_cryptogram
);

/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);

/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
// $mpgTxn->setForeignIndicator($foreign_indicator);

/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
// $mpgTxn->setInstallmentInfo($installmentInfo);

/***** Surcharge Info *OPTIONAL* *****/
$surchargeInfo = new SurchargeInfo();
$surchargeInfo->setSurchargeAmount("1.00");
$mpgTxn->setSurchargeInfo($surchargeInfo);

/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions

/***** HTTPS Post Object *****/
/* Status Check Example
$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);
*/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());

```

```

print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nStatusCode = " . $mpgResponse->getStatusCode());
print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
print("\nHostId = " . $mpgResponse->getHostId());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nAdviceCode = " . $mpgResponse->getAdviceCode());
// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());
?>

```

2.2 Pre-Authorization

Verifies and locks funds on the customer's credit card. The funds are locked for a specified amount of time based on the card issuer.

To retrieve the funds that have been locked by a Pre-Authorization transaction so that they may be settled in the merchant's account, a Pre-Authorization Completion transaction must be performed. A Pre-Authorization transaction may only be "completed" once.

Things to Consider:

- If a Pre-Authorization transaction is not followed by a Pre-Authorization Completion transaction, it must be reversed via a Pre-Authorization Completion transaction for 0.00. See 2.4 Pre-Authorization Completion
- A Pre-Authorization transaction may only be "completed" once

```

$txnArray = array('type'=>'preauth', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String	'api_token'=>\$api_token

Variable Name	Type and Limits	
	N/A	

Pre-Authorization transaction request fields – Required

Variable Name	Type and Limits	
order ID	String 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
credit card number	String max 20-character alphanumeric	'pan'=>\$pan
expiry date	String 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

Pre-Authorization transaction request fields – Optional

Variable Name	Type and Limits	
dynamic descriptor	String 20-character alphanumeric	'dynamic_descriptor'=>\$dynamic_descriptor

Variable Name	Type and Limits	
NOTE: For Pre-Authorization transactions: the value in the dynamic descriptor field will only be carried over to a Pre-Authorization Completion when executing the latter via the Merchant Resource Center; otherwise, the value for dynamic descriptor must be sent again in the Pre-Authorization Completion	total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	
is incremental	<i>Boolean</i>	'is_incremental'=>\$is_incremental
is_incremental	true/false	Indicates if this preauthorization is using an estimated amount. Estimations allow for incrementing the amount held via subsequent incrementalAuth requests. Defaults to false. NOTE: Please note that if this field is true, the preauthorization is only eligible for a single Preauthorization Completion. Any completion sent for partial completion is treated as a full completion (ship_indicator= P is treated as = F when is_incremental= true on the original preauth)
foreign indicator	<i>Boolean</i> true or false	'foreign_indicator'=>\$foreign_indicator
Customer Information	<i>Object</i> N/A	\$mpgTxn->setCustInfo(\$mpgCustInfo);
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);

Variable Name	Type and Limits	
NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only—merchants must not store CVD information.		
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
Surcharge Information	<i>Object</i> N/A NOTE: This object requires use of the Surcharge Lookup transaction prior to confirm card eligibility.	<pre>\$mpgTxn->setSurchargeInfo(\$surchargeInfo);</pre> Contains fields related to Surcharge feature.
wallet indicator	<i>String</i> 3-character alphanumeric NOTE: For basic Purchase and Preauthorization, the wallet indicator applies to Visa Checkout and MasterCard MasterPass only. For more, see Definition of Request Fields.	'wallet_indicator'=>\$wallet_indicator
Surcharge Information	<i>Object</i> N/A NOTE: This object requires use of the Vault Surcharge Lookup transaction prior to confirm card eligibility.	preauth <pre>\$mpgTxn->setSurchargeInfo(\$surchargeInfo);</pre> Contains fields related to Surcharge feature.

Sample Pre-Authorization

```
<?php
require "../mpgClasses.php";
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
/***** Transactional Variables *****/
$type='preauth';
$cust_id='cust id';
$order_id='ord-'.date("dmy-G:i:s");
$amount='6000.00';
$pan='4622943127023886';
$expdate='2212';
$crypt='7';
$dynamic_descriptor='123';
$status_check = 'false';
$foreign_indicator='true';
$is_incremental= 'true';

// TrId and TokenCryptogram are optional, refer documentation for more details.
$tr_id = '50189815682';
$token_cryptogram = 'APmbM/41le0uAAH+s6xMAAADFA==';
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expdate,
'crypt_type'=>$crypt,
'dynamic_descriptor'=>$dynamic_descriptor
//, 'cm_id' => '8nAK8712sGaAkls56' //set only for usage with Offlinx - Unique max 50
alphanumeric characters transaction id generated by merchant
//, 'tr_id' => $tr_id
//, 'token_cryptogram' => $token_cryptogram
'foreign_indicator'=>$foreign_indicator
);
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
//$mpgTxn->setInstallmentInfo($installmentInfo);

/***** Surcharge Info *OPTIONAL* *****/
$surchargeInfo = new SurchargeInfo();
$surchargeInfo->setSurchargeAmount("1.00");
$mpgTxn->setSurchargeInfo($surchargeInfo);

$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
```

```

print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nAdviceCode = " . $mpgResponse->getAdviceCode());

// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());
?>

```

2.3 Incremental Pre-Authorization

Increases the locked amount of funds in an existing pre-authorization for later settle by a single pre-authorization completion. There is no limit to the number of incremental pre-authorization transactions on the original estimated auth and each new incremental pre-authorization increases the hold on the customer's credit card.

Incremental Pre-authorizations require an estimated amount in the initial Pre-Authorization. This is set using the `<is_incremental>` field set to true.

For Mastercard only, an Incremental Pre-Authorization can be submitted with a \$0 value for the amount to request extending the allowable timeframe for completion (e.g, 30 days).

For additional details on using estimated amounts in Pre-Authorizations and using Incremental Pre-Authorizations to increase the locked amount of funds, see 1 Incremental Authorization Rules

Incremental Pre-Authorization transaction object definition

```

$txnArray = array('type'=>'incremental_preauth', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i>	'store_id'=>\$store_id

Variable Name	Type and Limits	
	N/A	
API token	<i>String</i>	'api_token'=>\$api_token
	N/A	

Incremental Preauthorization transaction request fields – Required

Variable Name	Type and Limits	
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
transaction number	<i>String</i> 255-character, alpha-numeric, hyphens or under-scores variable length	'txn_number'=>\$txn_number

Sample Incremental Pre-Authorization

```
<?php
require "../mpgClasses.php";
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
$orderid='ord-290824-5:27:32';
$txnnumber='16204-0_879';
$amount='20.20';
## step 1) create transaction array ###
$txnArray=array('type'=>'incremental_preauth',
```



```

'order_id'=>$orderid,
'txn_number'=>$txnnumber,
'amount'=>$amount
// 'ship_indicator'=>$ship_indicator, //optional
);
## step 2) create a transaction object passing the hash created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
?>

```

2.4 Pre-Authorization Completion

Retrieves funds that have been locked (by a Pre-Authorization transaction), and prepares them for settlement into the merchant's account.

Things to Consider:

- A Pre-Authorization transaction can only be completed once
- To reverse the full amount of a Pre-Authorization transaction, use the Pre-Authorization Completion transaction with the amount set to 0.00
- To process this transaction, you need the order ID and transaction number from the original Pre-Authorization transaction

```
$txnArray = array('type'=>'completion', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id Unique identifier provided by Moneris upon merchant account setup
API token	<i>String</i> N/A	'api_token'=>\$api_token Unique alphanumeric string assigned by Moneris upon merchant account activation To find your API token, refer to your test or production store's Admin settings in the Merchant Resource Center, at the following URLs: Testing: https://esqa.moneris.com/mpg/ Production: https://www3.moneris.com/mpg/

Pre-Authorization Completion transaction request fields – Required

Variable Name	Type and Limits	
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
completion amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'comp_amount'=>\$comp_amount

Variable Name	Type and Limits	
transaction number	<i>String</i> 255-character, alpha-numeric, hyphens or under-scores variable length	'txn_number'=>\$txn_number
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Pre-Authorization Completion transaction request fields – Optional

Variable Name	Type and Limits	Description
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
Surcharge Information	<i>Object</i> N/A NOTE: This object requires use of the Surcharge Lookup transaction prior to confirm card eligibility.	\$mpgTxn->setSurchargeInfo(\$surchargeInfo); Contains fields related to Surcharge feature.

Variable Name	Type and Limits	Description
shipping indicator	String 1-character alphanumeric	'ship_indicator'=>\$ship_indicator

Sample Pre-Authorization Completion

```
<?php
require "../mpgClasses.php";
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
$orderid='ord-290824-5:27:32';
$txnnumber='16204-0_879';
$compamount='200.00';
$dynamic_descriptor='123';
$ship_indicator = "F"; //optional
## step 1) create transaction array ###
$txnArray=array('type'=>'completion',
'txn_number'=>$txnnumber,
'order_id'=>$orderid,
'comp_amount'=>$compamount,
'crypt_type'=>'7',
'cust_id'=>'customer ID',
// 'ship_indicator'=>$ship_indicator, //optional
'dynamic_descriptor'=>$dynamic_descriptor
);
## step 2) create a transaction object passing the hash created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false or comment out this line for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
?>
```

2.5 Force Post

Retrieves the locked funds and prepares them for settlement into the merchant's account.

Used when a merchant obtains the authorization number directly from the issuer by a third-party authorization method (such as by phone).

Things to Consider:

- This transaction is an independent completion where the original Pre-Authorization transaction was not processed via the same Moneris Gateway merchant account.
- It is not required for the transaction that you are submitting to have been processed via the Moneris Gateway. However, a credit card number, expiry date and original authorization number are required.
- Force Post transactions are not supported for UnionPay

```
$txnArray = array('type'=>'forcepost', ...);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Force Post transaction request fields – Required

Variable Name	Type and Limits	
order ID	String 50-character alphanumeric	'order_id'=>\$order_id

Variable Name	Type and Limits	
	a-Z A-Z 0-9 _ - : . @ spaces	
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
authorization code	<i>String</i> 8-character alphanumeric	'auth_code'=>\$auth_code
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Force Post transaction request fields – Optional

Variable Name	Type and Limits	
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id

Variable Name	Type and Limits	
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor

Sample Force Post

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
//$status = 'false';
/***** Transactional Variables *****/
$type='forcepost';
$cust_id='CUST13343';
$order_id='ord-'.date("dmy-G:i:s");
$amount='10.00';
$pan='4242424242424242';
$expiry_date='0812';
$auth_code='123456';
$crypt='7';
$dynamic_descriptor='123456';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'auth_code'=>$auth_code,
'crypt_type'=>$crypt,
'dynamic_descriptor'=>$dynamic_descriptor
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
```

```

print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

2.6 Purchase Correction

Restores the full amount of a previous Purchase, Pre-Authorization Completion or Force Post transaction to the cardholder's card, and removes any record of it from the cardholder's statement.

This transaction can be used against a Purchase or Pre-Authorization Completion transaction that occurred same day provided that the batch containing the original transaction remains open.

Things to Consider:

- To process this transaction, you need the order ID and the transaction number from the original Completion, Purchase or Force Post transaction.

```

$txnArray = array('type'=>'purchasecorrection', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

Purchase Correction transaction request fields – Required

Variable Name	Type and Limits	
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
transaction number	<i>String</i> 255-character, alpha-numeric, hyphens or under-scores variable length	'txn_number'=>\$txn_number
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Purchase Correction transaction request fields – Optional

Variable Name	Type and Limits	
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
Surcharge Information	<i>Object</i> N/A NOTE: This object requires use of the Surcharge Lookup transaction prior to confirm card eligibility.	\$mpgTxn->setSurchargeInfo(\$surchargeInfo); Contains fields related to Surcharge feature.
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name	'dynamic_descriptor'=>\$dynamic_descriptor

Variable Name	Type and Limits	
	and separator	
	NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	

Sample Purchase Correction

```

<?php
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$orderid='ord-150816-12:36:20';
$txnnumber='117816-0_10';
$dynamic_descriptor='1234';
## step 1) create transaction hash ###
$txnArray=array('type'=>'purchasecorrection',
'txn_number'=>$txnnumber,
'order_id'=>$orderid,
'crypt_type'=>'7',
'cust_id'=>'customer ID',
'dynamic_descriptor'=>$dynamic_descriptor
);
## step 2) create a transaction object passing the array created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
//$mpgTxn->setInstallmentInfo($installmentInfo);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
// $installmentResults = $mpgResponse->getInstallmentResults();

```

```
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());
?>
```

2.7 Refund

Restores all or part of the funds from a Purchase, Pre-Authorization Completion or Force Post transaction to the cardholder's card.

Unlike a Purchase Correction, there is a record of both the initial charge and the refund on the cardholder's statement.

To process this transaction, you need the order ID and transaction number from the original Completion, Purchase or Force Post transaction.

```
$txnArray = array('type'=>'refund', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Refund transaction request fields – Required

Variable Name	Type and Limits	
order ID	String 50-character alpha-numeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id

Variable Name	Type and Limits	
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
transaction number	<i>String</i> 255-character, alpha-numeric, hyphens or underscores variable length	'txn_number'=>\$txn_number
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Pre-Authorization Completion transaction request fields – Optional

Variable Name	Type and Limits	Set Method
Surcharge Information NOTE: This object requires use of the Surcharge Lookup transaction prior to confirm card eligibility.	<i>Object</i> N/A	<pre>\$mpgTxn->setSurchargeInfo(\$surchargeInfo);</pre> Contains fields related to Surcharge feature.

Sample Refund

```
<?php
##
## This program takes 4 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
## 4. trans number
##
## Example php -q TestRefund.php store1 yesguy my_order_id 45109-89-0
##
require "../mpgClasses.php";
$store_id='store5';
```

```

$api_token='yesguy';
$orderid='ord-150816-11:56:58';
$txnnumber='117743-0_10';
$amount = '1.00';
$script_type = '7';
$dynamic_descriptor='123';
## step 1) create transaction array ###
$txnArray=array('type'=>'refund',
'txn_number'=>$txnnumber,
'order_id'=>$orderid,
'amount'=>$amount,
'crypt_type'=>$script_type,
'cust_id'=> 'Customer ID',
'dynamic_descriptor'=>$dynamic_descriptor
);
## step 2) create a transaction object passing the array created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
// $mpgTxn->setInstallmentInfo($installmentInfo);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());
?>

```

2.8 Independent Refund

Credits a specified amount to the cardholder's credit card. The credit card number and expiry date are mandatory.

It is not necessary for the transaction that you are refunding to have been processed via the Moneris Gateway.

Things to Consider:

- Because of the potential for fraud, permission for this transaction is not granted to all accounts by default. If it is required for your business, it must be requested via your account manager.

NOTE: Independent Refund transactions do not support Installments

```
$txnArray = array('type'=>'ind_refund', ...);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Independent Refund transaction request fields – Required

Variable Name	Type and Limits	
order ID	String 50-character alphanumeric	'order_id'=>\$order_id

Variable Name	Type and Limits	
	a-Z A-Z 0-9 _ - : . @ spaces	
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Independent Refund transaction request fields – Optional

Variable Name	Type and Limits	
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters includ-	'dynamic_descriptor'=>\$dynamic_descriptor

Variable Name	Type and Limits	
---------------	-----------------	--

ing your merchant name
and separator

NOTE:

Some special characters are not allowed:

<>\$%=?^{}[]\

Sample Independent Refund

```
<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
##
## Example php -q TestIndependentRefund.php store1 yesguy unique_order_id
##
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$orderid='ord-' . date("dmy-G:i:s");
$dynamic_descriptor='123456';
## step 1) create transaction array ###
$txnArray=array('type'=>'ind_refund',
'order_id'=>$orderid,
'cust_id'=>'my cust id',
'amount'=>'1.00',
'pan'=>'4242424242424242',
'expdate'=>'1103',
'crypt_type'=>'7',
'dynamic_descriptor'=>$dynamic_descriptor
);
## step 2) create a transaction object passing the array created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
```



```
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
?>
```

2.9 Card Verification with AVS and CVD

Verifies the validity of the credit card, expiry date and any additional details (such as the Card Verification Digits or Address Verification details). It does not verify the available amount or lock any funds on the credit card.

Things to Consider:

- The Card Verification transaction is only supported by Visa, Mastercard, Discover and American Express
- For some Credential on File transactions, Card Verification with AVS and CVD is used as a prior step to get the Issuer ID used in the subsequent transaction
- When testing Card Verification, please use the Visa and MasterCard test card numbers provided in the MasterCard Card Verification and Visa Card Verification tables available in CVD & AVS (E-Fraud) Simulator.
- For a full list of possible AVS & CVD result codes refer to the CVD and AVS Result Code tables.

```
$txnArray = array('type'=>'card_verification', ...);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Card Verification transaction request fields – Required

Variable Name	Type and Limits	
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);
NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only—merchants must not store CVD information.		

Account Name Verification object request fields

Request fields within the Account Name Verification object. The object can only be included in Card Verification transactions. Account name verification is only applicable to Visa credit cards.

Variable Name	Type and Limits	
First Name	String 32-character alphanumeric	Cardholder last name
Middle Name	String 32-character alphanumeric	Cardholder middle name
Last Name	String 32-character alphanumeric	Cardholder last name

Sample Card Verification

```

<?php
require "../mpgClasses.php";
$store_id='store5';
$api_token="yesguy";
// TrId and TokenCryptogram are optional, refer documentation for more details.
$str_id = '50189815682';
$token_cryptogram = 'APmbM/411e0uAAH+s6xMAAADFA==';

$txnArray=array('type'=>'card_verification',
'order_id'=>'ord-'.date("dmy-G:i:s"),
'cust_id'=>'my cust id',
'pan'=>'4242424242424242',
'expdate'=>'1512',
'crypt_type'=>'7'
//,'tr_id' => $str_id
//,'token_cryptogram' => $token_cryptogram
);
$mpgTxn = new mpgTransaction($txnArray);
/***** AVS Variables *****/
$avs_street_number = '201';
$avs_street_name = 'Michigan Ave';
$avs_zipcode = 'M1M1M1';
/***** CVD Variables *****/
$cvd_indicator = '1';
$cvd_value = '198';

/***** Account Name Variables *****/
$first_name = 'FIRST';
$middle_name = 'MIDDL';
$last_name = 'LAST';

/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number'=>$avs_street_number,
'avs_street_name' =>$avs_street_name,
'avs_zipcode' => $avs_zipcode
);
/***** CVD Associative Array *****/
$cvdTemplate = array(

```

```

'cvd_indicator' => $cvd_indicator,
'cvd_value' => $cvd_value
);

/***** AccountName Array *****/
$accountNameTemplate = array(
'first_name'=>$first_name,
'middle_name' =>$middle_name,
'last_name' => $last_name
);

/***** AVS Object *****/
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** CVD Object *****/
$mpgCvdInfo = new mpgCvdInfo ($cvdTemplate);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");

/***** AccountNameObject *****/
$mpgAccountNameInfo = new mpgAccountNameInfo ($accountNameTemplate);

/***** Set Objects *****/
$mpgTxn->setAccountNameVerification($mpgAccountNameInfo);
$mpgTxn->setAvsInfo($mpgAvsInfo);
$mpgTxn->setCvdInfo($mpgCvdInfo);
$mpgTxn->setCofInfo($cof);

$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nAccountNameVerificationResult = " . $mpgResponse->getAccountNameResult());
?>

```

2.10 Surcharge Lookup

Confirm eligibility of a card for merchant surcharges on a transaction.

Credit Surcharge is feature where merchants may append an additional charge to the amount of a transaction processed on an eligible credit card. Only eligible credit cards can be surcharged, so merchants must confirm eligibility of the card before processing the payment via surcharge lookup.

The following transactions support including surcharge amounts:

- Purchase (Basic)
- Pre-Authorization (Basic)
- Vault Purchase
- Vault Pre-Authorization
- 3DS Purchase
- 3DS Pre-Authorization
- 3DS & Vault Purchase
- 3DS & Vault Pre-Authorization
- Purchase Correction
- Pre-Authorization Completion
- Refund

Things to Consider:

- Card association rules require that cardholders must be informed of any surcharge by a merchant.
- The cardholder must be offered an option for canceling the transaction if they refuse the surcharge.

```
$txnArray = array('type'=>'surcharge_lookup', ...);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

Surcharge Lookup transaction request fields – Required

Variable Name	Type and Limits	
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point	'amount'=>\$amount
EXAMPLE: 1234567.89		

Sample Surcharge Lookup

```

<?php
require "../mpgClasses.php";
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
$amount = '50.00';
$pan='4242424242424242';
$txnArray=array('type'=>'surcharge_lookup',
'amount'=>$amount,
'pan'=>$pan
);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
//print the mpgrequest
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nMessage = " . $mpgResponse->getMessage());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nIsSurchargeEligible = " . $mpgResponse->getIsSurchargeEligible());
print("\nMaxSurchargeRate = " . $mpgResponse->getMaxSurchargeRate());
print("\nMaxSurchargeAmount = " . $mpgResponse->getMaxSurchargeAmount());
print("\nServiceType = " . $mpgResponse->getServiceType());
?>

```

2.11 Batch Close

Takes the funds from all Purchase, Completion, Refund and Force Post transactions so that they will be deposited or debited the following business day.

For funds to be deposited the following business day, the batch must close before 11 pm Eastern Time.

```

$txnArray = array('type'=>'batchclose', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

Batch Close transaction request fields – Required

Variable Name	Type and Limits	
ECR (electronic cash register) number	<i>String</i> No limit (value provided by Moneris)	ecr_number=>\$ecr_number

Sample Batch Close

```

<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. ecr number
##
## Example php -q TestBatchClose.php store1 yesguy 66002173
##
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$ecr_number='66013455';
## step 1) create transaction array ###
$txnArray=array('type'=>'batchclose',
'ecr_number'=>$ecr_number
);
$mpgTxn = new mpgTransaction($txnArray);
## step 2) create mpgRequest object ###
$mpgReq=new mpgRequest($mpgTxn);
$mpgReq->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgReq->setTestMode(true); //false for production transactions
## step 3) create mpgHttpPost object which does an https post ##
$mpgHttpPost=new mpgHttpPost($store_id,$api_token,$mpgReq);
## step 4) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();

```

```

##step 5) get array of all credit cards
$creditCards = $mpgResponse->getCreditCards($ecr_number);
## step 6) loop through the array of credit cards and get information
for($i=0; $i < count($creditCards); $i++)
{
    print "\nCard Type = $creditCards[$i]";
    print "\nPurchase Count = "
        . $mpgResponse->getPurchaseCount($ecr_number,$creditCards[$i]);
    print "\nPurchase Amount = "
        . $mpgResponse->getPurchaseAmount($ecr_number,$creditCards[$i]);
    print "\nRefund Count = "
        . $mpgResponse->getRefundCount($ecr_number,$creditCards[$i]);
    print "\nRefund Amount = "
        . $mpgResponse->getRefundAmount($ecr_number,$creditCards[$i]);
    print "\nCorrection Count = "
        . $mpgResponse->getCorrectionCount($ecr_number,$creditCards[$i]);
    print "\nCorrection Amount = "
        . $mpgResponse->getCorrectionAmount($ecr_number,$creditCards[$i]);
}
?>

```

2.12 Open Totals

Returns the details about the currently open batch.

Similar to the Batch Close; the difference is that it does not close the batch for settlement.

```
$txnArray = array('type'=>'opentotals', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Open Totals transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	
ECR (electronic cash register) number	No limit (value provided by Moneris)	ecr_number=>\$ecr_number

Sample Open Totals

```
<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. ecr number
##
## Example php -q TestOpenTotals.php store1 yesguy 66002163
##
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$ecr_number='66013455';
## step 1) create transaction array ###
$txnArray=array('type'=>'opentotals',
'ecr_number'=>$ecr_number
);
$mpgTxn = new mpgTransaction($txnArray);
## step 2) create mpgRequest object ###
$mpgReq= new mpgRequest($mpgTxn);
$mpgReq->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgReq->setTestMode(true); //false for production transactions
## step 3) create mpgHttpPost object which does an https post ##
$mpgHttpPost=new mpgHttpPost($store_id,$api_token,$mpgReq);
## step 4) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
##step 5) get array of all credit cards
$creditCards = $mpgResponse->getCreditCards($ecr_number);
## step 6) loop through the array of credit cards and get information
for($i=0; $i < count($creditCards); $i++)
{
print "\nCard Type = $creditCards[$i]";
print "\nPurchase Count = "
. $mpgResponse->getPurchaseCount($ecr_number,$creditCards[$i]);
print "\nPurchase Amount = "
. $mpgResponse->getPurchaseAmount($ecr_number,$creditCards[$i]);
print "\nRefund Count = "
. $mpgResponse->getRefundCount($ecr_number,$creditCards[$i]);
print "\nRefund Amount = "
. $mpgResponse->getRefundAmount($ecr_number,$creditCards[$i]);
print "\nCorrection Count = "
. $mpgResponse->getCorrectionCount($ecr_number,$creditCards[$i]);
print "\nCorrection Amount = "
. $mpgResponse->getCorrectionAmount($ecr_number,$creditCards[$i]);
}
?>
```

3 Credential on File

- 3.1 About Credential on File
- 3.2 Credential on File Info Object and Variables
- 3.3 Credential on File Transaction Types
- 3.5 Initial Transactions in Credential on File
- 3.9 Credential on File and Converting Temporary Tokens
- 3.8 Vault Tokenize Credit Card and Credential on File
- 3.10 Card Verification and Credential on File Transactions

3.1 About Credential on File

When storing customers' credit card credentials for use in future authorizations, or when using these credentials in subsequent transactions, card brands now require merchants to indicate this in the transaction request.

In the Moneris API, this is handled by the Moneris Gateway via the inclusion of the Credential on File info object and its variables in the transaction request.

While the requirements for handling Credential on File transactions relate to Visa, Mastercard and Discover only, in order to avoid confusion and prevent error, please implement these changes for all card types and the Moneris system will then correctly flow the relevant card data values as appropriate.

NOTE: If either the first transaction or a Card Verification authorization is declined when attempting to store cardholder credentials, those credentials cannot be stored —therefore the merchant must not use the credential for any subsequent transactions.

3.2 Credential on File Info Object and Variables

The Credential on File Info object is nested within the request for the applicable transaction types.

Credential on File Info Object:

cof

Variables in the cof object:

Payment Indicator

Payment Information

Issuer ID

For more information, see Definition of Request Fields – Credential on File.

3.3 Credential on File Transaction Types

The Credential on File Info object applies to the following transaction types:

- Purchase
- Pre-Authorization
- Purchase with 3-D Secure – cavv_purchase
- Purchase with 3-D Secure and Recurring Billing
- Pre-Authorization with 3-D Secure – cavv_preauth
- Purchase with Vault – ResPurchaseCC
- Pre-Authorization with Vault – ResPreauthCC
- Card Verification with AVS and CVD
- Card Verification with Vault – ResCardVerificationCC
- Vault Add Credit Card – ResAddCC
- Vault Update Credit Card – ResUpdateCC
- Vault Add Token – ResAddToken
- Vault Tokenize Credit Card – ResTokenizeCC
- Recurring Billing
- MCP Purchase
- MCP Pre-Authorization
- MCP Pre-Authorization Completion

- MCP Purchase with Vault
- MCP Pre-Authorization with Vault

3.4 Merchant- vs. Cardholder-Initiated COF Transactions

Transactions defined as Credential on File (COF) can be initiated one of two ways: by a merchant or by a cardholder. The initiator of a transaction is important because it determines which Credential on File indicator fields need to be sent in the transaction request.

Merchant-initiated Credential on File transactions: transactions in which the merchant intends to store cardholder credentials or use credentials that have already been stored. This includes sending the Credential on File Info object in the transaction request, and including all three of its fields: **issuer ID**, **payment indicator** and **payment information**.

Cardholder-initiated Credential on File transactions: transactions which are triggered by some action by the cardholder. For cardholder-initiated transactions, only the **payment indicator** and **payment information** fields are required.

For simplicity in developing your integration, the Moneris Gateway also allows cardholder-initiated transactions to be processed according to the same Credential on File rules as apply to merchant-initiated transactions. Technically, the **issuer ID** indicator is not required for cardholder-initiated transactions, but for convenience, if it is included in the transaction request, the Moneris Gateway will just ignore it when forwarding the request to the host.

3.5 Initial Transactions in Credential on File

When sending an *initial* transaction with the Credential on File Info object, i.e., a transaction request where the cardholder's credentials are being stored for the *first* time, it is important to understand the following:

- You must send the cardholder's Card Verification Digits (CVD)
- **Issuer ID** will be sent without a value on the initial transaction, because it is received in the response to that initial transaction; for all *subsequent* merchant-initiated transactions and all administrative transactions you send this **Issuer ID**
- The **payment information** field should always be set to a value of 0 on the first transaction
- The **payment indicator** field should be set to the value that is appropriate for the transaction

3.6 COF With Previously Stored Credentials

When processing a transaction with cardholder information that was already stored **prior to** the implementation of Credential on File requirements, you must:

- Include the Credential on File Info object in the transaction request, and
- Send the **payment information** value as 2, and
- Store the **issuer ID** returned in the transaction and associate it with the cardholder credentials for future use

Once the **issuer ID** has been stored and associated with the cardholder credentials, send it in all subsequent transactions going forward. **Issuer ID** is only required when sending merchant-initiated transactions.

3.7 Payment Indicator and E-Commerce Indicator Values

When sending Credential on File information in transaction requests that also include the **e-commerce indicator** request field (in code, referred to as `crypt_type`), acceptable values for the e-commerce indicator are dependent upon the value being sent for the **payment indicator**.

If Payment Indicator Value is:	Allowable e-commerce indicator values are:
R Fixed-Rate Recurring	2 – Mail Order / Telephone Order—Recurring 5 – Authenticated e-commerce transaction (3-D Secure) 6 – Non-authenticated e-commerce transaction (3-D Secure)
V Variable-Rate Recurring	2 – Mail Order / Telephone Order—Recurring (Variable) 5 – Authenticated e-commerce transaction (3-D Secure) 6 – Non-authenticated e-commerce transaction (3-D Secure)
NOTE: MasterCard Only	
C Credentials On File	1 – Mail Order / Telephone Order—Single 5 – Authenticated e-commerce transaction (3-D Secure) 6 – Non-authenticated e-commerce transaction (3-D Secure) 7 – SSL-enabled merchant
U	1 – Mail Order / Telephone Order—Single

If Payment Indicator Value is:	Allowable e-commerce indicator values are:
Unscheduled Stored Credential Transaction	7 – SSL-enabled merchant
Z	1 – Mail Order / Telephone Order—Single
Unscheduled Customer-Initiated Transaction	5 – Authenticated e-commerce transaction (3-D Secure)
	6 – Non-authenticated e-commerce transaction (3-D Secure)
	7 – SSL-enabled merchant

3.8 Vault Tokenize Credit Card and Credential on File

When you want to store cardholder credentials from previous transactions into the Vault, you use the Vault Tokenize Credit Card transaction request. Credential on File rules require that only previous transactions with the Credential on File Info object can be tokenized to the Vault.

For more information about this transaction, see 4.5.10 Vault Tokenize Credit Card – ResTokenizeCC.

3.9 Credential on File and Converting Temporary Tokens

In the event you decide to convert a temporary token representing cardholder credentials into a permanent token, these credentials become stored credentials, and therefore it is necessary to send Credential on File information.

For Vault Temporary Token Add transactions where you subsequently decide to convert the temporary token into a permanent token (stored credentials):

1. Send a transaction request that includes the Credential on File Info object to get the Issuer ID; this can be a Card Verification, Purchase or Pre-Authorization request
2. After completing the transaction, send the Vault Add Token request with the Credential on File object (Issuer ID only) in order to convert the temporary token to a permanent one.

3.10 Card Verification and Credential on File Transactions

In the absence of a Purchase or Pre-Authorization, a Card Verification transaction is used to get the unique issuer ID value (**issuerId**) that is used in subsequent Credential on File transactions. Issuer ID is a variable included in the nested Credential on File Info object.

For all first-time transactions, including Card Verification transactions, you must also request the cardholder's Card Verification Details (CVD). For more on CVD, see 9.2 Card Validation Digits (CVD).

For a complete list of these variables, see each transaction type or Definition of Request Fields – Credential on File

The Card Verification request, including the Credential on File Info object, must be sent immediately prior to storing cardholder credentials.

For information about Card Verification, see 2.9 Card Verification with AVS and CVD.

3.10.1 When to Use Card Verification With COF

If you are not sending a Purchase or Pre-Authorization transaction (i.e., you are not charging the customer immediately), you must use Card Verification (or in the case of Vault Add Token, Card Verification with Vault) first before running the transaction in order to get the Issuer ID.

Transactions this applies to:

Vault Add Credit Card – ResAddCC

Vault Update Credit Card – ResUpdateCC

Vault Add Token – ResAddToken

Recurring Billing transactions, if:

- the first transaction is set to start on a future date

3.10.2 Credential on File and Vault Add Token

For Vault Add Token transactions:

1. Send Card Verification with Vault transaction request including the Credential on File object to get the Issuer ID
2. Send the Vault Add Token request including the Credential on File object (with Issuer ID only; other fields are not applicable)

For more on this transaction type, see 4.5.9 Vault Add Token – ResAddToken.

3.10.3 Credential on File and Vault Update Credit Card

For Vault Update Credit Card transactions where you are updating the credit card number:

1. Send Card Verification transaction request including the Credential on File object to get the Issuer ID
2. Send the Vault Update Credit Card request including the Credential on File Info object (Issuer ID only).

For more on this transaction type, see 4.5.3 Vault Update Credit Card – ResUpdateCC.

3.10.4 Credential on File and Vault Add Credit Card

For Vault Add Credit Card transactions:

1. Send Card Verification transaction request including the Credential on File object to get the Issuer ID
2. Send the Vault Add Credit Card request including the Credential on File Info object (Issuer ID only)

For more on this transaction type, see 4.5.1 Vault Add Credit Card – ResAddCC.

3.10.5 Credential on File and Recurring Billing

NOTE: The value of the **payment indicator** field must be **R** when sending Recurring Billing transactions.

For Recurring Billing transactions which are set to start **immediately**:

1. Send a Purchase transaction request with both the Recurring Billing and Credential on File info objects (with Recurring Billing object field **start now** = true)

For Recurring Billing transactions which are set to start on a **future** date:

1. Send Card Verification transaction request including the Credential on File info object to get the Issuer ID
2. Send Purchase transaction request with the Recur and Credential on File info objects included

For updating a Recurring Billing series where you are updating the card number (does not apply if you are only modifying the schedule or amount in a recurring series):

1. Send Card Verification request including the Credential on File info object to get the Issuer ID
2. Send a Recurring Billing Update transaction

For more information about the Recurring Billing object, see [Definition of Request Fields – Recurring](#).

4 Vault

- 4.1 About the Vault Transaction Set
- 4.2 Vault Transaction Types
- 4.3 Vault Transactions That Support Temporary Tokens
- 4.5 Vault Administrative Transactions
- 4.6 Vault Financial Transactions
- 4.7 Hosted Tokenization

4.1 About the Vault Transaction Set

The Vault feature allows merchants to create customer profiles, edit those profiles, and use them to process transactions without having to enter financial information each time. Customer profiles store customer data essential to processing transactions, including credit and signature debit.

The Vault is a complement to the Recurring Billing module. It securely stores customer account information on Moneris secure servers. This allows merchants to bill customers for routine products or services when an invoice is due.

4.2 Vault Transaction Types

The Vault API supports both administrative and financial transactions.

4.2.1 Administrative Vault Transaction types

ResAddCC

Creates a new credit card profile, and generates a unique data key which can be obtained from the Receipt object.

This data key is the profile identifier that all future financial Vault transactions will use to associate with the saved information.

EncResAddCC

Creates a new credit card profile, but requires the card data to be either swiped or manually keyed in via a Moneris-provided encrypted mag swipe reader.

ResTempAdd

Creates a new temporary token credit card profile. This transaction requires a duration to be set to indicate how long the temporary token is to be stored for.

During the lifetime of this temporary token, it may be used for any other vault transaction before it is permanently deleted from the system.

ResUpdateCC

Updates a Vault profile (based on the data key) to contain credit card information.

All information contained within a credit card profile is updated as indicated by the submitted fields.

EncResUpdateCC

Updates a profile (based on the data key) to contain credit card information. The encrypted version of this transaction requires the card data to either be swiped or manually keyed in via a Moneris-provided encrypted mag swipe reader.

ResDelete

Deletes an existing Vault profile of any type using the unique data key that was assigned when the profile was added.

It is important to note that after a profile is deleted, the information which was saved within can no longer be retrieved.

ResLookupFull

Verifies what is currently saved under the Vault profile associated with the given data key. The response to this transaction returns the latest active data for that profile.

Unlike ResLookupMasked (which returns the masked credit card number), this transaction returns both the masked and the unmasked credit card numbers.

ResLookupMasked

Verifies what is currently saved under the Vault profile associated with the given data key. The response to this transaction returns the latest active data for that profile.

Unlike ResLookupFull (which only returns both the masked and the unmasked credit card numbers), this transaction only returns the masked credit card number.

ResGetExpiring

Verifies which profiles have credit cards that are expiring during the current and next calendar month. For example, if you are processing this transaction on September 30, then it will return all cards that expire(d) in September and October of this year.

When generating a list of profiles with expiring credit cards, only the **masked** credit card numbers are returned.

This transaction can be performed no more than 2 times on any given calendar day, and it only applies to credit card profiles.

ResIsCorporateCard

Determines whether a profile has a corporate card registered within it.

After sending the transaction, the response field to the Receipt object's getCorporateCard method is either `true` or `false` depending on whether the associated card is a corporate card.

ResAddToken

Converts a Hosted Tokenization temporary token to a permanent Vault token.

A temporary token is valid for 15 minutes after it is created.

ResTokenizeCC

Creates a new credit card profile using the credit card number, expiry date and e-commerce indicator that were submitted in a previous financial transaction. A transaction that was previously done in Moneris Gateway is taken, and the card data from that transaction is stored in the Moneris Vault.

As with ResAddCC, a unique data key is generated and returned to the merchant via the Receipt object. This is the profile identifier that all future financial Vault transactions will use to associate with the saved information.

4.2.2 Financial Vault Transaction types

ResPurchaseCC

Uses the data key to identify a previously registered credit card profile. The details saved within the profile are then submitted to perform a Purchase transaction.

ResPreauthCC

Uses the data key to identify a previously registered credit card profile. The details within the profile are submitted to perform a Pre-Authorization transaction.

ResIndRefundCC

Uses the unique data key to identify a previously registered credit card profile, and credits a specified amount to that credit card.

ResMpiTxn

Uses the data key (as opposed to a credit card number) in a VBV/SecureCode Txn MPI transaction. The merchant uses the data key with ResMpiTxn request, and then reads the response fields to verify whether the card is enrolled in Verified by Visa or MasterCard SecureCode. Retrieves the vault transaction value to pass on to Visa or MasterCard.

After it has been validated that the data key is enrolled in 3-D Secure, a window appears in which the customer can enter the 3-D Secure password. The merchant may initiate the forming of the validation form `getMpiInlineForm()`.

For more information on integrating with MonerisMPI, refer to 1 MPI.

4.3 Vault Transactions That Support Temporary Tokens

The following transactions support temporary as well as permanent tokens:

- Purchase with Vault – ResPurchaseCC
- Pre-Authorization with Vault – ResPreauthCC
- Card Verification with Vault – ResCardVerificationCC
- Purchase with Vault and 3-D Secure

- Pre-Authorization with Vault & 3-D Secure
- Force Post with Vault – ResForcePostCC
- Vault Independent Refund – ResIndRefundCC
- Vault Is Corporate Card - ResIsCorporateCard

4.4 Vault and Installments

Installments functionality is also available on transactions using cardholder credentials stored in the Moneris Vault. To offer this feature to the customer, send the Vault Installment Plan Lookup transaction prior to running a Purchase with Vault or Pre-Authorization with Vault.

For more about Installments, see 8 Installments by Visa

4.5 Vault Administrative Transactions

Administrative transactions allow you to perform such tasks as creating new Vault profiles, deleting existing Vault profiles and updating profile information.

Some Vault Administrative Transactions require the Credential on File object to be sent with the **issuer ID** field only.

4.5.1 Vault Add Credit Card – ResAddCC

Creates a new credit card profile, and generates a unique data key which can be obtained from the Receipt object.

This data key is the profile identifier that all future financial Vault transactions will use to associate with the saved information.

Vault Add Credit Card transaction object definition

```
$txnArray = array('type'=>'res_add_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Add Credit Card transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Vault Add Credit Card transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo(\$cof);
cof	N/A	
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		

Vault Add Credit Card transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
email address	<i>String</i> 30-character alphanumeric	'email'=>\$email

Variable Name	Type and Limits	Set Method
phone number	<i>String</i> 30-character alphanumeric	'phone'=>\$phone
note	<i>String</i> 30-character alphanumeric	'note'=>\$note
data key format	<i>String</i> 2-character alphanumeric	'data_key_format'=>\$data_key_format
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);

Credential on File Info object request fields

Variable Name	Type and Limits	Set Method
issuer ID NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in the transaction response and then used in subsequent transaction requests.	<i>String</i> 15-character alphanumeric variable length	\$cof->setIssuerId("VALUE_FOR_ISSUER_ID"); NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File

Sample Vault Add Credit Card

```

<?php
##
## Example php -q TestResAddCC.php store3 yesguy
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_add_cc';
$cust_id='customer1';

```

Sample Vault Add Credit Card

```

$phone = '5555551234';
$email = 'bob@smith.com';
$note = 'this is my note';
$pan='5454545454545454';
$expiry_date='1412';
$crypt_type='1';
$data_key_format = "0";
$avs_street_number = '123';
$avs_street_name = 'lakeshore blvd';
$avs_zipcode = '90210';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'cust_id'=>$cust_id,
'phone'=>$phone,
'email'=>$email,
'note'=>$note,
'pan'=>$pan,
'expdate'=>$expiry_date,
// 'data_key_format'=>$data_key_format, //optional
'crypt_type'=>$crypt_type
);
/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number' => $avs_street_number,
'avs_street_name' => $avs_street_name,
'avs_zipcode' => $avs_zipcode
);
/***** AVS Object *****/
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setIssuerId("139X3130ASCXAS9");
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setAvsInfo($mpgAvsInfo);
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\n\nPhone = " . $mpgResponse->getResDataPhone());
print("\n\nEmail = " . $mpgResponse->getResDataEmail());
print("\n\nNote = " . $mpgResponse->getResDataNote());

```


Sample Vault Add Credit Card

```
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.1.1 Vault Data Key

The ResAddCC sample code includes the following instruction from the Receipt object:

```
print("\nDataKey = " . $mpgResponse->getDataKey());
```

The data key response field is populated when you send a Vault Add Credit Card – ResAddCC (page 69), Vault Encrypted Add Credit Card – EncResAddCC (page 73), Vault Tokenize Credit Card – ResTokenizeCC (page 97), Vault Temporary Token Add – ResTempAdd (page 76) or Vault Add Token – ResAddToken (page 94) transaction. It is the profile identifier that all future financial Vault transactions will use to associate with the saved information.

The data key is a maximum 28-character alphanumeric string.

4.5.1.2 Vault Encrypted Add Credit Card – EncResAddCC

Vault Encrypted Add Credit Card transaction object definition

```
$txnArray = array('type'=>'enc_res_add_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Encrypted Add Credit Card transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Encrypted Add Credit Card transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
encrypted track 2 data	<i>String</i>	'enc_track2'=>\$enc_track2

Variable Name	Type and Limits	Set Method
	40-character numeric	
device type	String 30-character alphanumeric case sensitive	'device_type'=>\$device_type
electronic commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

Vault Encrypted Add Credit Card transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	String 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
AVS Information	Object N/A	\$mpgTxn->setAvsInfo (\$mpgAvsInfo);
email address	String 30-character alphanumeric	'email'=>\$email
phone number	String 30-character alphanumeric	'phone'=>\$phone
note	String 30-character alphanumeric	'note'=>\$note
data key format	String 2-character alphanumeric	'data_key_format'=>\$data_key_format

Sample Vault Encrypted Add Credit Card

```

<?php
require "../..//mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='enc_res_add_cc';
$cust_id='cust1';
$phone = '6479996999';
$email = 'bob@smith.com';
$note = 'this is my note';
$enc_track2 = 'ENCRYPTEDTRACK2DATA';
$device_type='idtech_bdk';
$data_key_format="0";
$crypt_type='7';
$avs_street_number = '11';
$avs_street_name = 'lakeshore blvd';
$avs_zipcode = 'm8x2x2';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'cust_id'=>$cust_id,
'phone'=>$phone,
'email'=>$email,
'note'=>$note,
'enc_track2'=>$enc_track2,
'device_type'=>$device_type,
// 'data_key_format'=>$data_key_format, //optional
'crypt_type'=>$crypt_type
);
/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number' => $avs_street_number,
'avs_street_name' => $avs_street_name,
'avs_zipcode' => $avs_zipcode
);
/***** AVS Object *****/
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setAvsInfo($mpgAvsInfo);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());

```

Sample Vault Encrypted Add Credit Card

```
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.2 Vault Temporary Token Add – ResTempAdd

Creates a new temporary token credit card profile. This transaction requires a duration to be set to indicate how long the temporary token is to be stored for.

During the lifetime of this temporary token, it may be used for any other Vault transaction before it is permanently deleted from the system.

Things to Consider:

- The duration, or lifetime, of the temporary token can be set to be a maximum of 15 minutes.

Vault Temporary Token Add transaction object definition

```
$txnArray = array('type'=>'res_temp_add', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Temporary Token Add transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Temporary Token Add transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
duration	<i>String</i> 3-character numeric maximum 900 seconds	'duration'=>\$duration
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Vault Temporary Token Add transaction request fields – Optional

Variable Name	Type and Limits	Set Method
data key format	<i>String</i> 2-character alphanumeric	'data_key_format'=>\$data_key_format

Sample Vault Temporary Token Add

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_temp_add';
$pan='5454545454545454';
$expiry_date='1509';
$duration='900';
$data_key_format = "0";
$crypt_type='7';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'pan'=>$pan,
'expdate'=>$expiry_date,
'duration'=>$duration,
// 'data_key_format'=>$data_key_format, //optional
'crypt_type'=>$crypt_type
```

Sample Vault Temporary Token Add

```
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
?>
'crypt_type'=>$crypt_type
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.3 Vault Update Credit Card – ResUpdateCC

Updates an existing Vault profile (referencing the profile's unique **data key**) with cardholder information.

Information contained within a credit card profile is updated as indicated by the submitted fields; if any field representing an item of cardholder information is not sent in this request, that item will remain unchanged in the profile.

If the Vault profile is being updated with a new credit card number, then you first need to send a Purchase, Pre-Authorization or Card Verification transaction, with the Credential on File Info object included, before performing Vault Update Credit Card. If the credit card number is not one of the profile items being updated, this step is not required.

Things to Consider:

- To update a specific element in the profile, set that element using the corresponding set method
- When updating a credit card number, first send a Purchase, Pre-Authorization, or Card Verification with the Credential on File Info object before sending this transaction; send the issuer ID received in the response in the subsequent Vault Update Credit Card request
- If the credit card number is not one of the profile items being updated, the Credential on File info object is not required

Vault Update Credit Card transaction object definition

```
$txnArray = array('type'=>'res_update_cc', ...);  
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Update Credit Card transaction

```
$mpgRequest = new mpgRequest($mpgTxn);  
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Update Credit Card transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
data key	<i>String</i>	'data_key'=>\$data_key

Variable Name	Type and Limits	Set Method
	25-character alphanumeric	

Optional values that are submitted to the ResUpdateCC object are updated. Unsubmitted optional values (with one exception) remain unchanged. This allows you to change only the fields you want.

If a profile contains AVS information, but a Vault Update Credit Card transaction is submitted without an AVS Info object, the existing AVS Info details are deactivated and the new credit card information is registered without AVS.

Vault Update Credit Card transaction request fields – Optional

Variable Name	Type and Limits	Set Method
credit card number	<i>String</i> 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
email address	<i>String</i> 30-character alphanumeric	'email'=>\$email
phone number	<i>String</i> 30-character alphanumeric	'phone'=>\$phone
note	<i>String</i> 30-character alphanumeric	'note'=>\$note

Variable Name	Type and Limits	Set Method
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo (\$mpgAvsInfo) ;
Credential on File Info cof	<i>Object</i> N/A	\$mpgTxn->setCofInfo (\$cof) ;
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		

Credential on File Info object request fields

Variable Name	Type and Limits	Set Method
issuer ID	<i>String</i> 15-character alphanumeric variable length	\$cof->setIssuerId ("VALUE_ FOR_ISSUER_ID") ;
NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in the transaction response and then used in subsequent transaction requests. NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File		

Sample Vault Update Credit Card

```

<?php
##
## Example php -q TestResUpdateCC.php store3 yesguy
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_update_cc';

```

Sample Vault Update Credit Card

```

$data_key='D8cpd4r7REXoN8NIJPI512xPh';
$cust_id='customer1';
$phone = '5555555555';
$email = 'bob@smith.com';
$note = 'stuff';
$pan='5454545454545454';
$expiry_date='0909';
$crypt_type='7';
$avs_street_number = '123';
$avs_street_name = 'stuff dr';
$avs_zipcode = '90215';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'data_key'=>$data_key,
'cust_id'=>$cust_id,
'phone'=>$phone,
'email'=>$email,
'note'=>$note,
'pan'=>$pan,
'expdate'=>$expiry_date,
'crypt_type'=>$crypt_type
);
/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number' => $avs_street_number,
'avs_street_name' => $avs_street_name,
'avs_zipcode' => $avs_zipcode
);
/***** AVS Object *****/
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setIssuerId("168451306048014");
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setAvsInfo($mpgAvsInfo);
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\n\nPhone = " . $mpgResponse->getResDataPhone());
print("\n\nEmail = " . $mpgResponse->getResDataEmail());
print("\n\nNote = " . $mpgResponse->getResDataNote());

```

Sample Vault Update Credit Card

```
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCryp Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.3.1 Vault Encrypted Update CC – EncResUpdateCC

Vault Encrypted Update CC transaction object definition

```
$txnArray = array('type'=>'enc_res_update_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Encrypted Update CC transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Encrypted Update CC transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
encrypted track 2 data	<i>String</i> 40-character numeric	'enc_track2'=>\$enc_track2
device type	<i>String</i> 30-character alphanumeric case sensitive	'device_type'=>\$device_type

Optional values that are submitted to the ResUpdateCC object are updated, while unsubmitted optional values (with one exception) remain unchanged. This allows you to change only the fields you want.

The exception is that if you are making changes to the payment type, **all** of the variables in the optional values table below must be submitted.

If you update a profile to a different payment type, it is automatically deactivated and a new credit card profile is created and assigned to the data key. The only values from the prior profile that will remain unchanged are the customer ID, phone number, email address, and note.

EXAMPLE: If a profile contains AVS information, but a ResUpdateCC transaction is submitted without an AVSInfo object, the existing AVSInfo details are deactivated and the new credit card information is registered without AVS.

Vault Encrypted Update CC transaction request fields – Optional

Variable Name	Type and Limits	Set Method
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id
NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \		
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
email address	<i>String</i> 30-character alphanumeric	'email'=>\$email
phone number	<i>String</i> 30-character alphanumeric	'phone'=>\$phone
note	<i>String</i> 30-character alphanumeric	'note'=>\$note

Sample Vault Encrypted Update CC

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='enc_res_update_cc';
$data_key='F91LyeEJjv8OvpOdmXYWKH7dV';
$cust_id='cust2';
$phone = '4169996999';
$email = 'bob@email.com';
$note = 'note4';
$enc_track2 = 'ENCRYPTEDTRACK2DATA';
$device_type='idtech_bdk';
$script_type='7';
$avs_street_number = '3300';
$avs_street_name = 'bloor street west';
$avs_zipcode = 'm8x2x3';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'data_key'=>$data_key,
'cust_id'=>$cust_id,
'phone'=>$phone,
'email'=>$email,
'note'=>$note,
'enc_track2'=>$enc_track2,
'device_type'=>$device_type,
'crypt_type'=>$script_type
);
/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number' => $avs_street_number,
'avs_street_name' => $avs_street_name,
'avs_zipcode' => $avs_zipcode
);
/***** AVS Object *****/
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setAvsInfo($mpgAvsInfo);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());

```

Sample Vault Encrypted Update CC

```
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCryp Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.4 Vault Delete – ResDelete

Deletes an existing Vault profile of any type using the unique data key that was assigned when the profile was added.

NOTE: Once a profile is deleted, the information that was saved within can no longer be retrieved.

Vault Delete transaction object definition

```
$txnArray = array('type'=>'res_delete', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Delete transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Delete transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
data key	String 25-character alphanumeric	'data_key'=>\$data_key

Sample Vault Delete

```

<?php
##
## Example php -q TestResDelete.php store3 yesguy
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_delete';
$data_key='YjNEwYw6U2pPwquXOkOme3G7g';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'data_key'=>$data_key
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.5 Vault Lookup Full – ResLookupFull

Verifies what is currently saved under the Vault profile associated with the given data key. The response to this transaction returns the latest active data for that profile.

Unlike Vault Lookup Masked (which returns a masked credit card number), this transaction returns both the masked and unmasked credit card number.

Vault Lookup Full transaction object definition

```
$txnArray = array('type'=>'res_lookup_full', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Lookup Full transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Lookup Full transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	String 25-character alphanumeric	'data_key'=>\$data_key

Sample Vault Lookup Full

```
<?php
##
## Example php -q TestResLookupFull.php store3 yesguy
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_lookup_full'; //will return both the full & masked card number
$data_key='t8RCndWBNFnt4Dx32CCnl2tlz';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'data_key'=>$data_key
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
```


Sample Vault Lookup Full

```

print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nPan = " . $mpgResponse->getResDataPan());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.6 Vault Lookup Masked – ResLookupMasked

Verifies what is currently saved under the Vault profile associated with the given data key. The response to this transaction returns the latest active data for that profile.

Unlike Vault Lookup Full (which returns both the masked and the unmasked credit card numbers), this transaction only returns the masked credit card number.

Vault Lookup Masked transaction object definition

```

$txnArray = array('type'=>'res_lookup_masked', ...);

$mpgTxn = new mpgTransaction($txnArray);

```

HttpPostRequest object for Vault Lookup Masked transaction

```

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);

```

Vault Lookup Masked transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
data key	String 25-character alphanumeric	'data_key'=>\$data_key

Sample Vault Lookup Masked

```
<?php
##
## Example php -q TestResLookupMasked.php store3 yesguy
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_lookup_masked'; //will only return the masked card number
$data_key='t8RCndWBNFnt4Dx32CCnl2tlz';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'data_key'=>$data_key
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.7 Vault Get Expiring – ResGetExpiring

Verifies which profiles have credit cards that are expiring during the current and next calendar month.

EXAMPLE: if you are processing this transaction on September 30, then it will return all cards that expire(d) in September and October of this year.

When generating a list of profiles with expiring credit cards, only the masked credit card numbers are returned. Can be performed no more than 2 times on any given calendar day.

Vault Get Expiring transaction object definition

```
$txnArray = array('type'=>'res_get_expiring', ...);

$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Get Expiring transaction

```
$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Vault Get Expiring transaction request fields – Required

This transaction has no required request fields.

Sample Vault Get Expiring

```
<?php
##
## Example php -q TestResGetExpiring.php store3 yesguy
##
//There is a max number of attempts set for this transaction per calendar day
//Can not surpass or will receive Invalid Transaction error
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_get_expiring';
/***** Transactional Associative Array *****/
$txnArray = array( 'type'=>$type );
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
```

Sample Vault Get Expiring

```
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
$DataKeys = $mpgResponse->getDataKeys();
for($i=0; $i < count($DataKeys); $i++)
{
    $mpgResponse->setResolveData($DataKeys[$i]);
    print("\n\nData Key = " . $DataKeys[$i]);
    print("\nCust ID = " . $mpgResponse->getResDataCustId());
    print("\nPhone = " . $mpgResponse->getResDataPhone());
    print("\nEmail = " . $mpgResponse->getResDataEmail());
    print("\nNote = " . $mpgResponse->getResDataNote());
    print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
    print("\nExp Date = " . $mpgResponse->getResDataExpDate());
    print("\nCryp Type = " . $mpgResponse->getResDataCryptType());
    print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
    print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
    print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
}
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.8 Vault Is Corporate Card - ResIsCorporateCard

Determines whether a profile has a corporate card registered within it.

After sending the transaction, the response field to the Receipt object's `getCorporateCard` method is either true or false depending on whether the associated card is a corporate card.

NOTE: This transaction supports both temporary and permanent tokens.

Vault Is Corporate Card transaction object definition

```
$txnArray = array('type'=>'res_iscorporatcard', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Is Corporate Card transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Is Corporate Card transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key

Sample Vault Is Corporate Card

```

<?php
##
## Example php -q TestResIsCorporateCard.php moneris hurgle
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_iscorporatcard';
$data_key='t8RCndWBNFnt4Dx32CCnl2tlz';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'data_key'=>$data_key
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nCorporateCard = " . $mpgResponse->getCorporateCard());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
?>

```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.9 Vault Add Token – ResAddToken

Converts a Hosted Tokenization temporary token to a permanent Vault token.

A temporary token is valid for 15 minutes after it is created. This transaction must be performed within that time frame if the token is to be changed to a permanent one for future use.

Using the temporary token, send either a Purchase with Vault, Pre-Authorization with Vault or Card Verification with Vault transaction request including the Credential on File object to get the issuer ID.

Vault Add Token transaction object definition

```
$txnArray = array('type'=>'res_add_token', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Add Token transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Add Token transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
Credential on File Info cof	<i>Object</i> N/A	\$mpgTxn->setCofInfo(\$cof);
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		

Vault Add Token transaction request fields – Optional

Table 1: Vault Add Token transaction optional values

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
email address	<i>String</i> 30-character alphanumeric	'email'=>\$email
phone number	<i>String</i> 30-character alphanumeric	'phone'=>\$phone
note	<i>String</i> 30-character alphanumeric	'note'=>\$note
data key format	<i>String</i> 2-character alphanumeric	'data_key_format'=>\$data_key_format

Credential on File Info object request fields

Variable Name	Type and Limits	Set Method
issuer ID NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in the transaction response and then used in subsequent	<i>String</i> 15-character alphanumeric variable length	\$cof->setIssuerId("VALUE_FOR_ISSUER_ID"); NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File

Variable Name	Type and Limits	Set Method
transaction requests.		

Sample Vault Add Token

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='res_add_token';
$temp_data_key='ot-mtNKdu8NcxDoChqOJKZJZ1BOB';
$cust_id='customer1';
$phone = '5555551234';
$email = 'bob@smith.com';
$note = 'this is my note';
$expiry_date='1811';
$data_key_format = "0";
$script_type='1';
$avs_street_number = '123';
$avs_street_name = 'lakeshore blvd';
$avs_zipcode = '90210';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'data_key'=>$temp_data_key,
'cust_id'=>$cust_id,
'phone'=>$phone,
'email'=>$email,
'note'=>$note,
'expdate'=>$expiry_date,
// 'data_key_format'=>$data_key_format, //optional
'crypt_type'=>$script_type
);
/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number' => $avs_street_number,
'avs_street_name' => $avs_street_name,
'avs_zipcode' => $avs_zipcode
);
/***** AVS Object *****/
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setIssuerId("168451306048014");
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setAvsInfo($mpgAvsInfo);
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());

```


Sample Vault Add Token

```
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.5.10 Vault Tokenize Credit Card – ResTokenizeCC

Creates a new credit card profile using the credit card number, expiry date and e-commerce indicator that were submitted in a previous financial transaction. Previous transactions to be tokenized must have included the Credential on File Info object.

The Issuer ID received in the previous transaction response is sent in the Vault Tokenize Credit Card request to reference that this is a stored credential. If you require this Issuer ID in the response to this request, include **return_issuer_ID** as **true**; this allows for retrieval of the Issuer ID from the previous financial transaction.

Basic transactions that can be tokenized are:

- Purchase
- Pre-Authorization
- Card Verification

The tokenization process is outlined below :

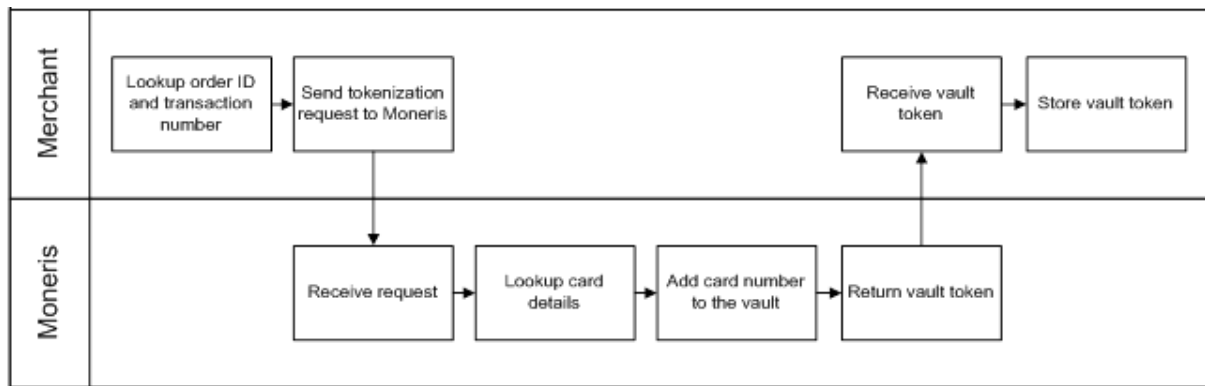


Figure 1: Tokenize process diagram

Vault Tokenize Credit Card transaction object definition

```
$txnArray = array('type'=>'res_tokenize_cc', ...);
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Vault Tokenize Credit Card transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Vault Tokenize Credit Card transaction request fields – Required

These mandatory values reference a previously processed credit card financial transaction. The credit card number, expiry date, and e-commerce indicator from the original transaction are registered in the Vault for future financial Vault transactions.

Variable Name	Type and Limits	Set Method
order ID	String 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
transaction number	String 255-character, alpha-numeric, hyphens or under-scores variable length	'txn_number'=>\$txn_number

Vault Tokenize Credit Card transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
return issuer ID	<i>Boolean</i> true/false	resTokenizeCC 'return_issuer_id'=>\$return_issuer_id
email address	<i>String</i> 30-character alphanumeric	'email'=>\$email
phone number	<i>String</i> 30-character alphanumeric	'phone'=>\$phone
note	<i>String</i> 30-character alphanumeric	'note'=>\$note
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo (\$mpgAvsInfo) ;
data key format	<i>String</i> 2-character alphanumeric	'data_key_format'=>\$data_key_format
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo (\$cof) ;
cof	N/A	
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Cre-		

Variable Name	Type and Limits	Set Method
credential on File Info Object and Variables.		

Credential on File Info object request fields

Variable Name	Type and Limits	Set Method
issuer ID NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in the transaction response and then used in subsequent transaction requests.	<i>String</i> 15-character alphanumeric variable length	<pre>\$cof->setIssuerId("VALUE_FOR_ISSUER_ID");</pre> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File

Any field that is not set in the tokenize request is not stored with the transaction. That is, Moneris Gateway does not automatically take the optional information that was part of the original transaction.

The ResolveData that is returned in the response fields indicates what values were registered for this profile.

Sample Vault Tokenize Credit Card
<pre><?php require "../mpgClasses.php"; /***** Request Variables *****/ \$store_id='store5'; \$api_token='yesguy'; /***** Transactional Variables *****/ \$type='res_tokenize_cc'; \$order_id='res-purch-110515-12:56:49'; \$txn_number='31570-0_10'; \$return_issuer_id = 'false'; \$data_key_format = "0"; \$cust_id='customer1'; \$phone = '4165555555'; \$email = 'bob@smith.com'; \$note = 'this is my note'; \$savs_street_number = '123'; \$savs_street_name = 'lakeshore blvd'; \$savs_zipcode = '90210'; /***** Transactional Associative Array *****/ \$txnArray=array('type'=>\$type,</pre>

Sample Vault Tokenize Credit Card

```

'order_id'=>$order_id,
'txn_number'=>$txn_number,
// 'data_key_format'=>$data_key_format, //optional
'cust_id'=>$cust_id,
'phone'=>$phone,
'email'=>$email,
'note'=>$note
);
/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number' => $avs_street_number,
'avs_street_name' => $avs_street_name,
'avs_zipcode' => $avs_zipcode
);
/***** AVS Object *****/
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setIssuerId("168451306048014");
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setAvsInfo($mpgAvsInfo);
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.6 Vault Financial Transactions

After a financial transaction is complete, the response fields indicate all the values that are currently saved under the profile that was used.

4.6.1 Customer ID Changes

Some financial transactions take the customer ID as an optional value. The customer ID may or may not already be in the Vault profile when the transaction is sent. Therefore, it is possible to change the value of the customer ID by performing a financial transaction

The table below shows what the customer ID will be in the response field after a financial transaction is performed.

Table 2: Customer ID use in response fields

Already in profile?	Passed in?	Version used in response
No	No	Customer ID not used in transaction
No	Yes	Passed in
Yes	No	Profile
Yes	Yes	Passed in

4.6.2 Purchase with Vault – ResPurchaseCC

NOTE: This transaction supports both temporary and permanent tokens.

Purchase with Vault transaction object definition

```
$txnArray = array('type'=>'res_purchase_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Purchase with Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Purchase with Vault transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
Credential on File Info cof	<i>Object</i> N/A NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.	\$mpgTxn->setCofInfo(\$cof);

Purchase with Vault transaction request fields – Optional

Variable Name	Type and Limits	Set Method
status check	<i>Boolean</i>	\$mpgHttpPost =new

Variable Name	Type and Limits	Set Method
	true/false	<code>mpgHttpPostStatus(\$store_id,\$api_token,\$status,\$mpgRequest);</code>
expiry date	<i>String</i> 4-character numeric YYMM format. (Note that this is reversed from the date displayed on the card, which is MMY)	<code>'expiry_date'=>\$expiry_date</code>
	NOTE: This field is required if referencing a temporary token in Installments by Visa	
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	<code>'cust_id'=>\$cust_id</code>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	<code>'dynamic_descriptor'=>\$dynamic_descriptor</code>
Customer Information	<i>Object</i> N/A	<code>\$mpgTxn->setCustInfo(\$mpgCustInfo);</code>
AVS Information	<i>Object</i> N/A	<code>\$mpgTxn->setAvsInfo(\$mpgAvsInfo);</code>
CVD Information	<i>Object</i> N/A	<code>\$mpgTxn->setCvdInfo(\$mpgCvdInfo);</code>

Variable Name	Type and Limits	Set Method
NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only—merchants must not store CVD information.		
Recurring Billing	<i>Object</i> N/A	<code>\$mpgTxn->setRecur(\$mpgRecur);</code>
Surcharge Information NOTE: This object requires use of the Vault Surcharge Lookup transaction prior to confirm card eligibility.	<i>Object</i> N/A	<code>\$mpgTxn->setSurchargeInfo(\$surchargeInfo);</code> Contains fields related to Surcharge feature.

Credential on File Info object request fields

Variable Name	Type and Limits	
issuer ID NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in the transaction response and then used in subsequent transaction requests.	<i>String</i> 15-character alphanumeric variable length	<code>\$cof->setIssuerId("VALUE_FOR_ISSUER_ID");</code> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File
payment indicator NOTE: In Credential on File transactions where the request field e-commerce indicator is also being sent,	<i>String</i> 1-character alphabetic	<code>\$cof->setPaymentIndicator("PAYMENT_INDICATOR_VALUE");</code> NOTE: For a list and explanation of the possible values to send for this variable, see

Variable Name	Type and Limits	
the acceptable values for e-commerce indicator are dependent on the value sent for payment indicator; see request field definitions for more information.		Definition of Request Fields – Credential on File
payment information	String 1-character numeric	<pre>\$cof->setPaymentInformation("PAYMENT_INFO_VALUE");</pre> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File

Sample Purchase with Vault

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00597';
$api_token='027AbCbxQorPggMQe6hU';
/***** Transaction Variables *****/
$data_key='4HIme0ZGURXE3NRBXHUj6nSc4';
$orderid='res-purch-' . date("dmy-G:i:s");
$amount='18.00';
$custid='customer1';
$cript_type='7';
$expdate='2301'; //For Temp Tokens only

//NT Response Option
$get_nt_response = 'false'; //Optional - set it true only if you want to get network
tokenization response.
/***** Transaction Array *****/
$txnArray=array('type'=>'res_purchase_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'amount'=>$amount,
'crypt_type'=>$cript_type,
'expdate'=>$expdate,
'dynamic_descriptor'=>'12484',
'get_nt_response'=>$get_nt_response
);

/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);

/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
```

Sample Purchase with Vault

```

$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);

/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
//$mpgTxn->setInstallmentInfo($installmentInfo);

/***** Surcharge Info *OPTIONAL* *****/
$surchargeInfo = new SurchargeInfo();
$surchargeInfo->setSurchargeAmount("1.00");
$mpgTxn->setSurchargeInfo($surchargeInfo);

/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions

/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);

/***** Response Object *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());

// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());

if($get_nt_response == 'true')
{
print("\nNTResponseCode = " . $mpgResponse->getNTResponseCode());
print("\nNTMessage = " . $mpgResponse->getNTMessage());
}

```

Sample Purchase with Vault

```
print("\nNTUsed = " . $mpgResponse->getNTUsed());
print("\nNTTokenBin = " . $mpgResponse->getNTTokenBin());
print("\nNTTokenLast4 = " . $mpgResponse->getNTTokenLast4());
print("\nNTTokenExpDate = " . $mpgResponse->getNTTokenExpDate());
}
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCryt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.6.3 Pre-Authorization with Vault – ResPreauthCC

NOTE: This transaction supports both temporary and permanent tokens.

Pre-Authorization with Vault transaction object definition

```
$txnArray = array('type'=>'res_preauth_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Pre-Authorization with Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Pre-Authorization with Vault transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	String 25-character alphanumeric	'data_key'=>\$data_key
order ID	String	'order_id'=>\$order_id

Variable Name	Type and Limits	Set Method
	50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
Credential on File Info cof	<i>Object</i> N/A NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.	\$mpgTxn->setCofInfo(\$cof);

Pre-Authorization with Vault transaction request fields – Optional

Value	Limits	Set method
status check	<i>Boolean</i> true/false	<pre>\$mpgHttpPost =new mpgHttpPostStatus(\$store_ id,\$api_ token,\$status,\$mpgRequest);</pre>
dynamic descriptor	<i>String</i> 20-character alphanumeric	'dynamic_descriptor'=>\$dynamic_descriptor

Value	Limits	Set method
NOTE: For Pre-Authorization transactions: the value in the dynamic descriptor field will only be carried over to a Pre-Authorization Completion when executing the latter via the Merchant Resource Center; otherwise, the value for dynamic descriptor must be sent again in the Pre-Authorization Completion	total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	
is incremental is_incremental	<i>Boolean</i> true/false	'is_incremental'=>\$is_incremental Indicates if this preauthorization is using an estimated amount. Estimations allow for incrementing the amount held via subsequent incrementalAuth requests. Defaults to false. NOTE: Please note that if this field is true, the preauthorization is only eligible for a single Preauthorization Completion. Any completion sent for partial completion is treated as a full completion (ship_indicator= P is treated as = F when is_incremental= true on the original preauth)
expiry date NOTE: This field is required if referencing a temporary token in Installments by Visa	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
final authorization	<i>String</i>	'final_auth' => 'true'

Value	Limits	Set method
NOTE: Applies to Mastercard transactions only	true/false	
Customer Information	<i>Object</i> N/A	<code>\$mpgTxn->setCustInfo(\$mpgCustInfo);</code>
AVS Information	<i>Object</i> N/A	<code>\$mpgTxn->setAvsInfo(\$mpgAvsInfo);</code>
CVD Information NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only— merchants must not store CVD information.	<i>Object</i> N/A	<code>\$mpgTxn->setCvdInfo(\$mpgCvdInfo);</code>
Surcharge Information NOTE: This object requires use of the Surcharge Lookup transaction prior to confirm card eligibility.	<i>Object</i> N/A	<code>\$mpgTxn->setSurchargeInfo(\$surchargeInfo);</code> Contains fields related to Surcharge feature.

Credential on File Info object request fields

Variable Name	Type and Limits	
issuer ID NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in	<i>String</i> 15-character alphanumeric variable length	<code>\$cof->setIssuerId("VALUE_FOR_ISSUER_ID");</code> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File

Variable Name	Type and Limits	
the transaction response and then used in subsequent transaction requests.		
payment indicator	String 1-character alphabetic	<pre>\$cof->setPaymentIndicator("PAYMENT_INDICATOR_VALUE");</pre> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File
NOTE: In Credential on File transactions where the request field e-commerce indicator is also being sent, the acceptable values for e-commerce indicator are dependent on the value sent for payment indicator; see request field definitions for more information.	payment information	String 1-character numeric <pre>\$cof->setPaymentInformation("PAYMENT_INFO_VALUE");</pre> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File

Sample Pre-Authorization with Vault

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00597';
$api_token='027AbCbxQorPggMQe6hU';
/***** Transaction Variables *****/
$data_key='4HIme0ZGURXE3NRBXHUj6nSc4';
$orderid='res-preauth-'.date("dmy-G:i:s");
$amount='1.00';
$custid='customer1'; //if sent will be submitted, otherwise cust_id from profile will be used
$crypt_type='1';
$expdate='2301';
$is_incremental= 'true';

//NT Response Option
$get_nt_response = 'false';//Optional - set it true only if you want to get network tokenization response.
/***** Transaction Array *****/
$txnArray =array('type'=>'res_preauth_cc',
```


Sample Pre-Authorization with Vault

```

'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'amount'=>$amount,
'crypt_type'=>$crypt_type,
'expdate'=>$expdate,
'dynamic_descriptor'=>'12424',
'get_nt_response'=>$get_nt_response
);

/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);

/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);

/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
//$mpgTxn->setInstallmentInfo($installmentInfo);

/***** Surcharge Info *OPTIONAL* *****/
$surchargeInfo = new SurchargeInfo();
$surchargeInfo->setSurchargeAmount("1.00");
$mpgTxn->setSurchargeInfo($surchargeInfo);

/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions

/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);

/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());

```

Sample Pre-Authorization with Vault

```
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\n\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());

// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());

if($get_nt_response == 'true')
{
print("\n\nNTResponseCode = " . $mpgResponse->getNTResponseCode());
print("\nNTMessage = " . $mpgResponse->getNTMessage());
print("\nNTUsed = " . $mpgResponse->getNTUsed());
print("\nNTTokenBin = " . $mpgResponse->getNTTokenBin());
print("\nNTTokenLast4 = " . $mpgResponse->getNTTokenLast4());
print("\nNTTokenExpDate = " . $mpgResponse->getNTTokenExpDate());
}
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypT Type = " . $mpgResponse->getResDataCrypTType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>
```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.6.4 Vault Independent Refund – ResIndRefundCC

NOTE: This transaction supports both temporary and permanent tokens.

Vault Independent Refund transaction object definition

```
$txnArray = array('type'=>'res_ind_refund_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpsPostRequest object for Vault Independent Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Vault Independent Refund transaction values

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Vault Independent Refund transaction request fields – Optional

Value	Limits	Set method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	'cust_id'=>\$cust_id
expiry date	<i>String</i>	'expiry_date'=>\$expiry_date

Value	Limits	Set method
	4-character alphanumeric YYMM	
status check	<i>Boolean</i> true/false	<code>\$mpgHttpPost =new mpgHttpsPostStatus(\$store_ id,\$api_ token,\$status,\$mpgRequest);</code>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	<code>'dynamic_descriptor'=>\$dynamic_ descriptor</code>

Sample Vault Independent Refund

```
<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
##
## Example php -q TestResIndRefundCC.php store3 yesguy unique_order_id cust_id 15.00 1
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transaction Variables *****/
$data_key='t8RCndWBNFnt4Dx32CCnl2tlz';
$orderid='res-ind-refund-' . date("dmy-G:i:s");
$amount='1.00';
$custid='';
$script_type='1';

//NT Response Option
$get_nt_response = 'true'; //Optional - set it true only if you want to get network
tokenization response.
/***** Transaction Array *****/
$txnArray = array('type'=>'res_ind_refund_cc',
```

Sample Vault Independent Refund

```

    'data_key'=>$data_key,
    'order_id'=>$orderid,
    'cust_id'=>$custid,
    'amount'=>$amount,
    'crypt_type'=>$crypt_type,
    'dynamic_descriptor'=>'12346'
    'get_nt_response'=>$get_nt_response
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\n\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());

if($get_nt_response == 'true')
{
    print("\n\nNTResponseCode = " . $mpgResponse->getNTResponseCode());
    print("\n\nNTMessage = " . $mpgResponse->getNTMessage());
    print("\n\nNTUsed = " . $mpgResponse->getNTUsed());
    print("\n\nNTTokenBin = " . $mpgResponse->getNTTokenBin());
    print("\n\nNTTokenLast4 = " . $mpgResponse->getNTTokenLast4());
    print("\n\nNTTokenExpDate = " . $mpgResponse->getNTTokenExpDate());
}
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\n\nPhone = " . $mpgResponse->getResDataPhone());
print("\n\nEmail = " . $mpgResponse->getResDataEmail());
print("\n\nNote = " . $mpgResponse->getResDataNote());
print("\n\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\n\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\n\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\n\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\n\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\n\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

Vault response fields

For a list and explanation of (Receipt object) response fields that are available after sending this Vault transaction, see Definitions of Response Fields (page 561).

4.6.5 Force Post with Vault – ResForcePostCC

NOTE: This transaction supports both temporary and permanent tokens.

Force Post with Vault transaction object definition

```
$txnArray = array('type'=>'res_forcepost_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Force Post with Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Force Post with Vault transaction request fields – Required

Variable Name	Type and Limits	Set Method
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
authorization code	<i>String</i> 8-character alphanumeric	'auth_code'=>\$auth_code
electronic commerce indicator	<i>String</i>	'crypt_type'=>\$crypt

Variable Name	Type and Limits	Set Method
	1-character alphanumeric	

Force Post with Vault transaction object definition

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	'dynamic_descriptor'=>\$dynamic_descriptor
status check	<i>Boolean</i> true/false	<pre>\$mpgHttpPost =new mpgHttpsPostStatus(\$store_id,\$api_token,\$status,\$mpgRequest);</pre>

Sample Force Post with Vault

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transaction Variables *****/
$data_key='uroyVNSxzjk5hHoT0kpQDBCw4';
```

Sample Force Post with Vault

```

$orderid='res-forcepost-' . date("dmy-G:i:s");
$amount='1.00';
$custid='cust';
$crypt_type='7';
$auth_code='256452';
$dynamic_descriptor='my descriptor';

//NT Response Option
$get_nt_response = 'true'; //Optional - set it true only if you want to get network
tokenization response.
/***** Transaction Array *****/
$txnArray=array('type'=>'res_forcepost_cc',
'order_id'=>$orderid,
'cust_id'=>$custid,
'amount'=>$amount,
'data_key'=>$data_key,
'crypt_type'=>$crypt_type,
'auth_code'=>$auth_code,
'dynamic_descriptor'=>$dynamic_descriptor,
'get_nt_response'=>$get_nt_response
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\n\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());

if($get_nt_response == 'true')
{
print("\n\nNTResponseCode = " . $mpgResponse->getNTResponseCode());
print("\n\nNTMessage = " . $mpgResponse->getNTMessage());
print("\n\nNTUsed = " . $mpgResponse->getNTUsed());
print("\n\nNTTokenBin = " . $mpgResponse->getNTTokenBin());
print("\n\nNTTokenLast4 = " . $mpgResponse->getNTTokenLast4());
}

```


Sample Force Post with Vault

```

print("\nNTTokenExpDate = " . $mpgResponse->getNTTokenExpDate());
}
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypT Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

4.6.6 Card Verification with Vault – ResCardVerificationCC

NOTE: This transaction supports both temporary and permanent tokens.

Things to Consider:

- This transaction type only applies to Visa, Mastercard, American Express and Discover transactions
- The card number and expiry date for this transaction are passed using a token, as represented by the data key value
- When using a temporary token (e.g., such as with Hosted Tokenization) **and** you intend to store the cardholder credentials, this transaction must be run prior to running the Vault Add Token transaction

Card Verification with Vault object definition

```
$txnArray = array('type'=>'res_card_verification_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Card Verification with Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Card Verification with Vault transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo (\$mpgAvsInfo);
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo (\$mpgCvdInfo);
NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only—merchants must not store CVD information.		
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo (\$cof);
cof	N/A	
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		

Credential on File Info object request fields

Variable Name	Type and Limits	
issuer ID NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in the transaction response and then used in subsequent transaction requests.	<i>String</i> 15-character alphanumeric variable length	<pre>\$cof->setIssuerId("VALUE_FOR_ISSUER_ID");</pre> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File
payment indicator NOTE: In Credential on File transactions where the request field e-commerce indicator is also being sent, the acceptable values for e-commerce indicator are dependent on the value sent for payment indicator; see request field definitions for more information.	<i>String</i> 1-character alphabetic	<pre>\$cof->setPaymentIndicator("PAYMENT_INDICATOR_VALUE");</pre> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File
payment information	<i>String</i> 1-character numeric	<pre>\$cof->setPaymentInformation("PAYMENT_INFO_VALUE");</pre> NOTE: For a list and explanation of the possible values to send for this variable, see Definition of Request Fields – Credential on File

Sample Card Verification with Vault

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00597';
$api_token='O27AbCbXQorPggMQe6hU';
```

Sample Card Verification with Vault

```

/***** Transaction Variables *****/
$data_key='4HIme0ZGURXE3NRBXHUj6nSc4';
$orderid='res-purch-'.date("dmy-G:i:s");
$crypt_type='1';
$expdate='2301'; //for temp token

//NT Response Option
$get_nt_response = 'false'; //Optional - set it true only if you want to get network
tokenization response.
/***** Transaction Array *****/
$txnArray=array('type'=>'res_card_verification_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'crypt_type'=>$crypt_type,
'expdate'=>$expdate,
'get_nt_response'=>$get_nt_response
);
/***** CVD Variables *****/
$cvd_indicator = '1';
$cvd_value = '198';
/***** CVD Associative Array *****/
$cvdTemplate = array(
'cvd_indicator' => $cvd_indicator,
'cvd_value' => $cvd_value
);
$mpgCvdInfo = new mpgCvdInfo ($cvdTemplate);
/***** AVS Variables *****/
//The AVS portion is optional if AVS details are already stored in this profile
//If AVS details are resnet in Purchase transaction, they will replace stored details
$avs_street_number = '';
$avs_street_name = 'bloor st';
$avs_zipcode = '111111';
/***** AVS Associative Array *****/
$avsTemplate = array(
'avs_street_number' => $avs_street_number,
'avs_street_name' => $avs_street_name,
'avs_zipcode' => $avs_zipcode
);
$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setCvdInfo($mpgCvdInfo);
$mpgTxn->setAvsInfo($mpgAvsInfo);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());

```

Sample Card Verification with Vault

```

print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCVDResponse = " . $mpgResponse->getCvdResultCode());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\n\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());

if($get_nt_response == 'true')
{
print("\n\nNTResponseCode = " . $mpgResponse->getNTResponseCode());
print("\nNTMessage = " . $mpgResponse->getNTMessage());
print("\nNTUsed = " . $mpgResponse->getNTUsed());
print("\nNTTokenBin = " . $mpgResponse->getNTTokenBin());
print("\nNTTokenLast4 = " . $mpgResponse->getNTTokenLast4());
print("\nNTTokenExpDate = " . $mpgResponse->getNTTokenExpDate());
}
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCryp Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

4.6.7 Surcharge Lookup with Vault

Confirm eligibility of a tokenized card for merchant surcharges on a transaction.

Credit Surcharge is feature where merchants may append an additional charge to the amount of a transaction processed on an eligible credit card. Only eligible credit cards can be surcharged, so merchants must confirm eligibility of the card before processing the payment via surcharge lookup.

The following transactions support including surcharge amounts:

- Purchase (Basic)
- Pre-Authorization (Basic)

- Vault Purchase
- Vault Pre-Authorization

Things to Consider:

- Card association rules require that cardholders must be informed of any surcharge by a merchant.
- The cardholder must be offered an option for canceling the transaction if they refuse the surcharge.

```
$txnArray = array('type'=>'res_surcharge_lookup', ...);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Surcharge Lookup transaction request fields – Required

Variable Name	Type and Limits	
data key	String 25-character alphanumeric	'data_key'=>\$data_key
amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point	'amount'=>\$amount

Variable Name	Type and Limits
	EXAMPLE: 1234567.89

Sample Vault Surcharge Lookup

```
<?php
require "../mpgClasses.php";
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
$amount = '50.00';
$pan='4242424242424242';
$txnArray=array('type'=>'surcharge_lookup',
'amount'=>$amount,
'pan'=>$pan
);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
//print the mpgrequest
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nMessage = " . $mpgResponse->getMessage());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nIsSurchargeEligible = " . $mpgResponse->getIsSurchargeEligible());
print("\nMaxSurchargeRate = " . $mpgResponse->getMaxSurchargeRate());
print("\nMaxSurchargeAmount = " . $mpgResponse->getMaxSurchargeAmount());
print("\nServiceType = " . $mpgResponse->getServiceType());
?>
```

4.7 Hosted Tokenization

Moneris Hosted Tokenization is a solution for online e-commerce merchants who do not want to handle credit card numbers directly on their websites, yet want the ability to fully customize their check-out web page appearance.

When an hosted tokenization transaction is initiated, the Moneris Gateway displays (on the merchant's behalf) a single text box on the merchant's checkout page. The cardholder can then securely enter the credit card information into the text box. Upon submission of the payment information on the checkout page, Moneris Gateway returns a temporary token representing the credit card number to the merchant. This is then used in an API call to process a financial transaction directly with Moneris to charge the card. After receiving a response to the financial transaction, the merchant generates a receipt and allows the cardholder to continue with online shopping.

For more details on how to implement the Moneris Hosted Tokenization feature, see the Hosted Solutions Integration Guide. The guide can be downloaded from the Moneris Developer Portal at

developer.moneris.com

5 Level 2/3 Transactions

- 5.1 About Level 2/3 Transactions
- 5.2 Level 2/3 Visa Transactions
- 5.3 Level 2/3 Mastercard Transactions
- 5.4 Level 2/3 American Express Transactions

5.1 About Level 2/3 Transactions

The Moneris Gateway API supports passing Level 2/3 purchasing card transaction data for Visa, MasterCard and American Express corporate cards.

All Level 2/3 transactions use the same Pre-Authorization transaction as described in the topic Pre-Authorization (page 26).

5.2 Level 2/3 Visa Transactions

- 5.2.1 Level 2/3 Transaction Types for Visa
- 5.2.2 Level 2/3 Transaction Flow for Visa
- 5.2.3 VS Completion
- 5.2.5 VS Force Post
- 5.2.4 VS Purchase Correction
- 5.2.6 VS Refund
- 5.2.7 VS Independent Refund
- 5.2.8 VS Corpais
- 1 VS Corpais Invoice
- 1 VS Corpais – Passenger Itinerary

5.2.1 Level 2/3 Transaction Types for Visa

This transaction set includes a suite of corporate card financial transactions as well as a transaction that allows for the passing of Level 2/3 data. Please ensure that Visa Level 2/3 support is enabled on your

merchant account. Batch Close, Open Totals and Pre-authorization are identical to the transactions outlined in the section Basic Transaction Set (page 21).

- When the Pre-authorization response contains CorporateCard equal to true then you can submit the Visa transactions.
- If CorporateCard is false then the card does not support Level 2/3 data and non Level 2/3 transaction are to be used. If the card is not a corporate card, please refer to the section 2 Basic Transaction Set for the appropriate non-corporate card transactions.

NOTE: This transaction set is intended for transactions where Corporate Card is true and Level 2/3 data will be submitted. If the credit card is found to be a corporate card but you do not wish to send any Level 2/3 data then you may submit Visa transactions using the basic transaction set outlined in 2 Basic Transaction Set.

Pre-authorization– (authorization/pre-authorization)

Pre-authorization verifies and locks funds on the customer's credit card. The funds are locked for a specified amount of time, based on the card issuer. To retrieve the funds from a preauth so that they may be settled in the merchant account a capture must be performed. CorporateCard will return as true if the card supports Level 2/3.

VS Completion – (Capture/Pre-authorization Completion)

Once a Pre-authorization is obtained the funds that are locked need to be retrieved from the customer's credit card. The capture retrieves the locked funds and readies them for settlement into the merchant account. Prior to performing a VS Completion, a Pre-authorization must be performed. Once the transaction is completed, VS Corpais must be used to process the Level 2/3 data.

VS Force Post – (Force Capture/Pre-authorization Completion)

This transaction is an alternative to VS Completion to obtain the funds locked on Pre-auth obtained from IVR or equivalent terminal. The VS Force Post retrieves the locked funds and readies them for settlement in to the merchant account. Once the transaction is completed, VS Corpais must be used to process the Level 2/3 data.

VS Purchase Correction (Void, Correction)

VS Completion and VS Force Post can be voided the same day* that they occur. A VS Purchase Correction must be for the full amount of the transaction and will remove any record of it from the cardholder statement.

VS Refund – (Credit)

A VS Refund can be performed against a VS Completion to refund any part or all of the transaction. Once the transaction is completed, VS Corpais must be used to process the Level 2/3 data.

VS Independent Refund – (Credit)

A VS Independent Refund can be performed against a purchase or a capture to refund any part, or all of the transaction. Independent refund is used when the originating transaction was not performed through Moneris Gateway. Once the transaction is completed, VS Corpais must be used to process the Level 2/3 data.

NOTE: the Independent Refund transaction may or may not be supported on your account. If you receive a transaction not allowed error when attempting an independent refund, it may mean the transaction is not supported on your account. If you wish to have the Independent Refund transaction type temporarily enabled (or re-enabled), please contact the Service Centre at 1-866-319-7450.

VS Corpais – (Level 2/3 Data)

VS Corpais will contain all the required and optional data fields for Level 2/3 Business to Business data. VS Corpais data can be sent when the card has been identified in the Pre-authorization transaction request as being a corporate card.

* A VS Purchase Correction can be performed against a transaction as long as the batch that contains the original transaction remains open. When using the automated closing feature, the batch close occurs daily between 10 – 11 pm EST.

5.2.2 Level 2/3 Transaction Flow for Visa

Pre-authorization/Completion Transaction Flow

Purchase Correction Transaction Flow

5.2.3 VS Completion

Once a Pre-authorization is obtained, the funds that are locked need to be retrieved from the customer's credit card. This VS Completion transaction is used to secure the funds locked by a pre-authorization transaction and readies them for settlement into the merchant account.

NOTE: Once you have completed this transaction successfully, to submit the complete supplemental level 2/3 data, please proceed to VS Corpais.

VS Completion transaction object definition

```
$txnArray = array('type'=>'vscompletion', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for VS Completion transaction object

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

VS Completion transaction request fields – Required

Variable Name	Limits	Set Method
Order ID	String 50-character alphanumeric	'order_id'=>\$order_id
Completion amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'comp_amount'=>\$comp_amount
Transaction number	String 255-character alpha-numeric	'txn_number'=>\$txn_number
E-Commerce Indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

Table 1 Visa - Corporate Card Common Data - Level 2 Request Fields

Req*	Value	Limits	Set Method	Description
Y	National Tax	12-character decimal	'national_tax'=>\$national_tax	Must reflect the amount of National Tax (GST or HST) appearing on

Req*	Value	Limits	Set Method	Description
				the invoice. Minimum - 0.01 Maximum - 999999.99. Must have 2 decimal places.
Y	Merchant VAT Registration/Single Business Reference	20-character alphanumeric	'merchant_vat_no'=>\$merchant_vat_no	Merchant's Tax Registration Number must be provided if tax is included on the invoice NOTE: Must not be all spaces or all zeroes
C	Local Tax	12-character decimal	'local_tax'=>\$local_tax	Must reflect the amount of Local Tax (PST or QST) appearing on the invoice If Local Tax included then must not be all spaces or all zeroes; Must be provided if Local Tax (PST or QST) applies

Req*	Value	Limits	Set Method	Description
				Minimum = 0.01 Maximum = 999999.99 Must have 2 decimal places
C	Local Tax (PST or QST) Registration Number	15-character alphanumeric	'local_tax_no'=>\$local_tax_no	Merchant's Local Tax (PST/QST) Registration Number Must be provided if tax is included on the invoice; If Local Tax included then must not be all spaces or all zeroes Must be provided if Local Tax (PST or QST) applies
C	Customer VAT Registration Number	13-character alphanumeric	'customer_vat_no'=>\$customer_vat_no	If the Customer's Tax Registration Number appears on the invoice to support tax exempt transactions it must be provided here
C	Customer Code/Customer Reference Identi-	16-character alphanumeric	'cri'=>\$cri	Value which the customer

Req*	Value	Limits	Set Method	Description
	tifier (CRI)			may choose to provide to the supplier at the point of sale – must be provided if given by the customer
N	Customer Code	17-character alphanumeric	'customer_code'=>\$customer_code	Optional customer code field that will not be passed along to Visa, but will be included on Moneris reporting
N	Invoice Number	17-character alphanumeric	'invoice_number'=>\$invoice_number	Optional invoice number field that will not be passed along to Visa, but will be included on Moneris reporting

*Y = Required, N = Optional, C = Conditional

Sample VS Completion

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
// $status = 'false';
/***** Transactional Variables *****/
$type='vscompletion';
$order_id='ord-210916-15:14:46';
$comp_amount='5.00';
$txn_number = '19002-0_11';
$script='7';
$national_tax = "1.23";
$merchant_vat_no = "gstno111";
$local_tax = "2.34";
$customer_vat_no = "gstno999";
```

Sample VS Completion

```

$cri = "CUST-REF-002";
$customer_code="ccvsfp";
$invoice_number="invsfp";
$local_tax_no="ltaxno";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'comp_amount'=>$comp_amount,
'txn_number'=>$txn_number,
'crypt_type'=>$crypt,
'national_tax'=>$national_tax,
'merchant_vat_no'=>$merchant_vat_no,
'local_tax'=>$local_tax,
'customer_vat_no'=>$customer_vat_no,
'cri'=>$cri,
'local_tax_no'=>$local_tax_no
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.2.4 VS Purchase Correction

The VS Purchase Correction (also known as a "void") transaction is used to cancel a transaction that was performed in the current batch. No amount is required because a void is always for 100% of the original transaction. The only transaction that can be voided using VS Purchase Correction is a VS Completion or VS Force Post. To send a void the order_id and txn_number from the VS Completion/VS Force Post are required.

VS Purchase Correction transaction object definition

```
$txnArray = array('type'=>'vspurchasecorrection', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for VS Purchase Correction transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

VS Purchase Correction transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	<i>String</i> 50-character alphanumeric	'order_id'=>\$order_id
Transaction number	<i>String</i> 255-character alphanumeric	'txn_number'=>\$txn_number
E-Commerce Indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Sample VS Purchase Correction

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
// $status = 'false';
/***** Transactional Variables *****/
$type='vspurchasecorrection';
$order_id='ord-210916-15:28:01';
$amount='5.00';
$txn_number = '19017-0_11';
$crypt='7';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'txn_number'=>$txn_number,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
```


Sample VS Purchase Correction

```

/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.2.5 VS Force Post

The VS Force Post transaction is used to secure the funds locked by a pre-authorization transaction performed over IVR or equivalent terminal. When sending a force post request, you will need Order ID, Amount, Credit Card Number, Expiry Date, E-commerce Indicator and the Authorization Code received in the pre-authorization response.

NOTE: Once you have completed this transaction successfully, to submit the complete supplemental level 2/3 data, please proceed to VS Corpnas.

VS Force Post transaction object definition

```
$txnArray = array('type'=>'vsforcepost', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for VS Force Post transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

VS Force Post transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	<i>String</i> 50-character alphanumeric	'order_id'=>\$order_id
Amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
Credit card number	<i>String</i> 20-character numeric	'pan'=>\$pan
Expiry Date	<i>String</i> 4-character numeric YYMM format	'expiry_date'=>\$expiry_date
Authorization code	<i>String</i> 8-character alphanumeric	'auth_code'=>\$auth_code
E-commerce Indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

VS Force Post transaction request fields – Optional

Variable Name	Type and Limits	Set Method
Customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id

Table 1 Visa - Corporate Card Common Data - Level 2 Request Fields

Req*	Value	Limits	Set Method	Description
Y	National Tax	12-character decimal	'national_tax'=>\$national_tax	Must reflect the amount of National Tax (GST or HST) appearing on the invoice. Minimum - 0.01 Maximum - 999999.99. Must have 2 decimal places.
Y	Merchant VAT Registration/Single Business Reference	20-character alphanumeric	'merchant_vat_no'=>\$merchant_vat_no	Merchant's Tax Registration Number must be provided if tax is included on the invoice NOTE: Must not be all spaces or all zeroes
C	Local Tax	12-character decimal	'local_tax'=>\$local_tax	Must reflect the amount of Local Tax (PST or QST) appearing on the invoice If Local Tax included then must not be all

Req*	Value	Limits	Set Method	Description
				spaces or all zeroes; Must be provided if Local Tax (PST or QST) applies Minimum = 0.01 Maximum = 999999.99 Must have 2 decimal places
C	Local Tax (PST or QST) Registration Number	15-character alphanumeric	'local_tax_no'=>\$local_tax_no	Merchant's Local Tax (PST/QST) Registration Number Must be provided if tax is included on the invoice; If Local Tax included then must not be all spaces or all zeroes Must be provided if Local Tax (PST or QST) applies
C	Customer VAT Registration Number	13-character alphanumeric	'customer_vat_no'=>\$customer_vat_no	If the Customer's Tax Registration Number appears on the invoice to support tax

Req*	Value	Limits	Set Method	Description
				exempt transactions it must be provided here
C	Customer Code/Customer Reference Identifier (CRI)	16-character alphanumeric	'cri'=>\$cri	Value which the customer may choose to provide to the supplier at the point of sale – must be provided if given by the customer
N	Customer Code	17-character alphanumeric	'customer_code'=>\$customer_code	Optional customer code field that will not be passed along to Visa, but will be included on Moneris reporting
N	Invoice Number	17-character alphanumeric	'invoice_number'=>\$invoice_number	Optional invoice number field that will not be passed along to Visa, but will be included on Moneris reporting

*Y = Required, N = Optional, C = Conditional

Sample VS Force Post

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
```

Sample VS Force Post

```

/***** Transactional Variables *****/
$type='vsforcepost';
$cust_id='CUST13343';
$order_id='ord-' . date("dmy-G:i:s");
$amount='5.00';
$pan='4242424254545454';
$expiry_date='2012';
$auth_code='123456';
$crypt='7';
$national_tax = "1.23";
$merchant_vat_no = "gstno111";
$local_tax = "2.34";
$customer_vat_no = "gstno999";
$cri = "CUST-REF-002";
$customerCode="ccvsfp";
$invoiceNumber="invsfp";
$local_tax_no="ltaxno";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'auth_code'=>$auth_code,
'crypt_type'=>$crypt,
'national_tax'=>$national_tax,
'merchant_vat_no'=>$merchant_vat_no,
'local_tax'=>$local_tax,
'customer_vat_no'=>$customer_vat_no,
'cri'=>$cri,
'local_tax_no'=>$local_tax_no
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());

```

Sample VS Force Post

```
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

5.2.6 VS Refund

VS Refund will credit a specified amount to the cardholder's credit card. A refund can be sent up to the full value of the original VS Completion or VS Force Post. To send a VS Refund you will require the Order ID and Transaction Number from the original VS Completion or VS Force Post.

NOTE: Once you have completed this transaction successfully, to submit the complete supplemental level 2/3 data, please proceed to VS Corpais.

VS Refund transaction object definition

```
$txnArray = array('type'=>'vsrefund', ...);
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for VS Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

VS Refund transaction object values

Variable Name	Type and Limits	Set Method
Order ID	<i>String</i> 50-character alphanumeric	'order_id'=>\$order_id
Transaction number	<i>String</i> 255-character alphanumeric	'txn_number'=>\$txn_number
Amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits	'amount'=>\$amount

Variable Name	Type and Limits	Set Method
	(cents) after the decimal point EXAMPLE: 1234567.89	
E-Commerce Indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

Table 1 Visa - Corporate Card Common Data - Level 2 Request Fields

Req*	Value	Limits	Set Method	Description
Y	National Tax	12-character decimal	'national_tax'=>\$national_tax	Must reflect the amount of National Tax (GST or HST) appearing on the invoice. Minimum - 0.01 Maximum - 999999.99. Must have 2 decimal places.
Y	Merchant VAT Registration/Single Business Reference	20-character alphanumeric	'merchant_vat_no'=>\$merchant_vat_no	Merchant's Tax Registration Number must be provided if tax is included on the invoice NOTE: Must

Req*	Value	Limits	Set Method	Description
				not be all spaces or all zeroes
C	Local Tax	12-character decimal	'local_tax'=>\$local_tax	Must reflect the amount of Local Tax (PST or QST) appearing on the invoice If Local Tax included then must not be all spaces or all zeroes; Must be provided if Local Tax (PST or QST) applies Minimum = 0.01 Maximum = 999999.99 Must have 2 decimal places
C	Local Tax (PST or QST) Registration Number	15-character alphanumeric	'local_tax_no'=>\$local_tax_no	Merchant's Local Tax (PST/QST) Registration Number Must be provided if tax is included on the invoice; If Local Tax included then

Req*	Value	Limits	Set Method	Description
				must not be all spaces or all zeroes Must be provided if Local Tax (PST or QST) applies
C	Customer VAT Registration Number	13-character alphanumeric	'customer_vat_no'=>\$customer_vat_no	If the Customer's Tax Registration Number appears on the invoice to support tax exempt transactions it must be provided here
C	Customer Code/Customer Reference Identifier (CRI)	16-character alphanumeric	'cri'=>\$cri	Value which the customer may choose to provide to the supplier at the point of sale – must be provided if given by the customer
N	Customer Code	17-character alphanumeric	'customer_code'=>\$customer_code	Optional customer code field that will not be passed along to Visa, but will be included on Moneris reporting
N	Invoice Number	17-character alphanumeric	'invoice_number'=>\$invoice_number	Optional invoice number field that

Req*	Value	Limits	Set Method	Description
				will not be passed along to Visa, but will be included on Moneris reporting

*Y = Required, N = Optional, C = Conditional

Sample VS Refund

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='vsrefund';
$order_id='ord-210916-15:14:46';
$amount='5.00';
$txn_number = '19003-1_11';
$crypt='7';
$national_tax = "1.23";
$merchant_vat_no = "gstno111";
$local_tax = "2.34";
$customer_vat_no = "gstno999";
$cri = "CUST-REF-002";
$customerCode="ccvsfp";
$invoiceNumber="invsfp";
$local_tax_no="ltaxno";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'txn_number'=>$txn_number,
'crypt_type'=>$crypt,
'national_tax'=>$national_tax,
'merchant_vat_no'=>$merchant_vat_no,
'local_tax'=>$local_tax,
'customer_vat_no'=>$customer_vat_no,
'cri'=>$cri,
'local_tax_no'=>$local_tax_no
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
```

Sample VS Refund

```
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

5.2.7 VS Independent Refund

VS Independent Refund will credit a specified amount to the cardholder's credit card. The independent refund does not require an existing order to be logged in the Moneris Gateway; however, the credit card number and expiry date will need to be passed. The transaction format is almost identical to a pre-authorization.

NOTE: Once you have completed this transaction successfully, to submit the complete supplemental level 2/3 data, please proceed to VS Corpais.

VS Independent Refund transaction object definition

```
$txnArray = array('type'=>'vsind_refund', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for VS Independent Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

VS Independent Refund transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	<i>String</i>	'order_id'=>\$order_id

Variable Name	Type and Limits	Set Method
	50-character alphanumeric	
Amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
Credit card number	<i>String</i> 20-character numeric	'pan'=>\$pan
Expiry date	<i>String</i> 4-character numeric YYMM format	'expiry_date'=>\$expiry_date
E-commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

VS Independent Refund transaction request fields – Optional

Variable Name	Type and Limits	Set Method
Customer ID	50-character alphanumeric	'cust_id'=>\$cust_id

Table 1 Visa - Corporate Card Common Data - Level 2 Request Fields

Req*	Value	Limits	Set Method	Description
Y	National Tax	12-character decimal	'national_tax'=>\$national_tax	Must reflect the amount of National Tax (GST or HST) appearing on the invoice.

Req*	Value	Limits	Set Method	Description
				Minimum - 0.01 Maximum - 999999.99. Must have 2 decimal places.
Y	Merchant VAT Registration/Single Business Reference	20-character alphanumeric	'merchant_vat_no'=>\$merchant_vat_no	Merchant's Tax Registration Number must be provided if tax is included on the invoice NOTE: Must not be all spaces or all zeroes
C	Local Tax	12-character decimal	'local_tax'=>\$local_tax	Must reflect the amount of Local Tax (PST or QST) appearing on the invoice If Local Tax included then must not be all spaces or all zeroes; Must be provided if Local Tax (PST or QST) applies Minimum = 0.01

Req*	Value	Limits	Set Method	Description
				Maximum = 9999999.99 Must have 2 decimal places
C	Local Tax (PST or QST) Registration Number	15-character alphanumeric	'local_tax_no'=>\$local_tax_no	Merchant's Local Tax (PST/QST) Registration Number Must be provided if tax is included on the invoice; If Local Tax included then must not be all spaces or all zeroes Must be provided if Local Tax (PST or QST) applies
C	Customer VAT Registration Number	13-character alphanumeric	'customer_vat_no'=>\$customer_vat_no	If the Customer's Tax Registration Number appears on the invoice to support tax exempt transactions it must be provided here
C	Customer Code/Customer Reference Identifier (CRI)	16-character alphanumeric	'cri'=>\$cri	Value which the customer may choose to provide to the supplier at the

Req*	Value	Limits	Set Method	Description
				point of sale – must be provided if given by the customer
N	Customer Code	17-character alphanumeric	'customer_code'=>\$customer_code	Optional customer code field that will not be passed along to Visa, but will be included on Moneris reporting
N	Invoice Number	17-character alphanumeric	'invoice_number'=>\$invoice_number	Optional invoice number field that will not be passed along to Visa, but will be included on Moneris reporting

*Y = Required, N = Optional, C = Conditional

Sample VS Independent Refund
<pre> <?php require "../mpgClasses.php"; /***** Request Variables *****/ \$store_id='moneris'; \$api_token='hurgle'; //\$status = 'false'; /***** Transactional Variables *****/ \$type='vsind_refund'; \$cust_id='CUST13343'; \$order_id='ord-'.date("dmy-G:i:s"); \$amount='5.00'; \$pan='4242424254545454'; \$expiry_date='2012'; \$script='7'; \$national_tax = "1.23"; \$merchant_vat_no = "gstnol11"; \$local_tax = "2.34"; \$customer_vat_no = "gstno999"; \$cri = "CUST-REF-002"; \$customerCode="ccvsfp"; </pre>

Sample VS Independent Refund

```

$invoiceNumber="invsfp";
$local_tax_no="ltaxno";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'crypt_type'=>$crypt,
'national_tax'=>$national_tax,
'merchant_vat_no'=>$merchant_vat_no,
'local_tax'=>$local_tax,
'customer_vat_no'=>$customer_vat_no,
'cri'=>$cri,
'local_tax_no'=>$local_tax_no
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
// Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.2.8 VS Corpais

VS Corpais will contain all the required and optional data fields for Level 2/3 Purchasing Card Addendum data. VS Corpais data can be sent when the card has been identified in the Pre-authorization transaction request as being a corporate card.

In addition to the order ID and transaction number, this transaction also contains two objects:

- VS Purcha – Corporate Card Common Data
- VS Purchl – Line Item Details

VS Corpais request must be preceded by a financial transaction (VS Completion, VS Force Post, VS Refund, VS Independent Refund) and the Corporate Card flag must be set to “true” in the Pre-authorization response.

VS Corpais transaction object definition

```
$txnArray = array('type'=>'vscorpais', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for VS Corpais transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

VS Corpais transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	<i>String</i> 50-character alphanumeric	'order_id'=>\$order_id
Transaction number	<i>String</i> 255-character alphanumeric	'txn_number'=>\$txn_number
vsPurcha For a list of the variables that appear in this object, see the table below	<i>String</i> n/a	\$vsPurcha = new vsPurcha(); \$mpgVsLevel23 = new mpgVsLevel23(); \$mpgVsLevel23->setVsPurch(\$vsPurcha, \$vsPurchl);
vsPurchl For a list of the variables that appear in this object, see the table below	<i>String</i> n/a	\$vsPurchl = new vsPurchl(); \$mpgVsLevel23 = new mpgVsLevel23(); \$mpgVsLevel23->setVsPurch(\$vsPurcha, \$vsPurchl);

*Y = Required, N = Optional, C = Conditional

5.2.8.1 VS Purcha – Corporate Card Common Data

VS Corpais transactions use the VS Purcha object to contain Level 2 data.

Variable Name	Type and Limits	Description
Buyer Name	<i>String</i> 30-character alphanumeric	Buyer/Recipient Name NOTE: Name required by CRA on transactions >\$150
Local Tax Rate	<i>String</i> 4-character decimal	Indicates the detailed tax rate applied in relationship to a local tax amount EXAMPLE: 8% PST should be 8.0 Minimum = 0.01 Maximum = 99.99 NOTE: Must be provided if Local Tax (PST or QST) applies.
Duty Amount	<i>String</i> 9-character decimal	Duty on total purchase amount A minus sign means 'amount is a credit', plus sign or no sign means 'amount is a debit' maximum without sign is 999999.99
Invoice Discount Treatment	<i>String</i> 1-character numeric	Indicates how the merchant is managing discounts Must be one of the following values: 0 - if no invoice level discounts apply for this invoice 1 - if Tax was calculated on Post-Discount totals 2 - if Tax was calculated on Pre-Discount totals
Invoice Level Discount Amount	<i>String</i> 9-character decimal	Amount of discount (if provided at the invoice level according to the Invoice Discount Treatment)

Variable Name	Type and Limits	Description
		Must be non-zero if Invoice Discount Treatment is 1 or 2 Minimum amount is 0.00 and maximum is 999999.99
Ship To Postal Code / Zip Code	<i>String</i> 10-character alphanumeric	The postal code or zip code for the destination where goods will be delivered NOTE: Required if shipment is involved Full alpha postal code - Valid ANA<space>NAN format required if shipping to an address within Canada
Ship From Postal Code / Zip Code	<i>String</i> 10-character alphanumeric	The postal code or zip code from which items were shipped For Canadian addresses, requires full alpha postal code for the merchant with Valid ANA<space>NAN format
Destination Country Code	2-character alphanumeric	Code of country where purchased goods will be delivered Use ISO 3166-1 alpha-2 format NOTE: Required if it appears on the invoice for an international transaction
Unique VAT Invoice Reference Number	<i>String</i> 25-character alphanumeric	Unique Value Added Tax Invoice Reference Number Must be populated with the invoice number and this cannot be all spaces or zeroes
Tax Treatment	<i>String</i> 1-character alphanumeric	Must be one of the following values: 0 = Net Prices with tax calculated at line item level; 1 = Net Prices with tax calculated at invoice level;

Variable Name	Type and Limits	Description
		2 = Gross prices given with tax information provided at line item level; 3 = Gross prices given with tax information provided at invoice level; 4 = No tax applies (small merchant) on the invoice for the transaction
Freight/Shipping Amount (Ship Amount)	<i>String</i> 9-character decimal	Freight charges on total purchase If shipping is not provided as a line item it must be provided here, if applicable Signed monetary amount: Minus (-) sign means 'amount is a credit', Plus (+) sign or no sign means 'amount is a debit' Maximum without sign is 999999.99
GST HST Freight Rate	<i>String</i> 4-character decimal	Rate of GST (excludes PST) or HST charged on the shipping amount (in accordance with the Tax Treatment) If Freight/Shipping Amount is provided then this (National GST or HST) tax rate must be provided. Monetary amount, maximum is 99.99. Such as 13% HST is 13.00
GST HST Freight Amount	<i>String</i> 9-character decimal	Amount of GST (excludes PST) or HST charged on the shipping amount If Freight/Shipping Amount is provided then this (National GST or HST) tax amount must be provided if taxTreatment is 0 or 2 Signed monetary amount: maximum without sign is 999999.99.

5.2.8.2 VS Purchl – Line Item Details

VS Corpais transactions use the VS Purchl object to contain Level 3 data.

Line Item Details for VS Purchl

```
$item_com_code = array("X3101", "X84802");

$product_code = array("CHR123", "DDSK200");

$item_description = array("Office Chair", "Disk Drive");

$item_quantity = array("3", "1");

$item_uom = array("EA", "EA");

$unit_cost = array("0.20", "0.40");

$vat_tax_amt = array("0.00", "0.00");

$vat_tax_rate = array("13.00", "13.00");

$discount_treatmentL = array("0", "0");

$discount_amtL = array("0.00", "0.00");
```

Setting VS Purchl Line Item Details

```
$vsPurchl->setVsPurchl($item_com_code[0], $product_code[0], $item_description
[0], $item_quantity[0], $item_uom[0], $unit_cost[0], $vat_tax_amt[0], $vat_
tax_rate[0], $discount_treatmentL[0], $discount_amtL[0]);
```

Table 1 Corporate Card Common Data - Level 3 Request Fields - VSPurchl

Req*	Value	Limits	Variable/Field	Description
C	Item Commodity Code	12-character alpha-numeric	item_com_code	Line item Comodity Code (if this field is not sent, then Product Code must be sent)
Y	Product Code	12-character alpha-numeric	product_code	Product code for this line item – merchant's product code, manufacturer's product code or buyer's product code Typically this will be the SKU or iden-

Req*	Value	Limits	Variable/Field	Description
				tifier by which the merchant tracks and prices the item or service This should always be provided for every line item
Y	Item Description	35-character alpha-numeric	item_description	Line item description
Y	Item Quantity	12-character decimal	item_quantity	Quantity invoiced for this line item Up to 4 decimal places supported, whole numbers are accepted Minimum = 0.0001 Maximum = 999999999999
Y	Item Unit of Measure	2-character alpha-numeric	item_uom	Unit of measure Use ANSI X-12 EDI Allowable Units of Measure and Codes
Y	Item Unit Cost	12-character decimal	unit_cost	Line item cost per unit 2-4 decimal places accepted Minimum = 0.0001 Maximum = 999999.9999
N	VAT Tax Amount	12-character decimal	vat_tax_amt	Any value-added tax or other sales tax amount

Req*	Value	Limits	Variable/Field	Description
				Must have 2 decimal places Minimum = 0.01 Maximum = 999999.99
N	VAT Tax Rate	4-character decimal	vat_tax_rate	Sales tax rate EXAMPLE: 8% PST should be 8.0 maximum 99.99
Y	Discount Treatment	1-character numeric	discount_treatmentL	Must be one of the following values: 0 if no invoice level discounts apply for this invoice 1 if Tax was calculated on Post-Discout totals 2 if Tax was calculated on Pre-Discout totals
C	Discount Amount	12-character decimal	discount_amtL	Amount of discount, if provided for this line item according to the Line Item Discount Treatment Must be non-zero if Line Item Discount Treatment is 1 or 2 Must have 2 decimal places Minimum = 0.01 Maximum = 999999.99

5.2.8.3 Sample Code for VS Corpsais

Sample VS Corpsais

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='vscorpais';
$cust_id='CUST13343';
$order_id='ord-160916-15:31:39';
$txn_number='18306-0_11';
$buyer_name = "Buyer Manager";
$local_tax_rate = "13.00";
$duty_amount = "0.00";
$discount_treatment = "0";
$discount_amt = "0.00";
$freight_amount = "0.20";
$ship_to_pos_code = "M8X 2W8";
$ship_from_pos_code = "M1K 2Y7";
$des_cou_code = "CAN";
$vat_ref_num = "VAT12345";
$tax_treatment = "3";//3 = Gross prices given with tax information provided at invoice
level
$gst_hst_freight_amount = "0.00";
$gst_hst_freight_rate = "13.00";
$item_com_code = array("X3101", "X84802");
$product_code = array("CHR123", "DDSK200");
$item_description = array("Office Chair", "Disk Drive");
$item_quantity = array("3", "1");
$item_uom = array("EA", "EA");
$unit_cost = array("0.20", "0.40");
$vat_tax_amt = array("0.00", "0.00");
$vat_tax_rate = array("13.00", "13.00");
$discount_treatmentL = array("0", "0");
$discount_amtL = array("0.00", "0.00");
//Create and set VsPurcha
$vsPurcha = new vsPurcha();
$vsPurcha->setBuyerName($buyer_name);
$vsPurcha->setLocalTaxRate($local_tax_rate);
$vsPurcha->setDutyAmount($duty_amount);
$vsPurcha->setDiscountTreatment($discount_treatment);
$vsPurcha->setDiscountAmt($discount_amt);
$vsPurcha->setFreightAmount($freight_amount);
$vsPurcha->setShipToPostalCode($ship_to_pos_code);
$vsPurcha->setShipFromPostalCode($ship_from_pos_code);
$vsPurcha->setDesCouCode($des_cou_code);
$vsPurcha->setVatRefNum($vat_ref_num);
$vsPurcha->setTaxTreatment($tax_treatment);
$vsPurcha->setGstHstFreightAmount($gst_hst_freight_amount);
$vsPurcha->setGstHstFreightRate($gst_hst_freight_rate);
//Create and set VsPurchl
$vsPurchl = new vsPurchl();
$vsPurchl->setVsPurchl($item_com_code[0], $product_code[0], $item_description[0], $item_
quantity[0], $item_uom[0], $unit_cost[0], $vat_tax_amt[0], $vat_tax_rate[0], $discount_
treatmentL[0], $discount_amtL[0]);
$vsPurchl->setVsPurchl($item_com_code[1], $product_code[1], $item_description[1], $item_
quantity[1], $item_uom[1], $unit_cost[1], $vat_tax_amt[1], $vat_tax_rate[1], $discount_

```

Sample VS Corpsais

```

treatmentL[1], $discount_amtL[1]);
//Create and set VsLevel23
$mpgVsLevel23 = new mpgVsLevel23();
$mpgVsLevel23->setVsPurch($vsPurcha, $vsPurchl);
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'txn_number'=>$txn_number,
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setLevel23Data($mpgVsLevel23);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.3 Level 2/3 Mastercard Transactions

- 5.3.1 Level 2/3 Transaction Types for Mastercard
- 5.3.2 Level 2/3 Transaction Flow for Mastercard
- 5.3.3 MC Completion
- 5.3.4 MC Force Post

- 5.3.5 MC Purchase Correction
- 5.3.6 MC Refund
- 5.3.7 MC Independent Refund
- 1 MC Corpais – Level 2/3 Transactions

5.3.1 Level 2/3 Transaction Types for Mastercard

This transaction set includes a suite of corporate card financial transactions as well as a transaction that allows for the passing of Level 2/3 data. Please ensure MC Level 2/3 processing support is enabled on your merchant account. Batch Close, Open Totals and Pre-authorization are identical to the transactions outlined in the section Basic Transaction Set (page 21).

When the Preauth response contains CorporateCard equal to true then you can submit the MC transactions.

If CorporateCard is false then the card does not support Level 2/3 data and non Level 2/3 transaction are to be used. If the card is not a corporate card, please refer to section 4 for the appropriate non-corporate card transactions.

NOTE: This transaction set is intended for transactions where Corporate Card is true and Level 2/3 data will be submitted. If the credit card is found to be a corporate card but you do not wish to send any Level 2/3 data then you may submit MC transactions using the transaction set outlined in Basic Transaction Set (page 21).

Pre-auth – (authorization/pre-authorization)

The pre-auth verifies and locks funds on the customer's credit card. The funds are locked for a specified amount of time, based on the card issuer. To retrieve the funds from a pre-auth so that they may be settled in the merchant account a capture must be performed. Level 2/3 data submission is not supported as part of a pre-auth as a pre-auth is not settled. When CorporateCard is returned true then Level 2/3 data may be submitted.

MC Completion – (Capture/Preauth Completion)

Once a Pre-authorization is obtained the funds that are locked need to be retrieved from the customer's credit card. The capture retrieves the locked funds and readies them for settlement in to the merchant account. Prior to performing an MCCompletion a Pre-auth must be performed.

MC Force Post – (Force Capture/Preauth Completion)

This transaction is an alternative to MC Completion to obtain the funds locked on Preauth obtained from IVR or equivalent terminal. The MC Force Post requires that the original Pre-authorization's auth code is provided and it retrieves the locked funds and readies them for settlement in to the merchant account.

MC Purchase Correction – (Void, Correction)

MC Completions can be voided the same day* that they occur. A void must be for the full amount of the transaction and will remove any record of it from the cardholder statement. * An MC Purchase Correction can be performed against a transaction as long as the batch that contains the original transaction remains open. When using the automated closing feature batch close occurs daily between 10 – 11 pm EST.

MC Refund – (Credit)

A MC Refund can be performed against an MC Completion or MC Force Post to refund an amount less than or equal to the amount of the original transaction.

MC Independent Refund – (Credit)

A MC Independent Refund can be performed against an completion to refund any part, or all of the transaction. Independent refund is used when the originating transaction was not performed through Moneris Gateway. Please note, the MC Independent Refund transaction may or may not be supported on your account. If you receive a transaction not allowed error when attempting an MC Independent Refund, it may mean the transaction is not supported on your account. If you wish to have the MC Independent Refund transaction type temporarily enabled (or re-enabled), please contact the Service Centre at 1-866-319-7450.

MC Corpais Common Line Item – (Level 2/3 Data)

MC Corpais Common Line Item will contain the entire required and optional data field for Level 2/3 data. MCCorpais Common Line Item data can be sent when the card has been identified in the transaction request as being a corporate card. This transaction supports multiple data types and combinations:

- Purchasing Card Data:
 - Corporate card common data with Line Item Details

5.3.2 Level 2/3 Transaction Flow for Mastercard

Pre-Authorization/Completion Transaction Flow

Purchase Correction Transaction Flow

5.3.3 MC Completion

The MC Completion transaction is used to secure the funds locked by a Pre-Authorization transaction. When sending a capture request you will need two pieces of information from the original pre-authorization– the order ID and the transaction number from the returned response.

Once you have completed this transaction successfully, to submit the complete supplemental level 2/3 data, please proceed to MC Corpais.

MC Completion transaction object definition

```
$txnArray = array('type'=>'mccompletion', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MC Completion transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

MC Completion transaction request fields – Required

Variable Name	Limits	Set Method
Order ID	String 50-character alphanumeric	'order_id'=>\$order_id
Completion amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'comp_amount'=>\$comp_amount
Transaction number	String 255-character alphanumeric	'txn_number'=>\$txn_number

Variable Name	Limits	Set Method
Merchant reference number	String 19-character alphanumeric	'merchant_ref_no'=>\$merchant_ref_no
E-commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

Sample MC Completion

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='mccompletion';
$order_id='ord-210916-16:13:11';
$comp_amount='5.00';
$txn_number='19021-0_11';
$crypt='7';
$merchant_ref_no = "319038";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'comp_amount'=>$comp_amount,
'txn_number'=>$txn_number,
'merchant_ref_no' => $merchant_ref_no,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
```

Sample MC Completion

```
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

5.3.4 MC Force Post

MC Force Post transaction is used to secure the funds locked by a Pre-Authorization transaction performed over IVR or equivalent terminal. When sending a Force Post request, you will need order_id, amount, pan (card number), expiry date, crypt type and the authorization code received in the Pre-Authorization response.

MC Force Post transaction object definition

```
$txnArray = array('type'=>'mcforcepost', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MC Force Post transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

MC Force Post transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	String 50-character alphanumeric	'order_id'=>\$order_id
Amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount

Variable Name	Type and Limits	Set Method
Credit card number	<i>String</i> 20-character alphanumeric	'pan'=>\$pan
Expiry date	<i>String</i> 4-character alphanumeric (YYMM format)	'expiry_date'=>\$expiry_date
Authorization code	<i>String</i> 8-character alphanumeric	'auth_code'=>\$auth_code
E-commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
Merchant reference number	<i>String</i> 19-character alphanumeric	'merchant_ref_no'=>\$merchant_ref_no

MC Force Post transaction request fields – Optional

Variable Name	Type and Limits	Set Method
Customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id

Sample MC Force Post

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='mcforcepost';
$cust_id='CUST13343';
$order_id='ord-' . date("dmy-G:i:s");
$amount='5.00';
$pan='5454545442424242';
$expiry_date='2012';
$auth_code='123456';
$crypt='7';

```


Sample MC Force Post

```

$merchant_ref_no = "319038";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'auth_code'=>$auth_code,
'merchant_ref_no' => $merchant_ref_no,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.3.5 MC Purchase Correction

The MC Purchase Correction (void) transaction is used to cancel a transaction that was performed in the current batch. No amount is required because a void is always for 100% of the original transaction. The only transaction that can be voided is completion. To send a void, the order ID and transaction number from the MC Completion or MC Force Post are required.

MC Purchase Correction transaction object definition

```

$txnArray = array('type'=>'mcpurchasecorrection', ...);

$mpgTxn = new mpgTransaction($txnArray);

```

HttpPostRequest object for MC Purchase Correction transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

MC Purchase Correction transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	<i>String</i> 50-character alphanumeric	'order_id'=>\$order_id
Transaction number	<i>String</i> 255-character alpha-numeric	'txn_number'=>\$txn_number
E-commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

Sample MC Purchase Correction

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='mcpurchasecorrection';
$order_id='ord-210916-16:15:50';
$txn_number='66011731642016265161550929-0_11';
$crypt='7';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'txn_number'=>$txn_number,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
```

Sample MC Purchase Correction

```
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

5.3.6 MC Refund

The MC Refund will credit a specified amount to the cardholder's credit card. A refund can be sent up to the full value of the original capture. To send a refund you will require the Order ID and Transaction Number from the original MC Completion or MC Force Post.

MC Refund transaction object definition

```
$txnArray = array('type'=>'mcrefund', ...);

$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MC Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

MC Refund transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	50-character alphanumeric	'order_id'=>\$order_id
Amount	10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point	'amount'=>\$amount

Variable Name	Type and Limits	Set Method
EXAMPLE: 1234567.89		
Transaction number	255-character alpha-numeric	'txn_number'=>\$txn_number
E-commerce indicator	1-character alphanumeric	'crypt_type'=>\$crypt
Merchant reference number	19-character alphanumeric	'merchant_ref_no'=>\$merchant_ref_no

Sample MC Refund

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='mcrefund';
$order_id='ord-210916-16:13:11';
$amount='5.00';
$txn_number='19021-1_11';
$crypt='7';
$merchant_ref_no = "319038";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'txn_number'=>$txn_number,
'merchant_ref_no' => $merchant_ref_no,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpsPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpsPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());

```

Sample MC Refund

```

print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.3.7 MC Independent Refund

MC Independent Refund is used when the originating transaction was not performed through Moneris Gateway and does not require an existing order to be logged in the Moneris Gateway; however, the credit card number and the expiry date will need to be passed. The transaction format is almost identical to a purchase or a pre-authorization.

NOTE: Independent refund transactions are not supported on all accounts. If you receive a transaction not allowed error when attempting an independent refund transaction, it may mean the feature is not supported on your account. To have Independent Refund transaction functionality temporarily enabled (or re-enabled), please contact the Moneris Customer Service Centre at 1-866-319-7450.

Once you have completed this transaction successfully, to submit the complete supplemental level 2/3 data, please proceed to MC Corpais.

MC Independent Refund transaction object definition

```

$txnArray = array('type'=>'mcind_refund', ...);

$mpgTxn = new mpgTransaction($txnArray);

```

HttpPostRequest object for MC Independent Refund transaction

```

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);

```

MC Independent Refund transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	50-character alphanumeric	'order_id'=>\$order_id
Amount	10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
E-commerce indicator	1-character alphanumeric	'crypt_type'=>\$crypt
Credit card number	20-character numeric	'pan'=>\$pan
Expiry date	4-character numeric (YYMM format)	'expiry_date'=>\$expiry_date
Merchant reference number	19-character alphanumeric	'merchant_ref_no'=>\$merchant_ref_no

MC Independent Refund transaction request fields – Optional**Table 1 MC Independent Refund transaction object optional values**

Variable Name	Type and Limits	Set Method
Customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id

Sample MC Independent Refund

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
// $status = 'false';
/***** Transactional Variables *****/
$type='mcind_refund';
$cust_id='CUST13343';

```

Sample MC Independent Refund

```

$order_id='ord-' . date("dmy-G:i:s");
$amount='5.00';
$pan='5454545442424242';
$expiry_date='2012';
$crypt='7';
$merchant_ref_no = "319038";
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'merchant_ref_no' => $merchant_ref_no,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.3.8 MC Corpais - Corporate Card Common Data with Line Item Details

This transaction example includes the following elements for Level 2 and 3 purchasing card corporate card data processing:

- Corporate Card Common Data (MC Corpac)
 - only 1 set of MC Corpac fields can be submitted
 - this data set includes data elements that apply to the overall order, e.g., the total overall taxes
- Line Item Details (MC Corpal)
 - 1-998 counts of MC Corpal line items can be submitted
 - This data set includes the details about each individual item or service purchased

The MC Corpais request must be preceded by a financial transaction (MC Completion, MC Force Post, MC Refund, MC Independent Refund) and the Corporate Card flag must be set to “true” in the Preauthorization response. The MC Corpais request will need to contain the Order ID of the financial transaction as well as the Transaction Number.

In addition, MC Corpais has a tax array object that can be sent via the Tax fields in MC Corpac and MC Corpal. For more about the tax array object, see 5.3.8.3 Tax Array Object - MC Corpais.

For descriptions of the Level 2/3 fields, please see Definition of Request Fields for Level 2/3 - Mastercard (page 528).

MC Corpais transaction object definition

```
$txnArray = array('type'=>'mccorpais', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MC Corpais transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```


MC Corpais transaction object values**Table 1 MC Corpais transaction object mandatory values**

Value	Type	Limits	Set Method
Order ID	String	50-character alpha-numeric	'order_id'=>\$order_id
Transaction number	String	255-character alpha-numeric	'txn_number'=>\$txn_number
MCCorpac	Object	n/a	<pre>\$mpgMcLevel23 = new mpgMcLevel23(); \$mpgMcLevel23->setMcCorpac (\$mcCorpac);</pre>
MC Corpal	Object	n/a	<pre>\$mpgMcLevel23 = new mpgMcLevel23(); \$mpgMcLevel23->setMcCorpall (\$mcCorpall);</pre>

*Y = Required, N = Optional, C = Conditional

5.3.8.1 MC Corpac - Corporate Card Common Data**Table 1 Corporate Card Common Data - Level 2 Request Fields - MCCorpac**

Re-q*	Value	Limits	Set Method	Description
N	Austin-Tetra Number	15-character alpha-numeric	<pre>\$mcCorpac->setAustinTetraNumber (\$austin_tetra_number);</pre>	The Austin-Tetra Number assigned to the card acceptor
N	NAICS Code	15-character alpha-numeric	<pre>\$mcCorpac->setNaicsCode (\$naics_code);</pre>	North American Industry Classification System (NAICS) code assigned to the card acceptor
N	Customer Code	25-character alpha-numeric	<pre>\$mcCorpac->setCustomerCode1 (\$customer_ code1_c);</pre>	A control number, such as purchase order number, project number, department allocation number or name that the purchaser supplied

Re-q*	Value	Limits	Set Method	Description
				the merchant Left-justified; may be spaces
N	Unique Invoice Number	17-character alpha-numeric	<code>\$mcCorpac->setUniqueInvoiceNumber(\$unique_invoice_number_c);</code>	Unique number associated with the individual transaction provided by the merchant
N	Commodity Code	15-character alpha-numeric	<code>\$mcCorpac->setCommodityCode(\$commodity_code);</code>	Code assigned by the merchant that best categorizes the item(s) being purchased
N	Order Date	6-character numeric YYMMDD format	<code>\$mcCorpac->setOrderDate(\$order_date_c);</code>	The date the item was ordered NOTE: If present, must contain a valid date
N	Corporation VAT Number	20-character alpha-numeric	<code>\$mcCorpac->setCorporationVatNumber(\$corporation_vat_number_c);</code>	Contains a corporation's value added tax (VAT) number
N	Customer VAT Number	20-character alpha-numeric	<code>\$mcCorpac->setCustomerVatNumber(\$customer_vat_number_c);</code>	Contains the VAT number for the customer / cardholder used to identify the customer when purchasing goods and services from the merchant
N	Freight Amount	12-character decimal	<code>\$mcCorpac->setFreightAmount1(\$freight_amount_c);</code>	The freight on the total purchase Must have 2 decimals Minimum = 0.00 Maximum = 999999.99

Re-q*	Value	Limits	Set Method	Description
N	Duty Amount	12-character decimal	<code>\$vsPurcha->setDutyAmount(\$duty_amount);</code>	The duty on the total purchase Must have 2 decimals Minimum = 0.00 Maximum = 999999.99
N	Destination State / Province Code	3-character alpha-numeric	<code>\$mcCorpac->setDestinationProvinceCode(\$destination_province_code);</code>	State or Province of the country where the goods will be delivered Left justified with trailing spaces EXAMPLE: ONT = Ontario
N	Destination Country Code	3-character alpha-numeric ISO 3166-1 alpha-3 format	<code>\$mcCorpac->setDestinationCountryCode(\$destination_country_code);</code>	The country code where goods will be delivered Left justified with trailing spaces ISO 3166-1 alpha-3 format EXAMPLE: CAN = Canada
N	Ship From Postal Code	10-character alpha-numeric ANA NAN format	<code>\$mcCorpac->setShipFromPosCode(\$ship_from_pos_code);</code>	The postal code or zip code from which items were shipped Full alpha postal code - Valid ANA<space>NAN format
N	Destination Postal Code	10-character alpha-numeric	<code>\$mcCorpac->setShipToPosCode(\$ship_to_pos_code_c);</code>	The postal code or zip code where goods will be delivered

Re-q*	Value	Limits	Set Method	Description
				Full alpha postal code - Valid ANA<space>NAN format if shipping to an address within Canada
N	Authorized Contact Name	36-character alpha-numeric	<code>\$mcCorpac->setAuthorizedContactName(\$authorized_contact_name_c);</code>	Name of an individual or company contacted for company authorized purchases
N	Authorized Contact Phone	17-character alpha-numeric	<code>\$mcCorpac->setAuthorizedContactPhone(\$authorized_contact_phone);</code>	Phone number of an individual or company contacted for company authorized purchases
N	Additional Card Acceptor Data	40-character alpha-numeric	<code>\$mcCorpac->setAdditionalCardAcceptorData(\$additional_card_acceptor_data);</code>	Information pertaining to the card acceptor
N	Card Acceptor Type	8-character alpha-numeric	<code>\$mcCorpac->setCardAcceptorType(\$card_acceptor_type);</code>	Various classifications of business ownership characteristics This field takes 8 characters. Each character represents a different component, as follows: 1st character represents 'Business Type' and contains a code to identify the specific classification or type of business: - Corporation - Not known - Individual/Sole Proprietorship - Partnership - Association/Estate/Trust

Re-q*	Value	Limits	Set Method	Description
				Tax Exempt Organizations (501C) International Organization Limited Liability Company (LLC) Government Agency 2nd character represents 'Business Owner Type'. Contains a code to identify specific characteristics about the business owner. 1 - No application classification 2 - Female business owner 3 - Physically handicapped female business owner 4 - Physically handicapped male business owner 0 - Unknown 3rd character represents 'Business Certification Type'. Contains a code to identify specific characteristics about the business certification type, such as small business, disadvantaged, or other certification type: 1 - Not certified 2 - Small Business Administration (SBA) certification small business 3 - SBA certification

Re-q*	Value	Limits	Set Method	Description
				as small disadvantaged business 4 - Other government or agency-recognized certification (such as Minority Supplier Development Council) 5 - Self-certified small business 6 - SBA certification as small and other government or agency-recognized certification 7 - SBA certification as small disadvantaged business and other government or agency-recognized certification 8 - Other government or agency-recognized certification and self-certified small business A - SBA certification as 8(a) B - Self-certified small disadvantaged business (SDB) C - SBA certification as HUBZone 0 - Unknown 4th character represents 'Business Racial/Ethnic Type'. Contains a code identifying

Re-q*	Value	Limits	Set Method	Description
				the racial or ethnic type of the majority owner of the business. 1 - African American 2 - Asian Pacific American 3 - Subcontinent Asian American 4 - Hispanic American 5 - Native American Indian 6 - Native Hawaiian 7 - Native Alaskan 8 - Caucasian 9 - Other 0 - Unknown 5th character represents 'Business Type Provided Code' Y - Business type is provided. N - Business type was not provided. R - Card acceptor refused to provide business type 6th character represents 'Business Owner Type Provided Code' Y - Business owner type is provided.

Re-q*	Value	Limits	Set Method	Description
				N - Business owner type was not provided. R - Card acceptor refused to provide business type 7th character represents 'Business Certification Type Provided Code' Y - Business certification type is provided. N - Business certification type was not provided. R - Card acceptor refused to provide business type 8th character represents 'Business Racial/Ethnic Type' Y - Business racial/ethnic type is provided. N - Business racial/ethnic type was not provided. R - Card acceptor refused to provide business racial/ethnic type
N	Card Acceptor Tax ID	20-character alpha- numeric	\$mcCorpac- >setCardAcceptorTaxTd (\$card_acceptor_tax_id_c);	US federal tax ID number or value-added tax (VAT) ID

Req*	Value	Limits	Set Method	Description
N	Card Acceptor Reference Number	25-character alpha-numeric	<code>\$mcCorpac->setCardAcceptorReferenceNumber(\$card_acceptor_reference_number);</code>	Code that facilitates card acceptor/corporation communication and record keeping
N	Card Acceptor VAT Number	20-character alpha-numeric	<code>\$mcCorpac->setCardAcceptorVatNumber(\$card_acceptor_vat_number_c);</code>	Value added tax (VAT) number for the card acceptor location Used to identify the card acceptor when collecting and reporting taxes
C	Tax	Up to 6 arrays	<code>\$mcCorpac->setTax(\$mcTax_c);</code>	Can have up to 6 arrays containing different tax details NOTE: If you use this variable, you must fill in all the fields of tax array mentioned below.

5.3.8.2 MC Corpal - Line Item Details

MC Corpal Object - Line Item Details

```
$mcCorpac->setMcCorpac($customer_code1_l[0], $line_item_date1_l[0], $ship_date1_l[0], $order_date1_l[0], $product_code1_l[0], $item_description1_l[0], $item_quantity1_l[0], $unit_cost1_l[0], $item_unit_measure1_l[0], $ext_item_amount1_l[0], $discount_amount1_l[0], $commodity_code1_l[0], $type_of_supply1_l[0], $vat_ref_num1_l[0], $mcTax1_l[0]);
```

Table 1 Line Item Details - Level 3 Request Fields - MC Corpal

Req*	Value	Limits	Variable	Description
N	Customer Code	25-character alpha-numeric	<code>customer_code1_l</code>	A control number, such as purchase order number, project number, department alloc-

Req*	Value	Limits	Variable	Description
				ation number or name that the purchaser supplied the merchant
N	Line Item Date	6-character numeric YYMMDD format	line_item_date_l	The purchase date of the line item referenced in the associated Corporate Card Line Item Detail Fixed length 6 Numeric, in YYMMDD format
N	Ship Date	6-character numeric YYMMDD format	ship_date_l	The date the merchandise was shipped to the destination Fixed length 6 Numeric, in YYMMDD format
N	Order Date	6-character numeric YYMMDD format	order_date1_l	The date the item was ordered Fixed length 6-character numeric, in YYMMDD format
Y	Product Code	12-character alphanumeric	product_code1_l	Line item Product Code Contains the non-fuel related product code of the individual item purchased

Req*	Value	Limits	Variable	Description
Y	Item Description	35-character alpha-numeric	item_description_l	Line Item description Contains the description of the individual item purchased
Y	Item Quantity	12-character alpha-numeric	item_quantity_l	Quantity of line item Up to 5 decimal places supported Minimum amount is 0.0 and maximum is 9999999.99999
Y	Unit Cost	12-character decimal	unit_cost_l	Line item cost per unit. Must contain a minimum of 2 decimal places, up to 5 decimal places supported. Minimum amount is 0.00001 and maximum is 999999.99999
Y	Item Unit Measure	12-character alpha-numeric	item_unit_measure_l	The line item unit of measurement code ANSI X-12 EDI Allowable Units of Measure and Codes
Y	Extended Item Amount	9-character decimal	ext_item_amount_l	Contains the individual item amount that is normally cal-

Req*	Value	Limits	Variable	Description
				culated as price multiplied by quantity Must contain 2 decimal places Minimum amount is 0.00 and maximum is 999999.99
N	Discount Amount	9-character decimal	discount_amount_l	Contains the item discount amount Must contain 2 decimal places Minimum amount is 0.00 and maximum is 999999.99
N	Commodity Code	15-character alphanumeric	commodity_code_l	Code assigned to the merchant that best categorizes the item(s) being purchased
C	Tax	Up to 6 arrays	tax_l	Can have up to 6 arrays containing different tax details –see Tax Array Request Fields table below for each field description NOTE: If you use this variable, you must fill in all the fields of tax array mentioned below.

5.3.8.3 Tax Array Object - MC Corpais

The tax array object is used when you use the Tax field of both MC Corpac and MC Corpal. If you use the tax array object, all of the array fields must be sent.

Setting the tax array differs slightly between the two objects.

Setting tax array for MC Corpac

```
//Tax Details

$tax_amount_c = array("1.19", "1.29");

$tax_rate_c = array("6.0", "7.0");

$tax_type_c = array("GST", "PST");

$tax_id_c = array("gst1298", "pst1298");

$tax_included_in_sales_c = array("Y", "N");

//Create and set Tax for McCorpac

$mcTax_c = new mcTax();

$mcTax_c->setTax($tax_amount_c[0], $tax_rate_c[0], $tax_type_c[0], $tax_id_c
[0], $tax_included_in_sales_c[0]);

$mcTax_c->setTax($tax_amount_c[1], $tax_rate_c[1], $tax_type_c[1], $tax_id_c
[1], $tax_included_in_sales_c[1]);
```

Setting tax array for MC Corpal

```
//Tax Details for Items

$tax_amount_l = array("0.52", "1.48");

$tax_rate_l = array("13.0", "13.0");

$tax_type_l = array("HST", "HST");

$tax_id_l = array("hst1298", "hst1298");

$tax_included_in_sales_l = array("Y", "Y");

//Create and set Tax for McCorpall

$mcTax_l = array(new mcTax(), new mcTax());

$mcTax_l[0]->setTax($tax_amount_l[0], $tax_rate_l[0], $tax_type_l[0], $tax_id_
l[0], $tax_included_in_sales_l[0]);

$mcTax_l[1]->setTax($tax_amount_l[1], $tax_rate_l[1], $tax_type_l[1], $tax_id_
l[1], $tax_included_in_sales_l[1]);
```

Table 1 MC Corpaix Tax Array Request Fields

Req*	Value	Limits	Variable	Description
Y	Tax Amount	12-character decimal	tax_amount_c/tax_amount_l	Contains detail tax amount for purchase of goods or services Must be 2 decimal places. Minimum amount is 0.00 and maximum is 999999.99
Y	Tax Rate	5-character decimal	tax_rate_c/tax_rate_l	Contains the detailed tax rate applied in relationship to a specific tax amount EXAMPLE: 5% GST should be '5.0' or 9.975% QST should be '9.975' May contain up to 3 decimals, minimum 0.001, maximum up to 9999.9
Y	Tax Type	4-character alpha-numeric	tax_type_c/tax_type_l	Contains tax type, such as GST,QST,PST,HST
Y	Tax ID	20-character alpha-numeric	tax_id_c/tax_id_l	Provides an identification number used by the card acceptor with the tax authority in relationship to a specific tax amount, such as GST/HST number
Y	Tax included in sales indicator	1-character alpha-numeric	tax_included_in_sales_c/tax_included_in_sales_l	This is the indicator used to reflect addi-

Req*	Value	Limits	Variable	Description
				tional tax capture and reporting Valid values are: Y = Tax included in total purchase amount N = Tax not included in total purchase amount

5.3.8.4 Sample Code for MC Corpsais

Sample MC Corpsais - Corporate Card Common Data with Line Item Details

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='mccorpais';
$cust_id='CUST13343';
$order_id='ord-200916-13:29:27';
$txn_number='66011731632016264132927986-0_11';
$customer_code1_c ="CustomerCode123";
$card_acceptor_tax_id_c ="UrTaxId";//Merchant tax id which is mandatory
$corporation_vat_number_c ="cvn123";
$freight_amount_c ="1.23";
$duty_amount_c ="2.34";
$ship_to_pos_code_c ="M1R 1W5";
$order_date_c ="141211";
$customer_vat_number_c ="customervn231";
$unique_invoice_number_c ="uin567";
$authorized_contact_name_c ="John Walker";
//Tax Details
$tax_amount_c = array("1.19", "1.29");
$tax_rate_c = array("6.0", "7.0");
$tax_type_c = array("GST", "PST");
$tax_id_c = array("gst1298", "pst1298");
$tax_included_in_sales_c = array("Y", "N");
//Item Details
$customer_code1_l = array("customer code", "customer code2");
$line_item_date_l = array("150114", "150114");
$ship_date_l = array("150120", "150122");
$order_date1_l = array("150114", "150114");
$medical_services_ship_to_health_industry_number_l = array(null, null);
$contract_number_l = array(null, null);
$medical_services_adjustment_l = array(null, null);
$medical_services_product_number_qualifier_l = array(null, null);
$product_code1_l = array("pc11", "pc12");
$item_description_l = array("Good item", "Better item");
$item_quantity_l = array("4", "5");
$unit_cost_l = array("1.25", "10.00");

```

Sample MC Corps - Corporate Card Common Data with Line Item Details

```

$item_unit_measure_l = array("EA", "EA");
$ext_item_amount_l = array("5.00", "50.00");
$discount_amount_l = array("1.00", "50.00");
$commodity_code_l = array("cCode11", "cCode12");
$type_of_supply_l = array(null, null);
$vat_ref_num_l = array(null, null);
//Tax Details for Items
$tax_amount_l = array("0.52", "1.48");
$tax_rate_l = array("13.0", "13.0");
$tax_type_l = array("HST", "HST");
$tax_id_l = array("hst1298", "hst1298");
$tax_included_in_sales_l = array("Y", "Y");
//Create and set Tax for McCorpac
$mcTax_c = new mcTax();
$mcTax_c->setTax($tax_amount_c[0], $tax_rate_c[0], $tax_type_c[0], $tax_id_c[0], $tax_
included_in_sales_c[0]);
$mcTax_c->setTax($tax_amount_c[1], $tax_rate_c[1], $tax_type_c[1], $tax_id_c[1], $tax_
included_in_sales_c[1]);
//Create and set McCorpac for common data - only set values that you know
$mcCorpac = new mcCorpac();
$mcCorpac->setCustomerCode($customer_code_c);
$mcCorpac->setCardAcceptorTaxTd($card_acceptor_tax_id_c);
$mcCorpac->setCorporationVatNumber($corporation_vat_number_c);
$mcCorpac->setFreightAmount1($freight_amount_c);
$mcCorpac->setDutyAmount1($duty_amount_c);
$mcCorpac->setShipToPosCode($ship_to_pos_code_c);
$mcCorpac->setOrderDate($order_date_c);
$mcCorpac->setCustomerVatNumber($customer_vat_number_c);
$mcCorpac->setUniqueInvoiceNumber($unique_invoice_number_c);
$mcCorpac->setAuthorizedContactName($authorized_contact_name_c);
$mcCorpac->setTax($mcTax_c);
//Create and set Tax for McCorpall
$mcTax_l = array(new mcTax(), new mcTax());
$mcTax_l[0]->setTax($tax_amount_l[0], $tax_rate_l[0], $tax_type_l[0], $tax_id_l[0], $tax_
included_in_sales_l[0]);
$mcTax_l[1]->setTax($tax_amount_l[1], $tax_rate_l[1], $tax_type_l[1], $tax_id_l[1], $tax_
included_in_sales_l[1]);
//Create and set McCorpall for each item
$mcCorpall = new mcCorpall();
$mcCorpall->setMcCorpall($customer_code_l[0], $line_item_date_l[0], $ship_date_l[0], $order_
date_l[0], $medical_services_ship_to_health_industry_number_l[0], $contract_number_l[0],
$medical_services_adjustment_l[0], $medical_services_product_number_qualifier_l[0],
$product_code_l[0], $item_description_l[0], $item_quantity_l[0],
$unit_cost_l[0], $item_unit_measure_l[0], $ext_item_amount_l[0], $discount_amount_l[0],
$commodity_code_l[0], $type_of_supply_l[0], $vat_ref_num_l[0], $mcTax_l[0]);
$mcCorpall->setMcCorpall($customer_code_l[1], $line_item_date_l[1], $ship_date_l[1], $order_
date_l[1], $medical_services_ship_to_health_industry_number_l[1], $contract_number_l[1],
$medical_services_adjustment_l[1], $medical_services_product_number_qualifier_l[1],
$product_code_l[1], $item_description_l[1], $item_quantity_l[1],
$unit_cost_l[1], $item_unit_measure_l[1], $ext_item_amount_l[1], $discount_amount_l[1],
$commodity_code_l[1], $type_of_supply_l[1], $vat_ref_num_l[1], $mcTax_l[1]);
//Create and set McLevel23
$mpgMcLevel23 = new mpgMcLevel23();
$mpgMcLevel23->setMcCorpac($mcCorpac);
$mpgMcLevel23->setMcCorpall($mcCorpall);
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'txn_number'=>$txn_number,

```


Sample MC Corpais - Corporate Card Common Data with Line Item Details

```

);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setLevel23Data($mpgMcLevel23);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
// Status check example
// $mpgHttpPost = new mpgHttpPostStatus($store_id, $api_token, $status, $mpgRequest);
/***** Response *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
// print("\nStatusCode = " . $mpgResponse->getStatusCode());
// print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.4 Level 2/3 American Express Transactions

- 1 American Express Level 2/3 Standard Transactions
- 1 American Express L23 Air and Rail Transactions

5.4.1 Level 2/3 Transaction Types for Amex

This transaction set includes a suite of corporate card financial transactions as well as a transaction that allows for the passing of Level 2/3 data. Please ensure American Express Level 2/3 processing support is enabled on your merchant account. Batch Close, Open Totals and Pre-authorization are identical to the transactions outlined in the section Basic Transaction Set (page 21).

- When the Pre-authorization response contains CorporateCard equal to true then you can submit the AX transactions.

- If CorporateCard is false then the card does not support Level 2/3 data and non Level 2/3 transaction are to be used. If the card is not a corporate card, please refer to 2 Basic Transaction Set for the appropriate non-corporate card transactions.

NOTE: This transaction set is intended for transactions where Corporate Card is true and Level 2/3 data will be submitted. If the credit card is found to be a corporate card but you do not wish to send any Level 2/3 data then you may submit AX transactions using the transaction set outlined in the section Basic Transaction Set (page 21).

Pre-authorization – (authorization)

The preauth verifies and locks funds on the customer's credit card. The funds are locked for a specified amount of time, based on the card issuer. To retrieve the funds from a pre-auth so that they may be settled in the merchant account a capture must be performed. CorporateCard will return as true if the card supports Level 2/3.

AX Completion – (Capture/Pre-authorization Completion)

Once a Pre-authorization is obtained the funds that are locked need to be retrieved from the customer's credit card. The capture retrieves the locked funds and readies them for settlement in to the merchant account. Prior to performing an AXCompletion a Preauth must be performed.

AX Force Post – (Force Capture/Pre-authorization Completion)

This transaction is an alternative to AX Completion to obtain the funds locked on a Pre-authorization obtained from IVR or equivalent terminal. The capture retrieves the locked funds and readies them for settlement in to the merchant account.

AX Purchase Correction – (Void, Correction)

AX Completion and AX Force Post can be voided the same day* that they occur. A void must be for the full amount of the transaction and will remove any record of it from the cardholder statement. * An AX Purchase Correction can be performed against a transaction as long as the batch that contains the original transaction remains open. When using the automated closing feature, the batch close occurs daily between 10 – 11 pm EST.

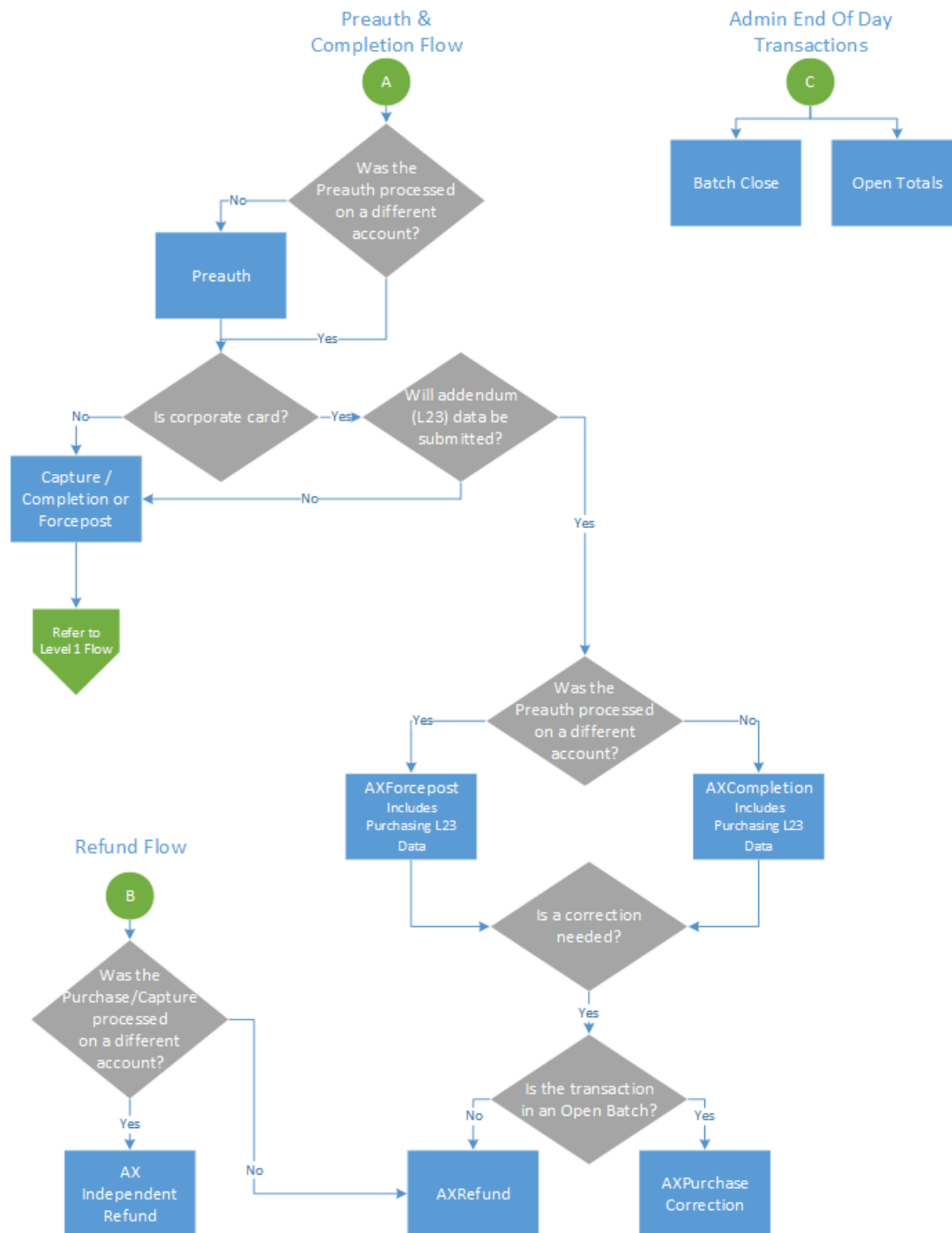
AX Refund – (Credit)

An AX Refund can be performed against an AX Completion and AX Force Post to refund any part, or all of the transaction.

AX Independent Refund – (Credit)

An AX Independent Refund can be performed against a purchase or a capture to refund any part, or all of the transaction. Independent refund is used when the originating transaction was not performed through Moneris Gateway. Please note, the Independent Refund transaction may or may not be supported on your account. If you receive a transaction not allowed error when attempting an independent refund, it may mean the transaction is not supported on your account. If you wish to have the AX Independent Refund transaction type temporarily enabled (or re-enabled), please contact the Service Centre at 1-866-319-7450.

5.4.2 Level 2/3 Transaction Flow for Amex



5.4.3 Level 2/3 Data Objects in Amex

- 5.4.3.1 About the Level 2/3 Data Objects for Amex
- 5.4.3.2 Definition of the AxLevel23 Object
- Table 1 Object
- Table 2 Object
- Table 3 Object

5.4.3.1 About the Level 2/3 Data Objects for Amex

Many of the Level 2/3 transaction requests using American Express also include a mandatory data object called AxLevel23. AxLevel23 is also comprised of other objects, also described in this section.

The Level 2/3 data objects within this section apply to all of the following transactions and are passed as part of the transaction request for:

- AX Completion
- AX Force Post
- AX Refund
- AX Independent Refund

Things to Consider:

- Please ensure the addendum data below is complete and accurate.
- Please ensure the math on quantities calculations, amounts, discounts, taxes, etc. properly adds up to the overall transaction amount. Incorrect amounts will cause the transaction to be rejected.

5.4.3.2 Definition of the AxLevel23 Object

AxLevel23 object definition

```
$mpgAxLevel23 = new mpgAxLevel23();
```

The AxLevel23 object itself has three objects, Table1, Table2 and Table3, all of which are mandatory.

Table 1 AxLevel23 Object

Variable Name	Type and Limits	Description	Set Method
Table1	Object	Refer below for further breakdown and definition of table1	\$mpgAxLevel23->setTable1(\$big04, \$big05, \$big10, \$axN1Loop);
Table2	Object	Refer below for further breakdown and definition of table2	\$mpgAxLevel23->setTable2(\$axItLoop);
Table3	Object	Refer below for further breakdown and definition of table3	\$mpgAxLevel23->setTable3(\$taxTbl3);

Table 1 Object

Table 1 contains the addendum data heading information. Contains information such as identification elements that uniquely identify an invoice (transaction), the customer name and shipping address.

Table 1 object definition

```
$mpgAxLevel23->setTable1($big04, $big05, $big10, $axN1Loop);
```

Table 1 AxLevel23 object - Table 1 object fields

Req*	Value	Limits	Set Method	Description
C	Purchase Order Number	22-character alphanumeric	'big04'=>\$big04	The cardholder supplied Purchase Order Number, which is entered by the merchant at the point-of-sale This entry is used in the Statement/Reporting process and may include accounting information specific to the client NOTE: This element is mandatory, if the merchant's customer provides a Purchase

Req*	Value	Limits	Set Method	Description
				Order Number.
N	Release Number	30-character alpha-numeric	'big05'=>\$big05	A number that identifies a release against a Purchase Order previously placed by the parties involved in the transaction
N	Invoice Number	8-character alpha-numeric	'big10'=>\$big10	Contains the Amex invoice/reference number
N	N1Loop	Object	'n1Loop'=>\$n1Loop	Refer below for further breakdown and definition of N1Loop object

*Y = Required, N = Optional, C = Conditional

Table 1 also has its own objects:

- N1Loop object
- AxRef object

Table 1 - Setting the N1Loop Object

The N1Loop data set contains the Requester names. It can also optionally contain the buying group, ship from, ship to and receiver details.

A minimum of at least 1 n1Loop must be set. Up to 5 n1Loop can be set.

N1Loop object definition

```
$axN1Loop = new axN1Loop();
```

```
$axN1Loop->setN1Loop($n101, $n102, $n301, $n401, $n402, $n403, $axRef1);
```

Table 1 AxLevel23 object - Table 1 object - N1Loop object fields

Req*	Value	Limits	Variable or Set Method	Description
Y	Entity Identifier Code	2-character alpha-numeric	n101	Supported values: R6 - Requester (required)

Req*	Value	Limits	Variable or Set Method	Description												
				BG - Buying Group (optional) SF - Ship From (optional) ST - Ship To (optional) 40 - Receiver (optional)												
Y	Name	40-character alpha-numeric	n102	<table><tr><th>n101 code</th><th>n102 meaning</th></tr><tr><td>R6</td><td>Requester Name</td></tr><tr><td>BG</td><td>Buying Group Name</td></tr><tr><td>SF</td><td>Ship From Name</td></tr><tr><td>ST</td><td>Ship To Name</td></tr><tr><td>40</td><td>Receiver Name</td></tr></table>	n101 code	n102 meaning	R6	Requester Name	BG	Buying Group Name	SF	Ship From Name	ST	Ship To Name	40	Receiver Name
n101 code	n102 meaning															
R6	Requester Name															
BG	Buying Group Name															
SF	Ship From Name															
ST	Ship To Name															
40	Receiver Name															
N	Address	40-character alpha-numeric	n301	Address												
N	City	30-character alpha-numeric	n401	City												
N	State or Province	2-character alpha-numeric	n402	State or province												
N	Postal Code	15-character alpha-numeric	n403	Postal Code												
N	AxRef	Object	<code>\$axRef1 = new axRef();</code>	Refer below for further breakdown and definition of AxRef object. This object contains the customer postal code (mandatory) and customer reference number (optional) A minimum of 1 axRef1 must be set; maximum of 2 axRef1's may be set												

*Y = Required, N = Optional, C = Conditional

Table 1 - Setting the AxRef Object

Setting AXRef object

```

$axRef1 = new axRef();

$ref01 = array("4C", "CR"); //Reference ID Qualifier

$ref02 = array("M5T3A5", "16802309004"); //Reference ID

$axRef1->setRef($ref01[0], $ref02[0]);

$axRef1->setRef($ref01[1], $ref02[1]);

```

Table 1 AxLevel23 object - Table 1 object - AxRef object fields

Req*	Value	Limits	Variable	Description												
Y	Reference Identification Qualifier	2-character alphanumeric	ref01	This element may contain the following qualifiers for the corresponding occurrences of the N1Loop: <table><tr><th>n101 value</th><th>ref01 denotation</th></tr><tr><td>R6</td><td>Supported values: 4C - Shipment Destination Code (mandatory) CR - Customer Reference Number (conditional)</td></tr><tr><td>BG</td><td>n/a</td></tr><tr><td>SF</td><td>n/a</td></tr><tr><td>ST</td><td>n/a</td></tr><tr><td>40</td><td>n/a</td></tr></table>	n101 value	ref01 denotation	R6	Supported values: 4C - Shipment Destination Code (mandatory) CR - Customer Reference Number (conditional)	BG	n/a	SF	n/a	ST	n/a	40	n/a
n101 value	ref01 denotation															
R6	Supported values: 4C - Shipment Destination Code (mandatory) CR - Customer Reference Number (conditional)															
BG	n/a															
SF	n/a															
ST	n/a															
40	n/a															
Y	Reference Identification	15-character alphanumeric	ref02	This field must be populated for each ref01 provided												

Req*	Value	Limits	Variable	Description	
				ref01 value	ref02 denotation
				4C (n101 value = R6)	This element must contain the Amex Ship-to Postal Code of the destination where the commodity was shipped. If the Ship-to Postal Code is unavailable, the postal code of the merchant location where the transaction took place may be substituted.
				CR (n101 value = R6):	This element must contain the Amex Card member Reference Number (e.g., purchase order, cost center, project number, etc.) that corresponds to this transaction, if provided by the Cardholder. This information may be displayed in the statement/reporting process and may include client-specific accounting information.

*Y = Required, N = Optional, C = Conditional

Table 2 Object

Table 2 includes the transaction's addendum detail. It contains transaction data including reference codes, debit or credit and tax amounts, line item detail descriptions, shipping information and much more. All transaction data in an invoice relate to a single transaction and cardholder account number.

Table 2 object definition

```
$mpgAxLevel23->setTable2 ($axItLoop) ;
```

Table 1 AxLevel23 object - Table 2 object fields

Req*	Value	Limits	Set Method	Description
N	It1loop	Object	'axIt1Loop'=>\$axIt1Loop	Refer below for further break-down and definition of object details.

*Y = Required, N = Optional, C = Conditional

Table 2 - Setting the AxIt1Loop Object

The AxIt1Loop data defines the baseline item data for the invoice. This data is defined for each item/service purchased and included within this invoice. This data set contains basic transaction data, including quantity, unit of measure, unit price and goods/services reference information.

- A minimum of 1 it1Loop required
- A maximum of 999 it1Loop's supported

AxIt1Loop object definition

```
$axItLoop = new axIt1Loop();
```

```
$axItLoop->setIt1Loop($it102[0], $it103[0], $it104[0], $it105[0], $it106s[0],  
$txi[0], $pam05[0], $pid05[0]);
```

```
$axItLoop->setIt1Loop($it102[1], $it103[1], $it104[1], $it105[1], $it106s[1],  
$txi[1], $pam05[1], $pid05[1]);
```

Table 1 AxLevel23 object - Table 2 object - AxIt1Loop object fields

Req*	Value	Limits	Variable	Description
Y	Line Item Quantity Invoiced	10-character decimal	it102	Quantity of line item Up to 2 decimal places supported Minimum amount is 0.0 and maximum is 9999999999

Req*	Value	Limits	Variable	Description
Y	Unit or Basis for Measurement Code	2-character alpha-numeric	it103	The line item unit of measurement code Must contain a code that specifies the units in which the value is expressed or the manner in which a measurement is taken EXAMPLE: EA = each, E5=inches See ANSI X-12 EDI Allowable Units of Measure and Codes for the list of codes
Y	Unit Price	15-character decimal	it104	Line item cost per unit Must contain 2 decimal places Minimum amount is 0.00 and maximum is 999999.99
N	Basis or Unit Price Code	2-character alpha-numeric	it105	Code identifying the type of unit price for an item EXAMPLE: DR = dealer, AP = advise price See ASC X12 004010 Element

Req*	Value	Limits	Variable	Description
				639 for list of codes
N	Axlt106s	object	it106s	Refer below for further break-down and definition of object details.
N	AxTxi	object	txi	Refer below for further break-down and definition of object details A maximum of 12 AxTxi (tax information data sets) may be defined NOTE: that if line item level tax information is populated in AxTxi in Table2, then tax totals for the entire invoice (transaction) must be entered in Table3.
Y	Line Item Extended Amount	8-character decimal	pam05	Contains the individual item amount that is normally calculated as price multiplied by quantity Must contain 2 decimal places Minimum amount is 0.00 and max-

Req*	Value	Limits	Variable	Description
				imum is 99999.99
Y	Line Item Description	80-character alphanumeric	pid05	Line Item description Contains the description of the individual item purchased This field pertain to each line item in the transaction

*Y = Required, N = Optional, C = Conditional

Table 2 - Setting the Axit106s Object

```

$it10618 = array("MG", "MG", "MG", "MG", "MG"); //Product/Service ID qualifier
$it10719 = array("DJFR4", "JFJ49", "FEF33", "FEE43", "DISCOUNT");
//Product/Service ID (corresponds to it10618)

$it106s = array();

$it106s[0] = new axIt106s($it10618[0], $it10719[0]);
$it106s[1] = new axIt106s($it10618[1], $it10719[1]);
$it106s[2] = new axIt106s($it10618[2], $it10719[2]);
$it106s[3] = new axIt106s($it10618[3], $it10719[3]);
$it106s[4] = new axIt106s($it10618[4], $it10719[4]);

```

Table 1 AxLevel23 object - Table 2 object - Axit106s object fields

Req*	Value	Limits	Set Method	Description
N	Product/Service ID Qualifier	2-character alphanumeric	'it10618'=>\$it10618	Supported values: MG - Manufacturer's Part Number VC - Supplier Catalog Number SK - Supplier Stock Keeping Unit Num-

Req*	Value	Limits	Set Method	Description								
				ber UP - Universal Product Code VP – Vendor Part Number PO – Purchase Order Number AN – Client Defined Asset Code								
N	Product/Service ID	<table><thead><tr><th>it10618</th><th>it10719 - size/type</th></tr></thead><tbody><tr><td>VC</td><td>20-character alphanumeric</td></tr><tr><td>PO</td><td>22-character alphanumeric</td></tr><tr><td>Other</td><td>30-character alphanumeric</td></tr></tbody></table>	it10618	it10719 - size/type	VC	20-character alphanumeric	PO	22-character alphanumeric	Other	30-character alphanumeric	'it10719'=>\$it10719	Product/Service ID corresponds to the preceding qualifier defined by it10618 The maximum length depends on the qualifier defined in it10618
it10618	it10719 - size/type											
VC	20-character alphanumeric											
PO	22-character alphanumeric											
Other	30-character alphanumeric											

*Y = Required, N = Optional, C = Conditional

Table 2 - Setting the AxTxi Object

Table 2 AxTxi object definition

```

$txi01_GST = array("GS", "GS", "GS", "GS", "GS"); //Tax type code

$txi02_GST = array("0.70", "1.75", "1.00", "0.80","0.00"); //Monetary amount

$txi03_GST = array("5.0", "5.0", "5.0", "5.0","5.0"); //Percent

$txi06_GST = array("", "", "", "", ""); //Tax exempt code

$txi01_PST = array("PG", "PG", "PG","PG","PG"); //Tax type code

$txi02_PST = array("0.80", "2.00", "1.00", "0.80","0.00"); //Monetary amount

$txi03_PST = array("7.0", "7.0", "7.0", "7.0","7.0"); //Percent

$txi06_PST = array("", "", "", "", ""); //Tax exempt code

$txi = array(new axTxi(), new axTxi(), new axTxi(), new axTxi(), new axTxi());

$txi[0]->setTxi($txi01_GST[0], $txi02_GST[0], $txi03_GST[0], $txi06_GST[0]);

```

```

$txi[0]->setTxi($txi01_PST[0], $txi02_PST[0], $txi03_PST[0], $txi06_PST[0]);
$txi[1]->setTxi($txi01_GST[1], $txi02_GST[1], $txi03_GST[1], $txi06_GST[1]);
$txi[1]->setTxi($txi01_PST[1], $txi02_PST[1], $txi03_PST[1], $txi06_PST[1]);
$txi[2]->setTxi($txi01_GST[2], $txi02_GST[2], $txi03_GST[2], $txi06_GST[2]);
$txi[2]->setTxi($txi01_PST[2], $txi02_PST[2], $txi03_PST[2], $txi06_PST[2]);
$txi[3]->setTxi($txi01_GST[3], $txi02_GST[3], $txi03_GST[3], $txi06_GST[3]);
$txi[3]->setTxi($txi01_PST[3], $txi02_PST[3], $txi03_PST[3], $txi06_PST[3]);
$txi[4]->setTxi($txi01_GST[4], $txi02_GST[4], $txi03_GST[4], $txi06_GST[4]);
$txi[4]->setTxi($txi01_PST[4], $txi02_PST[4], $txi03_PST[4], $txi06_PST[4]);

```

Table 1 AxLevel23 object - Table 2 object - AxiTxi object fields

Req*	Value	Limits	Variable	Description
C	Tax Type code	txi01	2-character alpha-numeric	Tax type code applicable to Canada and US only For Canada, this field must contain a code that specifies the type of tax If txi01 is used, then txi02, txi03 or txi06 must be populated Valid codes include the following: CT – County/Tax (optional) CA – City Tax (optional) EV – Environmental Tax (optional) GS – Good and Services Tax (GST)

Req*	Value	Limits	Variable	Description
				(optional) LS – State and Local Sales Tax (optional) LT – Local Sales Tax (optional) PG – Provincial Sales Tax (PST) (optional) SP – State/Provincial Tax a.k.a. Quebec Sales Tax (QST) (optional) ST – State Sales Tax (optional) TX – All Taxes (required) VA – Value-Added Tax a.k.a. Canadian Harmonized Sales Tax (HST) (optional)
C	Monetary Amount	txi02	6-character decimal	This element may contain the monetary tax amount that corresponds to the Tax Type Code in txi01 NOTE: If txi02 is used in mandatory occurrence txi01=TX, txi02 must contain the total tax amount applicable to the entire invoice (transaction) If taxes are not applicable for the entire invoice (transaction), txi02 must be 0.00. The maximum value that can be entered in this

Req*	Value	Limits	Variable	Description
				field is "9999.99", which is \$9,999.99 (CAD) A debit is entered as: 9999.99 A credit is entered as: – 9999.99
C	Percent	txi03	10-character decimal	Contains the tax percentage (in decimal format) that corresponds to the tax type code defined in txi01 Up to 2 decimal places supported
C	Tax Exempt Code	txi06	1-character alpha-numeric	This element may contain the Tax Exempt Code that identifies the exemption status from sales and tax that corresponds to the Tax Type Code in txi01 Supported values: 1 – Yes (Tax Exempt) 2 – No (Not Tax Exempt) 4 – Not Exempt/For Resale A – Labor Taxable, Material Exempt

Req*	Value	Limits	Variable	Description
				B – Material Taxable, Labor Exempt C – Not Taxable F – Exempt (Goods / Services Tax) G – Exempt (Provincial Sales Tax) L – Exempt Local Service R – Recurring Exempt U – Usage Exempt

*Y = Required, N = Optional, C = Conditional

Table 3 Object

Table 3 includes the transaction addendum summary. It contains the total invoice (transaction) amount, sales tax, freight and/or handling charges and invoice summary information, including total line items, number of segments in the invoice, and the transaction set control number (a.k.a., batch number).

Table 3 object definition

```
$mpgAxLevel23->setTable3 ($taxTbl3);
```

Table 1 AxLevel23 object - Table 3 object fields

Req*	Value	Limits	Set Method	Description
C	AxTxI	Object	'taxTbl3'=>\$taxTbl3	Refer below for further breakdown and definition of object details. NOTE: if line item level tax information is populated in AxTxI in Table2, then tax totals

Req*	Value	Limits	Set Method	Description
				for the entire invoice (transaction) must be entered in Table3. A maximum of 10 AxTxi's may be set in Table3.

*Y = Required, N = Optional, C = Conditional

Table 3 - Setting the AxTxi Object

The mandatory tax information data set must contain the total tax amount applicable to the entire invoice (transaction) which includes all line items identified in Table2. If taxes are not applicable for the entire invoice (transaction), then txi02 must be set to 0.00.

Tax totals must be entered in this mandatory tax information segment in Table 3, even if line item detail level tax data is reported in Table 2.

At least one occurrence of txi02, txi03 or txi06 is required.

Table 3 AxTxi object definition

```
$taxTbl3 = new axTxi();

$taxTbl3->setTxi("GS", "4.25","5.0",""); //sum of GST taxes
$taxTbl3->setTxi("PG", "4.60","7.0",""); //sum of PST taxes
$taxTbl3->setTxi("TX", "8.85","13.0",""); //sum of all taxes

$mpgAxLevel23->setTable3($taxTbl3);
```

Table 1 AxLevel23 object - Table 3 object - AxTxi object fields

Req*	Value	Limits	Variable	Description
C	Tax Type code	txi01	2-character alpha-numeric	Tax type code applicable to Canada and US only For Canada, this field must contain a code that specifies the type of tax

Req*	Value	Limits	Variable	Description
				If txi01 is used, then txi02, txi03 or txi06 must be populated Valid codes include the following: CT – County/Tax (optional) CA – City Tax (optional) EV – Environmental Tax (optional) GS – Good and Services Tax (GST) (optional) LS – State and Local Sales Tax (optional) LT – Local Sales Tax (optional) PG – Provincial Sales Tax (PST) (optional) SP – State/Provincial Tax a.k.a. Quebec Sales Tax (QST) (optional) ST – State Sales Tax (optional) TX – All Taxes (required) VA – Value-Added Tax a.k.a. Canadian Harmonized Sales Tax (HST) (optional)
C	Monetary Amount	txi02	6-character decimal	This element may contain the monetary tax amount that corresponds

Req*	Value	Limits	Variable	Description
				to the Tax Type Code in txi01 NOTE: If txi02 is used in mandatory occurrence txi01=TX, txi02 must contain the total tax amount applicable to the entire invoice (transaction) If taxes are not applicable for the entire invoice (transaction), txi02 must be 0.00. The maximum value that can be entered in this field is "9999.99", which is \$9,999.99 (CAD) A debit is entered as: 9999.99 A credit is entered as: – 9999.99
C	Percent	txi03	10-character decimal	Contains the tax percentage (in decimal format) that corresponds to the tax type code defined in txi01 Up to 2 decimal places supported
C	Tax Exempt Code	txi06	1-character alpha-numeric	This element may contain the Tax Exempt Code that identifies the

Req*	Value	Limits	Variable	Description
				exemption status from sales and tax that corresponds to the Tax Type Code in txi01 Supported values: 1 – Yes (Tax Exempt) 2 – No (Not Tax Exempt) 4 – Not Exempt/For Resale A – Labor Taxable, Material Exempt B – Material Taxable, Labor Exempt C – Not Taxable F – Exempt (Goods / Services Tax) G – Exempt (Provincial Sales Tax) L – Exempt Local Service R – Recurring Exempt U – Usage Exempt

*Y = Required, N = Optional, C = Conditional

5.4.4 AX Completion

The AX Completion transaction is used to secure the funds locked by a pre-authorization transaction. When sending a capture request you will need two pieces of information from the original pre-authorization – the Order ID and the transaction number from the returned response.

AX Completion transaction object definition

```
$txnArray = array('type'=>'axcompletion', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for AX Completion

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

AX Completion transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	String 50-character alphanumeric	'order_id'=>\$order_id
Completion amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'comp_amount'=>\$comp_amount
Transaction number	String 255-character alphanumeric	'txn_number'=>\$txn_number
E-commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt
Level 2/3 Data	Object n/a	\$mpgTxn->setLevel23Data(\$mpgAxLevel23);

Sample AX Completion

```

<?php
require "../..mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='axcompletion';
$order_id='ord-210916-12:06:38';
$comp_amount='62.37';
$txn_number = '18924-0_11';
$script = '7';
//Create AxLevel23 Object
$mpgAxLevel23 = new mpgAxLevel23();
//Create Table 1 with details
$n101 = "R6"; //Entity ID Code
$n102 = "Retailing Inc. International"; //Name
$n301 = "919 Oriole Rd."; //Address Line 1
$n401 = "Toronto"; //City
$n402 = "On"; //State or Province
$n403 = "H1T6W3"; //Postal Code
$ref01 = array("4C", "CR"); //Reference ID Qualifier
$ref02 = array("M5T3A5", "16802309004"); //Reference ID
$big04 = "P07758545"; //Purchase Order Number
$big05 = "RN0049858"; //Release Number
$big10 = "INV99870E"; //Invoice Number
$axRef1 = new axRef();
$axRef1->setRef($ref01[0], $ref02[0]);
$axRef1->setRef($ref01[1], $ref02[1]);
$axN1Loop = new axN1Loop();
$axN1Loop->setN1Loop($n101, $n102, $n301, $n401, $n402, $n403, $axRef1);
$mpgAxLevel23->setTable1($big04, $big05, $big10, $axN1Loop);
//Create Table 2 with details
//the sum of the extended amount field (pam05) must equal the level 1 amount field

$it102 = array("1", "1", "1", "1", "1"); //Line item quantity invoiced
$it103 = array("EA", "EA", "EA", "EA", "EA"); //Line item unit or basis of measurement code
$it104 = array("10.00", "25.00", "8.62", "10.00", "-10.00"); //Line item unit price
$it105 = array("", "", "", "", ""); //Line item basis of unit price code

$it10618 = array("MG", "MG", "MG", "MG", "MG"); //Product/Service ID qualifier
$it10719 = array("DJFR4", "JFJ49", "FEF33", "FEE43", "DISCOUNT"); //Product/Service ID
(corresponds to it10618)

$txi01_GST = array("GS", "GS", "GS", "GS", "GS"); //Tax type code
$txi02_GST = array("0.70", "1.75", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_GST = array("", "", "", "", ""); //Percent
$txi06_GST = array("", "", "", "", ""); //Tax exempt code

$txi01_PST = array("PG", "PG", "PG", "PG", "PG"); //Tax type code
$txi02_PST = array("0.80", "2.00", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_PST = array("", "", "", "", ""); //Percent
$txi06_PST = array("", "", "", "", ""); //Tax exempt code
$pam05 = array("11.50", "28.75", "10.62", "11.50", "-10.00"); //Extended line-item amount
$pid05 = array("Stapler", "Lamp", "Bottled Water", "Fountain Pen", "DISCOUNT"); //Line item
description
$it106s = array(new axIt106s(), new axIt106s(), new axIt106s(), new axIt106s(), new
axIt106s());
$it106s[0]->setIt10618($it10618[0]);

```

Sample AX Completion

```

$it106s[0]->setIt10719($it10719[0]);
$it106s[1]->setIt10618($it10618[1]);
$it106s[1]->setIt10719($it10719[1]);
$it106s[2]->setIt10618($it10618[2]);
$it106s[2]->setIt10719($it10719[2]);
$it106s[3]->setIt10618($it10618[3]);
$it106s[3]->setIt10719($it10719[3]);
$it106s[4]->setIt10618($it10618[4]);
$it106s[4]->setIt10719($it10719[4]);
$txi = array(new axTxi(), new axTxi(), new axTxi(), new axTxi(), new axTxi());
$txi[0]->setTxi($txi01_GST[0], $txi02_GST[0], $txi03_GST[0], $txi06_GST[0]);
$txi[0]->setTxi($txi01_PST[0], $txi02_PST[0], $txi03_PST[0], $txi06_PST[0]);
$txi[1]->setTxi($txi01_GST[1], $txi02_GST[1], $txi03_GST[1], $txi06_GST[1]);
$txi[1]->setTxi($txi01_PST[1], $txi02_PST[1], $txi03_PST[1], $txi06_PST[1]);
$txi[2]->setTxi($txi01_GST[2], $txi02_GST[2], $txi03_GST[2], $txi06_GST[2]);
$txi[2]->setTxi($txi01_PST[2], $txi02_PST[2], $txi03_PST[2], $txi06_PST[2]);
$txi[3]->setTxi($txi01_GST[3], $txi02_GST[3], $txi03_GST[3], $txi06_GST[3]);
$txi[3]->setTxi($txi01_PST[3], $txi02_PST[3], $txi03_PST[3], $txi06_PST[3]);
$txi[4]->setTxi($txi01_GST[4], $txi02_GST[4], $txi03_GST[4], $txi06_GST[4]);
$txi[4]->setTxi($txi01_PST[4], $txi02_PST[4], $txi03_PST[4], $txi06_PST[4]);
$axItLoop = new axItLoop();
$axItLoop->setItLoop($it102[0], $it103[0], $it104[0], $it105[0], $it106s[0], $txi[0],
$spam05[0], $pid05[0]);
$axItLoop->setItLoop($it102[1], $it103[1], $it104[1], $it105[1], $it106s[1], $txi[1],
$spam05[1], $pid05[1]);
$axItLoop->setItLoop($it102[2], $it103[2], $it104[2], $it105[2], $it106s[2], $txi[2],
$spam05[2], $pid05[2]);
$axItLoop->setItLoop($it102[3], $it103[3], $it104[3], $it105[3], $it106s[3], $txi[3],
$spam05[3], $pid05[3]);
$axItLoop->setItLoop($it102[4], $it103[4], $it104[4], $it105[4], $it106s[4], $txi[4],
$spam05[4], $pid05[4]);
$mpgAxLevel23->setTable2($axItLoop);
//Create Table 3 with details
$taxTbl3 = new axTxi();
$taxTbl3->setTxi("GS", "4.25","", ""); //sum of GST taxes
$taxTbl3->setTxi("PG", "4.60","", ""); //sum of PST taxes
$taxTbl3->setTxi("TX", "8.85","", ""); //sum of all taxes
$mpgAxLevel23->setTable3($taxTbl3);
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'comp_amount'=>$comp_amount,
'txn_number'=> $txn_number,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setLevel23Data($mpgAxLevel23);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());

```

Sample AX Completion

```
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

5.4.5 AX Force Post

The AX Force Post transaction is used to secure the funds locked by a pre-authorization transaction performed over IVR or equivalent terminal. When sending an AX Force Post request, you will need the order ID, amount, credit card number, expiry date, authorization code and e-commerce indicator.

AX Force Post transaction object definition

```
$txnArray = array('type'=>'axforcepost', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for AX Force Post transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

AX Force Post transaction request fields – Required

Value	Limits	Set Method
Order ID	String 50-character alphanumeric	'order_id'=>\$order_id
Amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal	'amount'=>\$amount

Value	Limits	Set Method
	point EXAMPLE: 1234567.89	
Credit card number	String 20-character alphanumeric	'pan'=>\$pan
Expiry date	String 4-character alphanumeric (YYMM format)	'expiry_date'=>\$expiry_date
Authorization code	String 8-character alphanumeric	'auth_code'=>\$auth_code
E-commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt
Level 2/3 Data	Object n/a	\$mpgTxn->setLevel23Data (\$mpgAxLevel23);

AX Force Post transaction request fields – Optional

Variable Name	Type and Limits	Set Method
Customer ID	String 50-character alphanumeric	'cust_id'=>\$cust_id

Sample AX Force Post

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$sapi_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='axforcepost';

```

Sample AX Force Post

```

$cust_id='CUST13343';
$order_id='ord-'.date("dmy-G:i:s");
$amount='62.37';
$pan='373269005095005';
$expiry_date='2012';
$auth_code='123456';
$script = '7';
//Create AxLevel23 Object
$mpgAxLevel23 = new mpgAxLevel23();
//Create Table 1 with details
$n101 = "R6"; //Entity ID Code
$n102 = "Retailing Inc. International"; //Name
$n301 = "919 Oriole Rd."; //Address Line 1
$n401 = "Toronto"; //City
$n402 = "On"; //State or Province
$n403 = "H1T6W3"; //Postal Code
$rref01 = array("4C", "CR"); //Reference ID Qualifier
$rref02 = array("M5T3A5", "16802309004"); //Reference ID
$b104 = "PO7758545"; //Purchase Order Number
$b105 = "RN0049858"; //Release Number
$b110 = "INV99870E"; //Invoice Number
$axRef1 = new axRef();
$axRef1->setRef($rref01[0], $rref02[0]);
$axRef1->setRef($rref01[1], $rref02[1]);
$axN1Loop = new axN1Loop();
$axN1Loop->setN1Loop($n101, $n102, $n301, $n401, $n402, $n403, $axRef1);
$mpgAxLevel23->setTable1($b104, $b105, $b110, $axN1Loop);
//Create Table 2 with details
//the sum of the extended amount field (pam05) must equal the level 1 amount field

$it102 = array("1", "1", "1", "1", "1"); //Line item quantity invoiced
$it103 = array("EA", "EA", "EA", "EA", "EA"); //Line item unit or basis of measurement code
$it104 = array("10.00", "25.00", "8.62", "10.00", "-10.00"); //Line item unit price
$it105 = array("", "", "", "", ""); //Line item basis of unit price code

$it10618 = array("MG", "MG", "MG", "MG", "MG"); //Product/Service ID qualifier
$it10719 = array("DJFR4", "JFJ49", "FEF33", "FEE43", "DISCOUNT"); //Product/Service ID
(corresponds to it10618)

$txi01_GST = array("GS", "GS", "GS", "GS", "GS"); //Tax type code
$txi02_GST = array("0.70", "1.75", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_GST = array("", "", "", "", ""); //Percent
$txi06_GST = array("", "", "", "", ""); //Tax exempt code

$txi01_PST = array("PG", "PG", "PG", "PG", "PG"); //Tax type code
$txi02_PST = array("0.80", "2.00", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_PST = array("", "", "", "", ""); //Percent
$txi06_PST = array("", "", "", "", ""); //Tax exempt code
$pam05 = array("11.50", "28.75", "10.62", "11.50", "-10.00"); //Extended line-item amount
$pid05 = array("Stapler", "Lamp", "Bottled Water", "Fountain Pen", "DISCOUNT"); //Line item
description
$it106s = array();
$it106s[0] = new axIt106s($it10618[0], $it10719[0]);
$it106s[1] = new axIt106s($it10618[1], $it10719[1]);
$it106s[2] = new axIt106s($it10618[2], $it10719[2]);
$it106s[3] = new axIt106s($it10618[3], $it10719[3]);
$it106s[4] = new axIt106s($it10618[4], $it10719[4]);
$txi = array(new axTxi(), new axTxi(), new axTxi(), new axTxi(), new axTxi());
$txi[0]->setTxi($txi01_GST[0], $txi02_GST[0], $txi03_GST[0], $txi06_GST[0]);

```

Sample AX Force Post

```

$txi[0]->setTxi($txi01_PST[0], $txi02_PST[0], $txi03_PST[0], $txi06_PST[0]);
$txi[1]->setTxi($txi01_GST[1], $txi02_GST[1], $txi03_GST[1], $txi06_GST[1]);
$txi[1]->setTxi($txi01_PST[1], $txi02_PST[1], $txi03_PST[1], $txi06_PST[1]);
$txi[2]->setTxi($txi01_GST[2], $txi02_GST[2], $txi03_GST[2], $txi06_GST[2]);
$txi[2]->setTxi($txi01_PST[2], $txi02_PST[2], $txi03_PST[2], $txi06_PST[2]);
$txi[3]->setTxi($txi01_GST[3], $txi02_GST[3], $txi03_GST[3], $txi06_GST[3]);
$txi[3]->setTxi($txi01_PST[3], $txi02_PST[3], $txi03_PST[3], $txi06_PST[3]);
$txi[4]->setTxi($txi01_GST[4], $txi02_GST[4], $txi03_GST[4], $txi06_GST[4]);
$txi[4]->setTxi($txi01_PST[4], $txi02_PST[4], $txi03_PST[4], $txi06_PST[4]);
$axItLoop = new axItLoop();
$axItLoop->setItLoop($it102[0], $it103[0], $it104[0], $it105[0], $it106s[0], $txi[0],
$spam05[0], $pid05[0]);
$axItLoop->setItLoop($it102[1], $it103[1], $it104[1], $it105[1], $it106s[1], $txi[1],
$spam05[1], $pid05[1]);
$axItLoop->setItLoop($it102[2], $it103[2], $it104[2], $it105[2], $it106s[2], $txi[2],
$spam05[2], $pid05[2]);
$axItLoop->setItLoop($it102[3], $it103[3], $it104[3], $it105[3], $it106s[3], $txi[3],
$spam05[3], $pid05[3]);
//$axItLoop->setItLoop($it102[4], $it103[4], $it104[4], $it105[4], $it106s[4], $txi[4],
$spam05[4], $pid05[4]);
$mpgAxLevel23->setTable2($axItLoop);
//Create Table 3 with details
$taxTbl3 = new axTxi();
$taxTbl3->setTxi("GS", "4.25","", ""); //sum of GST taxes
$taxTbl3->setTxi("PG", "4.60","", ""); //sum of PST taxes
$taxTbl3->setTxi("TX", "8.85","", ""); //sum of all taxes
$mpgAxLevel23->setTable3($taxTbl3);
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'auth_code'=>$auth_code,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setLevel23Data($mpgAxLevel23);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());

```

Sample AX Force Post

```

print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

5.4.6 AX Purchase Correction

The AX Purchase Correction (Void) transaction is used to cancel a transaction that was performed in the current batch. No amount is required because a void is always for 100% of the original transaction. The only transaction that can be voided using AX Purchase Correction is AX Completion and AX Force Post. To send an AX Purchase Correction the Order ID and transaction number from the AX Completion or AX Force Post are required.

AX Purchase Correction transaction object definition

```
$txnArray = array('type'=>'axpurchasecorrection', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for AX Purchase Correction transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

AX Purchase Correction transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	<i>String</i> 50-character alphanumeric	'order_id'=>\$order_id
Transaction number	<i>String</i> 255-character alphanumeric	'txn_number'=>\$txn_number
E-commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

AX Purchase Correction

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='axpurchasecorrection';
$order_id='ord-210916-12:12:18';
$txn_number = '66011731632016265121219276-0_11';
$crypt_type = '7';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'txn_number'=> $txn_number,
'crypt_type'=>$crypt_type
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

5.4.7 AX Refund

The AX Refund will credit a specified amount to the cardholder's credit card. A refund can be sent up to the full value of the original AX Completion or AX Force Post. To send an AX Refund you will require the order ID and transaction number from the original AX Completion or AX Force Post.

AX Refund transaction object definition

```
$txnArray = array('type'=>'axrefund', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for AX Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

AX Refund transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	String 50-character alphanumeric	'order_id'=>\$order_id
Transaction number	String 255-character alpha-numeric	'txn_number'=>\$txn_number
Amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
E-commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt
Level 2/3 Data	Object n/a	\$mpgTxn->setLevel23Data(\$mpgAxLevel23);

Sample AX Refund

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$sapi_token='hurgle';
// $status = 'false';
/***** Transactional Variables *****/
$type='axrefund';
$order_id='ord-210916-12:06:38';
$amount='62.37';
$txn_number = '18924-1_11';
$script = '7';
// Create AxLevel23 Object
$mpgAxLevel23 = new mpgAxLevel23();
// Create Table 1 with details
$n101 = "R6"; //Entity ID Code
$n102 = "Retailing Inc. International"; //Name
$n301 = "919 Oriole Rd."; //Address Line 1
$n401 = "Toronto"; //City
$n402 = "On"; //State or Province
$n403 = "H1T6W3"; //Postal Code
$ref01 = array("4C", "CR"); //Reference ID Qualifier
$ref02 = array("M5T3A5", "16802309004"); //Reference ID
$big04 = "P07758545"; //Purchase Order Number
$big05 = "RN0049858"; //Release Number
$big10 = "INV99870E"; //Invoice Number
$axRef1 = new axRef();
$axRef1->setRef($ref01[0], $ref02[0]);
$axRef1->setRef($ref01[1], $ref02[1]);
$axN1Loop = new axN1Loop();
$axN1Loop->setN1Loop($n101, $n102, $n301, $n401, $n402, $n403, $axRef1);
$mpgAxLevel23->setTable1($big04, $big05, $big10, $axN1Loop);
// Create Table 2 with details
// the sum of the extended amount field (pam05) must equal the level 1 amount field

$it102 = array("1", "1", "1", "1", "1"); //Line item quantity invoiced
$it103 = array("EA", "EA", "EA", "EA", "EA"); //Line item unit or basis of measurement code
$it104 = array("10.00", "25.00", "8.62", "10.00", "-10.00"); //Line item unit price
$it105 = array("", "", "", "", ""); //Line item basis of unit price code

$it10618 = array("MG", "MG", "MG", "MG", "MG"); //Product/Service ID qualifier
$it10719 = array("DJFR4", "JFJ49", "FEF33", "FEE43", "DISCOUNT"); //Product/Service ID
(corresponds to it10618)

$txi01_GST = array("GS", "GS", "GS", "GS", "GS"); //Tax type code
$txi02_GST = array("0.70", "1.75", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_GST = array("", "", "", "", ""); //Percent
$txi06_GST = array("", "", "", "", ""); //Tax exempt code

$txi01_PST = array("PG", "PG", "PG", "PG", "PG"); //Tax type code
$txi02_PST = array("0.80", "2.00", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_PST = array("", "", "", "", ""); //Percent
$txi06_PST = array("", "", "", "", ""); //Tax exempt code
$pam05 = array("11.50", "28.75", "10.62", "11.50", "-10.00"); //Extended line-item amount
$pid05 = array("Stapler", "Lamp", "Bottled Water", "Fountain Pen", "DISCOUNT"); //Line item
description
$it106s = array();
$it106s[0] = new axIt106s($it10618[0], $it10719[0]);
$it106s[1] = new axIt106s($it10618[1], $it10719[1]);

```

Sample AX Refund

```

$it106s[2] = new axIt106s($it10618[2], $it10719[2]);
$it106s[3] = new axIt106s($it10618[3], $it10719[3]);
$it106s[4] = new axIt106s($it10618[4], $it10719[4]);
$txi = array(new axTxi(), new axTxi(), new axTxi(), new axTxi(), new axTxi());
$txi[0]->setTxi($txi01_GST[0], $txi02_GST[0], $txi03_GST[0], $txi06_GST[0]);
$txi[0]->setTxi($txi01_PST[0], $txi02_PST[0], $txi03_PST[0], $txi06_PST[0]);
$txi[1]->setTxi($txi01_GST[1], $txi02_GST[1], $txi03_GST[1], $txi06_GST[1]);
$txi[1]->setTxi($txi01_PST[1], $txi02_PST[1], $txi03_PST[1], $txi06_PST[1]);
$txi[2]->setTxi($txi01_GST[2], $txi02_GST[2], $txi03_GST[2], $txi06_GST[2]);
$txi[2]->setTxi($txi01_PST[2], $txi02_PST[2], $txi03_PST[2], $txi06_PST[2]);
$txi[3]->setTxi($txi01_GST[3], $txi02_GST[3], $txi03_GST[3], $txi06_GST[3]);
$txi[3]->setTxi($txi01_PST[3], $txi02_PST[3], $txi03_PST[3], $txi06_PST[3]);
$txi[4]->setTxi($txi01_GST[4], $txi02_GST[4], $txi03_GST[4], $txi06_GST[4]);
$txi[4]->setTxi($txi01_PST[4], $txi02_PST[4], $txi03_PST[4], $txi06_PST[4]);
$axItLoop = new axItLoop();
$axItLoop->setItLoop($it102[0], $it103[0], $it104[0], $it105[0], $it106s[0], $txi[0],
$spam05[0], $pid05[0]);
$axItLoop->setItLoop($it102[1], $it103[1], $it104[1], $it105[1], $it106s[1], $txi[1],
$spam05[1], $pid05[1]);
$axItLoop->setItLoop($it102[2], $it103[2], $it104[2], $it105[2], $it106s[2], $txi[2],
$spam05[2], $pid05[2]);
$axItLoop->setItLoop($it102[3], $it103[3], $it104[3], $it105[3], $it106s[3], $txi[3],
$spam05[3], $pid05[3]);
//$axItLoop->setItLoop($it102[4], $it103[4], $it104[4], $it105[4], $it106s[4], $txi[4],
$spam05[4], $pid05[4]);
$mpgAxLevel23->setTable2($axItLoop);
//Create Table 3 with details
$taxTbl3 = new axTxi();
$taxTbl3->setTxi("GS", "4.25","", ""); //sum of GST taxes
$taxTbl3->setTxi("PG", "4.60","", ""); //sum of PST taxes
$taxTbl3->setTxi("TX", "8.85","", ""); //sum of all taxes
$mpgAxLevel23->setTable3($taxTbl3);
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'txn_number'=> $txn_number,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setLevel23Data($mpgAxLevel23);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());

```

Sample AX Refund

```
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

5.4.8 AX Independent Refund

The AX Independent Refund will credit a specified amount to the cardholder's credit card. The independent refund does not require an existing order to be logged in the Moneris Gateway; however, the credit card number and expiry date will need to be passed.

AX Independent Refund transaction object definition

```
$txnArray = array('type'=>'axind_refund', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for AX Independent Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

AX Independent Refund transaction request fields – Required

Variable Name	Type and Limits	Set Method
Order ID	String 50-character alphanumeric	'order_id'=>\$order_id
Amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point	'amount'=>\$amount
		EXAMPLE: 1234567.89

Variable Name	Type and Limits	Set Method
Credit card number	<i>String</i> 20-character alphanumeric	'pan'=>\$pan
Expiry date	<i>String</i> 4-character alphanumeric (YYMM format)	'expiry_date'=>\$expiry_date
E-commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

AX Independent Refund transaction request fields – Optional

Variable Name	Type and Limits	Set Method
Customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id

Sample AX Independent Refund

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
//$status = 'false';
/***** Transactional Variables *****/
$type='axind_refund';
$cust_id='CUST13343';
$order_id='ord-'.date("dmy-G:i:s");
$amount='62.37';
$pan='373269005095005';
$expiry_date='2012';
$crypt = '7';
//Create AxLevel23 Object
$mpgAxLevel23 = new mpgAxLevel23();
//Create Table 1 with details
$n101 = "R6"; //Entity ID Code
$n102 = "Retailing Inc. International"; //Name
$n301 = "919 Oriole Rd."; //Address Line 1
$n401 = "Toronto"; //City
$n402 = "On"; //State or Province
$n403 = "H1T6W3"; //Postal Code
$ref01 = array("4C", "CR"); //Reference ID Qualifier
$ref02 = array("M5T3A5", "16802309004"); //Reference ID
$big04 = "P07758545"; //Purchase Order Number

```

Sample AX Independent Refund

```

$big05 = "RN0049858"; //Release Number
$big10 = "INV99870E"; //Invoice Number
$axRef1 = new axRef();
$axRef1->setRef($ref01[0], $ref02[0]);
$axRef1->setRef($ref01[1], $ref02[1]);
$axN1Loop = new axN1Loop();
$axN1Loop->setN1Loop($n101, $n102, $n301, $n401, $n402, $n403, $axRef1);
$mpgAxLevel23->setTable1($big04, $big05, $big10, $axN1Loop);
//Create Table 2 with details
//the sum of the extended amount field (pam05) must equal the level 1 amount field

$it102 = array("1", "1", "1", "1", "1"); //Line item quantity invoiced
$it103 = array("EA", "EA", "EA", "EA", "EA"); //Line item unit or basis of measurement code
$it104 = array("10.00", "25.00", "8.62", "10.00", "-10.00"); //Line item unit price
$it105 = array("", "", "", "", ""); //Line item basis of unit price code

$it10618 = array("MG", "MG", "MG", "MG", "MG"); //Product/Service ID qualifier
$it10719 = array("DJFR4", "JFJ49", "FEF33", "FEE43", "DISCOUNT"); //Product/Service ID
(corresponds to it10618)

$txi01_GST = array("GS", "GS", "GS", "GS", "GS"); //Tax type code
$txi02_GST = array("0.70", "1.75", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_GST = array("", "", "", "", ""); //Percent
$txi06_GST = array("", "", "", "", ""); //Tax exempt code

$txi01_PST = array("PG", "PG", "PG", "PG", "PG"); //Tax type code
$txi02_PST = array("0.80", "2.00", "1.00", "0.80", "0.00"); //Monetary amount
$txi03_PST = array("", "", "", "", ""); //Percent
$txi06_PST = array("", "", "", "", ""); //Tax exempt code

$spam05 = array("11.50", "28.75", "10.62", "11.50", "-10.00"); //Extended line-item amount
$spid05 = array("Stapler", "Lamp", "Bottled Water", "Fountain Pen", "DISCOUNT"); //Line item
description
$it106s = array();
$it106s[0] = new axIt106s($it10618[0], $it10719[0]);
$it106s[1] = new axIt106s($it10618[1], $it10719[1]);
$it106s[2] = new axIt106s($it10618[2], $it10719[2]);
$it106s[3] = new axIt106s($it10618[3], $it10719[3]);
$it106s[4] = new axIt106s($it10618[4], $it10719[4]);
$txi = array(new axTxi(), new axTxi(), new axTxi(), new axTxi(), new axTxi());
$txi[0]->setTxi($txi01_GST[0], $txi02_GST[0], $txi03_GST[0], $txi06_GST[0]);
$txi[0]->setTxi($txi01_PST[0], $txi02_PST[0], $txi03_PST[0], $txi06_PST[0]);
$txi[1]->setTxi($txi01_GST[1], $txi02_GST[1], $txi03_GST[1], $txi06_GST[1]);
$txi[1]->setTxi($txi01_PST[1], $txi02_PST[1], $txi03_PST[1], $txi06_PST[1]);
$txi[2]->setTxi($txi01_GST[2], $txi02_GST[2], $txi03_GST[2], $txi06_GST[2]);
$txi[2]->setTxi($txi01_PST[2], $txi02_PST[2], $txi03_PST[2], $txi06_PST[2]);
$txi[3]->setTxi($txi01_GST[3], $txi02_GST[3], $txi03_GST[3], $txi06_GST[3]);
$txi[3]->setTxi($txi01_PST[3], $txi02_PST[3], $txi03_PST[3], $txi06_PST[3]);
$txi[4]->setTxi($txi01_GST[4], $txi02_GST[4], $txi03_GST[4], $txi06_GST[4]);
$txi[4]->setTxi($txi01_PST[4], $txi02_PST[4], $txi03_PST[4], $txi06_PST[4]);
$axItLoop = new axItLoop();
$axItLoop->setItLoop($it102[0], $it103[0], $it104[0], $it105[0], $it106s[0], $txi[0],
$spam05[0], $spid05[0]);
$axItLoop->setItLoop($it102[1], $it103[1], $it104[1], $it105[1], $it106s[1], $txi[1],
$spam05[1], $spid05[1]);
$axItLoop->setItLoop($it102[2], $it103[2], $it104[2], $it105[2], $it106s[2], $txi[2],
$spam05[2], $spid05[2]);
$axItLoop->setItLoop($it102[3], $it103[3], $it104[3], $it105[3], $it106s[3], $txi[3],
$spam05[3], $spid05[3]);
//$axItLoop->setItLoop($it102[4], $it103[4], $it104[4], $it105[4], $it106s[4], $txi[4],

```

Sample AX Independent Refund

```

$spam05[4], $pid05[4]);
$mpgAxLevel23->setTable2($axItLoop);
//Create Table 3 with details
$taxTbl3 = new axTxi();
$taxTbl3->setTxi("GS", "4.25","", ""); //sum of GST taxes
$taxTbl3->setTxi("PG", "4.60","", ""); //sum of PST taxes
$taxTbl3->setTxi("TX", "8.85","", ""); //sum of all taxes
$mpgAxLevel23->setTable3($taxTbl3);
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setLevel23Data($mpgAxLevel23);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

6 3-D Secure 2.2

- 6.1 About 3-D Secure 2.2
- 6.2 Building Your 3-D Secure 2.2 Integration
- 6.3 Implementing Card Lookup Request
- 6.5 Implementing MPI 3DS Authentication Request
- 6.6 Handling the Challenge Flow
- 6.8 Performing the Authorization
- 6.9 Testing Your 3-D Secure 2.2 Integration
- 6.10 Moving to Production With 3-D Secure 2.2
- 6.11 3-D Secure 2.2 TransStatus Codes
- 6.12 3-D Secure 2.2 Commons TransStatusReason Decline Codes
- 6.13 CAVV Result Codes

6.1 About 3-D Secure 2.2

3-D Secure 2.2 is an EMVCo payment authentication protocol designed to reduce card not present fraud by making a risk assessment based on transaction and device data, while also supporting further risk minimization measures, such as a challenge to the cardholder. In some cases, a liability shift takes effect for certain card-not-present fraud-related chargebacks enabling the merchant to provide goods and services with confidence.

The Moneris Gateway can enable transactions using the 3-D Secure protocol via Moneris 3DS Server and Access Control Server (ACS).

Moneris Gateway supports the following 3-D Secure implementations:

- Visa Secure (please note: Visa Secure does not support all the RI Indicators available in the 3D Secure 2.2. Check the RI Indicators status field to confirm the status Visa Secure support.)
- Mastercard Identity Check
- American Express SafeKey (please note: American Express only supports authentication requests for merchants who have an Amex OFI merchant account)

6.1.1 3-D Secure Implementations

Visa Secure, Mastercard Identity Check and American Express SafeKey are programs based on the 3-D Secure Protocol to improve the security of online transactions.

These programs involve authentication of the cardholder during an online e-commerce transaction.

Authentication is based on the issuer's selected method of authentication.

The following are examples of authentication methods:

- Risk-based authentication
- Dynamic passwords
- Static passwords

Some benefits of these programs are reduced risk of fraudulent transactions and protection against chargebacks for certain fraudulent transactions.

The PHP 3DS 2.2 API supports two message categories and two device channels from the 3-D Secure authentication protocol:

1. Message Categories:

- **Payment Authentication** – Cardholder authentication prior to an eCommerce transaction. After a successful 3DS authentication, you proceed with a purchase or pre-authorization.
- **Non-Payment Authentication (NPA)**– Identity verification and account confirmation performed without an accompanying financial transaction. After a successful 3DS authentication, you might proceed with:
 - Tokenizing the card for future payments
 - Allowing log-in for client portals
 - Any other activity relying on identity or account confirmation

2. Device Channels:

- **Browser** – The transaction originates from a website utilized via a browser on the cardholder's device.

- For example, an eCommerce transaction originating on the merchant's website with a check-out process that the cardholder is using via their personal computer or mobile phone's web browser (Chrome, Edge, Safari, etc.).
- **3DS Requestor Initiated** – Account confirmations and cardholder authentication with no direct cardholder originating the transaction.
 - 3RI can be used for authenticating Mail-Telephone Order (MOTO) transactions.
 - 3RI can be used to authenticate follow-on transactions as part of a subscription, such as recurring transactions. The first cardholder payment might use a browser-based authentication, with subsequent payments utilizing a 3RI authentication linking to the previous.
 - In situations where a merchant business model accommodates waiting before processing their payment, they can utilize Decoupled Authentication to allow the cardholder to authenticate directly with their issuer via a non-3DS challenge, such as a push notification to a banking app.

6.1.2 Out of Scope/Not Supported Check

- In-app

6.1.3 Version Compatibility

All development to the Moneris API must be able to support the addition of new fields in the response and new error conditions in the response. Otherwise any changes that affect backwards compatibility will be communicated by Moneris Solutions with an appropriate period of notice. When developing to the solution it is recommended to validate for success state of the request and then handle errors states separately and ensure there is a final catch for any unexpected/undocumented errors that are returned.

6.1.4 Upgrading from 3-D Secure 2.0 to 3-D Secure 2.2 Check

The 3DS 2.2 API is different from the 3DS 2.0 API therefore developers will have to complete the steps described in the section 6.2 Building Your 3-D Secure 2.2 Integration.

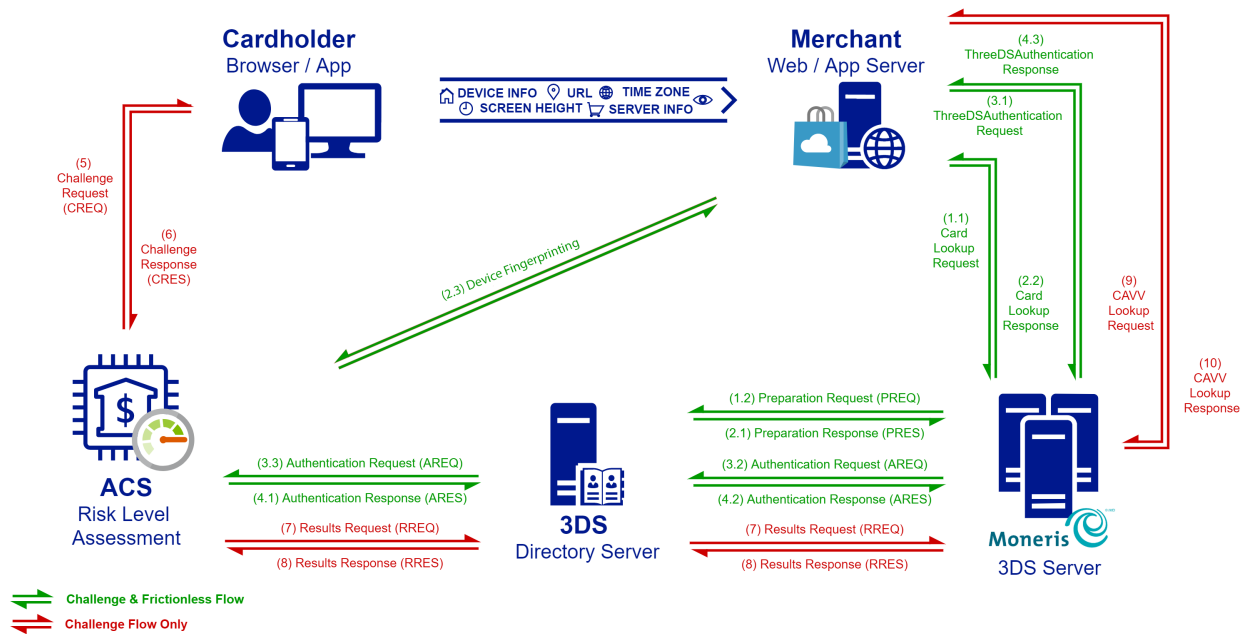
6.2 Building Your 3-D Secure 2.2 Integration

- 6.2.1 Activating 3-D Secure Functionality
- 6.2.2 Transaction Flow for 3-D Secure - Browser channel
- 6.2.3 Transaction Flow for 3-D Secure - 3RI channel

6.2.1 Activating 3-D Secure Functionality

To activate Visa Secure, Mastercard Identity Check and/or American Express SafeKey transaction functionality, call Moneris Sales Support at 1-855-465-4980 to have Moneris enroll you in the program(s) and enable the functionality on your account.

6.2.2 Transaction Flow for 3-D Secure - Browser channel



The 3DS 2.2 API is called when the customer wishes to checkout. An optional card lookup request can be performed to initiate cardholder browser fingerprinting. Once the fingerprint is complete, or as a first step if not performing a fingerprint, the transactional information can then be transmitted to the 3DS 2.2 service so a risk assessment may be initiated.

The flow can then proceed in one of two ways. The two different flows are referred to as “frictionless” and “challenge”.

The “frictionless” flow is invisible to a cardholder. If the issuing financial institution has enough information to make a risk assessment and assume liability, this will manifest itself as with an authentication attempt or success with an accompanying CAVV value. No cardholder challenge is presented.

In the “challenge” flow the issuing financial institution may wish to take a further step and issue a challenge to the cardholder. In this case the cardholder’s browser gets re-directed to the issuer’s 3DS platform for authentication. Once this challenge is complete, the cardholder browser is again re-directed back to the merchant’s site. The merchant’s server then issues a server-to-server request in order to obtain the CAVV value from Moneris.

Steps 1 – 2 (Optional)

An optional card lookup request can be performed to initiate cardholder browser fingerprinting. The merchant website collects device information and provides them to Moneris via the `card_lookup` request (1.1). Moneris submits this data to the 3DS Directory Server and returns with the `card_lookup` response containing the card’s supported 3DS version, an ACS URL, and 3DS Method Data representing the fingerprint (2.2). The merchant browser then submits an HTTP POST to the ACS URL with the method data. (2.3)

Once the fingerprint is complete, or as a first step if not performing a fingerprint, the transactional information can then be transmitted to the 3DS 2.2 service so a risk assessment may be initiated.

Steps 3 – 4 (Required)

The 3DS authentication request `threeDSAuthentication` is performed by the merchant website to initiate validating the cardholder identity. Moneris communicates with the 3DS Directory and the ACS system for that issuer to provide an initial risk assessment (3.2-4.2). Moneris returns a `threeDSAuthentication` response to the merchant with a `TransStatus` indicating the action for the website to perform:

- A `TransStatus` = “Y” or “A” means the website can proceed immediately to the financial transaction with the CAVV value provided. This is a frictionless transaction flow without presenting a challenge.
- A `TransStatus` = “C” indicates that the cardholder must be presented a challenge. To present the challenge, you must POST a `<form>` with a “creq” field, which contains the `ChallengeData`, to the URL defined in the `ChallengeURL` field.
- A `TransStatus` = “D” indicates that the cardholder must be presented a challenge via Decoupled Authentication. See Decoupled Authentication.

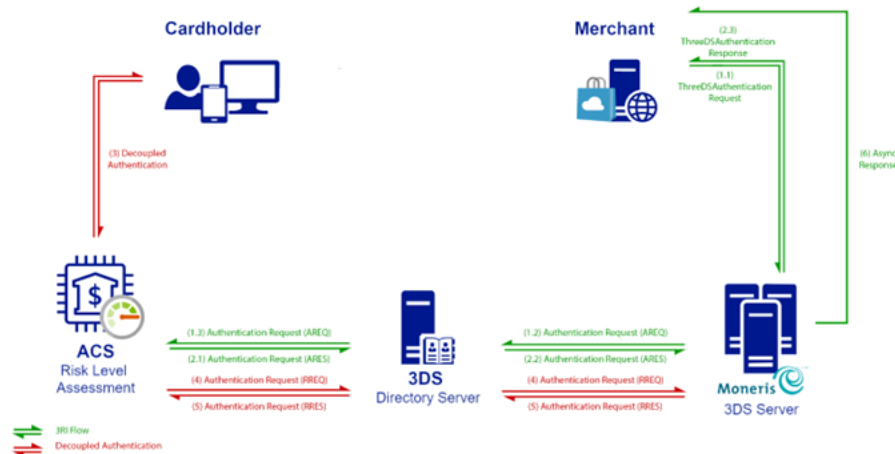
Steps 5 – 10 (Challenge Only)

In scenarios where a challenge is required, the merchant website sends an HTTP POST to the Challenge URL with the `ChallengeData` sent as a “CREQ” value (5). The ACS system will present a challenge to the cardholder, who will supply whatever credentials their issuer requires. The merchant website receives a

“CRES” value from the ACS via the HTTP POST response (6). Meanwhile, the ACS supplies the results to the 3DS Directory, which then forwards it to Moneris (7-8).

The merchant’s website then sends a CAVV Lookup to Moneris via a `cavv_lookup` request and includes their “CRES” (9). Moneris responds with the `cavv_lookup` response with the necessary ECI and CAVV values. With the 3DS authentication complete, you can proceed to the financial transaction.

6.2.3 Transaction Flow for 3-D Secure - 3RI channel



In a 3DS Requestor Initiated flow, the cardholder is not directly triggering the transaction flow via a browser experience as above. It is possible they are initiating the transaction outside the 3DS protocol, such as mailing or phoning the merchant (Mail-Telephone Order, aka MOTO), or it is possible the merchant is processing a recurring or installment plan on behalf of the cardholder’s subscription. It is also possible the merchant requires a non-payment authentication as part of tokenizing the card for later use.

3RI flows do not have direct cardholder interaction. The merchant sends their `<threeDSAuthentication>` request per steps 3-4 above but include additional fields to describe their 3RI usage scenario.

- If this is a Mail or Telephone (MOTO) payment authentication, the ACS may trigger a Decoupled Authentication between the issuer and cardholder (see Decoupled Authentication)
- If this is a follow-on payment from a previous 3DS authenticated transaction, you can include `PriorAuthenticationInfo` to link to the previous authentication and improve the likelihood of a successful result

Your server can utilize the fields `DeviceChannel`, `RiIndicator` and `MessageCategory` to inform Moneris if your merchant server is attempting to use the 3DS Requestor Initiated process.

6.2.3.1 Decoupled Authentication

For scenarios where a 3RI authentication requires challenge, instead of utilizing the standard challenge request and response the ACS authenticates the cardholder outside of the 3-D Secure protocol such as a banking app or mobile phone text to the cardholder. The Moneris 3DS Server waits for the ACS to authenticate the cardholder; this authentication can take up to 7 days. As this process relies on a cardholder action outside the 3DS flow, it occurs asynchronously to transaction processing.

Your server can utilize the fields `DecoupledRequestIndicator` and `DecoupledRequestAsyncUrl` to inform Moneris that you are opting in to accept a Decoupled Authentication attempt and where you want Moneris to POST the results asynchronously.

6.3 Implementing Card Lookup Request

The `CardLookup` request verifies the applicability of 3DS 2.2 on the card and returns the 3DS Method URL used for device fingerprinting if the card supports this feature. This request is optional, it may increase the chance of a frictionless flow.

The `threeDSMethodURL` & `threeDSMethodData` are returned to the merchant server on the `CardLookup` response, if supported.

- If you receive the `threeDSMethodURL`, you may send the `threeDSMethodData` to the `threeDSMethodURL` via a browser post in order to supplement the authentication request with device data pertaining to the cardholder's browser.
- If you do not receive the `threeDSMethodURL`, you may still proceed with 3DS Authentication.

The `threeDSMethodData` must be sent via HTTP POST to the `threeDSMethodURL` in a hidden `iFrame`.

In your implementation, use the following URLs as Host, depending on the development stage:

Testing:

`esqa.moneris.com`

Production:

`www3.moneris.com`

6.3.1 Card Lookup Request – `mpiCardLookup`

Card Lookup Request transaction object definition

```
$mpiCardLookup = new MpiCardLookup();
```

HttpPostRequest object for Card Lookup Request transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
$mpgTxn = new mpgTransaction($mpiCardLookup);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Card Lookup Request transaction request fields – Required

NOTE: Either a pan or a data_key must be passed in the request

Variable Name	Type and Limits	
order ID	String 50-character alpha- numerica-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	String max 20-character alpha- numeric	'pan'=>\$pan
data key	String 25-character alphanumeric	'data_key'=>\$data_key
notification URL	String 256-character alpha- numeric	\$mpiCardLookup->setNotificationURL ("HTTPS://YOURURL.COM");

Sample Card Lookup Request

```
<?php
```

```

require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = "moneris";
$api_token = "hurgle";
$order_id = 'ord-' . date("dmy-G:i:s");
$span = "347668693641199";

$mpiCardLookup = new MpiCardLookup();
$mpiCardLookup->setOrderId($order_id);
$mpiCardLookup->setPan($span);
// $mpiCardLookup->setDataKey("800XGiwXgvfbZngigVFeld9d2"); //Optional - For Moneris Vault and
// Hosted Tokenization tokens in place of setPan
$mpiCardLookup->setNotificationUrl("https://yournotificationurl.com"); // (Website URL that
// will receive 3DS Method Completion response from ACS)
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($mpiCardLookup);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false or comment out this line for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nMessageType = " . $mpgResponse->getMpiMessageType());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nThreeDSMethodURL = " . $mpgResponse->getMpiThreeDSMethodURL());
print("\nThreeDSMethodData = " . $mpgResponse->getMpiThreeDSMethodData());
print("\nThreeDSServerTransId = " . $mpgResponse->getMpiThreeDSServerTransId());
?>

```

6.4 Handling the 3DS Method for Device Fingerprinting

You can use the **threeDSMethodURL** & **threeDSMethodData** returned by a Card Lookup response to increase the probability of a frictionless 3DS flow for the cardholder. Transmitting the **threeDSMethodData** to the **threeDSMethodURL** via a browser HTTP POST allows the issuer use of a hidden iFrame on the merchant website to obtain details on the customer's device.

The results of the 3DS Method are returned to the merchant's **notificationURL** supplied in the preceding Card Lookup.

Below is a sample of a basic static form to help visualize the data and fields that need to be submitted.

Device Fingerprinting request form (Merchant browser to ACS):

```

<form name="frm" method="POST" action="Rendering URL">
<input type="hidden" name="threeDSMethodData" value=
="eyJ0aHJlZURTU2VydmVyVHJh-
bnNJRCI6IjNhYzdjYWE3LWFhNDItMjY2My03OTFiLTJhYzA1YTU0MmM0YSIsInRocmVlRFNNZ-
XRob2R0b3RpZmljYXRpb25VUkwioiJ0aHJlZURTU2V0aG9kTm90aWZpY2F0aW9uVWJMinO">
</form>

```

Decoded threeDSMethodData:

```

{"threeDSServerTransID":"3ac7caa7-aa42-2663-791b-2ac05a542c4a","-
threeDSMethodNotificationURL":"threeDSMethodNotificationURL"}

```


Device Fingerprinting response form (ACS to Merchant notificationURL):

```
<form name="frm" method="POST" action="threeDSMethodNotificationURL">
<input type="hidden" name="threeDSMethodData" value-
="eyJ0aHJlZURTU2VydmVyVHJh-
hbnNJRCI6IjNhYzdjYWE3LWFhNDItMjY2My03OTFiLTJhYzA1YTU0MmM0YSJ9">
</form>
```

Decoded threeDSMethodData:

```
{"threeDSServerTransID":"3ac7caa7-aa42-2663-791b-2ac05a542c4a"}
```

6.5 Implementing MPI 3DS Authentication Request

The MPI 3DS Authentication Request is used to start the validation process of the card. The result of this request determines whether 3DS 2.2 is supported by the card and what type of authentication is required.

In your implementation, use the following URLs as Host, depending on the development stage:

Testing URLs:

<https://mpg1t.moneris.io/mpi2/servlet/MpiServlet>

Production URLs:

<https://mpg1.moneris.io/mpi2/servlet/MpiServlet>

Below we detail three different scenarios for utilizing Moneris MPI 3DS Authentication. Each scenario has conditions for which fields are required or optional for the endpoint.

6.5.1 MPI 3DS Authentication Request - Browser Channel

The authentication request is used to start the validation process of the card.

The result of this request determines whether 3-D Secure 2.0 is supported by the card and what type of authentication is required.

In your implementation, use the following URLs as Host, depending on the development stage:

testing: mpg1t.moneris.io

production: mpg1.moneris.io

NOTE: Billing-related request fields are required to be sent for this transaction, or else the authentication process may fail

MPI 3DS Authentication Request transaction object definition

```
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
```

HttpPostRequest object for MPI 3DS Authentication Request transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
```

WARNING: Do not send fields related to 3RI on browser-based authentications.

Core connection object fields (all API transactions)

Variable Name	Type and Limits	Description
store ID	<i>String</i>	'store_id'=>\$store_id
<store_id>	N/A	Unique identifier provided by Moneris upon merchant account setup
API token	<i>String</i>	'api_token'=>\$api_token
<api_token>	N/A	Unique alphanumeric string assigned by Moneris upon merchant account activation
		To find your API token, refer to your test or production store's Admin settings in the Merchant Resource Center, at the following URLs:
		Testing: https://esqa.moneris.com/mpg/
		Production: https://www3.moneris.com/mpg/

MPI 3DS Authentication Request transaction request fields – Required

Variable Name	Type and Limits	Description
message category	<i>String</i>	\$mpiThreeDSAuthentication->setMessageCategory("02");
<MessageCategory>	2-character	

Variable Name	Type and Limits	Description
	numeric	Whether the authentication request is for a payment or non-payment use: 01 = payment authentication (PA) 02 = non-payment authentication (NPA)
device channel <DeviceChannel>	<i>String</i> 2-character numeric	<code>\$mpiThreeDSAuthentication->setDeviceChannel("02");</code> The interface used to initiate the authentication: 02 = Browser (BRW) 03 = 3DS Requestor Initiated (3RI)
request type <RequestType>	<i>String</i> 2-character alphanumeric	<code>\$mpiThreeDSAuthentication->setRequestType("REQUEST_TYPE_VALUE");</code> Indicates the type of browser-based authentication request: 01 = cardholder initiated payment 02 = recurring transaction 03 = installment transaction 04 = add card 05 = maintain card 06 = cardholder verification as part of EMV token ID & V Conditional. Required if device_channel = 02
order ID <order_id>	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	<code>'order_id'=>\$order_id;</code> Merchant-defined transaction identifier that must be unique for every Purchase, Pre-Authorization and Independent Refund transaction. No two transactions of these types may have the same order ID. For Refund, Completion and Purchase Correction transactions, the order ID must be the same as that of the original transaction.
data key	<i>String</i>	<code>'data_key'=>\$data_key;</code>

Variable Name	Type and Limits	Description
<data_key> OR credit card number <pan>	data key limits: 25-character alphanumeric credit card number limits: max 20-character alphanumeric	data key description: Unique identifier for a Vault profile, and used in future Vault financial transactions to associate a transaction with that profile Data key is generated by Moneris and returned to you in the Receipt object when the profile is first registered 'pan'=>\$pan credit card number description: Credit card number, usually 16 digits —field can be maximum 20 digits in support of future expansion of card number ranges. Carries the token for network tokenization transactions.
expiry date <expdate>	String 4-character alphanumeric YYYYMM	'expiry_date'=>\$expiry_date Expiry date of the credit card, in YYMM format. NOTE: This is the reverse of the MMY date format that is presented on the card.
amount <amount>	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount Transaction dollar amount This must contain at least 3 digits, two of which are penny values Minimum allowable value = \$0.01, maximum allowable value = \$9999999.99
cardholder name <CardholderName>	String 45-character	\$mpiThreeDSAuthentication->setCardholderName("CARDHOLDER_NAME_VALUE");

Variable Name	Type and Limits	Description
	alphanumeric NOTE: Accented characters are not allowable	Name of the cardholder
3DS completion indicator <ThreeDSCompletionInd>	<i>String</i> 1-character alphabetic	<pre>\$mpiThreeDSAuthentication->setThreeDSCompletionInd("THREEDSCOMPLETION_VALUE");</pre> indicates whether 3ds method MpiCardLookup was successfully completed Allowable values: Y = Successfully completed N = Did not successfully complete U = Unavailable Conditional. Required if card_lookup is used.
billing address <BillAddress1>	<i>String</i> 50-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBillAddress1("BILL_STREET_ADDRESS_VALUE");</pre> Cardholder billing address
billing province <BillProvince>	<i>String</i> 3-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBillProvince("BILL_PROV_VALUE");</pre> Cardholder province or state Defined in country subdivision ISO 3166-2
billing city <BillCity>	<i>String</i> 50-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBillCity("BILL_CITY_VALUE");</pre> Cardholder billing city
billing postal code <BillPostalCode>	<i>String</i> 16-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBillPostalCode("BILL_POSTAL_CODE_VALUE");</pre> Cardholder billing postal code
billing country	<i>String</i>	<pre>\$mpiThreeDSAuthentication->setBillCountry</pre>

Variable Name	Type and Limits	Description
<BillCountry>	3-character alphanumeric	("BILL_COUNTRY_VALUE"); Cardholder billing country Defined as 3 digit country code ISO 3166-1
shipping address <ShipAddress1>	<i>String</i> 50-character alphanumeric	\$mpiThreeDSAuthentication->setShipAddress1 ("SHIP_STREET_ADDRESS_VALUE"); Shipping destination address
shipping province <ShipProvince>	<i>String</i> 3-character alphanumeric	\$mpiThreeDSAuthentication->setShipProvince ("SHIP_PROV_VALUE"); Shipping destination province or state Defined in country subdivision ISO 3166-2
shipping city <ShipCity>	<i>String</i> 50-character alphanumeric	\$mpiThreeDSAuthentication->setShipCity ("SHIP_CITY_VALUE"); Shipping destination city
shipping postal code <ShipPostalCode>	<i>String</i> 16-character alphanumeric	\$mpiThreeDSAuthentication->setShipPostalCode ("SHIP_POSTAL_CODE_VALUE"); Shipping destination postal or ZIP code
shipping country <ShipCountry>	<i>String</i> 3-character alphanumeric	\$mpiThreeDSAuthentication->setShipCountry ("SHIP_COUNTRY_VALUE"); Shipping destination country Defined as 3-digit country code in ISO 3166-1
notification URL <NotificationURL>	<i>String</i> 256-character alphanumeric	\$mpiThreeDSAuthentication->setNotificationURL ("HTTPS://YOURURL.COM"); Notification URL for receiving the 3DS Method POST response from the issuer ACS. Conditional. Required if device_channel = 02
challenge window size <	<i>String</i> 2-character	\$mpiThreeDSAuthentication->setChallengeWindowSize ("CWS_VALUE");

Variable Name	Type and Limits	Description
ChallengeWindowSize>	alphanumeric	Relates to the rendering of the ACS challenge within the browser. Allowable values: 01 = 250 x 400 02 = 390 x 400 03 = 500 x 600 04 = 600 x 400 05 = Full screen Conditional. Required if device_channel = 02
browser IP Address <BrowserIP>	<i>String</i> Allows '.' and ':' 45-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBrowserIP("10.10.10.10") or ("011:0db8:85a3:0101:0101:8a2e:0370:7334"); // (IPv4 or IPv6)</pre> IP address of the browser as returned by the HTTP headers to the 3DS Requestor. NOTE: This field is not mandatory, but it is required. It is highly recommended to provide. Lack of providing this field, might increase the risk of rejects.
browser user agent <BrowserUserAgent>	<i>String</i> 2048-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBrowserUserAgent("BROWSER_USER_AGENT_VALUE");</pre> Browser User Agent Conditional. Required if device_channel = 02
browser java enabled <BrowserJavaEnabled>	<i>String</i> 1-character alphabetic	<pre>\$mpiThreeDSAuthentication->setBrowserJavaEnabled(BROWSER_JAVA_VALUE);</pre> Indicates whether Java is enabled in the browser Allowable values: T = True F = False Conditional. Required if device_channel = 02

Variable Name	Type and Limits	Description
browser screen height <BrowserScreenHeight>	<i>String</i> 6-character numeric	<pre>\$mpiThreeDSAuthentication->setBrowserScreenHeight (BROWSER_SCREEN_HEIGHT_VALUE) ;</pre> Pixel height of cardholder screen Conditional. Required if device_channel = 02 NOTE: This field is not mandatory, but it is required. It is highly recommended to provide. Lack of providing this field, might increase the risk of rejects.
browser screen width <BrowserScreenWidth>	<i>String</i> 6-character numeric	<pre>\$mpiThreeDSAuthentication->setBrowserScreenWidth (BROWSER_SCREEN_WIDTH_VALUE) ;</pre> Pixel width of cardholder screen Conditional. Required if device_channel = 02 NOTE: This field is not mandatory, but it is required. It is highly recommended to provide. Lack of providing this field, might increase the risk of rejects.
browser language <BrowserLanguage>	<i>String</i> 8-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBrowserLanguage (BROWSER_LANGUAGE_VALUE) ;</pre> As defined in IETF BCP47 Conditional. Required if device_channel = 02
email <Email>	<i>String</i> 254-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setEmail ("EMAIL_VALUE") ;</pre> Cardholder email address NOTE: This field is not mandatory, but it is required. It is highly recommended to provide the cardholder's email address. Lack of providing the cardholder's address, might increase the risk of rejects.
cardholder work phone number <WorkPhone>	<i>Object</i> N/A	<pre>\$mpiThreeDSAuthentication->setWorkPhone (\$workPhoneTemplate) ;</pre> Cardholder work phone number

Variable Name	Type and Limits	Description
		NOTE: This field is not mandatory, but it is required. It is highly recommended to provide at least one of the Cardholder Phone Number. Lack of providing at least one of the Cardholder Phone Number, might increase the risk of rejects. NOTE: This is a nested object within the transaction. For information about fields in the Cardholder Phone Number Info object, see Cardholder Phone Number Info Object and Variables.
cardholder home phone number <home_phone>	Object N/A	<pre>\$mpiThreeDSAuthentication->setHomePhone(\$homePhoneTemplate);</pre> Cardholder home phone number NOTE: This field is not mandatory, but it is required. It is highly recommended to provide at least one of the Cardholder Phone Number. Lack of providing at least one of the Cardholder Phone Number, might increase the risk of rejects. NOTE: This is a nested object within the transaction. For information about fields in the Cardholder Phone Number Info object, see Cardholder Phone Number Info Object and Variables.
cardholder mobile phone number <mobile_phone>	Object N/A	<pre>\$mpiThreeDSAuthentication->setMobilePhone(\$mobilePhoneTemplate);</pre> Cardholder mobile phone number NOTE: This field is not mandatory, but it is required. It is highly recommended to provide at least one of the Cardholder Phone Number. Lack of providing at least one of the Cardholder Phone Number, might increase the risk of rejects. NOTE: This is a nested object within the transaction. For information about fields in the Cardholder Phone Number Info object, see Cardholder Phone Number Info Object and Variables.

MPI 3DS Cardholder Phone Number

Variable Name	Type and Limits	Description
country code	String	Country Code of phone number provided

Variable Name	Type and Limits	Description
<country_code>	3-character numeric	by the Cardholder.
phone number	<i>String</i>	The phone number provided by the Cardholder.
<phone_number>	15-character numeric	

MPI 3DS Authentication Request transaction request fields – Optional

Variable Name	Type and Limits	Description
currency <currency>	<i>String</i> 3-character numeric	<code>\$mpiThreeDSAuthentication->SetCurrency("CURRENCY_VALUE");</code> ISO 4217 3 digit currency code CAD = 124 USD = 840 NOTE: This field should not be sent unless Multi Currency Pricing is enabled on your merchant account
request challenge <RequestChallenge>	<i>String</i> 2-character numeric	<code>\$mpiThreeDSAuthentication->setRequestChallenge ("CHALLENGE_VALUE");</code> Indicates whether a browser-based challenge is requested for this transaction. Standard is "01" • 01 = No preference • 02 = No challenge requested • 03 = Challenge requested: 3DS Requestor Preference • 04 = Challenge requested: Mandate Conditional. Required if device_channel = 02

Sample MPI 3DS Authentication Request - Browser Channel

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = "moneris";
```

```
$api_token = "hurgle";
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
$mpiThreeDSAuthentication->setOrderId("ord-110920-10:36:41"); //must be the same one that was
used in MpiCardLookup call
$mpiThreeDSAuthentication->setCardholderName("Moneris Test");
$mpiThreeDSAuthentication->setPan("347668693641199");
// $mpiThreeDSAuthentication->setDataKey("800XGiwxgvfbZngigVFeld9d2"); //Optional - For
Moneris Vault and Hosted Tokenization tokens in place of setPan
$mpiThreeDSAuthentication->setExpdate("2310");
$mpiThreeDSAuthentication->setAmount("1.00");
$mpiThreeDSAuthentication->setThreeDSCompletionInd("Y"); //(Y|N|U) indicates whether 3ds
method MpiCardLookup was successfully completed
$mpiThreeDSAuthentication->setRequestType("01"); //(01=payment|02=recur)
$mpiThreeDSAuthentication->setPurchaseDate("20200911035249"); //(YYYYMMDDHHMMSS)
$mpiThreeDSAuthentication->setNotificationURL("https://yournotificationurl.com"); //(Website
where response from RRes or CRes after challenge will go)
$mpiThreeDSAuthentication->setChallengeWindowSize("03"); //(01 = 250 x 400, 02 = 390 x 400,
03 = 500 x 600, 04 = 600 x 400, 05 = Full screen)
$mpiThreeDSAuthentication->setBrowserUserAgent("Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.132 Safari/537.36\\");

$mpiThreeDSAuthentication->setBrowserIP ("10.10.10.10") or
("011:0db8:85a3:0101:0101:8a2e:0370:7334"); //(IPv4 or IPv6)

$mpiThreeDSAuthentication->setBrowserJavaEnabled("true"); //(true|false)
$mpiThreeDSAuthentication->setBrowserScreenHeight("1000"); //(pixel height of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserScreenWidth("1920"); //(pixel width of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserLanguage("en-GB"); //(defined by IETF BCP47)
//Optional Methods
$mpiThreeDSAuthentication->setBillAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setBillProvince("ON");
$mpiThreeDSAuthentication->setBillCity("Toronto");
$mpiThreeDSAuthentication->setBillPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setBillCountry("124");
$mpiThreeDSAuthentication->setShipAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setShipProvince("ON");
$mpiThreeDSAuthentication->setShipCity("Toronto");
$mpiThreeDSAuthentication->setShipPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setShipCountry("124");
$mpiThreeDSAuthentication->setEmail("test@email.com");

$workPhoneTemplate = array('cc'=>'1','subscriber'=>'1111111111');
$mpiThreeDSAuthentication->setWorkPhone( $workPhoneTemplate );

$homePhoneTemplate = array('cc'=>'3','subscriber'=>'3333333333');
$mpiThreeDSAuthentication->setHomePhone( $homePhoneTemplate );

$mobilePhoneTemplate = array('cc'=>'2','subscriber'=>'2222222222');
$mpiThreeDSAuthentication->setMobilePhone( $mobilePhoneTemplate );

$mpiThreeDSAuthentication->setRequestChallenge("Y"); //(Y|N Requesting challenge regardless
of outcome)
$mpiThreeDSAuthentication->setMessageCategory("01");
$mpiThreeDSAuthentication->setDeviceChannel("03");
// $mpiThreeDSAuthentication->setDecoupledRequestIndicator("N");
// $mpiThreeDSAuthentication->setDecoupledRequestMaxTime("00010");
// $mpiThreeDSAuthentication->setDecoupledRequestAsyncUrl("localhost");
$mpiThreeDSAuthentication->setRiIndicator("08");
// $mpiThreeDSAuthentication->setRecurringFrequency("031");
// $mpiThreeDSAuthentication->setRecurringExpiry("20251230");
$paiTemplate = array(
'prior_request_auth_data'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340",
'prior_request_ref'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340",
```

```

'prior_request_auth_method'=>"01",
'prior_request_auth_timestamp'=>"201710282113"
);
//$mpiThreeDSAuthentication->setPriorAuthenticationInfo( $paiTemplate );
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions
//print_r($mpgRequest);
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nMessageType = " . $mpgResponse->getMpiMessageType());
print("\nTransStatus = " . $mpgResponse->getMpiTransStatus());
print("\nTransStatusReason = " . $mpgResponse->getMpiTransStatusReason());
print("\nChallengeURL = " . $mpgResponse->getMpiChallengeURL());
print("\nChallengeData = " . $mpgResponse->getMpiChallengeData());
print("\nThreeDSServerTransId = " . $mpgResponse->getMpiThreeDSServerTransId());
print("\nDSTransId = " . $mpgResponse->getMpiDSTransId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nThreeDSAcTransID = " . $mpgResponse->getMpiThreeDSAcTransID());
print("\nThreeDSAuthTimeStamp = " . $mpgResponse->getMpiThreeDSAuthTimeStamp());
print("\nAuthenticationType = " . $mpgResponse->getMpiAuthenticationType());
print("\nCardHolderInfo = " . $mpgResponse->getMpiCardholderInfo());
//In Frictionless flow, you may receive TransStatus as "Y",
//in which case you can then proceed directly to Cavv Purchase/Preauth with values below'
if($mpgResponse->getMpiTransStatus() == "Y")
{
print("\nCavv = " . $mpgResponse->getMpiCavv());
print("\nECI = " . $mpgResponse->getMpiEci());
}
?>

```

6.5.2 MPI 3DS Authentication Request - 3RI with recurring

NOTE: Billing address request fields are recommended to be sent for this transaction, or else the authentication process may fail

MPI 3DS Authentication Request transaction object definition

```
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
```

HttpPostRequest object for MPI 3DS Authentication Request transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	Description
store ID	<i>String</i>	'store_id'=>\$store_id
<store_id>	N/A	Unique identifier provided by Moneris upon merchant account setup
API token	<i>String</i>	'api_token'=>\$api_token
<api_token>	N/A	Unique alphanumeric string assigned by Moneris upon merchant account activation To find your API token, refer to your test or production store's Admin settings in the Merchant Resource Center, at the following URLs: Testing: https://esqa.moneris.com/mpg/ Production: https://www3.moneris.com/mpg/

MPI 3DS Authentication Request transaction request fields – Required

Variable Name	Type and Limits	Description
message category	<i>String</i>	<code>\$mpiThreeDSAuthentication->setMessageCategory("02");</code>
<MessageCategory>	2-character numeric	Whether the authentication request is for a payment or non-payment use: 01 = payment authentication (PA) 02 = non-payment authentication (NPA)
device channel	<i>String</i>	<code>\$mpiThreeDSAuthentication->setDeviceChannel("02");</code>
<DeviceChannel>	2-character numeric	The interface used to initiate the authentication: 02 = Browser (BRW)

Variable Name	Type and Limits	Description
		03 = 3DS Requestor Initiated (3RI)
recurring frequency <RecurringFrequency>	<i>String</i> 4-character numeric	<pre>\$mpiThreeDSAuthentication->setRecurringFrequency("031");</pre> The minimum number of days between recurring transactions. Numeric values between 1 and 9999, leading zeroes accepted. Conditional. Required if ri indicator = 01
recurring expiry <RecurringExpiry>	<i>String</i> 8-character numeric	<pre>\$mpiThreeDSAuthentication->setRecurringExpiry("20221230");</pre> End date after which no further recurring transactions shall be performed. Format is YYYYMMDD. Conditional. Required if ri indicator = 01
ri indicator <RiIndicator>	<i>String</i> 2-character numeric	<pre>\$mpiThreeDSAuthentication->setRiIndicator("03");</pre> The type of 3DS Requestor Initiated (3RI) request: 01 = Recurring 02 = Installment 03 = Add Card 04 = Maintain Card Information 05 = Account verification 06 = Split/Delayed Shipment 07 = Top-up 08 = Mail Order 09 = Telephone Order

NOTE: Visa Secure only support ri_Indicator = 01, 02, 06, 07, or 11 for Payment Transactions and ri Indicator = 03, 04, 05 and 10 for Non Payment Transactions

Variable Name	Type and Limits	Description
		10 = Whitelist 11 = Other Payment Conditional. Required if device_channel = 03
order ID	<i>String</i>	'order_id'=>\$order_id;
<order_id>	50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	Merchant-defined transaction identifier that must be unique for every Purchase, Pre-Authorization and Independent Refund transaction. No two transactions of these types may have the same order ID. For Refund, Completion and Purchase Correction transactions, the order ID must be the same as that of the original transaction.
data key	<i>String</i>	'data_key'=>\$data_key;
<data_key>	data key limits:	data key description:
OR	25-character alphanumeric	Unique identifier for a Vault profile, and used in future Vault financial transactions to associate a transaction with that profile
credit card number	credit card number limits:	Data key is generated by Moneris and returned to you in the Receipt object when the profile is first registered
<pan>	max 20-character alphanumeric	'pan'=>\$pan credit card number description: Credit card number, usually 16 digits —field can be maximum 20 digits in support of future expansion of card number ranges. Carries the token for network tokenization transactions.

Variable Name	Type and Limits	Description
expiry date <expdate>	String 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date Expiry date of the credit card, in YYMM format. NOTE: This is the reverse of the MMYM date format that is presented on the card.
amount <amount>	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount Transaction dollar amount This must contain at least 3 digits, two of which are penny values Minimum allowable value = \$0.01, maximum allowable value = \$9999999.99
cardholder name <CardholderName>	String 45-character alphanumeric NOTE: Accented characters are not allowable	\$mpiThreeDSAuthentication->setCardholderName ("CARDHOLDER_NAME_VALUE"); Name of the cardholder
billing address <BillAddress1>	String 50-character alphanumeric	\$mpiThreeDSAuthentication->setBillAddress1 ("BILL_STREET_ADDRESS_VALUE"); Cardholder billing address
billing province <BillProvince>	String 3-character alphanumeric	\$mpiThreeDSAuthentication->setBillProvince ("BILL_PROV_VALUE"); Cardholder province or state Defined in country subdivision ISO 3166-2
billing city <BillCity>	String 50-character alphanumeric	\$mpiThreeDSAuthentication->setBillCity ("BILL_CITY_VALUE");

Variable Name	Type and Limits	Description
		Cardholder billing city
billing postal code <BillPostalCode>	String 16-character alphanumeric	\$mpiThreeDSAuthentication->setBillPostalCode ("BILL_POSTAL_CODE_VALUE"); Cardholder billing postal code
billing country <BillCountry>	String 3-character alphanumeric	\$mpiThreeDSAuthentication->setBillCountry ("BILL_COUNTRY_VALUE"); Cardholder billing country Defined as 3 digit country code ISO 3166-1
shipping address <ShipAddress1>	String 50-character alphanumeric	\$mpiThreeDSAuthentication->setShipAddress1 ("SHIP_STREET_ADDRESS_VALUE"); Shipping destination address
shipping province <ShipProvince>	String 3-character alphanumeric	\$mpiThreeDSAuthentication->setShipProvince ("SHIP_PROV_VALUE"); Shipping destination province or state Defined in country subdivision ISO 3166-2
shipping city <ShipCity>	String 50-character alphanumeric	\$mpiThreeDSAuthentication->setShipCity ("SHIP_CITY_VALUE"); Shipping destination city
shipping postal code <ShipPostalCode>	String 16-character alphanumeric	\$mpiThreeDSAuthentication->setShipPostalCode ("SHIP_POSTAL_CODE_VALUE"); Shipping destination postal or ZIP code
shipping country <ShipCountry>	String 3-character alphanumeric	\$mpiThreeDSAuthentication->setShipCountry ("SHIP_COUNTRY_VALUE");

Variable Name	Type and Limits	Description
		Shipping destination country Defined as 3-digit country code in ISO 3166-1
email <Email>	<i>String</i> 254-character alpha-numeric	<code>\$mpiThreeDSAuthentication->setEmail("EMAIL_VALUE");</code> Cardholder email address NOTE: This field is not mandatory, but it is required. It is highly recommended to provide the cardholder's email address. Lack of providing the cardholder's address, might increase the risk of rejects.

MPI 3DS Authentication Request transaction request fields – Optional

Variable Name	Type and Limits	Description
currency <currency>	<i>String</i> 3-character numeric	<code>\$mpiThreeDSAuthentication->SetCurrency("CURRENCY_VALUE");</code> ISO 4217 3 digit currency code CAD = 124 USD = 840 NOTE: This field should not be sent unless Multi Currency Pricing is enabled on your merchant account
decoupled request indicator < DecoupledRequestIndicator >	<i>String</i> 1-character alphabetic	<code>\$mpiThreeDSAuthentication->setDecoupledRequestIndicator("Y");</code> Whether the request utilizes Decoupled Authentication or not, if the ACS confirms its use. Y = Decoupled Authentication is supported and preferred if challenge is necessary N = Do not use Decoupled Authentication (Default) Defaults to N if unused.
decoupled request max	<i>String</i>	<code>\$mpiThreeDSAuthentication-</code>

Variable Name	Type and Limits	Description
time <DecoupledRequestMaxTime>	5-character numeric	<pre>>setDecoupledRequestMaxTime("00010");</pre> The maximum minutes that Moneris waits for an ACS to provide results. Numeric values between 1 and 10080. The max is equivalent to 7 days. Conditional. Required if device_channel = 03 and decoupled_request_indicator = Y
decoupled request async URL <DecoupledRequestAsyncUrl>	<i>String</i> 256-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setDecoupledRequestAsyncUrl("https://yourasyncnotificationurl.com");</pre> Your URL where Moneris will POST the response back from ACS. Moneris reattempts 3 times to POST the response. Conditional. Only sent if decoupled_request_indicator = Y
prior request auth info <PriorAuthenticationInfo>	<i>Object</i> N/A	<pre>\$mpiThreeDSAuthentication->setPriorRequestAuthInfo(\$paiTemplate)</pre> Object containing details for a prior 3DS authentication for this series of transactions. This is a nested object within the authentication transaction, and required when storing or using the information about the prior authentication for that card. For information about fields in the Prior Authentication Info object, see MPI 3DS Prior Authentication Info Object and Variables.

MPI 3DS Prior Authentication Info

Variable Name	Type and Limits	Description
prior request auth data <prior_request_auth_data>	<i>String</i> 36-character alphanumeric	<pre>'prior_request_auth_data'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340"</pre> Refers to the DSTransID in the response of the previous 3DS authentication.
prior request ref	<i>String</i>	<pre>'prior_request_ref'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340"</pre>

Variable Name	Type and Limits	Description
<prior_request_auth_ref>	36-character alphanumeric	Refers to the 3DS ACS Transaction ID in the response of the previous 3DS authentication.
prior request auth method <prior_request_auth_method>	<i>String</i> 2-character numeric	'prior_request_auth_method'=>"01", Mechanism used by the cardholder to authenticate in the previous 3DS authentication: 01 = Frictionless authentication 02 = Challenge authentication 03 = AVS verified 04 = Other issuer methods
prior request auth timestamp <prior_request_auth_timestamp>	<i>String</i> 12-character numeric	'prior_request_auth_timestamp'=>"201710282113", Date and time in UTC of the prior cardholder authentication. Found in the previous 3DS authentication response as 3DS Auth TimeStamp. Format is YYYYMMDDHHMM.

Sample MPI 3DS Authentication Request - 3RI with recurring

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = "moneris";
$api_token = "hurgle";
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
$mpiThreeDSAuthentication->setOrderId("ord-110920-10:36:41"); //must be the same one that was
used in MpiCardLookup call
$mpiThreeDSAuthentication->setCardholderName("Moneris Test");
$mpiThreeDSAuthentication->setPan("343427006265962");
// $mpiThreeDSAuthentication->setDataKey("800XGiwXgvfbZngigVFeld9d2"); //Optional - For
Moneris Vault and Hosted Tokenization tokens in place of setPan
$mpiThreeDSAuthentication->setExpdate("2310");
$mpiThreeDSAuthentication->setAmount("1.00");
$mpiThreeDSAuthentication->setThreeDSCompletionInd("Y"); //(Y|N|U) indicates whether 3ds
method MpiCardLookup was successfully completed
$mpiThreeDSAuthentication->setRequestType("02"); //(01=payment|02=recur)
$mpiThreeDSAuthentication->setPurchaseDate("20200911035249"); //(YYYYMMDDHHMMSS)
$mpiThreeDSAuthentication->setNotificationURL("https://yournotificationurl.com"); //(Website
where response from RRes or CRes after challenge will go)
$mpiThreeDSAuthentication->setChallengeWindowSize("03"); //(01 = 250 x 400, 02 = 390 x 400,
03 = 500 x 600, 04 = 600 x 400, 05 = Full screen)
$mpiThreeDSAuthentication->setBrowserUserAgent("Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.132 Safari/537.36\\");
$mpiThreeDSAuthentication->setBrowserJavaEnabled("true"); //(true|false)
$mpiThreeDSAuthentication->setBrowserScreenHeight("1000"); //(pixel height of cardholder
screen)
```

```

$mpiThreeDSAuthentication->setBrowserScreenWidth("1920"); //(pixel width of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserLanguage("en-GB"); //(defined by IETF BCP47)
//Optional Methods
$mpiThreeDSAuthentication->setBillAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setBillProvince("ON");
$mpiThreeDSAuthentication->setBillCity("Toronto");
$mpiThreeDSAuthentication->setBillPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setBillCountry("124");
$mpiThreeDSAuthentication->setShipAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setShipProvince("ON");
$mpiThreeDSAuthentication->setShipCity("Toronto");
$mpiThreeDSAuthentication->setShipPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setShipCountry("124");
$mpiThreeDSAuthentication->setEmail("test@email.com");
$mpiThreeDSAuthentication->setRequestChallenge("Y"); //(Y|N Requesting challenge regardless
of outcome)
$mpiThreeDSAuthentication->setMessageCategory("01");
$mpiThreeDSAuthentication->setDeviceChannel("03");
//$mpiThreeDSAuthentication->setDecoupledRequestIndicator("N");
//$mpiThreeDSAuthentication->setDecoupledRequestMaxTime("00010");
//$mpiThreeDSAuthentication->setDecoupledRequestAsyncUrl("localhost");
$mpiThreeDSAuthentication->setRiIndicator("01");
$mpiThreeDSAuthentication->setRecurringFrequency("031");
$mpiThreeDSAuthentication->setRecurringExpiry("20251230");
$paiTemplate = array(
    'prior_request_auth_data'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340",
    'prior_request_ref'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340",
    'prior_request_auth_method'=>"01",
    'prior_request_auth_timestamp'=>"201710282113"
);
//$mpiThreeDSAuthentication->setPriorAuthenticationInfo( $paiTemplate );
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions
//print_r($mpgRequest);
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nMessageType = " . $mpgResponse->getMpiMessageType());
print("\nTransStatus = " . $mpgResponse->getMpiTransStatus());
print("\nTransStatusReason = " . $mpgResponse->getMpiTransStatusReason());
print("\nChallengeURL = " . $mpgResponse->getMpiChallengeURL());
print("\nChallengeData = " . $mpgResponse->getMpiChallengeData());
print("\nThreeDSServerTransId = " . $mpgResponse->getMpiThreeDSServerTransId());
print("\nDSTransId = " . $mpgResponse->getMpiDSTransId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nThreeDSAcTransID = " . $mpgResponse->getMpiThreeDSAcTransID());
print("\nThreeDSAuthTimeStamp = " . $mpgResponse->getMpiThreeDSAuthTimeStamp());
print("\nAuthenticationType = " . $mpgResponse->getMpiAuthenticationType());
print("\nCardHolderInfo = " . $mpgResponse->getMpiCardholderInfo());
//In Frictionless flow, you may receive TransStatus as "Y",
//in which case you can then proceed directly to Cavv Purchase/Preauth with values below'
if($mpgResponse->getMpiTransStatus() == "Y")
{
    print("\nCavv = " . $mpgResponse->getMpiCavv());
    print("\nECI = " . $mpgResponse->getMpiEci());
}
?>

```

6.5.3 MPI 3DS Authentication Request - 3RI, non-recurring

NOTE: Billing address request fields are recommended to be sent for this transaction, or else the authentication process may fail

MPI 3DS Authentication Request transaction object definition

```
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
```

HttpPostRequest object for MPI 3DS Authentication Request transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
```

WARNING: Do not send fields related to 3RI on browser-based authentications.

Core connection object fields (all API transactions)

Variable Name	Type and Limits	Description
store ID	<i>String</i>	'store_id'=>\$store_id
<store_id>	N/A	Unique identifier provided by Moneris upon merchant account setup
API token	<i>String</i>	'api_token'=>\$api_token
<api_token>	N/A	Unique alphanumeric string assigned by Moneris upon merchant account activation
		To find your API token, refer to your test or production store's Admin settings in the Merchant Resource Center, at the following URLs:
		Testing: https://esqa.moneris.com/mpg/
		Production: https://www3.moneris.com/mpg/

MPI 3DS Authentication Request transaction request fields – Required

Variable Name	Type and Limits	Description
message category <MessageCategory>	<i>String</i> 2-character numeric	<pre>\$mpiThreeDSAuthentication->setMessageCategory("02");</pre> Whether the authentication request is for a payment or non-payment use: 01 = payment authentication (PA) 02 = non-payment authentication (NPA)
device channel <DeviceChannel>	<i>String</i> 2-character numeric	<pre>\$mpiThreeDSAuthentication->setDeviceChannel("02");</pre> The interface used to initiate the authentication: 02 = Browser (BRW) 03 = 3DS Requestor Initiated (3RI)
ri indicator <RiIndicator>	<i>String</i> 2-character numeric	<pre>\$mpiThreeDSAuthentication->setRiIndicator("03");</pre> The type of 3DS Requestor Initiated (3RI) request: 01 = Recurring 02 = Installment 03 = Add Card 04 = Maintain Card Information 05 = Account verification 06 = Split/Delayed Shipment 07 = Top-up 08 = Mail Order 09 = Telephone Order 10 = Whitelist

NOTE: Visa Secure only support ri_Indicator = 01, 02, 06, 07, or 11 for Payment Transactions and ri Indicator = 03, 04, 05 and 10 for Non Payment Transactions

Variable Name	Type and Limits	Description
		11 = Other Payment
order ID <order_id>	String 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id; Merchant-defined transaction identifier that must be unique for every Purchase, Pre-Authorization and Independent Refund transaction. No two transactions of these types may have the same order ID. For Refund, Completion and Purchase Correction transactions, the order ID must be the same as that of the original transaction.
data key <data_key> OR credit card number <pan>	String data key limits: 25-character alphanumeric credit card number limits: max 20-character alphanumeric	'data_key'=>\$data_key; data key description: Unique identifier for a Vault profile, and used in future Vault financial transactions to associate a transaction with that profile Data key is generated by Moneris and returned to you in the Receipt object when the profile is first registered 'pan'=>\$pan credit card number description: Credit card number, usually 16 digits —field can be maximum 20 digits in support of future expansion of card number ranges. Carries the token for network tokenization transactions.
expiry date <expdate>	String 4-character alphanumeric	'expiry_date'=>\$expiry_date Expiry date of the credit card, in YYMM format.

Variable Name	Type and Limits	Description
	YYMM	NOTE: This is the reverse of the MMYM date format that is presented on the card.
amount <amount>	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount' => \$amount Transaction dollar amount This must contain at least 3 digits, two of which are penny values Minimum allowable value = \$0.01, maximum allowable value = \$9999999.99
cardholder name <CardholderName>	<i>String</i> 45-character alphanumeric NOTE: Accented characters are not allowable	<pre>\$mpiThreeDSAuthentication->setCardholderName("CARDHOLDER_NAME_VALUE");</pre> Name of the cardholder
billing address <BillAddress1>	<i>String</i> 50-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBillAddress1("BILL_STREET_ADDRESS_VALUE");</pre> Cardholder billing address
billing province <BillProvince>	<i>String</i> 3-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBillProvince("BILL_PROV_VALUE");</pre> Cardholder province or state Defined in country subdivision ISO 3166-2
billing city <BillCity>	<i>String</i> 50-character alphanumeric	<pre>\$mpiThreeDSAuthentication->setBillCity("BILL_CITY_VALUE");</pre> Cardholder billing city
billing postal code	<i>String</i>	<pre>\$mpiThreeDSAuthentication->setBillPostalCode("BILL_</pre>

Variable Name	Type and Limits	Description
<BillPostalCode>	16-character alphanumeric	POSTAL_CODE_VALUE"); Cardholder billing postal code
billing country <BillCountry>	String 3-character alphanumeric	\$mpiThreeDSAuthentication->setBillCountry ("BILL_COUNTRY_VALUE"); Cardholder billing country Defined as 3 digit country code ISO 3166-1
shipping address <ShipAddress1>	String 50-character alphanumeric	\$mpiThreeDSAuthentication->setShipAddress1 ("SHIP_STREET_ADDRESS_VALUE"); Shipping destination address
shipping province <ShipProvince>	String 3-character alphanumeric	\$mpiThreeDSAuthentication->setShipProvince ("SHIP_PROV_VALUE"); Shipping destination province or state Defined in country subdivision ISO 3166-2
shipping city <ShipCity>	String 50-character alphanumeric	\$mpiThreeDSAuthentication->setShipCity ("SHIP_CITY_VALUE"); Shipping destination city
shipping postal code <ShipPostalCode>	String 16-character alphanumeric	\$mpiThreeDSAuthentication->setShipPostalCode ("SHIP_POSTAL_CODE_VALUE"); Shipping destination postal or ZIP code
shipping country <ShipCountry>	String 3-character alphanumeric	\$mpiThreeDSAuthentication->setShipCountry ("SHIP_COUNTRY_VALUE"); Shipping destination country Defined as 3-digit country code in ISO 3166-1

Variable Name	Type and Limits	Description
email <Email>	<i>String</i> 254-character alpha-numeric	<pre>\$mpiThreeDSAuthentication->setEmail("EMAIL_VALUE");</pre> Cardholder email address NOTE: This field is not mandatory, but it is required. It is highly recommended to provide the cardholder's email address. Lack of providing the cardholder's address, might increase the risk of rejects.

MPI 3DS Authentication Request transaction request fields – Optional

Variable Name	Type and Limits	Description
currency <currency>	<i>String</i> 3-character numeric	<pre>\$mpiThreeDSAuthentication->SetCurrency("CURRENCY_VALUE");</pre> ISO 4217 3 digit currency code CAD = 124 USD = 840 NOTE: This field should not be sent unless Multi Currency Pricing is enabled on your merchant account
decoupled request indicator <DecoupledRequestIndicator>	<i>String</i> 1-character alphabetic	<pre>\$mpiThreeDSAuthentication->setDecoupledRequestIndicator("Y");</pre> Whether the request utilizes Decoupled Authentication or not, if the ACS confirms its use. Y = Decoupled Authentication is supported and preferred if challenge is necessary N = Do not use Decoupled Authentication (Default) Defaults to N if unused.
decoupled request max time <DecoupledRequestMaxTime>	<i>String</i> 5-character numeric	<pre>\$mpiThreeDSAuthentication->setDecoupledRequestMaxTime("00010");</pre> The maximum minutes that Moneris waits for an ACS to provide results. Numeric values between 1 and 10080. The max is

Variable Name	Type and Limits	Description
		equivalent to 7 days. Conditional. Required if device_channel = 03 and decoupled_request_indicator = Y
decoupled request async URL <DecoupledRequestAsyncUrl>	String 256-character alphanumeric	<code>\$mpiThreeDSAuthentication->setDecoupledRequestAsyncUrl("https://yourasyncnotificationurl.com");</code> Your URL where Moneris will POST the response back from ACS. Moneris reattempts 3 times to POST the response. Conditional. Only sent if decoupled_request_indicator = Y
prior request auth info <PriorAuthenticationInfo>	Object N/A	<code>\$mpiThreeDSAuthentication->setPriorRequestAuthInfo(\$paiTemplate)</code> Object containing details for a prior 3DS authentication for this series of transactions. This is a nested object within the authentication transaction, and required when storing or using the information about the prior authentication for that card. For information about fields in the Prior Authentication Info object, see MPI 3DS Prior Authentication Info Object and Variables.

MPI 3DS Prior Authentication Info

Variable Name	Type and Limits	Description
prior request auth data <prior_request_auth_data>	String 36-character alphanumeric	'prior_request_auth_data'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340" Refers to the DSTransID in the response of the previous 3DS authentication.
prior request ref <prior_request_auth_ref>	String 36-character alphanumeric	'prior_request_ref'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340" Refers to the 3DS ACS Transaction ID in the response of the previous 3DS authentication.
prior request auth method	String 2-character numeric	'prior_request_auth_method'=>"01", Mechanism used by the cardholder to authen-

Variable Name	Type and Limits	Description
<prior_request_auth_method>		authenticate in the previous 3DS authentication: 01 = Frictionless authentication 02 = Challenge authentication 03 = AVS verified 04 = Other issuer methods
prior request auth timestamp	String 12-character numeric	'prior_request_auth_timestamp'=>"201710282113",
<prior_request_auth_timestamp>		Date and time in UTC of the prior cardholder authentication. Found in the previous 3DS authentication response as 3DS Auth TimeStamp. Format is YYYYMMDDHHMM.

Sample MPI 3DS Authentication Request - 3RI without recurring

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = "moneris";
$api_token = "hurgle";
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
$mpiThreeDSAuthentication->setOrderId("ord-110920-10:36:41"); //must be the same one that was
used in MpiCardLookup call
$mpiThreeDSAuthentication->setCardholderName("Moneris Test");
$mpiThreeDSAuthentication->setPan("347668693641199");
//$mpiThreeDSAuthentication->setDataKey("800XGiwgfvfbZngigVFeld9d2"); //Optional - For
Moneris Vault and Hosted Tokenization tokens in place of setPan
$mpiThreeDSAuthentication->setExpdate("2310");
$mpiThreeDSAuthentication->setAmount("1.00");
$mpiThreeDSAuthentication->setThreeDSCompletionInd("Y"); //(Y|N|U) indicates whether 3ds
method MpiCardLookup was successfully completed
$mpiThreeDSAuthentication->setRequestType("01"); //(01=payment|02=recur)
$mpiThreeDSAuthentication->setPurchaseDate("20200911035249"); //(YYYYMMDDHHMMSS)
$mpiThreeDSAuthentication->setNotificationURL("https://yournotificationurl.com"); //(Website
where response from RRes or CRes after challenge will go)
$mpiThreeDSAuthentication->setChallengeWindowSize("03"); //(01 = 250 x 400, 02 = 390 x 400,
03 = 500 x 600, 04 = 600 x 400, 05 = Full screen)
$mpiThreeDSAuthentication->setBrowserUserAgent("Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.132 Safari/537.36\\");
$mpiThreeDSAuthentication->setBrowserJavaEnabled("true"); //(true|false)
$mpiThreeDSAuthentication->setBrowserScreenHeight("1000"); //(pixel height of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserScreenWidth("1920"); //(pixel width of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserLanguage("en-GB"); //(defined by IETF BCP47)
//Optional Methods
$mpiThreeDSAuthentication->setBillAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setBillProvince("ON");
$mpiThreeDSAuthentication->setBillCity("Toronto");
$mpiThreeDSAuthentication->setBillPostalCode("M8X 2X2");
```

```

$mpiThreeDSAuthentication->setBillCountry("124");
$mpiThreeDSAuthentication->setShipAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setShipProvince("ON");
$mpiThreeDSAuthentication->setShipCity("Toronto");
$mpiThreeDSAuthentication->setShipPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setShipCountry("124");
$mpiThreeDSAuthentication->setEmail("test@email.com");
$mpiThreeDSAuthentication->setRequestChallenge("Y"); // (Y|N Requesting challenge regardless
of outcome)
$mpiThreeDSAuthentication->setMessageCategory("01");
$mpiThreeDSAuthentication->setDeviceChannel("03");
// $mpiThreeDSAuthentication->setDecoupledRequestIndicator("N");
// $mpiThreeDSAuthentication->setDecoupledRequestMaxTime("00010");
// $mpiThreeDSAuthentication->setDecoupledRequestAsyncUrl("localhost");
$mpiThreeDSAuthentication->setRiIndicator("01");
// $mpiThreeDSAuthentication->setRecurringFrequency("031");
// $mpiThreeDSAuthentication->setRecurringExpiry("20251230");
$paiTemplate = array(
    'prior_request_auth_data'=>"d7clee99-9478-44a6-blf2-391e29c6b340",
    'prior_request_ref'=>"d7clee99-9478-44a6-blf2-391e29c6b340",
    'prior_request_auth_method'=>"01",
    'prior_request_auth_timestamp'=>"201710282113"
);
$mpiThreeDSAuthentication->setPriorAuthenticationInfo( $paiTemplate );
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false or comment out this line for production transactions
// print_r($mpgRequest);
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nMessageType = " . $mpgResponse->getMpiMessageType());
print("\nTransStatus = " . $mpgResponse->getMpiTransStatus());
print("\nTransStatusReason = " . $mpgResponse->getMpiTransStatusReason());
print("\nChallengeURL = " . $mpgResponse->getMpiChallengeURL());
print("\nChallengeData = " . $mpgResponse->getMpiChallengeData());
print("\nThreeDSServerTransId = " . $mpgResponse->getMpiThreeDSServerTransId());
print("\nDSTransId = " . $mpgResponse->getMpiDSTransId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nThreeDSAcSTransID = " . $mpgResponse->getMpiThreeDSAcSTransID());
print("\nThreeDSAuthTimeStamp = " . $mpgResponse->getMpiThreeDSAuthTimeStamp());
print("\nAuthenticationType = " . $mpgResponse->getMpiAuthenticationType());
print("\nCardHolderInfo = " . $mpgResponse->getMpiCardholderInfo());
// In Frictionless flow, you may receive TransStatus as "Y",
// in which case you can then proceed directly to Cavv Purchase/Preauth with values below'
if($mpgResponse->getMpiTransStatus() == "Y")
{
    print("\nCavv = " . $mpgResponse->getMpiCavv());
    print("\nECI = " . $mpgResponse->getMpiEci());
}
?>

```

6.6 Handling the Challenge Flow

If you get a TransStatus = “C” in your threeDSAuthentication Response, then a form must be built and POSTed to the URL provided.

The form can be dynamically generated and added to the DOM and submitted or created and submitted in a manner that suits your environment. This can be built as a full page redirect or presented as an inline iframe or as a lightbox.

If you wish for this to be loaded inside a defined space it must conform to the size specified in the ChallengeWindowSize from the request. The “action” is retrieved from the ChallengeURL and the “creq” field is retrieved from the ChallengeData.

Below is a sample of a basic static form to help visualize the data and fields that need to be submitted.

```
<form method="POST" action="https://3dsurl.example.com/do3DS">
<input name="creq" value="thisissamplechallengedata1234567890">
</form>
```

6.6.1 Cavv Lookup Request – mpiCavvLookup

(Challenge Flow Only)

In the challenge flow, the 3DS server will POST a **cres** value back to the notificationURL provided in the threeDSAuthentication request once the cardholder has completed the challenge. The “cres” is then posted to the Moneris 3DS server in the CavvLookup request, the response to this request will include the result of the challenge, which will include the eci and the cavv if the challenge was successful.

Cavv Lookup Request transaction object definition

```
$mpiCavvLookup = new MpiCavvLookup();
```

HttpPostRequest object for Cavv Lookup Request transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgTxn = new mpgTransaction($mpiCavvLookup);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Cavv Lookup Request transaction request fields – Required

Variable Name	Type and Limits	
cres	String 200-character alpha-numeric	\$mpiCavvLookup->setCRes(\$cres);

Sample Cavv Lookup Request

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = "moneris";
$api_token = "hurgle";
//BASE64 Encoded CRes value returned from response at completion of challenge flow.
$cres =
"eyJhY3NUcmFuc0lEiJoInzQ0ZDI2NjUtNjU2Yy00ZGNIbGt3MWUtYTBkYmMwODA0OTYzIiwibWVzc2FnZVR5cGUiOiJD
UmVzIiwiaWY2hhbGxlbmdlQ29tcGxldGlvbkluZCI6IlkiLCJtZXNzYWdlVmVyc2lubiI6IjIuMS4wIiwidHJhbnNTdGF0d
XMiOiJZiwiZGhyZWVEU1NlcnZlclRyYW5zSUQiOiJlMTFkNDk4NS04ZDI1LTQwZWQwOTlkNiJmZgM2ZlNWU2OGYifQ
==";

$mpiCavvLookup = new MpiCavvLookup();
$mpiCavvLookup->setCRes($cres);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($mpiCavvLookup);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false or comment out this line for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nMessageType = " . $mpgResponse->getMpiMessageType());
print("\nThreeDSServerTransId = " . $mpgResponse->getMpiThreeDSServerTransId());
print("\nTransStatus = " . $mpgResponse->getMpiTransStatus());
print("\nChallengeCompletionIndicator = " . $mpgResponse->getMpiChallengeCompletionIndicator());
print("\nCavv = " . $mpgResponse->getMpiCavv());
print("\nECI = " . $mpgResponse->getMpiEci());
?>
```

6.7 Handling the Decoupled Authentication Flow

If you get a TransStatus = "D" in your threeDSAuthentication Response, then your server must be prepared to accept a second asynchronous response from Moneris.

The cardholder will be engaged by their issuer for cardholder authentication outside the 3DS protocol. This may involve alternate authentication applications or SMS prompts to the cardholder to confirm.

The cardholder is given up to 7 days to complete this decoupled challenge. Once completed, the issuer will communicate to Moneris and our MPI system sends a second 3DS Authentication Response to the address you define in <decoupled_request_async_url>

Sample Authentication Request – 3RI With Decoupled Authentication for MOTO

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = "moneris";
$api_token = "hurgle";
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
$mpiThreeDSAuthentication->setOrderId("ord-110920-10:36:41"); //must be the same one that was
used in MpiCardLookup call
$mpiThreeDSAuthentication->setCardholderName("Moneris Test");
$mpiThreeDSAuthentication->setPan("347668693641199");
//$mpiThreeDSAuthentication->setDataKey("800XGiwgvgfbZngigVFeld9d2"); //Optional - For
Moneris Vault and Hosted Tokenization tokens in place of setPan
$mpiThreeDSAuthentication->setExpdate("2310");
$mpiThreeDSAuthentication->setAmount("1.00");
$mpiThreeDSAuthentication->setThreeDSCompletionInd("Y"); // (Y|N|U) indicates whether 3ds
method MpiCardLookup was successfully completed
$mpiThreeDSAuthentication->setRequestType("01"); // (01=payment|02=recur)
$mpiThreeDSAuthentication->setPurchaseDate("20200911035249"); // (YYYYMMDDHHMMSS)
$mpiThreeDSAuthentication->setNotificationURL("https://yournotificationurl.com"); // (Website
where response from RRes or CRes after challenge will go)
$mpiThreeDSAuthentication->setChallengeWindowSize("03"); // (01 = 250 x 400, 02 = 390 x 400,
03 = 500 x 600, 04 = 600 x 400, 05 = Full screen)
$mpiThreeDSAuthentication->setBrowserUserAgent("Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.132 Safari/537.36\\");
$mpiThreeDSAuthentication->setBrowserJavaEnabled("true"); // (true|false)
$mpiThreeDSAuthentication->setBrowserScreenHeight("1000"); // (pixel height of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserScreenWidth("1920"); // (pixel width of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserLanguage("en-GB"); // (defined by IETF BCP47)
//Optional Methods
$mpiThreeDSAuthentication->setBillAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setBillProvince("ON");
$mpiThreeDSAuthentication->setBillCity("Toronto");
$mpiThreeDSAuthentication->setBillPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setBillCountry("124");
$mpiThreeDSAuthentication->setShipAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setShipProvince("ON");
$mpiThreeDSAuthentication->setShipCity("Toronto");
$mpiThreeDSAuthentication->setShipPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setShipCountry("124");
$mpiThreeDSAuthentication->setEmail("test@email.com");
$mpiThreeDSAuthentication->setRequestChallenge("Y"); // (Y|N Requesting challenge regardless
of outcome)
$mpiThreeDSAuthentication->setMessageCategory("01");
$mpiThreeDSAuthentication->setDeviceChannel("03");
$mpiThreeDSAuthentication->setDecoupledRequestIndicator("N");
$mpiThreeDSAuthentication->setDecoupledRequestMaxTime("00010");
$mpiThreeDSAuthentication->setDecoupledRequestAsyncUrl("localhost");
$mpiThreeDSAuthentication->setRiIndicator("08");
// $mpiThreeDSAuthentication->setRecurringFrequency("031");
// $mpiThreeDSAuthentication->setRecurringExpiry("20251230");
$paiTemplate = array(
    'prior_request_auth_data'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340",
    'prior_request_ref'=>"d7c1ee99-9478-44a6-b1f2-391e29c6b340",
    'prior_request_auth_method'=>"01",
    'prior_request_auth_timestamp'=>"201710282113"
);
```

```
// $mpiThreeDSAuthentication->setPriorAuthenticationInfo( $paiTemplate );
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false or comment out this line for production transactions
// print_r($mpgRequest);
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nMessageType = " . $mpgResponse->getMpiMessageType());
print("\nTransStatus = " . $mpgResponse->getMpiTransStatus());
print("\nTransStatusReason = " . $mpgResponse->getMpiTransStatusReason());
print("\nChallengeURL = " . $mpgResponse->getMpiChallengeURL());
print("\nChallengeData = " . $mpgResponse->getMpiChallengeData());
print("\nThreeDSServerTransId = " . $mpgResponse->getMpiThreeDSServerTransId());
print("\nDSTransId = " . $mpgResponse->getMpiDSTransId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nThreeDSAcTransID = " . $mpgResponse->getMpiThreeDSAcTransID());
print("\nThreeDSAuthTimeStamp = " . $mpgResponse->getMpiThreeDSAuthTimeStamp());
print("\nAuthenticationType = " . $mpgResponse->getMpiAuthenticationType());
print("\nCardHolderInfo = " . $mpgResponse->getMpiCardholderInfo());
// In Frictionless flow, you may receive TransStatus as "Y",
// in which case you can then proceed directly to Cavv Purchase/Preauth with values below
if ($mpgResponse->getMpiTransStatus() == "Y")
{
    print("\nCavv = " . $mpgResponse->getMpiCavv());
    print("\nECI = " . $mpgResponse->getMpiEci());
}
?>
```

Sample Authentication Request – 3RI With Prior Authentication and Decoupled Authentication

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = "moneris";
$api_token = "hurgle";
$mpiThreeDSAuthentication = new MpiThreeDSAuthentication();
$mpiThreeDSAuthentication->setOrderId("ord-110920-10:36:41"); // must be the same one that was
used in MpiCardLookup call
$mpiThreeDSAuthentication->setCardholderName("Moneris Test");
$mpiThreeDSAuthentication->setPan("347668693641199");
// $mpiThreeDSAuthentication->setDataKey("800XGiwXgvfbZngigVFeld9d2"); // Optional - For
Moneris Vault and Hosted Tokenization tokens in place of setPan
$mpiThreeDSAuthentication->setExpdate("2310");
$mpiThreeDSAuthentication->setAmount("1.00");
$mpiThreeDSAuthentication->setThreeDSCompletionInd("Y"); // (Y|N|U) indicates whether 3ds
method MpiCardLookup was successfully completed
$mpiThreeDSAuthentication->setRequestType("01"); // (01=payment|02=recur)
$mpiThreeDSAuthentication->setPurchaseDate("20200911035249"); // (YYYYMMDDHHMMSS)
$mpiThreeDSAuthentication->setNotificationURL("https://yournotificationurl.com"); // (Website
where response from RRes or CRes after challenge will go)
$mpiThreeDSAuthentication->setChallengeWindowSize("03"); // (01 = 250 x 400, 02 = 390 x 400,
03 = 500 x 600, 04 = 600 x 400, 05 = Full screen)
$mpiThreeDSAuthentication->setBrowserUserAgent("Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.132 Safari/537.36");
$mpiThreeDSAuthentication->setBrowserJavaEnabled("true"); // (true|false)
```

```

$mpiThreeDSAuthentication->setBrowserScreenHeight("1000"); //(pixel height of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserScreenWidth("1920"); //(pixel width of cardholder
screen)
$mpiThreeDSAuthentication->setBrowserLanguage("en-GB"); //(defined by IETF BCP47)
//Optional Methods
$mpiThreeDSAuthentication->setBillAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setBillProvince("ON");
$mpiThreeDSAuthentication->setBillCity("Toronto");
$mpiThreeDSAuthentication->setBillPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setBillCountry("124");
$mpiThreeDSAuthentication->setShipAddress1("3300 Bloor St W");
$mpiThreeDSAuthentication->setShipProvince("ON");
$mpiThreeDSAuthentication->setShipCity("Toronto");
$mpiThreeDSAuthentication->setShipPostalCode("M8X 2X2");
$mpiThreeDSAuthentication->setShipCountry("124");
$mpiThreeDSAuthentication->setEmail("test@email.com");
$mpiThreeDSAuthentication->setRequestChallenge("Y"); //(Y|N Requesting challenge regardless
of outcome)
$mpiThreeDSAuthentication->setMessageCategory("01");
$mpiThreeDSAuthentication->setDeviceChannel("03");
$mpiThreeDSAuthentication->setDecoupledRequestIndicator("N");
$mpiThreeDSAuthentication->setDecoupledRequestMaxTime("00010");
$mpiThreeDSAuthentication->setDecoupledRequestAsyncUrl("localhost");
$mpiThreeDSAuthentication->setRiIndicator("08");
// $mpiThreeDSAuthentication->setRecurringFrequency("031");
// $mpiThreeDSAuthentication->setRecurringExpiry("20251230");
$paiTemplate = array(
    'prior_request_auth_data'=>"d7clee99-9478-44a6-b1f2-391e29c6b340",
    'prior_request_ref'=>"d7clee99-9478-44a6-b1f2-391e29c6b340",
    'prior_request_auth_method'=>"01",
    'prior_request_auth_timestamp'=>"201710282113"
);
$mpiThreeDSAuthentication->setPriorAuthenticationInfo( $paiTemplate );
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($mpiThreeDSAuthentication);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions
//print_r($mpgRequest);
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nMessageType = " . $mpgResponse->getMpiMessageType());
print("\nTransStatus = " . $mpgResponse->getMpiTransStatus());
print("\nTransStatusReason = " . $mpgResponse->getMpiTransStatusReason());
print("\nChallengeURL = " . $mpgResponse->getMpiChallengeURL());
print("\nChallengeData = " . $mpgResponse->getMpiChallengeData());
print("\nThreeDSServerTransId = " . $mpgResponse->getMpiThreeDSServerTransId());
print("\nDSTransId = " . $mpgResponse->getMpiDSTransId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nThreeDSAcTransID = " . $mpgResponse->getMpiThreeDSAcTransID());
print("\nThreeDSAuthTimeStamp = " . $mpgResponse->getMpiThreeDSAuthTimeStamp());
print("\nAuthenticationType = " . $mpgResponse->getMpiAuthenticationType());
print("\nCardHolderInfo = " . $mpgResponse->getMpiCardholderInfo());
//In Frictionless flow, you may receive TransStatus as "Y",
//in which case you can then proceed directly to Cavv Purchase/Preauth with values below'
if($mpgResponse->getMpiTransStatus() == "Y")
{
    print("\nCavv = " . $mpgResponse->getMpiCavv());
    print("\nECI = " . $mpgResponse->getMpiEci());
}

```

```
}  
?>
```

6.8 Performing the Authorization

Once the authentication is complete and a CAVV and ECI value are retrieved, these values can be sent to Moneris using the transactions Purchase with 3-D Secure – cavv_purchase or Pre-Authorization with 3-D Secure – cavv_preauth.

6.8.1 Purchase with 3-D Secure – cavv_purchase

The Purchase with 3-D Secure transaction follows a 3-D Secure MPI authentication. After receiving confirmation from the MPI ACS transaction, this Purchase verifies funds on the customer’s card, removes the funds and prepares them for deposit into the merchant’s account.

To perform the 3-D Secure authentication, the Moneris MPI or any third-party MPI may be used.

In addition to 3-D Secure transactions, this transaction can also be used to process Apple Pay and Google Pay™ transactions. This transaction is applicable only if choosing to integrate directly to Apple Wallet or Google Wallet (if not using the Moneris Apple Pay or Google Pay™ SDKs).

Refer to Apple or Google developer portals for details on integrating directly to their wallets to retrieve the payload data.

WARNING: Moneris strongly discourages the use of frames as part of a 3-D Secure implementation, and cannot guarantee their reliability when processing transactions in the production environment.

```
$txnArray = array('type'=>'cavv_purchase', ...);  
$mpgTxn = new mpgTransaction($txnArray);  
$mpgRequest = new mpgRequest($mpgTxn);  
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String	'api_token'=>\$api_token

Variable Name	Type and Limits	
	N/A	

Purchase with 3-D Secure transaction request fields – Required

Variable Name	Type and Limits	
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authorization transactions, CAVV field contains the decrypted cryptogram. For more, see Appendix A Definition of Request Fields.	cavv=>\$cavv

Variable Name	Type and Limits	
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authentication transactions, the e-commerce indicator is a mandatory field containing the value received from the decrypted payload or a default value of 5. If you get a 2-character value (e.g., 05 or 07) from the payload, remove the initial 0 and just send us the 2nd character. For more, see Appendix A Definition of Request Fields.		

3-D Secure 2.2 -specific fields – Required

Variable Name	Type and Limits	
3DS version	<i>String</i> 10-character numeric	'threeds_version'=>\$threeds_version Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services.		
3DS server transaction ID	<i>String</i> 36-character numeric	'threeds_server_trans_id'=>\$threeds_server_trans_id
NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services - obtained from the Cavv Lookup request or MPI 3DS Authentication request		

Following fields are required for Apple Pay and Google Pay only:

Variable Name	Type and Limits	
network	<i>String</i> alphanumeric	
data type	<i>String</i> 3-character alphanumeric	

Purchase with 3-D Secure transaction request fields – Optional

Variable Name	Type and Limits	
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
Surcharge Information	<i>Object</i> N/A NOTE: This object requires use of the Vault Surcharge Lookup transaction prior to confirm card eligibility.	\$mpgTxn->setSurchargeInfo(\$surchargeInfo); Contains fields related to Surcharge feature.
foreign indicator	<i>Boolean</i> true or false	'foreign_indicator'=>\$foreign_indicator

3-D Secure 2.2 -specific fields – Optional

Variable Name	Type and Limits	
DS transaction ID	<i>String</i> 36-character alphanumeric	'ds_trans_id' => \$ds_trans_id
NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.		

Sample Purchase with 3-D Secure – cavv_purchase

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
/***** Transactional Variables *****/
$type='cavv_purchase';
$order_id='ord-'.date("dmy-G:i:s");
$cust_id='CUST887763';
$amount='6000.00';
$pan='4622943127023886';
$expiry_date='2212';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA=';
$crypt_type = '7';
$wallet_indicator = "APP";
$dynamic_descriptor='123456';
$foreign_indicator='true';

// TrId and TokenCryptogram are optional, refer documentation for more details.
$tr_id = '50189815682';
$token_cryptogram = 'APmbM/41le0uAAH+s6xMAAADFA==';
'foreign_indicator'=>$foreign_indicator
/***** Transaction Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$pan,
    'expdate'=>$expiry_date,
    'cavv'=>$cavv,
    'crypt_type'=>$crypt_type, //mandatory for AMEX only
    //'wallet_indicator'=>$wallet_indicator, //set only for wallet transactions. e.g. APPLE PAY
    //'network'=> "Interac", //set only for Interac e-commerce
    //'data_type'=> "3DSecure", //set only for Interac e-commerce
    'dynamic_descriptor'=>$dynamic_descriptor,
    'threds_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
    Secure services
    'threds_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f' Mandatory for financial
    transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
    MpiThreeDSAuthentication
    //,'cm_id' => '8nAK8712sGaAkls56' //set only for usage with Offlinx - Unique max 50
    alphanumeric characters transaction id generated by merchant
    //,'ds_trans_id' => '12345' //Optional - to be used only if you are using 3rd party 3ds
    service
    //,'tr_id' => $tr_id
```



```

//,'token_cryptogram' => $token_cryptogram
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
// $mpgTxn->setInstallmentInfo($installmentInfo);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nAdviceCode = " . $mpgResponse->getAdviceCode());

// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());
?>

```

6.8.2 Pre-Authorization with 3-D Secure – cavv_preauth

The Pre-Authorization with 3-D Secure transaction follows a 3-D Secure MPI authentication. After receiving confirmation from the MPI ACS Request transaction, this Pre-Authorization verifies funds on the customer's card, removes the funds and prepares them for deposit into the merchant's account.

To perform the 3-D Secure authentication, the Moneris MPI or any third-party MPI may be used.

In addition to 3-D Secure transactions, this transaction can also be used to process Apple Pay and Google Pay™ transactions. This transaction is applicable only if choosing to integrate directly to Apple Wallet or Google Wallet (if not using the Moneris Apple Pay or Google Pay™ SDKs).

Refer to Apple or Google developer portals for details on integrating directly to their wallets to retrieve the payload data.

WARNING: Moneris strongly discourages the use of frames as part of a 3-D Secure implementation, and cannot guarantee their reliability when processing transactions in the production environment.

```
$txnArray = array('type'=>'cavv_preauth', ...);
$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

Pre-Authorization with 3-D Secure transaction request fields – Required

Variable Name	Type and Limits	
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits	'amount'=>\$amount

Variable Name	Type and Limits	
	(cents) after the decimal point	
	EXAMPLE: 1234567.89	
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authorization transactions, CAVV field contains the decrypted cryptogram. For more, see Appendix A Definition of Request Fields.		
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authorization transactions, the e-commerce indicator is a mandatory field containing the value received from the decrypted payload or a default value of 5. If you get a 2-character value (e.g., 05 or 07) from the payload, remove the initial 0 and just send us the 2nd character. For more, see Appendix A Definition of		

Variable Name	Type and Limits	
Request Fields.		

3-D Secure 2.2 -specific fields – Required

Variable Name	Type and Limits	
3DS version NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services.	<i>String</i> 10-character numeric	'threeds_version'=>\$threeds_version Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
3DS server transaction ID NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services - obtained from the Cavv Lookup request or MPI 3DS Authentication request	<i>String</i> 36-character numeric	'threeds_server_trans_id'=>\$threeds_server_trans_id

Pre-Authorization with 3-D Secure transaction request fields – Optional

Variable Name	Type and Limits	
status check	<i>Boolean</i> true/false	<pre>\$mpgHttpPost =new mpgHttpsPostStatus(\$store_ id,\$api_ token,\$status,\$mpgRequest);</pre>
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id

Variable Name	Type and Limits	
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
is incremental	<i>Boolean</i>	'is_incremental'=>\$is_incremental
is_incremental	true/false	Indicates if this preauthorization is using an estimated amount. Estimations allow for incrementing the amount held via subsequent incrementalAuth requests. Defaults to false. NOTE: Please note that if this field is true, the preauthorization is only eligible for a single Preauthorization Completion. Any completion sent for partial completion is treated as a full completion (ship_indicator= P is treated as = F when is_incremental= true on the original preauth)
Surcharge Information	<i>Object</i> N/A NOTE: This object requires use of the Vault Surcharge Lookup transaction prior to confirm card eligibility.	\$mpgTxn->setSurchargeInfo(\$surchargeInfo); Contains fields related to Surcharge feature.
foreign indicator	<i>Boolean</i> true or false	'foreign_indicator'=>\$foreign_indicator
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);

Variable Name	Type and Limits	
CVD Information	Object	\$mpgTxn->setCvdInfo (\$mpgCvdInfo);
NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only— merchants must not store CVD information.	N/A	
final authorization	String	'final_auth' => 'true'
NOTE: Applies to Mastercard transactions only	true/false	

3-D Secure 2.2 -specific fields – Optional

Variable Name	Type and Limits	
DS transaction ID	String	'ds_trans_id' => \$ds_trans_id
NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.	36-character alphanumeric	

Sample Pre-Authorization with 3-D Secure – cavv_preauth

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
/***** Transactional Variables *****/
$type='cavv_preauth';
$order_id='ord-' . date("dmy-G:i:s");
$cust_id='CUST887763';
$amount='4840.00';
$span='5454545454545454';
$expiry_date='2212';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA=';
$cript_type = '7';
$wallet_indicator = "APP";
$dynamic_descriptor='123456';
// TrId and TokenCryptogram are optional, refer documentation for more details.
$tr_id = '50189815682';
$token_cryptogram = 'APmbM/41le0uAAH+s6xMAAADFA==';
/***** Transaction Associative Array *****/

```

```

$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$pan,
    'expdate'=>$expiry_date,
    'cavv'=>$cavv,
    'crypt_type'=>$crypt_type, //mandatory for AMEX only
    //'wallet_indicator'=>$wallet_indicator, //set only for wallet transactions. e.g. APPLE PAY
    //'network'=> "Interac", //set only for Interac e-commerce
    //'data_type'=> "3DSecure", //set only for Interac e-commerce
    'dynamic_descriptor'=>$dynamic_descriptor,
    'threeds_version' => '2', //Mandatory for 3DS Version 2.0+
    'threeds_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f' //Mandatory for 3DS
    Version 2.0+ - obtained from MpiCavvLookup or MpiThreeDSAuthentication
    //'cm_id' => '8nAK8712sGaAkls56' //set only for usage with Offlinx - Unique max 50
    alphanumeric characters transaction id generated by merchant
    //'ds_trans_id' => '12345' //Optional - to be used only if you are using 3rd party 3ds 2.0
    service
    //'tr_id' => $tr_id
    //'token_cryptogram' => $token_cryptogram
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //US for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nAdviceCode = " . $mpgResponse->getAdviceCode());
?>

```

6.8.3 Purchase with Vault and 3-D Secure

NOTE: This transaction supports both temporary and permanent tokens.

WARNING: Moneris strongly discourages the use of frames as part of a 3-D Secure implementation, and cannot guarantee their reliability when processing transactions in the production environment.

```
$txnArray = array('type'=>'res_cavv_purchase_cc', ...);  
$mpgTxn = new mpgTransaction($txnArray);  
$mpgRequest = new mpgRequest($mpgTxn);  
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

Purchase with Vault and 3-D Secure transaction request fields – Required

Variable Name	Type and Limits	
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i>	'amount'=>\$amount

Variable Name	Type and Limits	
	10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	
Cardholder Authentication Verification Value (CAVV)	String 50-character alphanumeric	cavv=>\$cavv
electronic commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

3-D Secure 2.2 -specific fields – Required

Variable Name	Type and Limits	
3DS version NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services.	String 10-character numeric	'threads_version'=>\$threads_version Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
3DS server transaction ID NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services - obtained from the Cavv Lookup request or MPI 3DS Authentication request	String 36-character numeric	'threads_server_trans_id'=>\$threads_server_trans_id

Purchase with Vault and 3-D Secure transaction request fields – Optional

Variable Name	Type and Limits	
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
Surcharge Information	<i>Object</i> N/A NOTE: This object requires use of the Vault Surcharge Lookup transaction prior to confirm card eligibility.	\$mpgTxn->setSurchargeInfo (\$surchargeInfo); Contains fields related to Surcharge feature.
final authorization	<i>String</i> true/false NOTE: Applies to Mastercard transactions only	'final_auth' => 'true'

3-D Secure 2.2 -specific fields – Optional

Variable Name	Type and Limits	
DS transaction ID	<i>String</i> 36-character alphanumeric NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.	'ds_trans_id' => \$ds_trans_id

Sample Purchase with Vault and 3-D Secure

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00597';
$api_token='O27AbCbXQorPggMQe6hU';

```

```

/***** Transaction Variables *****/
$data_key='4HIme0ZGURXE3NRBXHUj6nSc4';
$orderid='res-preauth-' . date("dmy-G:i:s");
$amount='1.00';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA';
$custid='customer1'; //if sent will be submitted, otherwise cust_id from profile will be used
$expdate = '2301'; //YYMM - used only for temp token
//$crypt_type = '6'; //value obtained from MpiACS transaction

//NT Response Option
$get_nt_response = 'false'; //Optional - set it true only if you want to get network
tokenization response.
/***** Transaction Array *****/
$txnArray = array('type'=>'res_cavv_purchase_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'amount'=>$amount,
'cavv'=>$cavv,
'expdate'=>$expdate, //mandatory for temp tokens only
//'crypt_type'=>$crypt_type, //set for AMEX SafeKey only
//'dynamic_descriptor'=>'12346',
'threads_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
Secure services
'threads_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f', //Mandatory for
financial transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
MpiThreeDSAuthentication
'ds_trans_id' => '12345', //Optional - to be used only if you are using 3rd party 3ds service
'get_nt_response'=>$get_nt_response
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("00000000065");
$installmentInfo->setTacVersion("2");
//$mpgTxn->setInstallmentInfo($installmentInfo);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());

```

```

print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\n\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());

$installmentResults = $mpgResponse->getInstallmentResults();
print("\nPlanId = " . $installmentResults->getPlanId());
print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
print("\nTacVersion = " . $installmentResults->getTacVersion());
print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
print("\nPlanStatus = " . $installmentResults->getPlanStatus());
print("\nPlanResponse = " . $installmentResults->getPlanResponse());

if($get_nt_response == 'true')
{
    print("\n\nNTResponseCode = " . $mpgResponse->getNTResponseCode());
    print("\nNTMessage = " . $mpgResponse->getNTMessage());
    print("\nNTUsed = " . $mpgResponse->getNTUsed());
    print("\nNTTokenBin = " . $mpgResponse->getNTTokenBin());
    print("\nNTTokenLast4 = " . $mpgResponse->getNTTokenLast4());
    print("\nNTTokenExpDate = " . $mpgResponse->getNTTokenExpDate());
}
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

6.8.4 Pre-Authorization with Vault & 3-D Secure

NOTE: This transaction supports both temporary and permanent tokens.

WARNING: Moneris strongly discourages the use of frames as part of a 3-D Secure implementation, and cannot guarantee their reliability when processing transactions in the production environment.

Pre-Authorization with Vault & 3-D Secure transaction object definition

```

$txnArray = array('type'=>'res_cavv_preauth_cc', ...);

$mpgTxn = new mpgTransaction($txnArray);

```

HttpPostRequest object for Pre-Authorization with Vault & 3-D Secure

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

Pre-Authorization with Vault & 3-D Secure transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
electronic commerce indicator	<i>String</i>	'crypt_type'=>\$crypt

Variable Name	Type and Limits	Set Method
	1-character alphanumeric	

Pre-Authorization with Vault & 3-D Secure transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
is incremental is_incremental	<i>Boolean</i> true/false	'is_incremental'=>\$is_incremental Indicates if this preauthorization is using an estimated amount. Estimations allow for incrementing the amount held via subsequent incrementalAuth requests. Defaults to false. NOTE: Please note that if this field is true, the preauthorization is only eligible for a single Preauthorization Completion. Any completion sent for partial completion is treated as a full completion (ship_indicator= P is treated as = F when is_incre-

Variable Name	Type and Limits	Set Method
		mental= true on the original preauth)
Surcharge Information NOTE: This object requires use of the Vault Surcharge Lookup transaction prior to confirm card eligibility.	<i>Object</i> N/A	\$mpgTxn->setSurchargeInfo (\$surchargeInfo); Contains fields related to Surcharge feature.
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
DS transaction ID NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.	<i>String</i> 36-character alphanumeric	'ds_trans_id' => \$ds_trans_id
final authorization NOTE: Applies to Mastercard transactions only	<i>String</i> true/false	'final_auth' => 'true'

Sample Pre-Authorization with Vault & 3-D Secure

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00597';
$api_token='O27AbCbXQorPggMQe6hU';
/***** Transaction Variables *****/
$data_key='4HIme0ZGURXE3NRBXHUj6nSc4';
$orderid='res-preauth-'.date("dmy-G:i:s");
$amount='1.00';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA';
$custid='customer1'; //if sent will be submitted, otherwise cust_id from profile will be used
$expdate = '2301'; //YYMM - used only for temp token
//$crypt_type = '6'; //value obtained from MpiACS transaction

//NT Response Option
$get_nt_response = 'false'; //Optional - set it true only if you want to get network
tokenization response.
/***** Transaction Array *****/
$txnArray =array('type'=>'res_cavv_preauth_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,

```

```

'cust_id'=>$custid,
'amount'=>$amount,
'cavv'=>$cavv,
'expdate'=>$expdate, //mandatory for temp tokens only
//'crypt_type'=>$crypt_type, //set for AMEX SafeKey only
//'dynamic_descriptor'=>'12346',
'threads_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
Secure services
'threads_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f', //Mandatory for
financial transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
MpiThreeDSAuthentication
'ds_trans_id' => '12345', //Optional - to be used only if you are using 3rd party 3ds service
'get_nt_response'=>$get_nt_response
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("00000000065");
$installmentInfo->setTacVersion("2");
// $mpgTxn->setInstallmentInfo($installmentInfo);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\n\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());

// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());

```



```

if($get_nt_response == 'true')
{
    print("\n\nNTResponseCode = " . $mpgResponse->getNTResponseCode());
    print("\nNTMessage = " . $mpgResponse->getNTMessage());
    print("\nNTUsed = " . $mpgResponse->getNTUsed());
    print("\nNTTokenBin = " . $mpgResponse->getNTTokenBin());
    print("\nNTTokenLast4 = " . $mpgResponse->getNTTokenLast4());
    print("\nNTTokenExpDate = " . $mpgResponse->getNTTokenExpDate());
}
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCryp Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

6.8.5 Purchase with 3-D Secure and Recurring Billing

WARNING: Moneris strongly discourages the use of frames as part of a 3-D Secure implementation, and cannot guarantee their reliability when processing transactions in the production environment.

The example below illustrates the Purchase with 3-D Secure when also sending the Recurring Billing Info object in the transaction.

Purchase with 3-D Secure and Recurring Billing

```

<?php
## Example php -q TestPurchase-VBV.php "moneris" store
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='cavv_purchase';
$order_id='ord-' . date("dmy-G:i:s");
$cust_id='CUST887763';
$amount='10.00';
$pan="4242424242424242";
$expiry_date="1511";
$cavv='AAABBJg0VhI0VniQEjRWAAAAA=';
$crypt_type = '7';
$wallet_indicator = "APP";
$dynamic_descriptor='123456';
/***** Recur Variables *****/
$recurUnit = 'month'; //eom - end of month
$startDate = '2018/02/06';
$numRekurs = '4';
$recurInterval = '10';
$recurAmount = '31.00';

```

```

$startNow = 'true';
/***** Transaction Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$pan,
    'expdate'=>$expiry_date,
    'cavv'=>$cavv,
    'crypt_type'=>$crypt_type, //mandatory for AMEX only
    //'wallet_indicator'=>$wallet_indicator, //set only for wallet transactions. e.g. APPLE PAY
    //'network'=> "Interac", //set only for Interac e-commerce
    //'data_type'=> "3DSecure", //set only for Interac e-commerce
    'dynamic_descriptor'=>$dynamic_descriptor,
    'threeds_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
    Secure services
    'threeds_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f', //Mandatory for
    financial transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
    MpiThreeDSAAuthentication
    //'ds_trans_id' => '12345' //Optional - to be used only if you are using 3rd party 3ds
    service
);
/***** Recur Associative Array *****/
$recurArray = array('recur_unit'=>$recurUnit, // (day | week | month)
    'start_date'=>$startDate, //yyyy/mm/dd
    'num_recur'=>$numRecur,
    'start_now'=>$startNow,
    'period' => $recurInterval,
    'recur_amount'=> $recurAmount
);
$mpgRecur = new mpgRecur($recurArray);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Recur Object *****/
$mpgTxn->setRecur($mpgRecur);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("R");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());

```

```
print("\nIssuerId = " . $mpgResponse->getIssuerId());  
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());  
?>
```

6.9 Testing Your 3-D Secure 2.2 Integration

In the testing stage of development:

1. Use the testing URL as Host for your requests:
esqa.moneris.com
2. In all Card Lookup Request transactions, make sure that you are using the testing version of your credentials for store ID and API token
3. In all MPI 3DS Authentication Request transactions, make sure that you are using the testing version of your credentials for store ID and API token
4. In all Cavv Lookup Request transactions, make sure that you are using the testing version of your credentials for store ID and API token

6.10 Moving to Production With 3-D Secure 2.2

Once you have finished testing your 3D Secure 2.2 integration, do the following to move the integration into production:

1. Use the production URL as Host for your requests:
www3.moneris.com
2. In all Card Lookup Request transactions, make sure that you are using the production version of your credentials for store ID and API token
3. In all MPI 3DS Authentication Request transactions, make sure that you are using the production version of your credentials for store ID and API token
4. In all Cavv Lookup Request transactions, make sure that you are using the production version of your credentials for store ID and API token

6.11 3-D Secure 2.2 TransStatus Codes

Value	Description	Comments
Y	Authenticated	Cardholder has been fully authenticated
D	Challenge Required (Decoupled)	Cardholder requires a challenge using Decoupled Authentication
A	Authentication Attempt	A proof of authentication attempt was generated
C	Challenge Required	Cardholder requires a challenge to complete authentication
U	Not Authenticated	Authentication could not be performed due to technical or other issue
N	Not Authenticated	Not authenticated
R	Not Authenticated	Not authenticated because the Issuer is rejecting authentication and requesting that authorisation not be attempted

6.12 3-D Secure 2.2 Commons TransStatusReason Decline Codes

The following codes are returned by the 3-D Secure service in order to provide additional information about the 3-D Secure transaction status.

TransStatusReason Code	Description
01	Card authentication failed
02	Unknown Device
03	Unsupported Device
04	Exceeds authentication frequency limit
05	Expired card
06	Invalid card number

TransStatusReason Code	Description
07	Invalid transaction
08	No Card record
09	Security failure
10	Stolen card
11	Suspected fraud
12	Transaction not permitted to cardholder
13	Cardholder not enrolled in service
14	Transaction timed out at the ACS
15	Low confidence
16	Medium confidence
17	High confidence
18	Very High confidence
19	Exceeds ACS maximum challenges
20	Non-Payment transaction not supported
21	3RI transaction not supported
22	ACS technical issue
23	Decoupled Authentication required by ACS but not requested by 3DS Requestor
24	3DS Requestor Decoupled Max Expiry Time exceeded
25	Decoupled Authentication was provided insufficient time to authenticate cardholder. ACS will not make attempt
26	Authentication attempted but not performed by

TransStatusReason Code	Description
------------------------	-------------

the cardholder

NOTE: For a list of all TransStatus Decline Codes, please see Reference section of 3D Secure 2.2 at <https://developer.moneris.com>.

6.13 CAVV Result Codes

The Cardholder Authentication Verification Value (CAVV), the Accountholder Authentication Value (AAV), and the American Express Verification Value (AEVV), are the values that allows Visa, Mastercard and American Express to validate the integrity of the Visa Secure, Mastercard Identity Check and American Express SafeKey transaction data. These values are passed back from the issuer to the merchant after the authentication has taken place. The merchant then integrates the CAVV/AAV/AEVV value into the authorization request using the Purchase or Pre-Authorization with 3-D Secure transaction type.

To summarize this process:

1. Merchant conducts 3-D Secure authentication request and receives CAVV/AAV/AEVV value in response
2. Merchant sends the CAVV/AAV/AEVV value to Moneris using the Purchase or Pre-Authorization with 3-D Secure transaction type and receives the CAVV result code in the response

The following tables describe the contents of the CAVV data response and what it means to the merchant.

6.13.1 Visa CAVV Result Codes

Visa CAVV result codes

Result Code	Message	Significance to Merchants
Blank	CAVV not present or not verified	Not a Visa Secure transaction. No liability shift and merchant is not protected from chargebacks
0	CAVV authentication results invalid	Not a Visa Secure transaction. No liability shift and merchant is not protected from chargebacks
1	CAVV failed validation (authentication)	Provided that you have implemented the Visa Secureprocess correctly, the liability for this

Result Code	Message	Significance to Merchants
		transaction should remain with the Issuer for chargeback reason codes covered by Visa Secure.
2	CAVV passed validation (authentication)	Fully authenticated transaction. There is a liability shift and the merchant is protected from chargebacks.
3, 8, A	CAVV passed validation (attempt)	Visa Secure has been attempted. There is a liability shift and the merchant is protected from certain card fraud-related chargebacks.
4, 7, 9	CAVV failed validation (attempt)	Visa Secure has been attempted. There is a liability shift and the merchant is protected from certain card fraud-related chargebacks.
6	CAVV not validated - Issuer not participating	Visa Secure has been attempted. There is a liability shift and the merchant is protected from certain card fraud-related chargebacks.
B	CAVV passed validation; information only	Not a Visa Secure transaction. No liability shift and merchant is not protected from chargebacks
C	CAVV was not validated (attempt)	Visa Secure has been attempted. There is a liability shift and the merchant is protected from certain card fraud-related chargebacks.
D	CAVV was not validated (authentication)	Visa Secure has been attempted. There is a liability shift and the merchant is protected from certain card fraud-related chargebacks.

6.13.2 Mastercard CAVV Result Codes

Mastercard CAVV result codes

Result Code	Message	Significance to Merchants
0	Authentication failed	Not a Mastercard Identity Check transaction. No liability shift and merchant is not protected from chargebacks
1	Authentication attempted	Mastercard Identity Check has been attempted.

Result Code	Message	Significance to Merchants
		There is a liability shift and the merchant is protected from certain card fraud-related chargebacks (international commercial cards excluded).
2	Authentication successful	Fully authenticated transaction. There is a liability shift and the merchant is protected from chargebacks.

6.13.3 American Express CAVV Result Codes

American Express CAVV result codes

NOTE: American Express SafeKey is only available to American Express direct acquired merchants (i.e., not OptBlue merchants). Any questions pertaining to chargebacks, liability and disputes should be addressed to your American Express representative given that American Express is the acquirer of record for these merchants.

Result Code	Description
1	AEVV Failed - Authentication, Issuer Key
2	AEVV Passed - Authentication, Issuer Key
3	AEVV Passed - Attempt, Issuer Key
4	AEVV Failed - Attempt, Issuer Key
7	AEVV Failed - Attempt, Issuer not participating, Network Key
8	AEVV Passed - Attempt, Issuer not participating, Network Key
9	AEVV Failed - Attempt, Participating, Access Control Server (ACS) not available, Network Key
A	AEVV Passed - Attempt, Participating, Access Control Server (ACS) not available, Network Key

Result Code	Description
U	AEVV Unchecked

7 Multi-Currency Pricing (MCP)

- 7.1 About Multi-Currency Pricing (MCP)
- 7.2 Methods of Processing MCP Transactions
- 7.3 MCP Get Rate
- 7.4 MCP Purchase
- 7.5 MCP Purchase with 3-D Secure
- 7.6 MCP Purchase with 3-D Secure and Vault
- 7.7 MCP Pre-Authorization
- 7.8 MCP Pre-Authorization with 3-D Secure
- 7.9 MCP Pre-Authorization with 3-D Secure and Vault
- 7.10 MCP Pre-Authorization Completion
- 7.11 MCP Purchase Correction
- 7.12 MCP Refund
- 7.13 MCP Independent Refund
- 7.14 MCP Purchase with Vault
- 7.15 MCP Pre-Authorization with Vault
- 7.16 MCP Independent Refund with Vault
- 7.17 MCP Currency Codes
- 7.18 MCP Error Codes

7.1 About Multi-Currency Pricing (MCP)

Multi-currency pricing (MCP) is a financial service which allows businesses to price goods and services in a variety of foreign currencies, while continuing to receive settlement and reporting in Canadian dollars. MCP allows cardholders to shop, view prices and pay in the currency of their choice.

MCP is only available when processing Visa and Mastercard transactions.

NOTE: Use MCP only when processing transactions that involve foreign currency exchange; for transactions strictly in Canadian dollars, use the basic financial transaction requests

7.2 Methods of Processing MCP Transactions

There are two methods of processing multi-currency pricing transactions via the Moneris Gateway:

1. **Using the MCP Get Rate transaction** – this method is used to obtain a foreign exchange rate and locks that specific rate in for a limited time, and is applied in a subsequent transaction
2. **Without using MCP Get Rate** – this method sends a MCP transaction without performing the Get Rate request, and the foreign exchange rate is obtained at processing time

7.3 MCP Get Rate

Performs a foreign currency exchange rate look-up, and secures that exchange rate for use in a subsequent MCP financial transaction.

The exchange rate retrieved by this transaction request is represented in the response as the **RateToken**, and the underlying exchange rate is locked in for a limited time period.

MCP Get Rate transaction object definition

```
$txnArray = array('type'=>'mcp_get_rate', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Get Rate transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

MCP Get Rate transaction request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	String numeric current version is 1.0	'mcp_version'=> \$mcp_version
rate transaction type	String 1-character alphabetic	'rate_txn_type'=>\$rate_txn_type
MCP Rate Info	Object N/A	\$mpgTxn->setMCPRateInfo(\$mcpRate);

MCP Rate Info object request fields

At least one of the following variables must be sent:

Variable Name	Type and Limits	Set Method
add cardholder amount	<i>String array</i> 12-character numeric, 3-character numeric (smallest discrete unit of foreign currency, currency code)	<code>\$mcpRate->setCardholderAmount('FOREIGN_AMT', 'FOREIGN_CURRENCY_CODE');</code>
add merchant settlement amount	<i>String array</i> 12-character numeric, 3-character numeric (amount in CAD pennies, currency code)	<code>\$mcpRate->setMerchantSettlementAmount('CAD_AMOUNT', 'FOREIGN_CURRENCY_CODE');</code>

Sample MCP Get Rate

```
<?php
##
## Example php -q TestPurchase.php store1
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
$type = 'mcp_get_rate';
$mcp_version = '1.0';
$rate_txn_type = 'P';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'mcp_version'=>$mcp_version,
'rate_txn_type'=>$rate_txn_type
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$mcpRate = new MCPRate();
$mcpRate->setCardholderAmount('100', '840');
$mcpRate->setMerchantSettlementAmount('200', '826');
$mpgTxn->setMCPRateInfo($mcpRate);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
/* Status Check Example
$mpgHttpPost =new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);
*/
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nRateTxnType = " . $mpgResponse->getRateTxnType());
print("\nMCPRateToken = " . $mpgResponse->getMCPRateToken());
print("\nRateInqStartTime = " . $mpgResponse->getRateInqStartTime()); //The time (unix UTC)
of when the rate is requested
print("\nRateInqEndTime = " . $mpgResponse->getRateInqEndTime()); //The time (unix UTC) of
when the rate is returned
print("\nRateValidityStartTime = " . $mpgResponse->getRateValidityStartTime()); //The time
(unix UTC) of when the rate is valid from
print("\nRateValidityEndTime = " . $mpgResponse->getRateValidityEndTime()); //The time (unix
UTC) of when the rate is valid until
print("\nRateValidityPeriod = " . $mpgResponse->getRateValidityPeriod()); //The time in
minutes this rate is valid for
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
//RateData
for ($index = 0; $index < $mpgResponse->getRatesCount(); $index++)
{
    print("\nMCPRate = " . $mpgResponse->getMCPRate($index));
    print("\nMerchantSettlementCurrency = " . $mpgResponse->getMerchantSettlementCurrency
($index));
    print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount($index));
    //Domestic (CAD) amount
    print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode($index));
    print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount($index)); //Foreign amount

    print("\nMCPErrorStatusCode = " . $mpgResponse->getMCPErrorStatusCode($index));
    print("\nMCPErrorMessage = " . $mpgResponse->getMCPErrorMessage($index));
}
?>
```

7.4 MCP Purchase

Verifies funds on the customer's card, removes the funds and prepares them for deposit into the merchant's account.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

MCP Purchase transaction object definition

```
$txnArray = array('type'=>'mcp_purchase', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Purchase transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	Set Method
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

MCP Purchase transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i>	'cardholder_amount' => \$cardholder_amount

Variable Name	Type and Limits	Set Method
	12-character numeric	
	smallest discrete unit of foreign currency	
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Purchase transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
wallet indicator	<i>String</i> 3-character alphanumeric	'wallet_indicator'=>\$wallet_indicator
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo(\$cof);
cof	N/A	
NOTE: This is a nested object within the transaction, and		

Variable Name	Type and Limits	Set Method
required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		
AVS Information	Object N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	Object N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	String N/A	'mcp_rate_token' => \$mcp_rate_token

Sample MCP Purchase

```

<?php
##
## Example php -q TestPurchase.php store1
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='mcp_purchase';
$cust_id='cust id';
$order_id='ord-' . date("dmy-G:i:s");
$pan='4242424242424242';
$expiry_date='2011';
$crypt='7';
$dynamic_descriptor='123';
$status_check = 'false';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1536163325404090';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'pan'=>$pan,
'expdate'=>$expiry_date,
```



```

'crypt_type'=>$crypt,
'dynamic_descriptor'=>$dynamic_descriptor,
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
//,'wallet_indicator' => '' //Refer to documentation for details
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
/* Status Check Example
$mpgHttpPost =new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);
*/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nStatusCode = " . $mpgResponse->getStatusCode());
print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
print("\nHostId = " . $mpgResponse->getHostId());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrrorStatusCode = " . $mpgResponse->getMCPErrrorStatusCode());
print("\nMCPErrrorMessage = " . $mpgResponse->getMCPErrrorMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.5 MCP Purchase with 3-D Secure

MCP Purchase with 3-D Secure transaction object definition

```

$txnArray = array('type'=>'mcp_cavv_purchase', ...);

$mpgTxn = new mpgTransaction($txnArray);

```

HttpPostRequest object for MCP Purchase with 3-D Secure transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

MCP Purchase with 3-D Secure transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

3-D Secure 2.2 -specific fields – Required

Variable Name	Type and Limits	Set Method
3DS version NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services.	<i>String</i> 10-character numeric	'threeds_version'=>\$threeds_version Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
3DS server transaction ID NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services - obtained from the Cavv Lookup request or MPI 3DS Authentication request	<i>String</i> 36-character numeric	'threeds_server_trans_id'=>\$threeds_server_trans_id

MCP Purchase with 3-D Secure transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
wallet indicator	<i>String</i> 3-character alphanumeric	'wallet_indicator'=>\$wallet_indicator
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo(\$cof);
cof	N/A	
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	<i>Object</i>	\$mpgTxn->setCvdInfo

Variable Name	Type and Limits	Set Method
	N/A	(\$mpgCvdInfo);

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	String N/A	'mcp_rate_token' => \$mcp_rate_token

3-D Secure 2.2 -specific fields – Optional

Variable Name	Type and Limits	Set Method
DS transaction ID	String 36-character alphanumeric	'ds_trans_id' => \$ds_trans_id
NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.		

Sample MCP Purchase with 3-D Secure

```
<?php
## Example php -q TestPurchase-VBV.php "moneris" store
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca02932';
$api_token='CG8kYzGgzVU5z23irgMx';
/***** Transactional Variables *****/
$type='mcp_cavv_purchase';
$order_id='ord-' . date("dmy-G:i:s");
$cust_id='CUST887763';
$amount='10.00';
$pan="4740611374762707";
$expiry_date="2211";
$cavv='BwABApFSYyd4l2eQQFJjAAAAAAA=';
$crypt_type = '7';
$wallet_indicator = "APP";
$dynamic_descriptor='123456';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1623680755788776';
/***** Transaction Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$pan,
    'expdate'=>$expiry_date,
    'cavv'=>$cavv,
    'crypt_type'=>$crypt_type, //mandatory for AMEX only
```

```
//'wallet_indicator'=>$wallet_indicator, //set only for wallet transactions. e.g. APPLE PAY
//'network'=> "Interac", //set only for Interac e-commerce
'data_type'=> "3DSecure", //set only for Interac e-commerce
'dynamic_descriptor'=>$dynamic_descriptor,
'threads_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
Secure services
'threads_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f', //Mandatory for
financial transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
MpiThreeDSAuthentication
//'cm_id' => '8nAK8712sGaAkls56', //set only for usage with Offlinx - Unique max 50
alphanumeric characters transaction id generated by merchant
//'ds_trans_id' => '12345', //Optional - to be used only if you are using 3rd party 3ds
service
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrStatusCode = " . $mpgResponse->getMCPErrStatusCode());
print("\nMCPErrMessage = " . $mpgResponse->getMCPErrMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>
```

7.6 MCP Purchase with 3-D Secure and Vault

MCP Purchase with 3-D Secure and Vault transaction object definition

```
$txnArray = array('type'=>'mcp_cavv_res_purchase_cc', ...);  
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Purchase with 3-D Secure and Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);  
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

MCP Purchase with 3-D Secure and Vault transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	String 25-character alphanumeric	'data_key'=>\$data_key
order ID	String 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	String max 20-character alpha-numeric	'pan'=>\$pan
expiry date	String 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date

Variable Name	Type and Limits	Set Method
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

3-D Secure 2.2 -specific fields – Required

Variable Name	Type and Limits	Set Method
3DS version	<i>String</i> 10-character numeric	'threeds_version'=>\$threeds_version Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
3DS server transaction ID	<i>String</i>	'threeds_server_trans_

NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services.

Variable Name	Type and Limits	Set Method
NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services - obtained from the Cavv Lookup request or MPI 3DS Authentication request	36-character numeric	<code>id'=>\$threeds_server_trans_id</code>

MCP Purchase with 3-D Secure and Vault transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	<code>'cust_id'=>\$cust_id</code>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	<code>'dynamic_descriptor'=>\$dynamic_descriptor</code>
wallet indicator	<i>String</i> 3-character alphanumeric	<code>'wallet_indicator'=>\$wallet_indicator</code>
Credential on File Info	<i>Object</i>	<code>\$mpgTxn->setCofInfo(\$cof);</code>
cof	N/A	
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored		

Variable Name	Type and Limits	Set Method
credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		
AVS Information	Object N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	Object N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);

3-D Secure 2.2 -specific fields – Optional

Variable Name	Type and Limits	Set Method
DS transaction ID	String 36-character alphanumeric	'ds_trans_id' => \$ds_trans_id
NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.		

Sample MCP Purchase with 3-D Secure and Vault

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store1';
$api_token='yesguyl';
/***** Transaction Variables *****/
$orderid='res-purchase-' . date("dmy-G:i:s");
$data_key='4INQR1A8ocxD0oafSz50LADxy';
$amount='1.00';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA';
$custid='customer1'; //if sent will be submitted, otherwise cust_id from profile will be used
$expdate = '1901'; //YYMM - used only for temp token
$crypt_type = '7'; //value obtained from MpiACS transaction
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1623436475143166';
/***** Transaction Array *****/
$txnArray =array('type'=>'mcp_res_cavv_purchase_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'amount'=>$amount,
'cavv'=>$cavv,
'expdate'=>$expdate, //mandatory for temp tokens only
```

```

'crypt_type'=>$crypt_type, //set for AMEX SafeKey only
//'dynamic_descriptor'=>'12346',
'threads_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
Secure services.
'threads_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f', //Mandatory for
financial transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
MpiThreeDSAuthentication
//'ds_trans_id' => '12345', //Optional - to be used only if you are using 3rd party 3ds
service
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\n\nPhone = " . $mpgResponse->getResDataPhone());
print("\n\nEmail = " . $mpgResponse->getResDataEmail());
print("\n\nNote = " . $mpgResponse->getResDataNote());
print("\n\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\n\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\n\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\n\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\n\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\n\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

7.7 MCP Pre-Authorization

Verifies and locks funds on the customer's credit card. The funds are locked for a specified amount of time based on the card issuer. For situations where the merchant does not know the precise amount when the transaction begins, a Pre-Authorization transaction may use an estimated amount instead. If the cardholder incurs additional charges during the transaction period, the merchant is able to increase the total amount authorized via Incremental Pre-Authorization; these increments are added to the previously authorized amount. To retrieve the funds that have been locked by a Pre-Authorization transaction so that they may be settled in the merchant's account, a Pre-Authorization Completion transaction must be performed. A Pre-Authorization transaction may only be "completed" once. If the Pre-Authorization is not completed within the allowable chargeback protection period then a new authorization should be obtained. For Mastercard, the merchant can instead submit an Incremental Pre-Authorization with a \$0 amount field associated to the original Pre-Authorization transaction; if approved, this extends the protection period.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

MCP Pre-Authorization transaction object definition

```
$txnArray = array('type'=>'mcp_preauth', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Pre-Authorization transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

MCP Pre-Authorization transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	String 50-character alphanumeric	'order_id'=>\$order_id

Variable Name	Type and Limits	Set Method
	a-Z A-Z 0-9 _ - : . @ spaces	
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Pre-Authorization transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id

Variable Name	Type and Limits	Set Method
	NOTE: Some special characters are not allowed: <code>< > \$ % = ? ^ { } [] \</code>	
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: <code>< > \$ % = ? ^ { } [] \</code>	'dynamic_descriptor'=>\$dynamic_descriptor
	NOTE: For Pre-Authorization transactions: the value in the dynamic descriptor field will only be carried over to a Pre-Authorization Completion when executing the latter via the Merchant Resource Center; otherwise, the value for dynamic descriptor must be sent again in the Pre-Authorization Completion	
wallet indicator	<i>String</i> 3-character alphanumeric	'wallet_indicator'=>\$wallet_indicator
final authorization	<i>String</i> true/false	'final_auth' => 'true'
	NOTE: Applies to Mastercard transactions only	
Credential on File Info cof	<i>Object</i> N/A	\$mpgTxn->setCofInfo(\$cof);
	NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.	
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);

Variable Name	Type and Limits	Set Method
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo (\$mpgCvdInfo);

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	<i>String</i> N/A	'mcp_rate_token' => \$mcp_rate_token

Sample MCP Pre-Authorization

```

<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
##
## Example php -q TestPreAuth.php store1 yesguy
##
require "../mpgClasses.php";
$store_id='store5';
$api_token="yesguy";
/***** Transactional Variables *****/
$type='mcp_preauth';
$cust_id='cust id';
$order_id='ord-'.date("dmy-G:i:s");
$pan='4242424242424242';
$expiry_date='2011';
$crypt='7';
$dynamic_descriptor='123';
$status_check = 'false';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1536170825312107';
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'pan'=>$pan,
'expdate'=>$expiry_date,
'crypt_type'=>$crypt,
'dynamic_descriptor'=>$dynamic_descriptor,
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
//, 'wallet_indicator' => '' //Refer to documentation for details
// 'final_auth'=>'true'
);
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");

```

```

$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrrorStatusCode = " . $mpgResponse->getMCPErrrorStatusCode());
print("\nMCPErrrorMessage = " . $mpgResponse->getMCPErrrorMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.8 MCP Pre-Authorization with 3-D Secure

MCP Pre-Authorization with 3-D Secure transaction object definition

```
$txnArray = array('type'=>'mcp_cavv_preauth', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Pre-Authorization with 3-D Secure transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

MCP Pre-Authorization with 3-D Secure transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

3-D Secure 2.2 -specific fields – Required

Variable Name	Type and Limits	Set Method
3DS version NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services.	<i>String</i> 10-character numeric	<pre>'threads_version'=>\$threads_version</pre> Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
3DS server transaction ID NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services - obtained from the Cavv Lookup request or MPI 3DS Authentication request	<i>String</i> 36-character numeric	<pre>'threads_server_trans_id'=>\$threads_server_trans_id</pre>

MCP Pre-Authorization with 3-D Secure transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	<pre>'cust_id'=>\$cust_id</pre>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not	<pre>'dynamic_descriptor'=>\$dynamic_descriptor</pre>

Variable Name	Type and Limits	Set Method
	allowed: <>\$%=?^{}[]\	
wallet indicator	<i>String</i> 3-character alphanumeric	'wallet_indicator'=>\$wallet_indicator
Credential on File Info cof	<i>Object</i> N/A	\$mpgTxn->setCofInfo(\$cof);
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	<i>String</i> N/A	'mcp_rate_token' => \$mcp_rate_token

3-D Secure 2.2 -specific fields – Optional

Variable Name	Type and Limits	Set Method
DS transaction ID	<i>String</i> 36-character alphanumeric	'ds_trans_id' => \$ds_trans_id
NOTE: Only used in financial		

Variable Name	Type and Limits	Set Method
transactions using 3rd Party 3-D Secure services.		

Sample MCP Pre-Authorization with 3-D Secure

```
<?php
## Example php -q TestPurchase-VBV.php "moneris" store
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca02932';
$api_token='CG8kYzGgzVU5z23irgMx';
/***** Transactional Variables *****/
$type='mcp_cavv_preauth';
$order_id='ord-' . date("dmy-G:i:s");
$cust_id='CUST887763';
$amount='10.00';
$pan="4242424242424242";
$expiry_date="0812";
$cavv='AAABBJg0VhI0VniQEjRWAAAAA=';
$crypt_type = '7';
$wallet_indicator = "APP";
$dynamic_descriptor='123456';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1623437375175657';
/***** Transaction Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'cavv'=>$cavv,
'crypt_type'=>$crypt_type, //mandatory for AMEX only
// 'wallet_indicator'=>$wallet_indicator, //set only for wallet transactions. e.g. APPLE PAY
'dynamic_descriptor'=>$dynamic_descriptor,
'threads_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
Secure services
'threads_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f', //Mandatory for
financial transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
MpiThreeDSAuthentication
// 'cm_id' => '8nAK8712sGaAkls56', //set only for usage with Offlinx - Unique max 50
alphanumeric characters transaction id generated by merchant
// 'ds_trans_id' => '12345', //Optional - to be used only if you are using 3rd party 3ds
service
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
```

```

/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrorsStatusCode = " . $mpgResponse->getMCPErrorsStatusCode());
print("\nMCPErrorsMessage = " . $mpgResponse->getMCPErrorsMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.9 MCP Pre-Authorization with 3-D Secure and Vault

MCP Pre-Authorization with 3-D Secure and Vault transaction object definition

```
$txnArray = array('type'=>'mcp_res_cavv_preauth_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Pre-Authorization with 3-D Secure and Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String	'api_token'=>\$api_token

Variable Name	Type and Limits	
	N/A	

MCP Pre-Authorization with 3-D Secure and Vault transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i>	'cardholder_amount' => \$cardholder_amount

Variable Name	Type and Limits	Set Method
	12-character numeric	
	smallest discrete unit of foreign currency	
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

3-D Secure 2.2 -specific fields – Required

Variable Name	Type and Limits	Set Method
3DS version	<i>String</i> 10-character numeric	'threeds_version'=>\$threeds_version Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
3DS server transaction ID	<i>String</i> 36-character numeric	'threeds_server_trans_id'=>\$threeds_server_trans_id

NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services.

NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services - obtained from the Cavv Lookup request or MPI 3DS Authentication request

MCP Pre-Authorization with 3-D Secure and Vault transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id
	NOTE: Some special characters are not allowed:	

Variable Name	Type and Limits	Set Method
	< > \$ % = ? ^ { } [] \	
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
wallet indicator	<i>String</i> 3-character alphanumeric	'wallet_indicator'=>\$wallet_indicator
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo(\$cof);
cof	N/A	
	NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.	
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	String N/A	'mcp_rate_token' => \$mcp_rate_token

3-D Secure 2.2 -specific fields – Optional

Variable Name	Type and Limits	Set Method
DS transaction ID	String 36-character alphanumeric	'ds_trans_id' => \$ds_trans_id
NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.		

Sample MCP Pre-Authorization with 3-D Secure and Vault

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store1';
$api_token='yesguy1';
/***** Transaction Variables *****/
$data_key='4INQR1A8ocxD0oafSz50LADxy';
$orderid='res-preauth-' . date("dmy-G:i:s");
$amount='1.00';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA';
$custid='customer1'; //if sent will be submitted, otherwise cust_id from profile will be used
$expdate = '1901'; //YYMM - used only for temp token
$crypt_type = '7'; //value obtained from MpiACS transaction
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1623438275728112';
/***** Transaction Array *****/
$txnArray =array('type'=>'mcp_res_cavv_preauth_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'amount'=>$amount,
'cavv'=>$cavv,
'expdate'=>$expdate, //mandatory for temp tokens only
'crypt_type'=>$crypt_type, //set for AMEX SafeKey only
/'dynamic_descriptor'=>'12346',
'threads_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
Secure services
'threads_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f', //Mandatory for
financial transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
MpiThreeDSAuthentication
/'ds_trans_id' => '12345', //Optional - to be used only if you are using 3rd party 3ds
service
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
```

```

'mcp_rate_token' => $mcp_rate_token
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
//----- ResolveData -----
print("\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCryp Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
?>

```

7.10 MCP Pre-Authorization Completion

Retrieves funds that have been locked by an MCP Pre-Authorization transaction, and prepares them for settlement into the merchant's account.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

MCP Pre-Authorization Completion transaction object definition

```
$txnArray = array('type'=>'mcp_completion', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Pre-Authorization Completion transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

MCP Pre-Authorization Completion transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
transaction number	<i>String</i> 255-character, alpha-numeric, hyphens or under-scores variable length	'txn_number'=>\$txn_number
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	String numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	String 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	String 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Pre-Authorization Completion transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	String 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	String 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor

NOTE: For Pre-Authorization transactions: the value in the dynamic descriptor field will only be carried over to a Pre-Authorization Completion when executing the latter via the Merchant Resource Center; otherwise, the value for dynamic descriptor must be sent again in the Pre-Authorization Completion

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	<i>String</i> N/A	'mcp_rate_token' => \$mcp_rate_token

Sample MCP Pre-Authorization Completion

```

<?php
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$orderid='ord-050918-14:11:30';
$txnnumber='407995-0_11';
$dynamic_descriptor='123';
$ship_indicator = "F"; //optional
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1536170825312107';
## step 1) create transaction array ###
$txnArray= array('type'=>'mcp_completion',
'txn_number'=>$txnnumber,
'order_id'=>$orderid,
'crypt_type'=>'7',
'cust_id'=>'customer ID',
// 'ship_indicator'=>$ship_indicator, //optional
'dynamic_descriptor'=>$dynamic_descriptor,
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
);
## step 2) create a transaction object passing the hash created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());

```

```

print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrorsStatusCode = " . $mpgResponse->getMCPErrorsStatusCode());
print("\nMCPErrorsMessage = " . $mpgResponse->getMCPErrorsMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.11 MCP Purchase Correction

Restores the full amount of a previous MCP Purchase or MCP Pre-Authorization Completion transaction to the cardholder's card, and removes any record of it from the cardholder's statement.

This transaction can be used against a Purchase or Pre-Authorization Completion transaction that occurred same day provided that the batch containing the original transaction remains open.

MCP processing uses the automated closing feature, and Batch Close occurs daily between 10 and 11 pm Eastern Time.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

MCP Purchase Correction transaction object definition

```
$txnArray = array('type'=>'mcp_purchaseCorrection', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Purchase Correction transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

MCP Purchase Correction transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alpha-numeric-A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
transaction number	<i>String</i> 255-character, alpha-numeric, hyphens or under-scores variable length	'txn_number'=>\$txn_number
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP Purchase Correction transaction request fields – Optional

Variable Name	Type and Limits	Description
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id

Sample MCP Purchase Correction

```
<?php
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$orderid='ord-150816-12:36:20';
$txnnumber='117816-0_10';
$dynamic_descriptor='1234';
## step 1) create transaction hash ###
$txnArray=array('type'=>'mcp_purchaseCorrection',
'txn_number'=>$txnnumber,
'order_id'=>$orderid,
'crypt_type'=>'7',
'cust_id'=>'customer ID',
'dynamic_descriptor'=>$dynamic_descriptor
);
## step 2) create a transaction object passing the array created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrorsStatusCode = " . $mpgResponse->getMCPErrorsStatusCode());
print("\nMCPErrorsMessage = " . $mpgResponse->getMCPErrorsMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>
```

7.12 MCP Refund

Restores all or part of the funds from a MCP Purchase or MCP Pre-Authorization Completion transaction to the cardholder's card.

Unlike a MCP Purchase Correction, there is a record of both the initial charge and the refund on the card-holder's statement.

For processing refunds on a different card than the one used in the original transaction, the MCP Independent Refund transaction should be used instead.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

MCP Refund transaction object definition

```
$txnArray = array('type'=>'mcp_refund', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

MCP Refund transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alpha- numerica-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
transaction number	<i>String</i> 255-character, alpha- numeric, hyphens or under- scores variable length	'txn_number'=>\$txn_number

Variable Name	Type and Limits	Set Method
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Refund transaction request fields – Optional

Variable Name	Type and Limits	Description
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
customer ID	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id

Variable Name	Type and Limits	Description
---------------	-----------------	-------------

NOTE:
Some special characters are not allowed:
<>\$%=?^{}[]\

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	String N/A	'mcp_rate_token' => \$mcp_rate_token

Sample MCP Refund

```
<?php
##
## This program takes 4 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
## 4. trans number
##
## Example php -q TestRefund.php store1 yesguy my_order_id 45109-89-0
##
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$orderid='ord-050918-12:03:26';
$txnnumber='407512-0_11';
$crypt_type = '7';
$dynamic_descriptor='123';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'R1536163085399771';
## step 1) create transaction array ###
$txnArray=array('type'=>'mcp_refund',
'txn_number'=>$txnnumber,
'order_id'=>$orderid,
'crypt_type'=>$crypt_type,
'cust_id'=> 'Customer ID',
'dynamic_descriptor'=>$dynamic_descriptor,
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token //optional
);
## step 2) create a transaction object passing the array created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
## step 4) create mpgHttpPost object which does an https post ##
```

```

$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrStatusCode = " . $mpgResponse->getMCPErrStatusCode());
print("\nMCPErrMessage = " . $mpgResponse->getMCPErrMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.13 MCP Independent Refund

Credits a specified amount to the cardholder's credit card. The credit card number and expiry date are mandatory.

It is not necessary for the transaction that you are refunding to have been processed via the Moneris Gateway.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

Things to Consider:

- Because of the potential for fraud, permission for this transaction is not granted to all accounts by default. If it is required for your business, it must be requested via your account manager.

MCP Independent Refund transaction object definition

```
$txnArray = array('type'=>'mcp_ind_refund', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Independent Refund transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

MCP Independent Refund transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alpha- numerica-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
credit card number	<i>String</i> max 20-character alpha- numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i>	'mcp_version'=> \$mcp_version

Variable Name	Type and Limits	Set Method
	numeric current version is 1.0	
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Independent Refund transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	<i>String</i> N/A	'mcp_rate_token' => \$mcp_rate_token

Sample MCP Independent Refund

```

<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
##
## Example php -q TestIndependentRefund.php store1 yesguy unique_order_id
##
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$orderid='ord-'.date("dmy-G:i:s");
$pan='4242424242424242';
$expiry_date='2011';
$script='7';
$dynamic_descriptor='123456';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'R1536163085399771';
## step 1) create transaction array ###
$txnArray=array('type'=>'mcp_ind_refund',
'order_id'=>$orderid,
'cust_id'=>'my cust id',
'pan'=>$pan,
'expdate'=>$expiry_date,
'crypt_type'=>$script,
'dynamic_descriptor'=>$dynamic_descriptor,
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
);
## step 2) create a transaction object passing the array created in
## step 1.
$mpgTxn = new mpgTransaction($txnArray);
## step 3) create a mpgRequest object passing the transaction object created
## in step 2
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
## step 4) create mpgHttpPost object which does an https post ##
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
## step 5) get an mpgResponse object ##
$mpgResponse=$mpgHttpPost->getMpgResponse();
## step 6) retrieve data using get methods
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());

```

```

print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrorStatusCode = " . $mpgResponse->getMCPErrorStatusCode());
print("\nMCPErrorMessage = " . $mpgResponse->getMCPErrorMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.14 MCP Purchase with Vault

This transaction uses the data key to identify a previously registered credit card profile in Vault. The details saved within the profile are then submitted to perform a Purchase transaction.

The data key may be a temporary one generated used Hosted Tokenization, or may be a permanent one from the Vault.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

MCP Purchase with Vault transaction object definition

```

$txnArray = array('type'=>'mcp_res_purchase_cc', ...);

$mpgTxn = new mpgTransaction($txnArray);

```

HttpPostRequest object for MCP Purchase with Vault transaction

```

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

MCP Purchase with Vault transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key
order ID	<i>String</i> 50-character alpha- numerica-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	<i>String</i> numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	<i>String</i> 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Purchase with Vault transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE:	'cust_id'=>\$cust_id

Variable Name	Type and Limits	Set Method
	Some special characters are not allowed: < > \$ % = ? ^ { } [] \	
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo (\$cof) ;
cof	N/A	
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		
AVS Information	<i>Object</i>	\$mpgTxn->setAvsInfo (\$mpgAvsInfo) ;
	N/A	
CVD Information	<i>Object</i>	\$mpgTxn->setCvdInfo (\$mpgCvdInfo) ;
	N/A	

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	<i>String</i>	'mcp_rate_token' => \$mcp_rate_token
	N/A	

Sample MCP Purchase with Vault

```
<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
##
## Example php -q TestResPurchaseCC.php store3 yesguy unique_order_id 1.00
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transaction Variables *****/
```

```
$data_key='uX6jFwbvCytmG7oT70YVNm0e2';
$orderid='res-purch-' .date("dmy-G:i:s");
$custid='cust';
$script_type='1';
$expdate='1911'; //For Temp Tokens only
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1536163745116323';
/***** Transaction Array *****/
$txnArray=array('type'=>'mcp_res_purchase_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'crypt_type'=>$script_type,
// 'expdate'=>$expdate,
'dynamic_descriptor'=>'12484',
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
```

```

print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrorsStatuscode = " . $mpgResponse->getMCPErrorsStatuscode());
print("\nMCPErrorsMessage = " . $mpgResponse->getMCPErrorsMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.15 MCP Pre-Authorization with Vault

This transaction uses the data key to identify a previously registered credit card profile in Vault. The details saved within the profile are then submitted to perform a Pre-Authorization transaction.

The data key may be a temporary one generated used Hosted Tokenization, or may be a permanent one from the Vault.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

MCP Pre-Authorization with Vault transaction object definition

```
$txnArray = array('type'=>'mcp_res_preauth_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Pre-Authorization with Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i> N/A	'store_id'=>\$store_id
API token	<i>String</i> N/A	'api_token'=>\$api_token

MCP Pre-Authorization with Vault transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	<i>String</i>	'data_key'=>\$data_key

Variable Name	Type and Limits	Set Method
	25-character alphanumeric	
order ID	String 50-character alpha- numerica-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
electronic commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	String numeric current version is 1.0	'mcp_version'=> \$mcp_version
cardholder amount	String 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	String 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Pre-Authorization with Vault transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	String 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id

Variable Name	Type and Limits	Set Method
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator	'dynamic_descriptor'=>\$dynamic_descriptor
NOTE: For Pre-Authorization transactions: the value in the dynamic descriptor field will only be carried over to a Pre-Authorization Completion when executing the latter via the Merchant Resource Center; otherwise, the value for dynamic descriptor must be sent again in the Pre-Authorization Completion NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \		
final authorization	<i>String</i> true/false	'final_auth' => 'true'
NOTE: Applies to Mastercard transactions only		
Credential on File Info	<i>Object</i>	\$mpgTxn->setCofInfo(\$cof);
cof	N/A	
NOTE: This is a nested object within the transaction, and required when storing or using the customer's stored credentials. For information about fields in the Credential on File Info object, see Credential on File Info Object and Variables.		
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	<i>String</i> N/A	'mcp_rate_token' => \$mcp_rate_token

Sample MCP Pre-Authorization with Vault

```

<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
##
## Example php -q TestResPreauthCC.php store3 yesguy unique_order_id cust_id 15.00 1
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transaction Variables *****/
$data_key='uX6jFwbvCytmG7oT70YVNm0e2';
$orderid='res-preauth-'.date("dmy-G:i:s");
$amount='1.00';
$custid='cust'; //if sent will be submitted, otherwise cust_id from profile will be used
$crypt_type='1';
//$expdate='1512';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'P1536170825312107';
/***** Transaction Array *****/
$txnArray =array('type'=>'mcp_res_preauth_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'amount'=>$amount,
'crypt_type'=>$crypt_type,
//'expdate'=>$expdate,
'dynamic_descriptor'=>'12424',
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
//'final_auth'=>'true'
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

```

```

/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nAVSResponse = " . $mpgResponse->getAvsResultCode());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrorStatusCode = " . $mpgResponse->getMCPErrorStatusCode());
print("\nMCPErrorMessage = " . $mpgResponse->getMCPErrorMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.16 MCP Independent Refund with Vault

This transaction uses the data key to identify a previously registered credit card profile in Vault. The details saved within the profile are then submitted to perform an Independent Refund transaction.

This transaction request is the multi-currency pricing (MCP) enabled version of the equivalent financial transaction.

Things to Consider:

- Because of the potential for fraud, permission for this transaction is not granted to all accounts by default. If it is required for your business, it must be requested via your account manager.

MCP Independent Refund with Vault transaction object definition

```
$txnArray = array('type'=>'mcp_res_ind_refund_cc', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for MCP Independent Refund with Vault transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

MCP Independent Refund with Vault transaction request fields – Required

Variable Name	Type and Limits	Set Method
data key	String 25-character alphanumeric	'data_key'=>\$data_key
order ID	String 50-character alpha- numerica-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
electronic commerce indicator	String 1-character alphanumeric	'crypt_type'=>\$crypt

MCP-specific request fields – Required

Variable Name	Type and Limits	Set Method
MCP version number	String numeric	'mcp_version'=> \$mcp_version

Variable Name	Type and Limits	Set Method
	current version is 1.0	
cardholder amount	String 12-character numeric smallest discrete unit of foreign currency	'cardholder_amount' => \$cardholder_amount
cardholder currency code	String 3-character numeric	'cardholder_currency_code' => \$cardholder_currency_code

MCP Independent Refund with Vault transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	String 50-character alphanumeric NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	'cust_id'=>\$cust_id

MCP-specific request fields – Optional

Variable Name	Type and Limits	Set Method
MCP rate token	String N/A	'mcp_rate_token' => \$mcp_rate_token

Sample MCP Independent Refund with Vault

```
<?php
##
## This program takes 3 arguments from the command line:
## 1. Store id
## 2. api token
## 3. order id
##
## Example php -q TestResIndRefundCC.php store3 yesguy unique_order_id cust_id 15.00 1
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transaction Variables *****/
```

```

$data_key='uX6jFwbvCytmG7oT70YVNm0e2';
$orderid='res-ind-refund-' . date("dmy-G:i:s");
$custid='';
$script_type='1';
$mcp_version = '1.0';
$cardholder_amount = '100';
$cardholder_currency_code = '840';
$mcp_rate_token = 'R1536165545730980';
/***** Transaction Array *****/
$txnArray = array('type'=>'mcp_res_ind_refund_cc',
'data_key'=>$data_key,
'order_id'=>$orderid,
'cust_id'=>$custid,
'crypt_type'=>$script_type,
'dynamic_descriptor'=>'12346',
'mcp_version'=> $mcp_version,
'cardholder_amount' => $cardholder_amount,
'cardholder_currency_code' => $cardholder_currency_code,
'mcp_rate_token' => $mcp_rate_token
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nResSuccess = " . $mpgResponse->getResSuccess());
print("\nPaymentType = " . $mpgResponse->getPaymentType());
//----- ResolveData -----
print("\n\nCust ID = " . $mpgResponse->getResDataCustId());
print("\nPhone = " . $mpgResponse->getResDataPhone());
print("\nEmail = " . $mpgResponse->getResDataEmail());
print("\nNote = " . $mpgResponse->getResDataNote());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nCrypt Type = " . $mpgResponse->getResDataCryptType());
print("\nAvs Street Number = " . $mpgResponse->getResDataAvsStreetNumber());
print("\nAvs Street Name = " . $mpgResponse->getResDataAvsStreetName());
print("\nAvs Zipcode = " . $mpgResponse->getResDataAvsZipcode());
print("\nMerchantSettlementAmount = " . $mpgResponse->getMerchantSettlementAmount());
print("\nCardholderAmount = " . $mpgResponse->getCardholderAmount());
print("\nCardholderCurrencyCode = " . $mpgResponse->getCardholderCurrencyCode());
print("\nMCPRate = " . $mpgResponse->getMCPRate());
print("\nMCPErrrorStatusCode = " . $mpgResponse->getMCPErrrorStatusCode());
print("\nMCPErrrorMessage = " . $mpgResponse->getMCPErrrorMessage());
print("\nHostId = " . $mpgResponse->getHostId());
?>

```

7.17 MCP Currency Codes

For currency symbols, see <https://justforex.com/education/currencies>

NOTE: This documentation contains links to websites owned and operated by third parties. If you use these links, you leave our website. These links are provided for your information and convenience only and are not an endorsement by Moneris Solutions of the content of such linked websites or third parties. Moneris Solutions has no control over the contents of any linked website and is not responsible for these websites or their content or availability. If you decide to access any third party websites and make use of the information contained on them, you do so entirely at your own risk.

Numeric Currency Code (ISO)	Currency Name/Acronym
008	Albanian Lek (ALL)
012	Algerian Dinar (DZD)
032	Argentine Peso (ARS)
036	Australian Dollar (AUD)
048	Bahraini Dinar (BHD)
050	Bangladeshi Taka (BDT)
052	Barbados Dollar (BBD)
060	Bermudian Dollar (BMD)
064	Bhutan Ngultrum (BTN)
068	Bolivia Boliviano (BOB)
084	Belize Dollar (BZD)
090	Solomon Islands Dollar (SBD)
096	Brunei Dollar (BND)
108	Burundi Franc (BIF)
132	Cabo Verde Escudo (CVE)

Numeric Currency Code (ISO)	Currency Name/Acronym
136	Cayman Islands Dollar (KYD)
144	Sri Lanka Rupee (LKR)
152	Chilean Peso (CLP)
156	Chinese Yuan (CNY)
170	Colombian Peso (COP)
174	Comorian Franc (KMF)
188	Costa Rican Colon (CRC)
191	Croatian Kuna (HRK)
192	Cuban Peso (CUP)
203	Czech Koruna (CZK)
208	Danish Krone (DKK)
214	Dominican Republic Peso
222	Salvadoran Colon (SVC)
242	Fijian Dollar (FJD)
262	Djiboutian Franc (DJF)
270	Gambian Dalasi (GMD)
292	Gibraltar Pound (GIP)
320	Guatemala Quetzal (GTQ)
324	Guinean Franc (GNF)
328	Guyanese Dollar (GYD)
332	Haitian Gourde (HTG)
340	Honduran Lempira (HNL)

Numeric Currency Code (ISO)	Currency Name/Acronym
344	Hong Kong Dollar (HKD)
348	Hungarian Forint (HUF)
352	Iceland Krona (ISK)
356	Indian Rupee (INR)
360	Indonesian Rupiah (IDR)
376	Israeli Shekel (ILS)
388	Jamaican Dollar (JMD)
392	Japanese Yen (JPY)
398	Kazakh Tenge (KZT)
400	Jordanian Dinar (JOD)
404	Kenyan Shilling (KES)
410	South Korean Won (KRW)
414	Kuwaiti Dinar (KWD)
418	Laotian Kip (LAK)
426	Lesotho Loti (LSL)
430	Liberian Dollar (LRD)
446	Macanese Pataca (MOP)
454	Malawian Kwacha (MWK)
458	Malaysian Ringgit (MYR)
462	Maldivian Rufiyaa (MVR)
480	Mauritius Rupee (MUR)
484	Mexican Peso (MXN)

Numeric Currency Code (ISO)	Currency Name/Acronym
498	Moldovan Leu (MDL)
504	Moroccan Dirham (MAD)
512	Omani Rial (OMR)
516	Namibian Dollar (NAD)
524	Nepalese Rupee (NPR)
532	Netherlands Antillean Guilder (ANG)
533	Aruban Guilder (AWG)
548	Vanuatu Vatu (VUV)
554	New Zealand Dollar (NZD)
558	Nicaraguan Cordoba (NIO)
566	Nigerian Naira (NGN)
578	Norwegian Krone (NOK)
586	Pakistan Rupee (PKR)
598	Papua New Guinean Kina (PGK)
600	Paraguayan Guarani (PYG)
604	Peruvian Nuevo Sol (PEN)
608	Philippine Peso (PHP)
634	Qatari Rial (QAR)
643	Russian Ruble (RUB)
646	Rwandan Franc (RWF)
654	Saint Helena Pound (SHP)
682	Saudi Riyal (SAR)

Numeric Currency Code (ISO)	Currency Name/Acronym
690	Seychelles Rupee (SCR)
694	Sierra Leonean Leone (SLL)
702	Singapore Dollar (SGD)
704	Vietnamese Dong (VND)
710	South African Rand (ZAR)
748	Swaziland Lilangeni (SZL)
752	Swedish Krona (SEK)
756	Swiss Franc (CHF)
764	Thai Baht (THB)
780	Trinidad & Tobago Dollar (TTD)
784	UAE Dirham (AED)
788	Tunisian Dinar (TND)
800	Ugandan Shilling (UGX)
807	Macedonian Denar (MKD)
818	Egyptian Pound (EGP)
826	UK Pound Sterling (GBP)
834	Tanzanian Shilling (TZS)
840	US Dollar (USD)
858	Uruguayan Peso (UYU)
860	Uzbekistani Sum (UZS)
882	Samoan Tala (WST)
901	New Taiwan Dollar (TWD)

Numeric Currency Code (ISO)	Currency Name/Acronym
929	Mauritanian Ouguiya (MRU)
933	Belarusian Ruble (BYN)
934	Turkmenistan Manat (TMT)
941	Serbian Dinar (RSD)
943	Mozambique Metical (MZN)
944	Azerbaijani Manat (AZN)
946	Romanian New Leu (RON)
949	New Turkish Lira (TRY)
951	East Caribbean Dollar (XCD)
952	West African CFA Franc BCEAO (XOF)
953	CFP Franc (XPF)
967	Zambian Kwacha (ZMW)
968	Surinamese Dollar (SRD)
969	Malagasy Ariary (MGA)
971	Afghan Afghani (AFN)
972	Tajikistan Somoni (TJS)
973	Angola Kwanza (AOA)
975	Bulgarian Lev (BGN)
977	Bosnia and Herzegovina Convertible Mark (BAM)
978	Euro (EUR)
981	Georgian Lari (GEL)
985	Polish New Zloty (PLN)

Numeric Currency Code (ISO)	Currency Name/Acronym
986	Brazilian Real (BRL)

7.18 MCP Error Codes

Error Code	Description
200	OK (there will be no value returned in the MCP error message)
500	Upstream error
1000	Invalid JSON format
1003	Invalid txnType detected: <invalid txnType> please enter PURCHASE or REFUND
1005	Invalid rateInquiryId-txnType combination.
1007	Warning: at least one of cardHolderCurrency or merchantSettlementCurrency must be non-zero.
1008	Card-holder amount must be non-zero.
1009	Negative amounts detected
1010	Unsupported cardholder currency detected: <unsupported currency>
1015	invalid rateInquiryId
1016	Unsupported merchant id

8 Installments by Visa

- 8.1 About Installments by Visa
- 8.2 Installments by Visa Transaction Types
- 8.3 Sending Transactions with Installments by Visa
- 8.4 Installment Plan Lookup
- 8.5 Vault Installment Plan Lookup
- 8.6 Installment Info Object

8.1 About Installments by Visa

Installments by Visa enables issuers the ability to offer cardholders installment payment plans at the time of purchase. When a cardholder accepts an installment plan option, the merchant receives the payment in full, and the cardholder pays the issuer according to the plan.

For a full list of definitions of the request and response fields see B.3 Definition of Response Fields – Installments by Visa

8.2 Installments by Visa Transaction Types

Financial transactions that support Installments by Visa include the following:

- Purchase
- Pre-Authorization
- Pre-Authorization Completion
- Purchase Correction
- Refund

- Purchase with Vault – ResPurchaseCC
- Pre-Authorization with Vault – ResPreauthCC

NOTE: Independent Refund transactions do not support Installments by Visa

WARNING: Do not send the Installment Info object on any transaction that is not intended to offer Installments by Visa functionality; doing so may cause the transaction to fail.

8.3 Sending Transactions with Installments by Visa

Sending transactions with Installments by Visa functionality involves the following steps:

1. Send the Installment Plan Lookup or Vault Installment Plan Lookup (for Vault transactions) transaction request to obtain the **installment plan ID**, **installment plan reference** and **terms and conditions version** data in the response
2. Present the offered installment plan(s) to the cardholder and obtain their agreement to a particular plan.
3. Using the data obtained in the response above, send the Installment Info object in the Purchase or Pre-Authorization; for Vault transactions, use Purchase with Vault or Pre-Authorization with Vault

When completing the transaction with a Pre-Authorization Completion, or when doing a Purchase Correction or Refund, as in the rest of the Unified API, the previous transactions are referenced using the **order ID** and **transaction number**, or for Vault transactions, using the **data key**.

NOTE: Independent Refund transactions do not support Installments by Visa

8.4 Installment Plan Lookup

Used to obtain information required to do financial transactions with Installments by Visa.

Installment Plan Lookup transaction object definition

```
$txnArray = array('type'=>'installmentLookup', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Installment Plan Lookup transaction request fields – Required

Variable Name	Type and Limits	
order ID	String 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
credit card number	String max 20-character alpha-numeric	'pan'=>\$pan
expiry date	String	'expiry_date'=>\$expiry_date

Variable Name	Type and Limits
---------------	-----------------

4-character alphanumeric

YYMM

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
/***** Transaction Variables *****/
$type='installment_lookup';
$order_id='Test'.date("dmy-G:i:s");
$amount='600.00';
$pan='4761270070000310';
$expdate='2212';
/***** Transaction Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expdate
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
/* Status Check Example
$mpgHttpPost =new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);
*/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nBankTotals = " . $mpgResponse->getBankTotals());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
$eligibleInstallmentPlans = $mpgResponse->getEligibleInstallmentPlans();

$planCount = $eligibleInstallmentPlans->getPlanCount();

```

```

$installmentPlans = $eligibleInstallmentPlans->getInstallmentPlans();

for ($i = 0; $i < $planCount; $i++)
{
    print("\nPlanId = " . $installmentPlans[$i]->getPlanId());
    print("\nPlanIdRef = " . $installmentPlans[$i]->getPlanIdRef());
    print("\nName = " . $installmentPlans[$i]->getName());
    print("\nType = " . $installmentPlans[$i]->getType());
    print("\nNumInstallments = " . $installmentPlans[$i]->getNumInstallments());
    print("\nInstallmentFrequency = " . $installmentPlans[$i]->getInstallmentFrequency());
    $tac = $installmentPlans[$i]->getTac();
    $tacDetailsList = $tac->getTacDetailsList();
    $tacCount = $tac->getTacCount();
    print("\ntacCount = " . $tacCount);
    for ($j = 0; $j < $tacCount; $j++)
    {
        $tacDetails = $tacDetailsList[$j];

        print("\nText = " . $tacDetails->getText());
        print("\nUrl = " . $tacDetails->getUrl());
        print("\nVersion = " . $tacDetails->getVersion());
        print("\nLanguageCode = " . $tacDetails->getLanguageCode());
    }
    $promotionInfo = $installmentPlans[$i]->getPromotionInfo();
    print("\nPromotionCode = " . $promotionInfo->getPromotionCode());
    print("\nPromotionId = " . $promotionInfo->getPromotionId());
    $firstInstallment = $installmentPlans[$i]->getFirstInstallment();
    print("\nUpfrontFee = " . $firstInstallment->getUpfrontFee());
    print("\nInstallmentFee = " . $firstInstallment->getInstallmentFee());
    print("\nAmount = " . $firstInstallment->getAmount());
    $lastInstallment = $installmentPlans[$i]->getLastInstallment();
    print("\nInstallmentFee = " . $lastInstallment->getInstallmentFee());
    print("\nAmount = " . $lastInstallment->getAmount());
    print("\nAPR = " . $installmentPlans[$i]->getAPR());
    print("\nTotalFees = " . $installmentPlans[$i]->getTotalFees());
    print("\nTotalPlanCost = " . $installmentPlans[$i]->getTotalPlanCost());
}
?>

```

8.5 Vault Installment Plan Lookup

Used to obtain information required to do financial transactions with installments when using a token stored in the Moneris Vault.

Vault Installment Plan Lookup transaction object definition

```

$txnArray = array('type'=>'resInstallmentLookup', ...);

$mpgTxn = new mpgTransaction($txnArray);

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);

```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	<i>String</i>	'store_id'=>\$store_id

Variable Name	Type and Limits	
	N/A	
API token	String	'api_token'=>\$api_token
	N/A	

Vault Installment Plan Lookup transaction request fields – Required

Variable Name	Type and Limits	
order ID	String 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	String 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
data key	String 25-character alphanumeric	'data_key'=>\$data_key
expiry date	String 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
NOTE: Only send this field if using a temporary token; if not, omit this field		

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
/***** Transaction Variables *****/
$type='res_installment_lookup';
$order_id='Test'.date("dmy-G:i:s");
$amount='600.00';
$data_key='8cwY6hotzkM362ygNiytlBtY0';
$expdate='2212'; //For Temp Token
```



```

/***** Transaction Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'amount'=>$amount,
'data_key'=>$data_key,
'expdate'=>$expdate
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
/* Status Check Example
$mpgHttpPost =new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);
*/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nBankTotals = " . $mpgResponse->getBankTotals());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
$eligibleInstallmentPlans = $mpgResponse->getEligibleInstallmentPlans();

$planCount = $eligibleInstallmentPlans->getPlanCount();
$installmentPlans = $eligibleInstallmentPlans->getInstallmentPlans();

for ($i = 0; $i < $planCount; $i++)
{
print("\nPlanId = " . $installmentPlans[$i]->getPlanId());
print("\nPlanIdRef = " . $installmentPlans[$i]->getPlanIdRef());
print("\nName = " . $installmentPlans[$i]->getName());
print("\nType = " . $installmentPlans[$i]->getType());
print("\nNumInstallments = " . $installmentPlans[$i]->getNumInstallments());
print("\nInstallmentFrequency = " . $installmentPlans[$i]->getInstallmentFrequency());
$tac = $installmentPlans[$i]->getTac();
$tacDetailsList = $tac->getTacDetailsList();
$tacCount = $tac->getTacCount();
print("\ntacCount = " . $tacCount);
for ($j = 0; $j < $tacCount; $j++)
{
$tacDetails = $tacDetailsList[$j];

print("\nText = " . $tacDetails->getText());
print("\nUrl = " . $tacDetails->getUrl());
print("\nVersion = " . $tacDetails->getVersion());
print("\nLanguageCode = " . $tacDetails->getLanguageCode());
}
$promotionInfo = $installmentPlans[$i]->getPromotionInfo();
print("\nPromotionCode = " . $promotionInfo->getPromotionCode());
print("\nPromotionId = " . $promotionInfo->getPromotionId());
}

```

```

$firstInstallment = $installmentPlans[$i]->getFirstInstallment();
print("\nUpfrontFee = " . $firstInstallment->getUpfrontFee());
print("\nInstallmentFee = " . $firstInstallment->getInstallmentFee());
print("\nAmount = " . $firstInstallment->getAmount());
$lastInstallment = $installmentPlans[$i]->getLastInstallment();
print("\nInstallmentFee = " . $lastInstallment->getInstallmentFee());
print("\nAmount = " . $lastInstallment->getAmount());
print("\nAPR = " . $installmentPlans[$i]->getAPR());
print("\nTotalFees = " . $installmentPlans[$i]->getTotalFees());
print("\nTotalPlanCost = " . $installmentPlans[$i]->getTotalPlanCost());
}
//Resolve Data
print("\ncust_id = " . $mpgResponse->getResDataCustId());
print("\nphone = " . $mpgResponse->getResDataPhone());
print("\nemail = " . $mpgResponse->getResDataEmail());
print("\nnote = " . $mpgResponse->getResDataNote());
print("\nexdate = " . $mpgResponse->getResDataExpdate());
print("\nmasked_pan = " . $mpgResponse->getResDataMaskedPan());
?>

```

8.6 Installment Info Object

When sending Purchase or Pre-Authorization transactions with Installments by Visa, the Installment Info object is included in the request. The Installment Info object uses information received in the response to the Installment Plan Lookup transaction.

For a full list of definitions of the request and response fields see B.3 Definition of Response Fields – Installments by Visa

Installment Info object request fields

Variable Name	Type and Limits	
installment plan ID	<i>String</i> 36-character alphanumeric fixed length	<code>\$installmentInfo->setPlanId("VALUE");</code>
installment plan reference	<i>String</i> 10-character alphanumeric fixed length	<code>\$installmentInfo->setPlanIdRef("VALUE");</code>
terms and conditions version	<i>String</i> 10-character alphanumeric variable length (1-10 characters)	<code>\$installmentInfo->setTacVersion("VALUE");</code>

WARNING: Do not send the Installment Info object on any transaction that is not intended to offer Installments by Visa functionality; doing so may cause the transaction to fail.

9 e-Fraud Tools

- 9.1 Address Verification Service
- 9.2 Card Validation Digits (CVD)
- 9.3 Transaction Risk Management Tool

9.1 Address Verification Service

- 9.1.1 About Address Verification Service (AVS)
- 9.1.2 AVS Info Object
- 9.1.3 AVS Response Codes
- 9.1.4 AVS Sample Code

9.1.1 About Address Verification Service (AVS)

Address Verification Service (AVS) is an optional fraud-prevention tool offered by issuing banks whereby a cardholder's address is submitted as part of the transaction authorization. The AVS address is then compared to the address kept on file at the issuing bank. AVS checks whether the street number, street name and zip/postal code match. The issuing bank returns an AVS result code indicating whether the data was matched successfully. Regardless of the AVS result code returned, the credit card is authorized by the issuing bank.

The response that is received from AVS verification is intended to provide added security and fraud prevention, but the response itself does not affect the completion of a transaction. Upon receiving a response, the choice to proceed with a transaction is left entirely to the merchant. The responses is **not** a strict guideline of whether a transaction will be approved or declined.

The following transactions support AVS:

- Purchase (Basic and Mag Swipe)
- Pre-Authorization (Basic)
- Re-Authorization (Basic)
- ResAddCC (Vault)
- ResUpdateCC (Vault)

Things to Consider:

- AVS is supported by Visa, MasterCard, American Express, Discover and JCB.
- When testing AVS, you must **only** use the Visa test card numbers 4242424242424242 or 4005554444444403, and the amounts described in the Simulator eFraud Response Codes document available at the Moneris developer portal (<https://developer-moneris.com>).
- Store ID "store5" is set up to support AVS testing.

9.1.2 AVS Info Object

AVSInfo object definition

```
$avsTemplate = array(

'avs_street_number'=>$avs_street_number,

'avs_street_name' =>$avs_street_name,

'avs_zipcode' => $avs_zipcode,

'avs_hostname'=>$avs_hostname,

'avs_browser' =>$avs_browser,

'avs_shiptocountry' => $avs_shiptocountry,

'avs_merchprodsku' => $avs_merchprodsku,

'avs_custip'=>$avs_custip,

'avs_custphone' => $avs_custphone

);

$mpgAvsInfo = new mpgAvsInfo ($avsTemplate);
```

Transaction object set method

```
$mpgTxn->setAvsInfo ($mpgAvsInfo);
```

Variable Name	Type and Limits	Set Method	Description
AVS street number	String 19-character alpha-numeric NOTE: this character limit is a combined total allowed for AVS street number and AVS street name	'avs_street_number'=>'212'	Cardholder street number
AVS street name	String 19-character alpha-numeric NOTE: this character	'avs_street_name'=>'Payton Street'	Cardholder street name

Variable Name	Type and Limits	Set Method	Description
	limit is the combined total allowed for AVS street number and AVS street name		
AVS zip/postal code	<i>String</i> 9-character alpha-numeric	'avs_zipcode' => 'M1M1M1 '	Cardholder zip/postal code

9.1.3 AVS Response Codes

Below is a full list of possible AVS response codes. These can be returned when you call the `$mp-gResponse->getAvsResultCode()` method.

Code	Visa	Mastercard/Discover	American Express/ JCB
A	AVS street address only (partial match)	Address matches, zip/ postal code does not	Billing address matches, zip/postal code does not
D	No longer applicable to Visa	N/A	Customer name incorrect; zip/postal code matches
E	N/A	N/A	Customer name incorrect, billing address and zip/postal code match
F	No longer applicable to Visa	N/A	Customer name incorrect; billing address matches
G	No longer applicable to Visa	Address information not verified for international transaction	N/A
K	N/A	N/A	Customer name matches

Code	Visa	Mastercard/Discover	American Express/ JCB
L	N/A	N/A	Customer name and zip/postal code match
M	No longer applicable to Visa	N/A	Customer name, billing address, and zip/postal code match
N	AVS non-match	Neither address nor zip/postal code matches	Billing address and zip/postal code do not match
O	N/A	N/A	Customer name and billing address match
R	AVS indeterminate outcome (retry)	Retry; system unable to process	System unavailable; retry
S	No longer applicable to Visa	AVS currently not supported	AVS currently not supported
T	N/A	Nine-digit zip code matches; address does not match	N/A
U	AVS unable to verify	No data from issuer-authorization system	Information is unavailable
W	No longer applicable to Visa	For U.S. addresses, nine-digit postal code matches, address does not For addresses outside the U.S., postal code matches, address does not	Customer name, billing address, and zip/postal code are all correct matches
X	No longer applicable to Visa	For U.S. addresses, nine-digit postal code and address match For addresses outside the U.S., postal code and address match	N/A
Y	AVS full match	Billing address and zip/postal code both match	Billing address and zip/postal code both match

Code	Visa	Mastercard/Discover	American Express/ JCB
Z	AVS zip/postal code only (partial match)	For U.S. addresses, five-digit zip code matches, address does not match	Zip/postal code matches, billing address does not

9.1.4 AVS Sample Code

This is a sample of Java code illustrating how AVS is implemented with a Purchase transaction. Purchase object information that is not relevant to AVS has been removed.

For more about Purchase transactions, see 2.1 Purchase.

Sample Purchase with AVS information
<pre> \$avs_street_number = '201'; \$avs_street_name = 'Michigan Ave'; \$avs_zipcode = 'M1M1M1'; \$avs_email = 'test@host.com'; \$avs_hostname = "www.testhost.com"; \$avs_browser = 'Mozilla'; \$avs_shiptocountry = 'Canada'; \$avs_merchprodsku = '123456'; \$avs_custip = '192.168.0.1'; \$avs_custphone = '5556667777'; \$avsTemplate = array('avs_street_number'=>\$avs_street_number, 'avs_street_name' =>\$avs_street_name, 'avs_zipcode' => \$avs_zipcode, 'avs_hostname'=>\$avs_hostname, 'avs_browser' =>\$avs_browser, 'avs_shiptocountry' => \$avs_shiptocountry, 'avs_merchprodsku' => \$avs_merchprodsku, 'avs_custip'=>\$avs_custip, 'avs_custphone' => \$avs_custphone); \$mpgAvsInfo = new mpgAvsInfo (\$avsTemplate); \$txnArray=array('type'=>'purchase', 'order_id'=>\$order_id, 'cust_id'=>\$cust_id, 'amount'=>\$amount, 'pan'=>\$pan, 'expdate'=>\$expiry_date, 'crypt_type'=>\$crypt); \$mpgTxn = new mpgTransaction(\$txnArray); \$mpgTxn->setAvsInfo(\$mpgAvsInfo); </pre>

9.2 Card Validation Digits (CVD)

- 9.2.1 About Card Validation Digits (CVD)
- 9.2.3 CVD Information Object
- 9.2.4 CVD Result Codes
- 9.2.5 Sample Purchase with CVD Info Object

9.2.1 About Card Validation Digits (CVD)

The Card Validation Digits (CVD) value is an additional number printed on credit cards that is used as an additional check when verifying cardholder credentials during a transaction.

The response that is received from CVD verification is intended to provide added security and fraud prevention, but the response itself does not affect the completion of a transaction. Upon receiving a response, the choice whether to proceed with a transaction is left entirely to the merchant. The responses is **not** a strict guideline of which transaction will approve or decline.

The following transactions support CVD:

- Purchase (Basic, Vault and Mag Swipe)
- Pre-Authorization (Basic and Vault)
- Re-Authorization

Things to Consider:

- CVD is only supported by Visa, MasterCard, American Express, Discover, JCB and UnionPay.
- For UnionPay cards, the CVD response will not be returned; the issuer will approve or decline based on the CVD result.
- When testing CVD, you must **only** use the Visa test card numbers 4242424242424242 or 4005554444444403, and the amounts described in the Simulator eFraud Response Codes document available at the Moneris developer portal (<https://developer-moneris.com>).
- Test store_id "store5" is set up to support CVD testing.

9.2.2 Transactions Where CVD Is Required

The Card Validation Digits (CVD) object is required in transaction requests in the following scenarios:

- Initial transactions when storing cardholder credentials in Credential on File scenarios; subsequent follow-on transactions do not use CVD
- Any Purchase, Pre-Authorization or Card Verification where you are not storing cardholder credentials

9.2.3 CVD Information Object

NOTE: The CVD value must only be passed to the Moneris Gateway. Under **no** circumstances may it be stored for subsequent uses or displayed as part of the receipt information.

CvdInfo object definition

```
CvdInfo cvdCheck = new CvdInfo();  
  
$cvdTemplate = array(  
    'cvd_indicator' => $cvd_indicator,  
    'cvd_value' => $cvd_value  
);  
  
$mpgCvdInfo = new mpgCvdInfo ($cvdTemplate);
```

Transaction object set method

```
transaction.setCvdInfo(cvdCheck);  
  
$mpgTxn->setCvdInfo($mpgCvdInfo);
```

Table 1 CVD Info Object – Required Fields

Variable Name	Type and Limits	Set Method	Description
CVD indicator	<i>String</i> 1-character numeric	'cvd_indicator' =>'1'	Indicates presence of CVD Possible values: 0: CVD value is deliberately bypassed or is not provided by the merchant. 1: CVD value is present.

Variable Name	Type and Limits	Set Method	Description
			2: CVD value is on the card, but is illegible. 9: Cardholder states that the card has no CVD imprint.

CVD value

String

4-character numeric

'cvd_value'
=>'123'

CVD value located on credit card

NOTE: The CVD value must only be passed to the Moneris Gateway. Under **no** circumstances may it be stored for subsequent uses or displayed as part of the receipt information.

9.2.4 CVD Result Codes

CVD verification is available for Visa, Mastercard, Discover, American Express, JCB and UnionPay transactions.

Code	Description
M	Match
N	No match
P	Not processed
S	CVD should be on the card, but Merchant has indicated that CVD is not present
U	Issuer is not a CVD participant
Y	Match for American Express/JCB only
D	Invalid security code for American Express or JCB only
Other	Invalid response code

9.2.5 Sample Purchase with CVD Info Object

This is a sample of Java code illustrating how CVD is implemented with a Purchase transaction. Purchase object information that is not relevant to CVD has been removed.

Sample Purchase with CVD Information

```
$cvdTemplate = array(
    'cvd_indicator' => '1',
    'cvd_value' => '123'
);
$mpgCvdInfo = new mpgCvdInfo ($cvdTemplate);
$txnArray=array(
    'type'=>'purchase',
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$pan,
    'expdate'=>$expiry_date,
    'crypt_type'=>$crypt
);
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setCvdInfo ($mpgCvdInfo);
```

9.3 Transaction Risk Management Tool

- 9.3.1 About the Transaction Risk Management Tool
- 9.3.2 Introduction to Queries
- 9.3.3 Session Query
- 9.3.4 Attribute Query
- 9.3.6 Inserting the Profiling Tags Into Your Website
- 9.3.6 Inserting the Profiling Tags Into Your Website

The Transaction Risk Management Tool (TRMT) is available to **Canadian integrations** only.

9.3.1 About the Transaction Risk Management Tool

The Transaction Risk Management Tool provides additional information to assist in identifying fraudulent transactions. To maximize the benefits from the Transaction Risk Management Tool, it is highly recommended that you:

- Carefully consider the business logic and processes that you need to implement surrounding the handling of response information the Transaction Risk Management Tool provides.
- Implement the other fraud tools available through Moneris Gateway (such as AVS, CVD, Verified by Visa, MasterCard SecureCode and American Express SafeKey).

9.3.2 Introduction to Queries

There are two types of transactions associated with the Transaction Risk Management Tool (TRMT):

- Session Query (page 392)
- Attribute Query (page 399)

The Session Query and Attribute Query are used at the time of the transaction to obtain the risk assessment.

Moneris recommends that you use the Session Query as much as possible for obtaining your risk assessment because it uses the device fingerprint as well as other transaction information when providing the risk scores.

To use the Session Query, you must implement two components:

- Tags on your website to collect the device fingerprinting information
- Session Query transaction.

If you are not able to collect the necessary information for the Session Query (such as the device fingerprint), then use the Attribute Query.

9.3.3 Session Query

Once a device profiling session has been initiated upon a client device, the Session Query API is used at the time of the transaction or even to obtain a device identifier or 'fingerprint', attribute list and risk assessment for the client device.

Session Query transaction object definition

```
$riskTxn = new riskTransaction($txnArray);
```

HttpPostRequest object for Session Query transaction

```
$riskHttpPost = new riskHttpPost($store_id, $api_token, $riskRequest);
```

Session Query transaction values

Table 3: Session Query transaction object mandatory values

Value	Type	Limits	Set method
	Description		
Session ID	String	9-character decimal Permitted characters: [a-z], [A-Z], 0-9, _, -	'session_id'=>\$session_id
	Web server session identifier generated when device profiling was initiated.		
Service type	String	9-character decimal	'service_type'=>\$service_type
	Which output fields are returned. session -- returns IP and device related attributes.		
Event type	String	payment	'event_type'=>\$event_type
	Defines the type of transaction or event for reporting purposes. payment - Purchasing of goods/services.		

Table 3: Session Query transaction object mandatory values (continued)

Value	Type	Limits	Set method
	Description		
Credit card number (PAN)	String	20-character numeric No spaces or dashes	'pan'=>\$pan
	Most credit card numbers today are 16 digits, but some 13-digit numbers are still accepted by some issuers. This field has been intentionally expanded to 20 digits in consideration for future expansion and potential support of private label card ranges.		
Account address street 1	String	32-character alphanumeric	'account_address_street1'=>\$account_address_street1
	First portion of the street address component of the billing address.		
Account Address street 2	String	32-character alphanumeric	'account_address_street2'=>\$account_address_street2
	Second portion of the street address component of the billing address.		
Account address city	String	50-character alphanumeric	'account_address_city'=>\$account_address_city
	The city component of the billing address.		
Account address state/- province	String	64-character alphanumeric	'account_address_state'=>\$account_address_state
	The state/province component of the billing address.		
Account address country	String	2-character alphanumeric	'account_address_country'=>\$account_address_country
	ISO2 country code of the billing addresses.		
Account address ZIP/- postal code	String	8-character alphanumeric	'account_address_zip'=>\$account_address_zip
	ZIP/postal code of the billing address.		
Shipping address street 1	String	32-character alphanumeric	'shipping_address_street1'=>\$shipping_address_street1
	First portion of the street address component of the shipping address.		

Table 3: Session Query transaction object mandatory values (continued)

Value	Type	Limits	Set method
	Description		
Shipping address street 2	String	32-character alphanumeric	'shipping_address_street2'=>\$shipping_address_street2
	Second portion of the street address component of the shipping address.		
Shipping address city	String	50-character alphanumeric	'shipping_address_city'=>\$shipping_address_city
	City component of the shipping address.		
Shipping address state/- province	String	64-character alphanumeric	'shipping_address_state'=>\$shipping_address_state
	The state/province component of the shipping address.		
Shipping address country	String	2-character alphanumeric	'shipping_address_country'=>\$shipping_address_country
	ISO2 country code of the account address country.		
Shipping address ZIP	String	8-character alphanumeric	'shipping_address_zip'=>\$shipping_address_zip
	The ZIP/postal code component of the shipping address.		
Local attribute 1-5	String	255-character alphanumeric	
	These five attributes can be used to pass custom attribute data. These are used if you wish to correlate some data with the returned device information.		
Transaction amount	String	255-character alphanumeric Must contain 2 decimal places	
	The numeric currency amount.		
Transaction currency	String	10-character numeric	
	The currency type that the transaction was denominated in. If TransactionAmount is passed, the TransactionCurrency is required. Values to be used are: • CAD – 124 • USD – 840		

Table 4: Session Query transaction object optional values

Value	Type	Limits	Set method
	Description		
Account login	String	255-character alphanumeric	'account_login'=>\$account_login
	The Account Login name.		
Password hash	String	40-character alphanumeric	'password_hash' =>\$password_hash
	The input must be a SHA-2 hash of the password in hexadecimal format. Used to check if it is on a watch list.		
Account number	String	255-character alphanumeric	'account_number' => \$account_number
	The account number for the account.		
Account name	String	255-character alphanumeric	'account_name' => \$account_name
	Account name (or concatenation of first and last name of account holder).		
Account email	String	100-character alphanumeric	'account_email'=>\$account_email
	The email address entered into the form for this contact. Used to check if this is a high risk account email id.		
Account telephone	String	32-character alphanumeric	
	Contact telephone number including country and city codes. All whitespace is removed. Must be in format: 0..9,<space>,(,),[,] braces must be matched.		
Address street 1	String	32-character alphanumeric	
	The first portion of the street address component of the account address.		
Address street 2	String	32-character alphanumeric	
	The second portion of the street address component of the account address.		
Address city	String	50-character alphanumeric	
	The city component of the account address.		
Address state/- province	String	64-character alphanumeric	
	The state/province component of the account address		

Table 4: Session Query transaction object optional values (continued)

Value	Type	Limits	Set method
	Description		
Address country	String	2-character alphanumeric	
	The 2 character ISO2 country code of the account address country		
Address ZIP	String	8-character alphanumeric	
	The ZIP/postal code of the account address.		
Ship Address Street 1	String	32-character alphanumeric	
	The first portion of the street address component of the shipping address		
Ship Address Street 2	String	32-character alphanumeric	
	The second portion of the street address component of the shipping address		
Ship Address City	String	50-character alphanumeric	
	The city component of the shipping address		
Ship Address State/Province	String	64-character alphanumeric	
	The state/province component of the shipping address		
Ship Address Country	String	2-character alphanumeric	
	The 2 character ISO2 country code of the shipping address country		
Ship Address ZIP	String	8-character alphanumeric	
	The ZIP/postal code of the shipping address		
CC Number Hash	String	255-character alphanumeric	
	This is a SHA-2 hash (in hexadecimal format) of the credit card number.		
Custom Attribute 1-8	String	255-character alphanumeric	
	These 8 attributes can be used to pass custom attribute data which can be used within the rules.		

Sample Session Query - CA

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$api_token='hurgle';
/***** Transactional Variables *****/
$type='session_query';
$order_id='risktest-' . date("dmy-G:i:s");
$session_id='abc123';
$service_type='session';
//$event_type='login';
/***** SessionAccountInfo Variables *****/
$policy = '';
$device_id = '4EC40DE5-0770-4fa0-BE53-981C067C598D';
$account_login = '13195417-8CA0-46cd-960D-14C158E4DBB2';
$password_hash = '489c830f10f7c601d30599a0deaf66e64d2aa50a';
$account_number = '3E17A905-AC8A-4c8d-A417-3DADA2A55220';
$account_name = '4590FCC0-DF4A-44d9-A57B-AF9DE98B84DD';
$account_email = '3CAE72EF-6B69-4a25-93FE-2674735E78E8@test.threatmetrix.com';
$account_telephone = '5556667777';
$pan = '4242424242424242';
$account_address_street1 = '3300 Bloor St W';
$account_address_street2 = '4th Flr West Tower';
$account_address_city = 'Toronto';
$account_address_state = 'Ontario';
$account_address_country = 'CA';
$account_address_zip = 'M8X2X2';
$shipping_address_street1 = '3300 Bloor St W';
$shipping_address_street2 = '4th Flr West Tower';
$shipping_address_city = 'Toronto';
$shipping_address_state = 'Ontario';
$shipping_address_country = 'CA';
$shipping_address_zip = 'M8X2X2';
$local_attrib_1 = 'a';
$local_attrib_2 = 'b';
$local_attrib_3 = 'c';
$local_attrib_4 = 'd';
$local_attrib_5 = 'e';
$online_tld = 'Facebook';
$online_id_handle = 'Moneris';
$transaction_amount = '1.00';
$transaction_currency = '124';
/***** SessionAccountInfo Associative Array *****/
$sessionAccountInfoTemplate = array
(
    'account_login'=>$account_login,
    'password_hash' =>$password_hash,
    'account_number' => $account_number,
    'account_name' => $account_name,
    'account_email'=>$account_email,
    'pan' =>$pan
);
/***** SessionAccountInfo Object *****/
$mpgSessionAccountInfo = new mpgSessionAccountInfo ($sessionAccountInfoTemplate);
/***** Transactional Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'session_id'=>$session_id,
```

Sample Session Query - CA

```

'service_type'=>$service_type
);
/***** Transaction Object *****/
$riskTxn = new riskTransaction($txnArray);
/***** Set SessionAccountInfo *****/
$riskTxn->setSessionAccountInfo($mpgSessionAccountInfo);
/***** Request Object *****/
$riskRequest = new riskRequest($riskTxn);
$riskRequest->setTestMode(true);
/***** HTTPS Post Object *****/
$riskHttpPost = new riskHttpPost($store_id,$api_token,$riskRequest);
/***** Response *****/
$riskResponse=$riskHttpPost->getRiskResponse();
//print("\nResponse = " . $riskResponse);
print("\nResponseCode = " . $riskResponse->getResponseCode());
print("\nMessage = " . $riskResponse->getMessage());
$results = $riskResponse->getResults();
foreach($results as $key => $value)
{
print("\n".$key ." = ". $value);
}
$rules = $riskResponse->getRules();
//print_r($rules);
foreach ($rules as $i)
{
foreach ($i as $key => $value)
{
echo "\n$key = $value";
}
}
}
?>

```

9.3.3.1 Session Query Transaction Flow

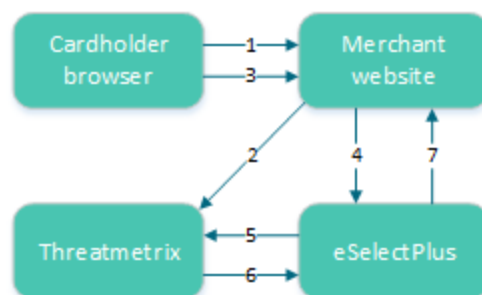


Figure 2: Session Query transaction flow

1. Cardholder logs onto the merchant website.
2. When the page has loaded in the cardholder's browser, special tags within the site allow information from the device to be gathered and sent to ThreatMetrix as the device fingerprint.
The HTML tags should be placed where the cardholder is resident on the page for a couple of seconds to get the broadest data possible.
3. Customer submits a transaction.

4. Merchant's web application makes a Session Query transaction to the Moneris Gateway using the same session id that was included in the device fingerprint. This call must be made within 30 minutes of profiling (2).
5. Moneris Gateway submits the Session Query data to ThreatMetrix.
6. ThreatMetrix uses the Session Query data and the device fingerprint information to assess the transaction against the rules. A score is generated based on the rules.
7. The merchant uses the returned device information in its risk analysis to make a business decision. The merchant may wish to continue or cancel with the cardholder's payment transaction.

9.3.4 Attribute Query

The Attribute Query is used to obtain a risk assessment of transaction-related identifiers such as the email address and the card number. Unlike the Session Query, the Attribute Query does not require the device fingerprinting information to be provided.

AttributeQuery transaction object definition

```
$riskTxn = new riskTransaction($txnArray);
```

HttpPostRequest object for AttributeQuery transaction

```
$riskHttpPost = new riskHttpPost($store_id, $api_token, $riskRequest);
```

Attribute Query transaction values

Table 5: Attribute Query transaction object mandatory values

Value	Type		Limits	Set method
	Description			
Service type	String	N/A		'service_type'=>\$service_type
	Which output fields are returned. session -- returns IP and device related attributes.			
Device ID	String	36-character alphanumeric		'device_id'=>\$device_id
	Unique device identifier generated by a previous call to the ThreatMetrix session-query API.			
Credit card number	String	20-character numeric No spaces or dashes		'pan'=>\$pan
	Most credit card numbers today are 16 digits, but some 13-digit numbers are still accepted by some issuers. This field has been intentionally expanded to 20 digits in consideration for future expansion and potential support of private label card ranges.			

Table 5: Attribute Query transaction object mandatory values (continued)

Value	Type	Limits	Set method
	Description		
IP address	String	64-character alphanumeric	'ip_address'=>\$ip_address
	True IP address. Results will be returned as true_ip_geo, true_ip_score and so on.		
IP forwarded	String	64-character alphanumeric	'ip_forwarded'=>\$ip_forwarded
	The IP address of the proxy. If the IPAddress is supplied, results will be returned as proxy_ip_geo and proxy_ip_score. If the IP Address is not supplied, this IP address will be treated as the true IP address and results will be returned as true_ip_geo, true_ip_score and so on		
Account address street 1	String	32-character alphanumeric	'account_address_street1'=>\$account_address_street1
	First portion of the street address component of the billing address.		
Account Address Street 2	String	32-character alphanumeric	'account_address_street2'=>\$account_address_street2
	Second portion of the street address component of the billing address.		
Account address city	String	50-character alphanumeric	'account_address_city'=>\$account_address_city
	The city component of the billing address.		
Account address state/- province	String	64-character alphanumeric	'account_address_state'=>\$account_address_state
	The state component of the billing address.		
Account address country	String	2-character alphanumeric	'account_address_country'=>\$account_address_country
	ISO2 country code of the billing addresses.		
Account address zip/- postal code	String	8-character alphanumeric	'account_address_zip'=>\$account_address_zip
	Zip/postal code of the billing address.		
Shipping address street 1	String	32-character alphanumeric	'shipping_address_street1'=>\$shipping_address_street1
	Account address country		

Table 5: Attribute Query transaction object mandatory values (continued)

Value	Type	Limits	Set method
	Description		
Shipping Address Street 2	String	32-character alphanumeric	'shipping_address_street2'=>\$shipping_address_street2
	Second portion of the street address component of the shipping address.		
Shipping Address City	String	50-character alphanumeric	'shipping_address_city'=>\$shipping_address_city
	City component of the shipping address.		
Shipping Address State/Province	String	64-character alphanumeric	'shipping_address_state'=>\$shipping_address_state
	State/Province component of the shipping address.		
Shipping Address Country	String	2-character alphanumeric	'shipping_address_country'=>\$shipping_address_country
	ISO2 country code of the account address country.		
Shipping Address zip/-postal code	String	8-character alphanumeric	'shipping_address_zip'=>\$shipping_address_zip
	The zip/postal code component of the shipping address.		

Sample Attribute Query

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='moneris';
$sapi_token='hurgle';
/***** Transactional Variables *****/
$type='session_query';
$order_id='risktest-'.date("dmy-G:i:s");
$session_id='abc123';
$service_type='session';
//$event_type='login';
/***** SessionAccountInfo Variables *****/
$policy = '';
$device_id = '4EC40DE5-0770-4fa0-BE53-981C067C598D';
$account_login = '13195417-8CA0-46cd-960D-14C158E4DBB2';
$password_hash = '489c830f10f7c601d30599a0deaf66e64d2aa50a';
$account_number = '3E17A905-AC8A-4c8d-A417-3DADA2A55220';
$account_name = '4590FCC0-DF4A-44d9-A57B-AF9DE98B84DD';
$account_email = '3CAE72EF-6B69-4a25-93FE-2674735E78E8@test.threatmetrix.com';
$account_telephone = '5556667777';
$pan = '4242424242424242';
$account_address_street1 = '3300 Bloor St W';
$account_address_street2 = '4th Flr West Tower';
```

Sample Attribute Query

```

$account_address_city = 'Toronto';
$account_address_state = 'Ontario';
$account_address_country = 'CA';
$account_address_zip = 'M8X2X2';
$shipping_address_street1 = '3300 Bloor St W';
$shipping_address_street2 = '4th Flr West Tower';
$shipping_address_city = 'Toronto';
$shipping_address_state = 'Ontario';
$shipping_address_country = 'CA';
$shipping_address_zip = 'M8X2X2';
$local_attr1 = 'a';
$local_attr2 = 'b';
$local_attr3 = 'c';
$local_attr4 = 'd';
$local_attr5 = 'e';
$online_tld = 'Facebook';
$online_id_handle = 'Moneris';
$transaction_amount = '1.00';
$transaction_currency = '124';
/***** SessionAccountInfo Associative Array *****/
$sessionAccountInfoTemplate = array
(
    'account_login'=>$account_login,
    'password_hash' =>$password_hash,
    'account_number' => $account_number,
    'account_name' => $account_name,
    'account_email'=>$account_email,
    'pan' =>$pan
);
/***** SessionAccountInfo Object *****/
$mpgSessionAccountInfo = new mpgSessionAccountInfo ($sessionAccountInfoTemplate);
/***** Transactional Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'session_id'=>$session_id,
    'service_type'=>$service_type
);
/***** Transaction Object *****/
$riskTxn = new riskTransaction($txnArray);
/***** Set SessionAccountInfo *****/
$riskTxn->setSessionAccountInfo($mpgSessionAccountInfo);
/***** Request Object *****/
$riskRequest = new riskRequest($riskTxn);
$riskRequest->setTestMode(true);
/***** HTTPS Post Object *****/
$riskHttpPost =new riskHttpPost($store_id,$api_token,$riskRequest);
/***** Response *****/
$riskResponse=$riskHttpPost->getRiskResponse();
//print("\nResponse = " . $riskResponse);
print("\nResponseCode = " . $riskResponse->getResponseCode());
print("\nMessage = " . $riskResponse->getMessage());
$results = $riskResponse->getResults();
foreach($results as $key => $value)
{
    print("\n".$key ." = ". $value);
}
$rules = $riskResponse->getRules();
//print_r($rules);

```

Sample Attribute Query
<pre>foreach (\$rules as \$i) { foreach (\$i as \$key => \$value) { echo "\n\$key = \$value"; } } ?></pre>

9.3.4.1 Attribute Query Transaction Flow

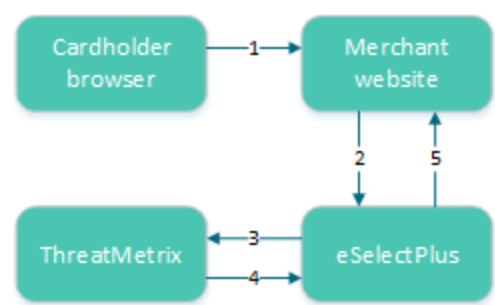


Figure 3: Attribute query transaction flow

1. Cardholder logs onto merchant website and submits a transaction.
2. The merchant’s web application makes an Attribute Query transaction that includes the session ID to the Moneris Gateway.
3. Moneris Gateway submits Attribute Query data to ThreatMetrix.
4. ThreatMetrix uses the Attribute Query data to assess the transaction against the rules. A score is generated based on the rules.
5. The merchant uses the returned device information in its risk analysis to make a business decision. The merchant may wish to continue or cancel with the cardholder's payment transaction.

9.3.5 Handling Response Information

When reviewing the response information and determining how to handle the transaction, it is recommended that you (either manually or through automated logic on your site) use the following pieces of information:

- Risk score
- Rules triggered (such as Rule Codes, Rule Names, Rule Messages)
- Results obtained from Verified by Visa, MasterCard Secure Code, AVS, CVD and the financial transaction authorization

- Response codes for the Transaction Risk Management Transaction that are included by automated processes.

9.3.5.1 TRMT Response Fields

Table 6: Receipt object response values for TRMT

Value	Type	Limits	Get method
	Definition		
Response Code	String	3-character alphanumeric	<code>\$mpgResponse->getResponseCode () ;</code>
	001 – Success 981 – Data error 982 – Duplicate Order ID 983 – Invalid Transaction 984 – Previously asserted 985 – Invalid activity description 986- Invalid impact description 987 – Invalid Confidence description 988 - Cannot find Previous		
Message	String	N/A	<code>\$mpgResponse->getMessage () ;</code>
	Response message		
Event type	String	N/A	
	Type of transaction or event returned in the response.		
Org ID	String	N/A	
	ThreatMetrix-defined unique transaction identifier		
Policy	String	N/A	
	Policy used for the Session Query will be returned with the return request. If the Policy was not included, then the Policy name default is returned.		

Table 6: Receipt object response values for TRMT (continued)

Value	Type	Limits	Get method
	Definition		
Policy score	String	N/A	
	The sum of all the risks weights from triggered rules within the selected policy in the range [-100...100]		
Request duration	String	N/A	
	Length of time it takes for the transaction to be processed.		
Request ID	String	N/A	
	Unique number and will always be returned with the return request.		
Request result	String	N/A	
	See 9.3.5.1 (page 404).		
Review status	String	N/A	
	The transaction status based on the assessments and risk scores.		
Risk rating	String	N/A	
	The rating based on the assessments and risk scores.		
Service type	String	N/A	
	The service type will be returned in the attribute query response.		
Session ID	String	N/A	
	Temporary identifier unique to the visitor will be returned in the return request.		
Summary risk score	String	N/A	
	Based on all of the returned values in the range [-100 ... 100]		
Transaction ID	String	N/A	
	This is the transaction identifier and will always be returned in the response when supplied as input.		
Unknown session	String	N/A	
	If present, the value is "yes". It indicates the session ID that was passed was not found.		

Table 7: Response code descriptions

Value	Definition
001	Success

Value	Definition
981	Data error
982	Duplicate order ID
983	Invalid transaction
984	Previously asserted
985	Invalid activity description
986	Invalid impact description
987	Invalid confidence description
988	Cannot find previous

Table 8: Request result values and descriptions

Value	Definition
fail_duplicate_entities_of_same_type	More than one entity of the same was specified, e.g. password_hash was specified twice.
fail_incomplete	ThreatMetrix was unable to process the request due to incomplete or incorrect input data
fail_invalid_account_number	The format of the supplied account number was invalid
fail_invalid_characters	Invalid characters submitted
fail_invalid_charset	The value of character set was invalid
fail_invalid_currency_code	The format of the currency_code was invalid
fail_invalid_currency_format	The format of the currency_format was invalid
fail_invalid_telephone_number	Format of the supplied telephone number was invalid
fail_access	ThreatMetrix was unable to process the request because of API verification failing
fail_internal_error	ThreatMetrix encountered an error while processing the request
fail_invalid_device_id	Format of the supplied device_id was invalid

Value	Definition
fail_invalid_email_address	Format of the supplied email address was invalid
fail_invalid_fuzzy_device_id	The format of fuzzy_device_id was invalid
fail_invalid_ip_address_parameter	Format of a supplied ip_address parameter was invalid
fail_invalid_parameter	The format of the parameter was invalid, or the value is out of boundary
fail_invalid_sha_hash	The format of a parameter specified as a sha hash was invalid, sha hash included sha1/2/3 hash
fail_invalid_submitter_id	The format of the submitter id was invalid or the value is out of boundary
fail_no_policy_configured	No policy was configured against the org_id
fail_not_enough_params	Not enough device attributes were collected during profiling to perform a fingerprint match
fail_parameter_overlength	The value of the parameter was overlength
fail_temporarily_unavailable	Request failed because the service is temporarily unavailable
fail_too_many_instances_of_same_parameter	Multiple values for some parameters which only allow one instance
fail_verification	API query limit reached
success	ThreatMetrix was able to process the request successfully

9.3.5.2 Understanding the Risk Score

For each Session Query or Attribute Query, a score with a value between -100 and +100 is returned based on the rules that were triggered for the transaction.

Table 9 defines the risk scores ranges.

Table 9: Session Query and Attribute Query risk score definitions

Risk score	Visa definition
-100 to -1	A lower score indicates a higher probability that the transaction is fraudulent.
0	Neutral transaction
1 to 100	A higher score indicates a lower probability that the transaction is fraudulent. Note: All e-commerce transactions have some level of risk associated with them. Therefore, it is rare to see risk score in the high positive values.

When evaluating the risk of a transaction, the risk score gives an initial indicator of the potential risk that the transaction is fraudulent. Because some of the rules that are evaluated on each transaction may not be relevant to your business scenario, review the rules that were triggered for the transaction before determining how to handle the transaction.

9.3.5.3 Understanding the Rule Codes, Rule Names and Rule Messages

The rule codes, rule names and rule messages provide details about what rules were triggered during the assessment of the information provided in the Session or Attribute Query. Each rule code has a rule name and rule message. The rule name and rule message are typically similar. Table 10 provides additional information on each rule.

When evaluating the risk of a transaction, it is recommended that you review the rules that were triggered for the transaction and assess the relevance to your business. (That is, how does it relate to the typical buying habits of your customer base?)

If you are automating some or all of the decision-making processes related to handling the responses, you may want to use the rule codes. If you are documenting manual processes, you may want to refer to the more user-friendly rule name or rule message.

Table 10: Rule names, numbers and messages

Rule name	Rule number	Rule message
	Rule explanation	
White lists		
DeviceWhitelisted	WL001	Device White Listed
	Device is on the white list. This indicates that the device has been flagged as always "ok". Note: This rule is currently not in use.	

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number		Rule message
	Rule explanation		
IPWhitelisted	WL002	IP White Listed	
	IP address is on the white list. This indicates the device has been flagged as always "ok".		
	Note: This rule is currently not in use.		
EmailWhitelisted	WL003	Email White Listed	
	Email address is on the white list. This indicates that the device has been flagged as always "ok".		
	Note: This rule is currently not in use.		
Event velocity			
2DevicePayment	EV003	2 Device Payment Velocity	
	Multiple payments were detected from this device in the past 24 hours.		
2IPPaymentVelocity	EV006	2 IP Payment Velocity	
	Multiple payments were detected from this IP within the past 24 hours.		
2ProxyPaymentVelocity	EV008	2 Proxy Payment Velocity	
	The device has used 3 or more different proxies during a 24 hour period. This could be a risk or it could be someone using a legitimate corporate proxy.		
Email			
3EmailPerDeviceDay	EM001	3 Emails for the Device ID in 1 Day	
	This device has presented 3 different email IDs within the past 24 hours.		
3EmailPerDeviceWeek	EM002	3 emails for the Device ID in 1 week	
	This device has presented 3 different email IDs within the past week.		
3DevciePerEmailDay	EM003	3 Device Ids for email address in 1 day	
	This email has been presented from three different devices in the past 24 hours.		

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number	Rule message
	Rule explanation	
3DevciePerEmailWeek	EM004	3 Device Ids for email address in 1 week
	This email has been presented from three different devices in the past week.	
EmailDistanceTravelled	EM005	Email Distance Travelled
	This email address has been associated with different physical locations in a short period of time.	
3EmailPerSmartIDHour	EM006	3 Emails for SmartID in 1 Hour
	The SmartID for this device has been associated with 3 different email addresses in 1 hour.	
GlobalEMailOverOneMonth	EM007	Global Email over 1 month
	The e-mail address involved in the transaction over 30 days ago. This generally indicates that the transaction is less risky. Note: This rule is set so that it does not impact the policy score or risk rating.	
ComputerGeneratedEmailAddress	EM008	Computer Generated Email Address
	This transaction used a computer-generated email address.	
Account Number		
3AccountNumberPerDeviceDay	AN001	3 Account Numbers for device in 1 day
	This device has presented 3 different user accounts within the past 24 hours.	
3AccountNumberPerDeviceWeek	AN002	3 Account Numbers for device in 1 week
	This device has presented 3 different user accounts within the past week.	
3DevciePerAccountNumberDay	AN003	3 Device IDs for account number in 1 day
	This user account been used from three different devices in the past 24 hours.	

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number		Rule message	
	Rule explanation			
3DevciePerAccountNumberWeek	AN004	3 Device IDs for account number in 1 week		
	This card number has been used from three different devices in the past week.			
AccountNumberDistanceTravelled	AN005	Account Number distance travelled		
	This card number has been used from a number of physically different locations in a short period of time.			
Credit card/payments				
3CreditCardPerDeviceDay	CP001	3 credit cards for device in 1 day		
	This device has used three credit cards within 24 hours.			
3CreditCardPerDeviceWeek	CP002	3 credit cards for device in 1 week		
	This device has used three credit cards within 1 week.			
3DevicePerCreditCardDay	CP003	3 device ids for credit card in 1 day		
	This credit card has been used on three different devices in 24 hours.			
3DevciePerCreditCardWeek	CP004	3 device ids for credit card in 1 week		
	This credit card has been used on three different devices in 1 week.			
CredtCardDistanceTravelled	CP005	Credit Card has travelled		
	The credit card has been used at a number of physically different locations in a short period of time.			
CreditCardShipAddressGeoMismatch	CP006	Credit Card and Ship Address do not match		
	The credit card was issued in a region different from the Ship To Address information provided.			
CreditCardBillAddressGeoMismatch	CP007	Credit Card and Billing Address do not match		
	The credit card was issued in a region different from the Billing Address information provided.			
CreditCardDeviceGeoMismatch	CP008	Credit Card and device location do not match		
	The device is located in a region different from where the card was issued.			

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number	Rule message
	Rule explanation	
CreditCardBINShipAddressGeoMismatch	CP009	Credit Card issuing location and Shipping address do not match
	The credit card was issued in a region different from the Ship To Address information provided.	
CreditCardBINBillAddressGeoMismatch	CP010	Credit Card issuing location and Billing address do not match
	The credit card was issued in a region different from the Billing Address information provided.	
CreditCardBINDeviceGeoMismatch	CP011	Credit Card issuing location and location of the device do not match
	The device is located in a region different from where the card was issued.	
TransactionValueDay	CP012	Daily Transaction Value Threshold
	The transaction value exceeds the daily threshold.	
TransactionValueWeek	CP013	Weekly Transaction Value Threshold
	The transaction value exceeds the weekly threshold.	
Proxy rules		
3ProxyPerDeviceDay	PX001	3 Proxy Ips in 1 day
	This device has used three different proxy servers in the past 24 hours.	
AnonymousProxy	PX002	Anonymous Proxy IP
	This device is using an anonymous proxy	
UnusualProxyAttributes	PX003	Unusual Proxy Attributes
	This transaction is coming from a source with unusual proxy attributes.	
AnonymousProxy	PX004	Anonymous Proxy
	This device is connecting through an anonymous proxy connection.	
HiddenProxy	PX005	Hidden Proxy
	This device is connecting via a hidden proxy server.	

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number		Rule message
	Rule explanation		
OpenProxy	PX006	Open Proxy	
	This transaction is coming from a source that is using an open proxy.		
TransparentProxy	PX007	Transparent Proxy	
	This transaction is coming from a source that is using a transparent proxy.		
DeviceProxyGeoMismatch	PX008	Proxy and True GEO Match	
	This device is connecting through a proxy server that didn't match the devices geo-location.		
ProxyTrueISPMismatch	PX009	Proxy and True ISP Match	
	This device is connecting through a proxy server that doesn't match the true IP address of the device.		
ProxyTrueOrganizationMismatch	PX010	Proxy and True Org Match	
	The Proxy information and True ISP information for this source do not match.		
DeviceProxyRegionMismatch	PX011	Proxy and True Region Match	
	The proxy and device region location information do not match.		
ProxyNegativeReputation	PX012	Proxy IP Flagged Risky in Reputation Network	
	This device is connecting from a proxy server with a known negative reputation.		
SatelliteProxyISP	PX013	Satellite Proxy	
	This transaction is coming from a source that is using a satellite proxy.		
GEO			
DeviceCountriesNotAllowed	GE001	True GEO in Countries Not Allowed blacklist	
	This device is connecting from a high-risk geographic location.		

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number		Rule message
	Rule explanation		
DeviceCountriesNotAllowed	GE002	True GEO in Countries Not Allowed (negative whitelist)	
	The device is from a region that is not on the whitelist of regions that are accepted.		
DeviceProxyGeoMismatch	GE003	True GEO different from Proxy GEO	
	The true geographical location of this device is different from the proxy geographical location.		
DeviceAccountGeoMismatch	GE004	Account Address different from True GEO	
	This device has presented an account billing address that doesn't match the devices geolocation.		
DeviceShipGeoMismatch	GE005	Device and Ship Geo mismatch	
	The location of the device and the shipping address do not match.		
DeviceShipGeoMismatch	GE006	Device and Ship Geo mismatch	
	The location of the device and the shipping address do not match.		
Device			
SatelliteISP	DV001	Satellite ISP	
	This transaction is from a source that is using a satellite ISP.		
MidsessionChange	DV002	Session Changed Mid-session	
	This device changed session details and identifiers in the middle of a session.		
LanguageMismatch	DV003	Language Mismatch	
	The language of the user does not match the primary language spoken in the location where the True IP is registered.		
NoDeviceID	DV004	No Device ID	
	No device ID was available for this transaction.		
Dial-upConnection	DV005	Dial-up connection	
	This device uses a less identifiable dial-up connection.		

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number	Rule message
	Rule explanation	
DeviceNegativeReputation	DV006	Device Blacklisted in Reputational Network
	This device has a known negative reputation as reported to the fraud network.	
DeviceGlobalBlacklist	DV007	Device on the Global Black List
	This device has been flagged on the global blacklist of known problem devices.	
DeviceCompromisedDay	DV008	Device compromised in last day
	This device has been reported as compromised in the last 24 hours.	
DeviceCompromisedHour	DV009	Device compromised in last hour
	This device has been reported as compromised in the last hour.	
FlashImagesCookiesDisabled	DV010	Flash Images Cookies Disabled
	Key browser functions/identifiers have been disabled on this device.	
FlashCookiesDisabled	DV011	Flash Cookies Disabled
	Key browser functions/identifiers have been disabled on this device.	
FlashDisabled	DV012	Flash Disabled
	Key browser functions/identifiers have been disabled on this device.	
ImagesDisabled	DV013	Images Disabled
	Key browser functions/identifiers have been disabled on this device.	
CookiesDisabled	DV014	Cookies Disabled
	Key browser functions/identifiers have been disabled on this device.	
DeviceDistanceTravelled	DV015	Device Distance Travelled
	The device has been used from multiple physical locations in a short period of time.	

Table 10: Rule names, numbers and messages (continued)

Rule name	Rule number		Rule message	
	Rule explanation			
PossibleCookieWiping	DV016		Cookie Wiping	
	This device appears to be deleting cookies after each session.			
PossibleCookieCopying	DV017		Possible Cookie Copying	
	This device appears to be copying cookies.			
PossibleVPNConnection	DV018		Possibly using a VPN Connection	
	This device may be using a VPN connection			

9.3.5.4 Examples of Risk Response

Session Query

Sample Risk Response - Session Query
<pre><?xml version="1.0"?> <response> <receipt> <ResponseCode>001</ResponseCode> <Message>Success</Message> <Result> <session_id>abc123</session_id> <unknown_session>yes</unknown_session> <event_type>payment</event_type> <service_type>session</service_type> <policy_score>-25</policy_score> <transaction_id>riskcheck42</transaction_id> <org_id>11kue096</org_id> <request_id>91C1879B-33D4-4D72-8FCB-B60A172B3CAC</request_id> <risk_rating>medium</risk_rating> <request_result>success</request_result> <summary_risk_score>-25</summary_risk_score> <Policy>default</policy> <review_status>review</review_status> </Result> <Rule> <RuleName>ComputerGeneratedEMail</RuleName> <RuleCode>UN001</RuleCode> <RuleMessageEn>Unknown Rule</RuleMessageEn> <RuleMessageFr>Regle Inconnus</RuleMessageFr> </Rule> <Rule> <RuleName>NoDeviceID</RuleName> <RuleCode>DV004</RuleCode> <RuleMessageEn>No Device ID</RuleMessageEn> <RuleMessageFr>null</RuleMessageFr> </Rule> </receipt> </response></pre>

Attribute Query

Sample Risk Response - Attribute Query
<pre><?xml version="1.0"?> <response> <receipt> <ResponseCode>001</ReponseCode> <Message = Success</Message> <Result> <org_id>11kue096</org_id> <request_id>443D7FB5-CC5C-4917-A57E-27EAC824069C</request_id> <service_type>session</service_type> <risk_rating>medium</risk_rating> <summary_risk_score>-25</summary_risk_score> <request_result>success</request_result></pre>

Sample Risk Response - Attribute Query

```

<policy>default</policy>
<policy_score>-25</policy_score>
<transaction_id>riskcheck19</transaction_id>
<review_status>review</review_status>
</Result>
<Rule>
<RuleName>ComputerGeneratedEmail</RuleName>
<RuleCode>UN001</RuleCode>
<RuleMessageEn>Unknown Rule</RuleMessageEn>
<RuleMessageFr>Regle Inconnus</RuleMessageFr>
</Rule>
<Rule>
<RuleName>NoDeviceID</RuleName>
<RuleCode>DV004</RuleCode>
<RuleMessageEn>No Device ID</RuleMessageEn>
<RuleMessageFr>null</RuleMessageFr>
</Rule>
</receipt>
</response>

```

9.3.6 Inserting the Profiling Tags Into Your Website

Place the profiling tags on an HTML page served by your web application such that ThreatMetrix can collect device information from the customer's web browser. The tags must be placed on a page that a visitor would display in a browser window for 3-5 seconds (such as a page that requires a user to input data). After the device is profiled, a Session Query may be used to obtain the detail device information for risk assessment before submitting a financial payment transaction.

There are two profiling tags that require two variables. Those tags are `org_id` and `session_id`. `session_id` must match the session ID value that is to be passed in the Session Query transaction. The valid `org_id` values are:

11kue096

QA testing environment.

lbhqgx47

Production environment.

Below is an HTML sample of the profiling tags.

NOTE: Your site must replace `<my_session_id>` in the sample code with a unique alphanumeric value each time you fingerprint a new customer.

```

<p style="background:url(https://h.online-metrix.net/fp/clear.png?org_id=11kue096&session_id=<my_session_id>&m=1)">
</p>

```

```



```

```
<script src="https://h.onlinemetrix.net/fp/check.js?org_id=11kue096&session_id=<my_session_id>"
type="text/javascript">
</script>

<object type="application/x-shockwave-flash"

data="https://h.onlinemetrix.net/fp/fp.swf?org_id=11kue096&session_id=<my_session_id>"
width="1" height="1" id="obj_id">
<param name="movie"
value="https://h.onlinemetrix.net/fp/fp.swf?org_id=11kue096&session_id=<my_session_id>" />
<div></div>
</object>
```

9.4 Incorporating All Available Fraud Tools

- 9.4.1 Implementation Options for TRMT
- 9.4.2 Implementation Checklist
- 9.4.3 Making a Decision

To minimize fraudulent activity in online transactions, Moneris recommends that you implement all of the fraud tools available through the Moneris Gateway. These are explained below:

Address Verification Service (AVS)

Verifies the cardholder's billing address information.

Verified by Visa, MasterCard Secure Code and Amex SafeKey (VbV/MCSC/SafeKey)

Authenticates the cardholder at the time of an online transaction.

Card Validation Digit (CVD)

Validates that cardholder is in possession of a genuine credit card during the transaction.

Note that all responses that are returned from these verification methods are intended to provide added security and fraud prevention. The response itself does not affect the completion of a transaction. Upon receiving a response, the choice to proceed with a transaction is left entirely to the merchant.

9.4.1 Implementation Options for TRMT

Option A

Process a Transaction Risk Management Tool query and obtain the response. You can then decide whether to continue with the transaction, abort the transaction, or use additional eFraud features.

If you want to use additional eFraud features, perform one or both of the following to help make your decision about whether to continue with the transaction or abort it:

- Process a VbV/MCSC/SafeKey transaction and obtain the response. The merchant then makes the decision whether to continue with the transaction or to abort it.
- Process a financial transaction including AVS/CVD details and obtain the response. The merchant then makes a decision whether to continue with the transaction or to abort it.

Option B

1. Process a Transaction Risk Management Tool query and obtain the response.
2. Process a VbV/MCSC/SafeKey transaction and obtain the response.
3. Process a financial transaction including AVS/CVD details and obtain the response.
4. Merchant then makes a one-time decision based on the responses received from the eFraud tools.

9.4.2 Implementation Checklist

The following checklists provide high-level tasks that are required as part of your implementation of the Transaction Risk Management Tool. Because each organization has certain project requirements for implementing system and process changes, this list is only a guideline, and does not cover all aspects of your project.

Download and review all of the applicable APIs and Integration Guides

Please review the sections outlined within this document that refers to the following feature

Table 11: API documentation

Document/API	Use the document if you are....
Transaction Risk Management Tool Integration Guide (Section #)	Implementing or updating your integration for the Transaction Risk Management Tool
Moneris MPI – Verified by Visa/MasterCard SecureCode/American Express SafeKey – Java API Integration Guide	Implementing or updating Verified by Visa, MasterCard SecureCode or American Express SafeKey
Basic transaction with VS and CVD (Section#)	Implementing or updating transaction processing, AVS or CVD

Design your transaction flow and business processes

When designing your transaction flow, think about which scenarios you would like to have automated, and which scenarios you would like to have handled manually by your employees.

The “Understand Transaction Risk Management Transaction Flow” and Handling Response Information (page 403) sections can help you work through the design of your transaction and process flows.

Things to consider when designing your process flows:

- Processes for notifying people within your organization when there is scheduled maintenance for Moneris Gateway.
- Handling refunds, canceled orders and so on.
- Communicating with customers when you will not be shipping the goods because of suspected fraud, back-ordered goods and so on.

Complete your development and testing

- The Moneris Gateway API - Integration Guide - PHP provides the technical details required for the development and testing. Ensure that you follow the testing instructions and data provided.

If you are an integrator

- Ensure that your solution meets the requirements for PCI-DSS/PA-DSS as applicable.
- Send an email to eproducts@moneris.com with the subject line "Certification Request".
- Develop material to set up your customers as quickly as possible with your solution and a Moneris account. Include information such as:
 - Steps they must take to enter their store ID or API token information into your solution.
 - Any optional services that you support via Moneris Gateway (such as TRMT, AVS, CVD, VBV/MCSC/SafeKey and so on) so that customers can request these features.

9.4.3 Making a Decision

Depending on your business policies and processes, the information obtained from the fraud tools (such as AVS, CVD, VbV/MCSC/SafeKey and TRMT) can help you make an informed decision about whether to accept a transaction or deny it because it is potentially fraudulent.

If you do not want to continue with a likely fraudulent transaction, you must inform the customer that you are not proceeding with their transaction.

If you are attempting to do further authentication by using the available fraud tools, but you have received an approval response instead, cancel the financial transaction by doing one of the following:

- If the original transaction is a Purchase, use a Purchase Correction or Refund transaction. You will need the original order ID and transaction number.
- If the original transaction is a Pre-Authorization, use a Completion transaction for \$0.00.

10 Apple Pay and Google Pay™ Integrations

- 10.1 About Apple Pay and Google Pay™ Integration
- 10.2 Apple Pay Transaction Process Overview
- 1 Google Pay™ Transaction Process Overview
- 10.4 About API Integration of Apple Pay and Google Pay™
- 10.5 Cavv Purchase – Apple Pay and Google Pay™
- 10.6 Cavv Pre-Authorization – Apple Pay & Google Pay™

10.1 About Apple Pay and Google Pay™ Integration

The Moneris Gateway enables merchants to process in-app transactions within applications running on iOS (Apple Pay) or Android (Google Pay™) mobile devices, and in-browser when using the Safari web browser (Apple Pay, using Apple devices only) or the Chrome web browser on (Google Pay™).

Moneris Solutions offers two integration methods for processing Apple Pay and Google Pay™ transactions. Merchants can choose to use one of two methods:

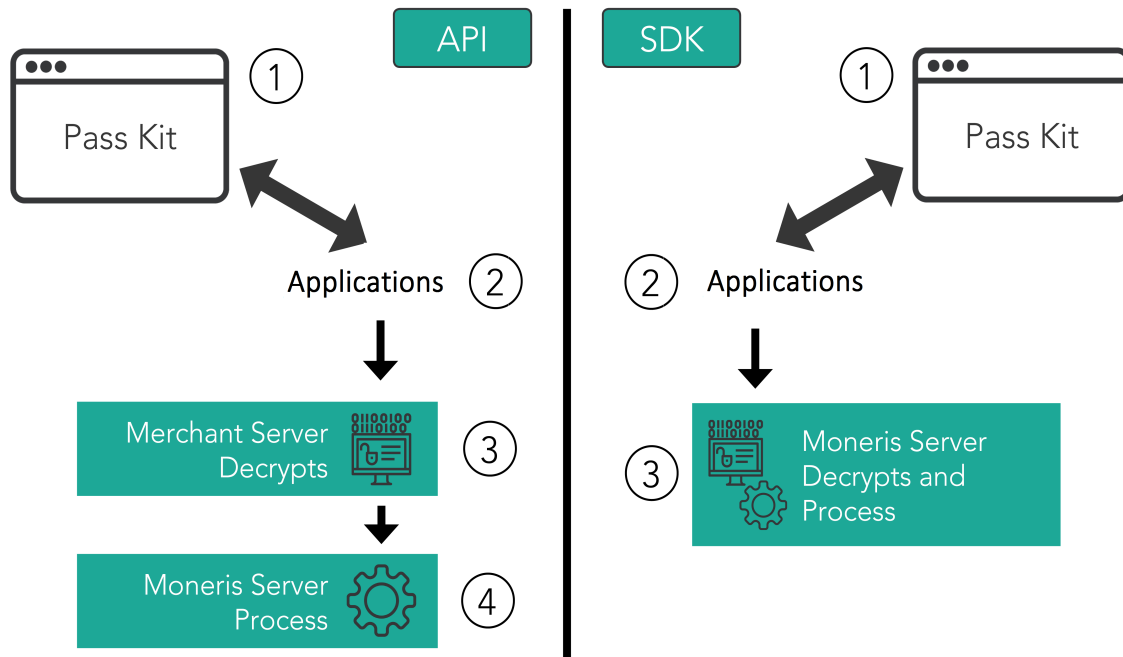
- Software Development Kit (SDK), or
- API method (where decryption of the transaction payload is handled by merchants)

While both methods provide the same basic payment features, there are differences in their implementations.

This guide mainly focuses on the API method, but some of its API requests are utilized in SDK implementation as well; for detailed information about the SDK method of integration, see the Moneris Developer Portal at <https://developer.moneris.com>.

10.2 Apple Pay Transaction Process Overview

For both API and SDK methods of mobile in-app integration, the merchant's iOS app uses Apple's PassKit Framework to request and receive encrypted payment details from Apple. When payment details are returned in their encrypted form, they can be decrypted and processed by the Moneris Gateway in one of two ways: SDK or API.



Steps in the Apple Pay payment process

API

1. Merchant's mobile application or web page requests and receives the encrypted payload
2. Encrypted payload is sent to the merchant's server, where it is decrypted
3. Moneris Gateway receives the decrypted payload from the merchant's server, and processes the Cavv Purchase – Apple Pay and Google Pay™ or "Cavv Pre-Authorization – Apple Pay & Google Pay™" on page 435transaction
 - a. Please ensure the wallet indicator is properly populated with the correct value (APP for Apple Pay In-App or APW for Apple Pay on the Web)

SDK

1. Merchant's mobile application or web page requests and receives the encrypted payload
2. Encrypted payload is sent from the merchant's server to the Moneris Gateway, and the payload is decrypted and processed

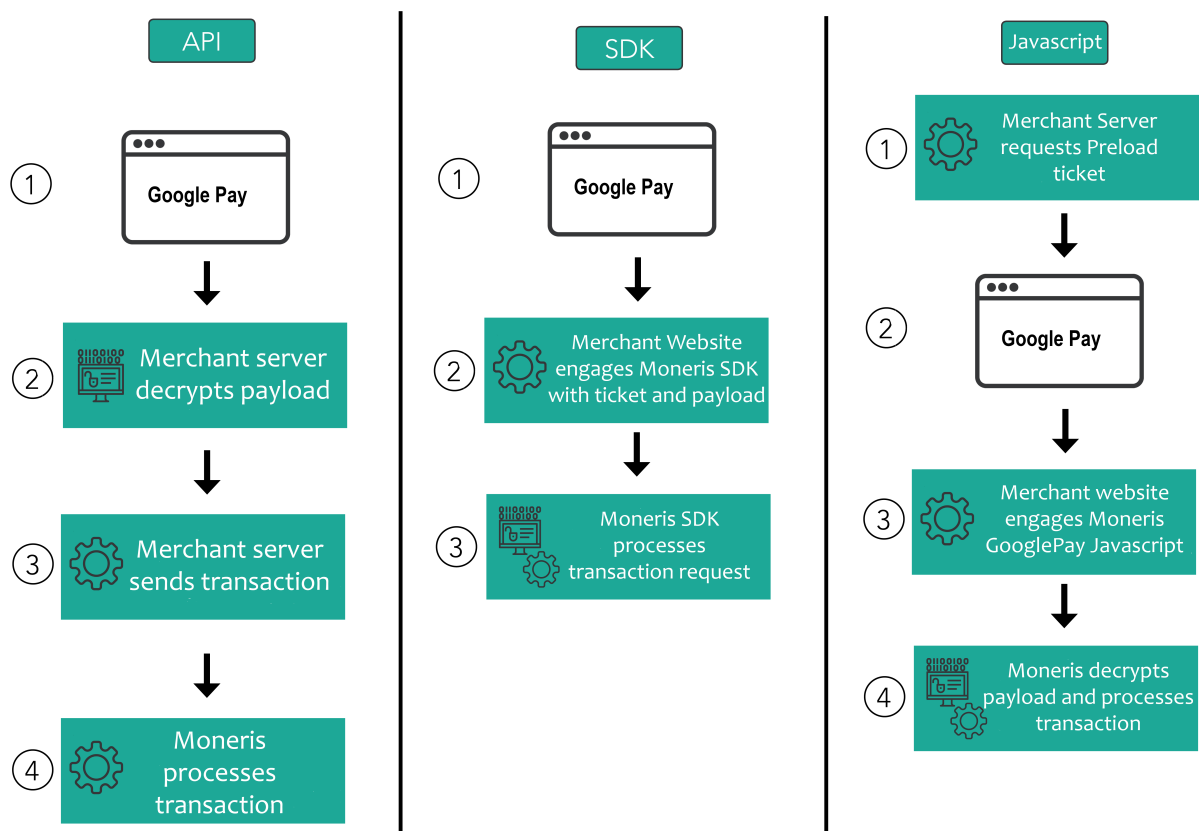
This guide mainly focuses on the API method, but some of its API requests are utilized in SDK implementation as well; for detailed information about the SDK method of integration, see the Moneris Developer Portal at <https://developer.moneris.com>.

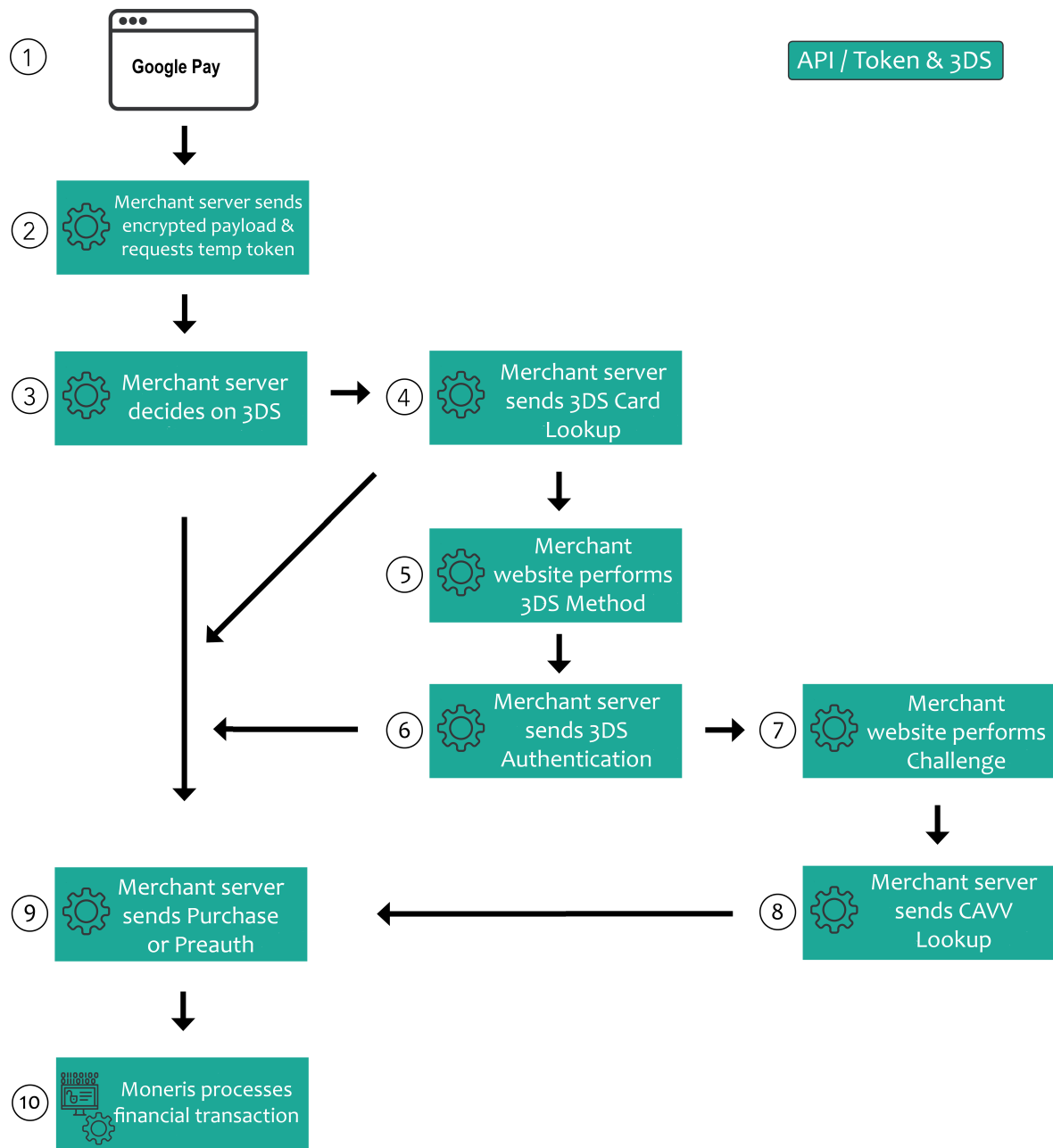
10.3 Google Pay™ Transaction Process Overview

Moneris offers four integration methods for processing transactions with Google Pay wallets. All integrations use the Google Pay™ Framework to request and receive encrypted payment details from Google. When payment details are returned in their encrypted form, the merchant can decrypt the payload on their server or transmit the encrypted payload to Moneris for decryption.

Moneris recommends merchants utilize the 3DS process on Google Pay temporary tokens with an underlying card type of FPAN to reduce risk of fraud and chargebacks. Merchants may attempt or skip the 3DS process if the underlying card type is DPAN; the card issuer may not support 3DS for the device PAN, however, so Moneris recommends using the 3DS Card Lookup to ensure support for 3DS.

Google Pay™ Integration Methods & Process





Decrypted Payload (Merchant server to Moneris Gateway API)

Merchant decrypts the Google Pay encrypted payload locally then processes a standard financial transaction via Moneris Gateway API call with card data. Used by both in-app and web solutions.

NOTE: In this API scenario where merchant's server is responsible for decrypting the payload, merchants must sign agreement with Google directly. Google can then provide you with the keys to decrypt the payload.

1. Merchant's mobile application or web page requests and receives the encrypted payload from Google
2. Encrypted payload is sent from the merchant's website to Moneris Gateway via the SDK, and the payload is decrypted and processed

Encrypted Payload (Merchant website to Moneris SDK)

Merchant passes the encrypted payload to the Moneris Google Pay SDK. Used for in-app solutions only. The SDK files are located on the [Moneris Github](#) and instructions on integration found on our [Moneris Developer Portal](#).

1. Merchant's mobile application or web page requests and receives the encrypted payload from Google
2. Encrypted payload is sent from the merchant's website to Moneris Gateway via the SDK, and the payload is decrypted and processed

Encrypted Payload (Merchant website to Moneris JavaScript)

Merchant edits the Google Pay Javascript to utilize Moneris Google Pay for processing the payment. Includes an optional Preload via Moneris Gateway API specific to this integration method. See the full guide located on our [Moneris Github](#) or instructions on our [Moneris Developer Portal](#).

1. Merchant's server populates a Preload request and receives a ticket (optional step)
2. Merchant's mobile application or web page requests and receives the encrypted payload from Google
3. Encrypted payload is sent from the merchant's application or website to the via embedded Moneris Javascript, and the payload is decrypted and processed.

Encrypted Payload With 3DS (Merchant server to Moneris Gateway API)

Merchant transmits the encrypted payload via Moneris Gateway API to decrypt and tokenize the card data temporarily. This temporary token is usable for performing 3D-Secure authentication and the subsequent financial transaction.

NOTE: Moneris recommends merchants utilize the 3DS process on Google Pay temporary tokens with an underlying card type of FPAN to reduce risk of fraud and chargebacks. Merchants may attempt or skip the 3DS process if the underlying card type is DPAN; the card issuer may not support 3DS for the device PAN, however, so Moneris recommends using the 3DS Card Lookup to ensure support for 3DS.

1. Merchant's app or web page requests and receives the encrypted payload from Google
2. Encrypted payload is sent from the merchant's server to the via a GooglePay Temporary Token Add. Moneris returns a temporary payment token in the response and a GooglePayPaymentMethod indicating the type of underlying card data (FPAN or DPAN)
3. Merchant server elects whether to perform 3DS Authentication or not. If electing to skip 3DS Authentication, the merchant server can skip to Step 9 and immediately perform a financial transaction.

4. Merchant server sends the temporary token in a 3DS Card Lookup request. Moneris responds with whether the underlying card supports 3DS authentication or not and details for the 3DS Method, if available.

For cards that do not support 3DS, the merchant server should skip the rest of the 3DS Authentication flow and move to Step 9.

5. If available, merchant server and app performs the 3DS Method using the 3DSMethodData and 3DSMethodURL. See "Handling the 3DS Method for Device Fingerprinting" on page 240
6. Merchant server performs a 3DS Authentication request (Browser Channel) to the Moneris Gateway. See "Implementing MPI 3DS Authentication Request" on page 241

If Moneris responds with a successful result (frictionless), the merchant server receives the CAVV and ECI values from the 3DS authentication response itself. Skip the challenge flow and move to Step 9.

If Moneris responds that a challenge prompt (friction) is required, continue with the next step.

7. Merchant server and application proceed with the 3DS Challenge. See "Handling the Challenge Flow" on page 271
8. Merchant server sends CAVV Lookup request to retrieve the authentication value (CAVV) and e-commerce indicator (crypt_type) after the challenge is completed.
9. Merchant server performs either a GooglePayTokenPreauth or GooglePayTokenPurchase as the financial transaction. The CAVV and electronic commerce indicator (crypt_type) are included as follows:

If 3DS was skipped earlier, omit the CAVV field and use the appropriate ECI for your transaction type.

If 3DS authentication was performed successfully, supply the CAVV from your 3DS Authentication or CAVV Lookup response.

10.4 About API Integration of Apple Pay and Google Pay™

An API integration works to provide a communication link between your merchant server and Moneris' server. APIs are required to complete any transaction, and therefore the APIs for Apple Pay and Google Pay™ are also included in SDK integration.

If the merchant chooses to use the API-only integration method, the merchant must decrypt payload information themselves before sending the decrypted information to the Moneris Gateway to be processed. Because this process is complicated, Moneris recommends only businesses with expertise and a previously integrated payment processing system use the API integration method; all other merchants should use the Moneris Apple Pay or Google Pay™ SDK as the integration method.

10.4.1 Transaction Types Used for Apple Pay and Google Pay™

In the Moneris Gateway API, there are two transaction types that allow you to process decrypted transaction payload information from Apple Pay and Google Pay™:

- 10.5 Cavv Purchase – Apple Pay and Google Pay™
- 10.6 Cavv Pre-Authorization – Apple Pay & Google Pay™

NOTE: INTERAC® e-Commerce functionality is currently available using the Cavv Purchase transaction type only.

Note: In the event that a decrypted payment token contains a Device Primary Account Number (DPAN), a 3DS authentication attempt is not recommended as it may not be supported. A DPAN is a device-specific token from the wallet provider to identify the underlying card that is associated with a cardholder's Funding Primary Account Number (FPAN).

Once you have processed the initial transaction using Cavv Purchase or Cavv Pre-Authorization if required you can then process any of the following transactions:

- Refund
- Pre-Authorization Completion
- Purchase Correction

10.5 Cavv Purchase – Apple Pay and Google Pay™

The Cavv Purchase for Apple Pay and Google Pay™ transaction follows a 3-D Secure model but it does not require an MPI. Once the transaction payload has been decrypted, this transaction verifies funds on the customer's card, removes the funds and prepares them for deposit into the merchant's account.

To perform the 3-D Secure authentication, the Moneris MPI or any third-party MPI may be used.

In addition to 3-D Secure transactions, this transaction can also be used to process Apple Pay and Google Pay™ transactions. This transaction is applicable only if choosing to integrate directly to Apple Wallet or Google Wallet (if not using the Moneris Apple Pay or Google Pay™ SDKs).

Refer to Apple or Google developer portals for details on integrating directly to their wallets to retrieve the payload data.

WARNING: Moneris strongly discourages the use of frames as part of a 3-D Secure implementation, and cannot guarantee their reliability when processing transactions in the production environment.

Cavv Purchase for Apple Pay & Google Pay™ transaction object definition

```
$txnArray = array('type'=>'cavv_purchase', ...);  
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Cavv Purchase for Apple Pay & Google Pay™ transaction

```
$mpgRequest = new mpgRequest($mpgTxn);  
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Cavv Purchase for Apple Pay & Google Pay™ transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal	'amount'=>\$amount

Variable Name	Type and Limits	Set Method
	point	
	EXAMPLE: 1234567.89	
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authorization transactions, CAVV field contains the decrypted cryptogram. For more, see Appendix A Definition of Request Fields.		
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authorization transactions, the e-commerce indicator is a mandatory field containing the value received from the decrypted payload or a default value of 5. If you get a 2-character value (e.g., 05 or 07) from the payload, remove the initial 0 and just send us the 2nd character. For more, see Appendix A Definition of Request Fields.		

Following fields are required for Apple Pay and Google Pay only:

Variable Name	Type and Limits	
network	<i>String</i> alphanumeric	
data type	<i>String</i> 3-character alphanumeric	

Cavv Purchase for Apple Pay & Google Pay™ transaction request fields – Optional

Variable Name	Type and Limits	Set Method
status check	<i>Boolean</i> true/false	<code>\$mpgHttpPost =new mpgHttpsPostStatus(\$store_ id,\$api_ token,\$status,\$mpgRequest);</code>
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	<code>'cust_id'=>\$cust_id</code>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	<code>'dynamic_descriptor'=>\$dynamic_ descriptor</code>
card match ID	<i>String</i> 50-character alphanumeric NOTE: Applies to Offlinx™ only; must be unique value for each transaction	<code>'cm_id' => \$transaction_id</code>

Variable Name	Type and Limits	Set Method
Customer Information	<i>Object</i> N/A	\$mpgTxn->setCustInfo (\$mpgCustInfo);

Sample Cavv Purchase for Apple Pay & Google Pay™

```
<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
/***** Transactional Variables *****/
$type='cavv_purchase';
$order_id='ord-' .date("dmy-G:i:s");
$cust_id='CUST887763';
$amount='6000.00';
$pan='4622943127023886';
$expiry_date='2212';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA=';
$crypt_type = '7';
$wallet_indicator = "APP";
$dynamic_descriptor='123456';
$foreign_indicator='true';

// TrId and TokenCryptogram are optional, refer documentation for more details.
$tr_id = '50189815682';
$token_cryptogram = 'APmbM/41le0uAAH+s6xMAAADFA==';
'foreign_indicator'=>$foreign_indicator
/***** Transaction Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$pan,
    'expdate'=>$expiry_date,
    'cavv'=>$cavv,
    'crypt_type'=>$crypt_type, //mandatory for AMEX only
    //'wallet_indicator'=>$wallet_indicator, //set only for wallet transactions. e.g. APPLE PAY
    //'network'=> "Interac", //set only for Interac e-commerce
    //'data_type'=> "3DSecure", //set only for Interac e-commerce
    'dynamic_descriptor'=>$dynamic_descriptor,
    'threeds_version' => '2.2.0', //Mandatory for financial transactions using 3rd Party 3-D
    Secure services
    'threeds_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f' Mandatory for financial
    transactions using 3rd Party 3-D Secure services - obtained from MpiCavvLookup or
    MpiThreeDSAuthentication
    //'cm_id' => '8nAK8712sGaAkls56' //set only for usage with Offlinx - Unique max 50
    alphanumeric characters transaction id generated by merchant
    //'ds_trans_id' => '12345' //Optional - to be used only if you are using 3rd party 3ds
    service
    //'tr_id' => $tr_id
    //'token_cryptogram' => $token_cryptogram
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
```

```

$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Installment Info *OPTIONAL* *****/
$installmentInfo = new InstallmentInfo();
$installmentInfo->setPlanId("ae859ef1-eb91-b708-8b80-1dd481746401");
$installmentInfo->setPlanIdRef("0000000065");
$installmentInfo->setTacVersion("2");
// $mpgTxn->setInstallmentInfo($installmentInfo);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nAdviceCode = " . $mpgResponse->getAdviceCode());

// $installmentResults = $mpgResponse->getInstallmentResults();
// print("\nPlanId = " . $installmentResults->getPlanId());
// print("\nPlanIDRef = " . $installmentResults->getPlanIDRef());
// print("\nTacVersion = " . $installmentResults->getTacVersion());
// print("\nPlanAcceptanceId = " . $installmentResults->getPlanAcceptanceId());
// print("\nPlanStatus = " . $installmentResults->getPlanStatus());
// print("\nPlanResponse = " . $installmentResults->getPlanResponse());
?>

```

10.6 Cavv Pre-Authorization – Apple Pay & Google Pay™

The Cavv Pre-Authorization for Apple Pay and Google Pay™ transaction follows a 3-D Secure model but it does not require an MPI. Once the transaction payload has been decrypted, this transaction verifies funds on the customer's card, and holds the funds. To prepare the funds for deposit into the merchant's account please process a Pre-Authorization Completion transaction.

To perform the 3-D Secure authentication, the Moneris MPI or any third-party MPI may be used.

In addition to 3-D Secure transactions, this transaction can also be used to process Apple Pay and Google Pay™ transactions. This transaction is applicable only if choosing to integrate directly to Apple Wallet or Google Wallet (if not using the Moneris Apple Pay or Google Pay™ SDKs).

Refer to Apple or Google developer portals for details on integrating directly to their wallets to retrieve the payload data.

NOTE: INTERAC® e-Commerce functionality is currently available using the Cavv Purchase transaction type only.

WARNING: Moneris strongly discourages the use of frames as part of a 3-D Secure implementation, and cannot guarantee their reliability when processing transactions in the production environment.

Cavv Pre-Authorization for Apple Pay & Google Pay™ transaction object definition

```
$txnArray = array('type'=>'cavv_preauth', ...);  
  
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Cavv Pre-Authorization for Apple Pay & Google Pay™ transaction

```
$mpgRequest = new mpgRequest($mpgTxn);  
  
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Cavv Pre-Authorization for Apple Pay&Google Pay™ transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point	'amount'=>\$amount
EXAMPLE: 1234567.89		

Variable Name	Type and Limits	Set Method
credit card number	<i>String</i> max 20-character alpha-numeric	'pan'=>\$pan
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authorization transactions, CAVV field contains the decrypted cryptogram. For more, see Appendix A Definition of Request Fields.		
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
NOTE: For Apple Pay and Google Pay™ Cavv Purchase and Cavv Pre-Authorization transactions, the e-commerce indicator is a mandatory field containing the value received from the decrypted payload or a default value of 5. If you get a 2-character value (e.g., 05 or 07) from the payload, remove the initial 0 and just send us the 2nd character. For more, see Appendix A Definition of Request Fields.		

Cavv Pre-Authorization for Apple Pay & Google Pay™ transaction request fields – Optional

Variable Name	Type and Limits	Set Method
status check	<i>Boolean</i> true/false	<pre>\$mpgHttpPost =new mpgHttpPostStatus(\$store_ id,\$api_ token,\$status,\$mpgRequest);</pre>
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	<pre>'cust_id'=>\$cust_id</pre>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: <>\$%=?^{}[]\	<pre>'dynamic_descriptor'=>\$dynamic_ descriptor</pre>
card match ID	<i>String</i> 50-character alphanumeric NOTE: Applies to Offlinx™ only; must be unique value for each transaction	<pre>'cm_id' => \$transaction_id</pre>
network	<i>String</i> alphabetic NOTE: This request variable is mandatory for INTERAC® e-Commerce transactions conducted via Apple Pay or Google Pay™ only, and is not for use with credit card transactions.	

Variable Name	Type and Limits	Set Method
data type	String	
NOTE: This request variable is mandatory for INTERAC® e-Commerce transactions conducted via Apple Pay or Google Pay™ only, and is not for use with credit card transactions.		
	3-character alphanumeric	

Sample Cavv Pre-Authorization for Apple Pay & Google Pay™

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca03650';
$api_token='7Yw0MPTlhjBRcZiE6837';
/***** Transactional Variables *****/
$type='cavv_preauth';
$order_id='ord-' . date("dmy-G:i:s");
$cust_id='CUST887763';
$amount='4840.00';
$span='5454545454545454';
$expiry_date='2212';
$cavv='AAABBJg0VhI0VniQEjRWAAAAA=';
$crypt_type = '7';
$wallet_indicator = "APP";
$dynamic_descriptor='123456';
// TrId and TokenCryptogram are optional, refer documentation for more details.
$tr_id = '50189815682';
$token_cryptogram = 'APmbM/41le0uAAH+s6xMAAADFA==';
/***** Transaction Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$span,
    'expdate'=>$expiry_date,
    'cavv'=>$cavv,
    'crypt_type'=>$crypt_type, //mandatory for AMEX only
    //'wallet_indicator'=>$wallet_indicator, //set only for wallet transactions. e.g. APPLE PAY
    //'network'=> "Interac", //set only for Interac e-commerce
    //'data_type'=> "3DSecure", //set only for Interac e-commerce
    'dynamic_descriptor'=>$dynamic_descriptor,
    'threeds_version' => '2', //Mandatory for 3DS Version 2.0+
    'threeds_server_trans_id' => 'e11d4985-8d25-40ed-99d6-c3803fe5e68f' //Mandatory for 3DS
    Version 2.0+ - obtained from MpiCavvLookup or MpiThreeDSAuthentication
    //'cm_id' => '8nAK8712sGaAkls56' //set only for usage with Offlinx - Unique max 50
    alphanumeric characters transaction id generated by merchant
    //'ds_trans_id' => '12345' //Optional - to be used only if you are using 3rd party 3ds 2.0
    service
    //'tr_id' => $tr_id
    //'token_cryptogram' => $token_cryptogram
);

```

```

/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("U");
$cof->setPaymentInformation("2");
$cof->setIssuerId("139X3130ASCXAS9");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false or comment out this line for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse = $mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nAdviceCode = " . $mpgResponse->getAdviceCode());
?>
```

10.7 GooglePay Temporary Token Add – GooglePayTokenTempAdd

Creates a new temporary token credit card profile from an encrypted GooglePay payload. During the life-time of this temporary token, it may be used to perform 3DS authentication and financial transactions via GooglePay Token Preauthorization or GooglePay Token Purchase.

The response field `GooglePaymentMethod` returned by this request will inform you if the underlying card within GooglePay is the funding card number ("FPAN") or a tokenized card number ("DPAN"). If a `GoogleTokenTempAdd` returns an FPAN, you may perform 3DS authentication with it; if it returns a DPAN, 3DS is not required.

Refer to Apple or Google developer portals for details on integrating directly to their wallets to retrieve the payload data.

Things to Consider:

- The duration, or lifetime, of the temporary token can be set to be a maximum of 15 minutes.

GooglePay Temporary Token Add transaction object definition

```
$txnArray = array('type'=>'googlePayTokenTempAdd', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for GooglePay Temporary Token Add transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

GooglePay Temporary Token Add transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
network	String alphabetic	<pre>\$network = "MASTERCARD";</pre> Card Brand name. Field is case sensitive Possible values: Visa Mastercard American Express Interac

Variable Name	Type and Limits	Set Method
		Discover
payment token	Object N/A	<pre>\$googlePayTokenTempAdd->setPaymentToken(\$signature, \$protocol_version, \$signed_message);</pre> Payment details returned by Google in their <code>PaymentData</code> object for GooglePay transactions. See GooglePay Payment Token object request fields – Required below for field details.

GooglePay Payment Token object request fields – Required

Variable Name	Type and Limits	Set Method
signature	String N/A	<pre>\$signature = "ME...";</pre> Verifies that the message came from Google. It's base64-encoded, and created with ECDSA by the intermediate signing key. Returned by Google in their <code>PaymentData</code> object for GooglePay transactions
protocol version	String N/A	<pre>\$protocol_version = "ECv1";</pre> Identifies the encryption or signing scheme under which the message is created. It allows the protocol to evolve over time, if needed. Returned by Google in their <code>PaymentData</code> object for GooglePay transactions
signed message	String N/A	<pre>\$signed_message = "{\"encryptedMessage\...\"}";</pre> A JSON object serialized as an HTML-safe string that contains the encryptedMessage, ephemeralPublicKey, and tag. It's serialized to simplify the signature verification process. Returned by Google in their <code>PaymentData</code>

Variable Name	Type and Limits	Set Method
		object for GooglePay transactions

Sample GooglePay Temporary Token Add

```
<?php

require "../mpgClasses.php";

/***** Request Variables *****/

$store_id='intuit_spd';
$sapi_token='spedguy';

/***** Transactional Variables *****/

$network = "MASTERCARD";
$signature =
"MEYCIQCdjfGZ1k/8h+eH9Ue5UxJsgDEFimp6YIrWhptpe+W3kAIhAOKikmz1B/C4WB5g3mXy139euOHHnSQ7bQWl2c
hgwole";
$protocol_version = "ECv1";
$signed_message = '
{"encryptedMessage":"nAEP5f0pzvU+cJHwxCwrCVPRr196NugevgfrdidPOB5B+WG7+yrsYoUVA7HopRD5y5GCld
QwrKnP2h2w/Qc2HBFn+G/g2IXqPBzMjguhpGitr6lV0tRLaYimxrgrbh/Xn8DxfW++pTHHoo+0xJiON6o3JC4vM6wMA
uhjjwEOgiDeKpgxJKEl8NULR2RK1OtvongkR80K8Et7CT+W0lXoMCoYrH3tJDKMtoVyFNfHPMAXLeV4NfVV+ZwhwD3F
+tGm7bQkPFMy2xUQxzdj7/H03vmyxwsblSKXhVG3hWKPmnY/+Gkb2K0pAicOHAB/SZuwaxHQ1l30jaNafUIm96R9T2Y
c3p5gmnGiR03R9H5R8JqLL9Wb7LncvfIwuQppgbAKa6HdbuSjbehNOTW8S34VqxvpeSfQFUNYgkQ+fVEU/VaClE17Py
F8AMZKN10ZIZ1jj7jntqoD", "ephemeralPublicKey":"BD5snQM3HF2gdCyERaF9XBPdGOXL8fNyTM9QY/xNTi9Vh
WTtq5sg7dYgPXdLmQuwIhBN9OyLULAMsNcmSV2TT7k\u003d", "tag":"hyG7Ty/qQAZelt2INIMtDQPMafODVhUinW
451hJrcP4\u003d"}';

/***** Transactional Associative Array *****/

$googlePayTokenTempAdd = new GooglePayTokenTempAdd();
$googlePayTokenTempAdd->setPaymentToken($signature, $protocol_version, $signed_message);

/***** Transaction Object *****/

$mpgTxn = new mpgTransaction($googlePayTokenTempAdd);

/***** Request Object *****/

$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production
transactions

/***** HTTPS Post Object *****/

/* Status Check Example
$mpgHttpPost =new mpgHttpPostStatus($store_id,$sapi_token,$status_check,$mpgRequest);
*/
$mpgHttpPost = new mpgHttpPost($store_id,$sapi_token,$mpgRequest);

/***** Response *****/
```

Sample GooglePay Temporary Token Add

```
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nMasked Pan = " . $mpgResponse->getResDataMaskedPan());
print("\nExp Date = " . $mpgResponse->getResDataExpDate());
print("\nPayment Type = " . $mpgResponse->getPaymentType());
print("\nGooglepayPaymentMethod = " . $mpgResponse->getGooglepayPaymentMethod());
?>
```

10.8 Google Pay™ Token Preauth

The Google Pay™ Token Preauth transaction is utilized after passing a GooglePay account into a temporary token using our GooglePay Token Temporary Add then performing 3DS authentication with the token. This transaction verifies funds on the customer's card and locks those funds for a time period specified by the card issuer.

To perform the 3-D Secure authentication, the Moneris MPI or any third-party MPI may be used.

Refer to Apple or Google developer portals for details on integrating directly to their wallets to retrieve the payload data.

Google Pay™ Token Preauth for transaction object definition

```
$txnArray = array('type'=>'googlePayTokenPreauth', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Google Pay™ Token Preauth transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String N/A	'store_id'=>\$store_id
API token	String N/A	'api_token'=>\$api_token

Google Pay™ Token Preauth transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
data key	<i>String</i> 25-character alphanumeric	The temporary token returned by a <code>GooglePayTokenTempAdd</code> request.
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv The 3DS cryptogram. Sent in all financial transactions with 3-D Secure, including Verified By Visa, MasterCard SecureCode, American Express SafeKey
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt Describes the category of e-commerce transaction being processed. Allowable values are: 1 – Mail Order / Telephone Order—Single 2 – Mail Order / Telephone Order—Recurring 3 – Mail Order / Telephone Order—Instalment

NOTE: For Google Pay™ Token Pre-Authorization transactions, CAVV field contains the 3DS cryptogram only when 3DS is used prior. If you elected to skip 3DS Authentication, you may omit the CAVV field.

NOTE: For Google Pay™ Token Purchase and Token Pre-Authorization transactions using 3DS Authentication, use the e-commerce indicator obtained from your 3DS Authentication.

Variable Name	Type and Limits	Set Method
		4 – Mail Order / Telephone Order—Unknown classification 5 – Authenticated e-commerce transaction (3-D Secure) 6 – Non-authenticated e-commerce transaction (3-D Secure) 7 – SSL-enabled merchant In Credential on File transactions where the request field e-commerce indicator is also being sent: the allowable values for e-commerce indicator are dependent on the value sent for payment indicator, as follows: if payment indicator = R, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = V, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = C, then allowable values for e-commerce indicator: 1, 5, 6 or 7 if payment indicator = U, then allowable values for e-commerce indicator: 1 or 7 if payment indicator = Z, then allowable values for e-commerce indicator: 1, 5, 6 or 7
3DS server transaction ID NOTE: Obtained from the Cavv Lookup request or MPI 3DS Authentication request. For Google Pay™ Token Purchase and Token Pre-Authorization transactions that do not use 3DS Authentication, you may omit the 3DS Server Transaction ID.	<i>String</i> 36-character numeric	<pre>'threeds_server_trans_id'=>\$threeds_server_trans_id</pre> Data is obtained from a Cavv Lookup Request or MPI 3DS Authentication Request transaction
3DS version	<i>String</i> 10-character numeric	<pre>'threeds_version'=>\$threeds_version</pre> Acceptable values:

Variable Name	Type and Limits	Set Method
NOTE: Mandatory for financial transactions using 3rd Party 3-D Secure services. If you elected to skip 3DS Authentication, you may omit the 3DS Version field.		2.0.0 = 3DS protocol 2.0.0
		2.1.0 = 3DS protocol 2.1.0
		2.2.0 = 3DS protocol 2.2.0
		2.3.0 = 3DS protocol 2.3.0
network	String	Card Brand name.
	alphanumeric	Field is case sensitive
		Possible values:
		Visa
		Mastercard
		American Express
		Interac
		Discover

Google Pay™ Token preauth transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	String 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	String 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor

SampleGoogle Pay™ Token Preauth

```

<?php

require "../mpgClasses.php";

/***** Request Variables *****/

$store_id='intuit_sped';
$api_token='spedguy';

/***** Transactional Variables *****/

$order_id='ord-' . date("dmy-G:i:s");
$cust_id='nqa cust id';
$amount='1.00';
$crypt_type='2';
$network = "MASTERCARD";
$data_key = "ot-PLa88dGe5uSy8TwbI9BeMqurl";
$threeds_server_trans_id = "de1b97ee-c610-4877-b53f-clc5ecd99bf0";
$threeds_version = "2.2";
$cavv = "kAABApFSYyd4l2eQQFJjAAAAAA=";
$dynamic_descriptor = "nqa-dd";

/***** Transactional Associative Array *****/

$googlePayTokenPreauth = new GooglePayTokenPreauth();
$googlePayTokenPreauth->setOrderId($order_id);
$googlePayTokenPreauth->setCustId($cust_id);
$googlePayTokenPreauth->setAmount($amount);
$googlePayTokenPreauth->setCryptType($crypt_type);
$googlePayTokenPreauth->setNetwork($network);
$googlePayTokenPreauth->setDataKey($data_key);
$googlePayTokenPreauth->setThreeDSServerTransId($threeds_server_trans_id);
$googlePayTokenPreauth->setThreeDSVersion($threeds_version);
$googlePayTokenPreauth->setCavv($cavv);
$googlePayTokenPreauth->setDynamicDescriptor($dynamic_descriptor);

/***** Transaction Object *****/

$mpgTxn = new mpgTransaction($googlePayTokenPreauth);

/***** Request Object *****/

$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions

/***** HTTPS Post Object *****/

/* Status Check Example
$mpgHttpPost =new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);
*/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

/***** Response *****/

$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());

```



```

print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nStatusCode = " . $mpgResponse->getStatusCode());
print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
print("\nHostId = " . $mpgResponse->getHostId());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nPayment Type = " . $mpgResponse->getPaymentType());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nPar = " . $mpgResponse->getPar());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nGooglepayPaymentMethod = " . $mpgResponse->getGooglepayPaymentMethod());

?>

```

10.9 Google Pay™ Token Purchase

The Google Pay™ Token Purchase transaction is utilized after passing a GooglePay account into a temporary token using our GooglePay Token Temporary Add then performing 3DS authentication with the token. This transaction verifies funds on the customer's card, removes the funds and prepares them for deposit into the merchant's account.

To perform the 3-D Secure authentication, the Moneris MPI or any third-party MPI may be used.

Refer to Apple or Google developer portals for details on integrating directly to their wallets to retrieve the payload data.

Google Pay™ Token Purchase for transaction object definition

```

$txnArray = array('type'=>'googlePayTokenPurchase', ...);

$mpgTxn = new mpgTransaction($txnArray);

```

HttpPostRequest object for Google Pay™ Token Purchase transaction

```

$mpgRequest = new mpgRequest($mpgTxn);

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

```

Core connection object fields (all API transactions)

Variable Name	Type and Limits	
store ID	String	'store_id'=>\$store_id
	N/A	
API token	String	'api_token'=>\$api_token

Variable Name	Type and Limits	
	N/A	

Google Pay™ Token Purchase transaction request fields – Required

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
data key	<i>String</i> 25-character alphanumeric	'data_key'=>\$data_key The temporary token returned by a <code>GooglePayTokenTempAdd</code> request.
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
Cardholder Authentication Verification Value (CAVV) NOTE: For Google Pay™ Token Purchase transactions, CAVV field contains the 3DS cryptogram only when 3DS is used prior. If you elected to skip 3DS Authentication, you may omit the CAVV field.	<i>String</i> 50-character alphanumeric	cavv=>\$cavv The 3DS cryptogram. Sent in all financial transactions with 3-D Secure, including Verified By Visa, MasterCard SecureCode, American Express SafeKey
electronic commerce indicator NOTE: For Google Pay™ Token Purchase and Token	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt Describes the category of e-commerce transaction being processed. Allowable values are:

Variable Name	Type and Limits	Set Method
Pre-Authorization transactions using 3DS Authentication, use the e-commerce indicator obtained from your 3DS Authentication.		1 – Mail Order / Telephone Order—Single 2 – Mail Order / Telephone Order—Recurring 3 – Mail Order / Telephone Order—Instalment 4 – Mail Order / Telephone Order—Unknown classification 5 – Authenticated e-commerce transaction (3-D Secure) 6 – Non-authenticated e-commerce transaction (3-D Secure) 7 – SSL-enabled merchant In Credential on File transactions where the request field e-commerce indicator is also being sent: the allowable values for e-commerce indicator are dependent on the value sent for payment indicator, as follows: if payment indicator = R, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = V, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = C, then allowable values for e-commerce indicator: 1, 5, 6 or 7 if payment indicator = U, then allowable values for e-commerce indicator: 1 or 7 if payment indicator = Z, then allowable values for e-commerce indicator: 1, 5, 6 or 7
3DS server transaction ID NOTE: Obtained from the Cavv Lookup request or MPI 3DS Authentication request. For Google Pay™ Token Purchase and Token Pre-Authorization transactions that do not use 3DS Authentication, you may omit the 3DS Server Transaction ID.	<i>String</i> 36-character numeric	'threads_server_trans_id'=>\$threads_server_trans_id Data is obtained from a Cavv Lookup Request or MPI 3DS Authentication Request transaction

Variable Name	Type and Limits	Set Method
3DS version NOTE: If you elected to skip 3DS Authentication, you may omit the 3DS Version field.	<i>String</i> 10-character numeric	<pre>'threeds_version'=>\$threeds_version</pre> Acceptable values: 2.0.0 = 3DS protocol 2.0.0 2.1.0 = 3DS protocol 2.1.0 2.2.0 = 3DS protocol 2.2.0 2.3.0 = 3DS protocol 2.3.0
network	<i>String</i> alphabetic	<pre>\$network = "MASTERCARD";</pre> Card Brand name. Field is case sensitive Possible values: Visa Mastercard American Express Interac Discover

Google Pay™ Token Purchase transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	<pre>'cust_id'=>\$cust_id</pre>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name	<pre>'dynamic_descriptor'=>\$dynamic_descriptor</pre>

Variable Name	Type and Limits	Set Method
---------------	-----------------	------------

and separator

NOTE:
Some special characters are not allowed:
<>\$%=?^{}[]\

Sample Google Pay™ Token Purchase

```
<?php

require "../mpgClasses.php";

/***** Request Variables *****/

$store_id='intuit_spd';
$api_token='spdguy';

/***** Transactional Variables *****/

$order_id='ord-' . date("dmy-G:i:s");
$cust_id='nqa cust id';
$amount='1.00';
$crypt_type='2';
$network = "MASTERCARD";
$data_key = "ot-eJ7J1FxbUrLkUow8Kko7Djie1";
$threeds_server_trans_id = "delb97ee-c610-4877-b53f-c1c5ecd99bf0";
$threeds_version = "2.2";
$cavv = "kAABApFSYyd4l2eQQFJjAAAAAA=";
$dynamic_descriptor = "nqa-dd";

/***** Transactional Associative Array *****/

$googlePayTokenPurchase = new googlePayTokenPurchase();
$googlePayTokenPurchase->setOrderId($order_id);
$googlePayTokenPurchase->setCustId($cust_id);
$googlePayTokenPurchase->setAmount($amount);
$googlePayTokenPurchase->setCryptType($crypt_type);
$googlePayTokenPurchase->setNetwork($network);
$googlePayTokenPurchase->setDataKey($data_key);
$googlePayTokenPurchase->setThreeDSSTransId($threeds_server_trans_id);
$googlePayTokenPurchase->setThreeDSVersion($threeds_version);
$googlePayTokenPurchase->setCavv($cavv);
$googlePayTokenPurchase->setDynamicDescriptor($dynamic_descriptor);

/***** Transaction Object *****/

$mpgTxn = new mpgTransaction($googlePayTokenPurchase);

/***** Request Object *****/

$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false or comment out this line for production transactions

/***** HTTPS Post Object *****/

/* Status Check Example
$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);
*/
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);

/***** Response *****/

$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nStatusCode = " . $mpgResponse->getStatusCode());
print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
print("\nHostId = " . $mpgResponse->getHostId());
print("\nIssuerId = " . $mpgResponse->getIssuerId());
print("\nPayment Type = " . $mpgResponse->getPaymentType());
print("\nSourcePanLast4 = " . $mpgResponse->getSourcePanLast4());
print("\nDataKey = " . $mpgResponse->getDataKey());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nPar = " . $mpgResponse->getPar());
print("\nThreeDSVersion = " . $mpgResponse->getThreeDSVersion());
print("\nGooglepayPaymentMethod = " . $mpgResponse->getGooglepayPaymentMethod());

?>
```

11 Offlinx™

- What Is a Pixel Tag?
- Offlinx™ and API Transactions

11.1 What Is a Pixel Tag?

A pixel tag is a piece of code that goes on a web page and requests an image file (a tiny transparent image or pixel) when loaded, which, while not visible to the user, allows Offlinx™ to gather relevant information about the user.

The data collected by our pixel tag is:

- Anonymous (not personally identifiable) and compliant with privacy standards
- Secure — utilizes SSL communication to transmit the data securely
- Not shared with anyone

11.2 Offlinx™ and API Transactions

The Offlinx™ Card Match pixel tag feature can be implemented via the Unified API with the Card Match ID variable, which corresponds to the Transaction ID in Offlinx™. The Card Match ID must be a unique value for each transaction.

For more information about the Offlinx™ solution, consult the Offlinx™ Pixel Tag Setup Guide available from your account/service manager.

API transactions where this applies:

- Purchase
- Pre-Authorization
- Purchase with 3-D Secure – cavv_purchase
- Pre-Authorization with 3-D Secure – cavv_preauth

- Cavv Purchase – Apple Pay and Google Pay™
- Cavv Pre-Authorization – Apple Pay & Google Pay™

12 Convenience Fee

- 12.1 About Convenience Fee
- 12.3 Purchase with Convenience Fee
- 12.4 Purchase with Customer Info and Convenience Fee
- 12.5 Purchase with 3-D Secure and Convenience Fee

12.1 About Convenience Fee

The Convenience Fee program was designed to allow merchants to offer the convenience of an alternative payment channel to the cardholder at a charge. This applies only when providing a true "convenience" in the form of an alternative payment channel outside the merchant's customary face-to-face payment channels. The convenience fee will be a separate charge on top of what the consumer is paying for the goods and/or services they were given, and this charge will appear as a separate line item on the consumer's statement.

Convenience fee transactions do not support MCP or digital wallets.

NOTE: The Convenience Fee program is only offered to certain supported Merchant Category Codes (MCCs). Please speak to your account manager for further details.

12.2 Convenience Fee Information Object

Any transaction that supports Convenience Fee has an available set method for the Convenience Fee Information object.

The Convenience Fee Information object contains one request field, **convenience fee amount**.

Convenience Fee Info object definition

```
$mpgConvFee = new mpgConvFeeInfo($convFeeTemplate);
```

Convenience Fee Info object set method

```
$mpgTxn->setConvFeeInfo($mpgConvFee);
```

Convenience Fee Information object request fields

Variable Name	Type and Limits	Description
Convenience Fee Information	<i>Object</i> N/A	Contains fields related to the Convenience Fee feature

Variable Name	Type and Limits	Description
convenience fee amount	<i>String</i> 9-character decimal	Dollar amount charged to the customer as a convenience fee

12.3 Purchase with Convenience Fee

Information below describes a Purchase transaction request that also includes the Convenience Fee Info object.

Purchase with Convenience Fee transaction object definition

```
$txnArray = array('type'=>'purchase', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Purchase with Convenience Fee transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
```

Purchase with Convenience Fee transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
Convenience Fee Information	<i>Object</i> N/A	<code>\$mpgConvFee = new mpgConvFeeInfo(\$convFeeTemplate);</code>
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	<code>'order_id'=>\$order_id</code>
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	<code>'amount'=>\$amount</code>

Variable Name	Type and Limits	Set Method
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
NOTE: In Credential on File transactions where the request field e-commerce indicator is also being sent, the acceptable values for e-commerce indicator are dependent on the value sent for payment indicator; see request field definitions for more information.		
convenience fee amount	<i>String</i> 9-character decimal	\$convFeeTemplate = array (convenience_fee=>\$convfee_amount) ;

Purchase with Convenience Fee transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'cust_id'=>\$cust_id
dynamic descriptor	<i>String</i> 20-character alphanumeric	'dynamic_descriptor'=>\$dynamic_descriptor

Variable Name	Type and Limits	Set Method
	total of 22 characters including your merchant name and separator	
	NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo (\$mpgAvsInfo);
CVD Information	<i>Object</i> N/A	\$mpgTxn->setCvdInfo (\$mpgCvdInfo);

Sample Purchase with Convenience Fee

```

<?php
/* Moneris Gateway Canada Convenience Fee Account Required this transaction*/
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00392';
$api_token='qYdISUhHiOdfTrlCLNpN';
// $status = 'false';
/***** Transaction Variables *****/
$orderid='ord-' . date("dmy-G:i:s");
$amount='10.00';
$pan='4242424242424242';
$expiry_date='1812';
$dynamic_descriptor='test';
/***** Transaction Array *****/
$txnArray=array(type=>'purchase',
order_id=>$orderid,
cust_id=>'cust',
amount=>$amount,
pan=>$pan,
expdate=>$expiry_date,
crypt_type=>'7',
dynamic_descriptor=>$dynamic_descriptor
);
/***** ConvFee Associative Array *****/
$convFeeTemplate = array(
convenience_fee=>'1.00'
);
/***** ConvFee Object *****/
$mpgConvFee = new mpgConvFeeInfo($convFeeTemplate);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Set ConvFee *****/
$mpgTxn->setConvFeeInfo($mpgConvFee);
/***** Request Object *****/

```

Sample Purchase with Convenience Fee

```
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"CA" for sending transaction to Canadian
environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** mpgHttpPost Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
// $mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response Object *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nISO = " . $mpgResponse->getISO());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCardLevelResult = " . $mpgResponse->getCardLevelResult());
print("\nCfSuccess = " . $mpgResponse->getCfSuccess());
print("\nCfStatus = " . $mpgResponse->getCfStatus());
print("\nFeeAmount = " . $mpgResponse->getFeeAmount());
print("\nFeeRate = " . $mpgResponse->getFeeRate());
print("\nFeeType = " . $mpgResponse->getFeeType());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

12.4 Purchase with Customer Info and Convenience Fee

Information below is a Purchase transaction request that also includes the Convenience Fee Info and Customer Information objects.

Purchase with Customer Info and Convenience Fee transaction object definition

```
$txnArray = array('type'=>'purchase', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Purchase with Customer Info and Convenience Fee transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Purchase with Customer Info and Convenience Fee transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
Convenience Fee Information	<i>Object</i> N/A	<code>\$mpgConvFee = new mpgConvFeeInfo (\$convFeeTemplate);</code>
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	<code>'order_id'=>\$order_id</code>
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	<code>'amount'=>\$amount</code>
credit card number	<i>String</i> max 20-character alpha- numeric	<code>'pan'=>\$pan</code>
expiry date	<i>String</i> 4-character alphanumeric YYMM	<code>'expiry_date'=>\$expiry_date</code>
electronic commerce indicator	<i>String</i> 1-character alphanumeric	<code>'crypt_type'=>\$crypt</code>
convenience fee amount	<i>String</i> 9-character decimal	<code>\$mpgConvFee = new mpgConvFeeInfo (\$convFeeTemplate);</code>

Purchase with Customer Info and Convenience Fee transaction request fields – Optional

Variable Name	Type and Limits	Set Method
customer ID	<i>String</i>	<code>'cust_id'=>\$cust_id</code>

Variable Name	Type and Limits	Set Method
	50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	'dynamic_descriptor'=>\$dynamic_descriptor
Customer Information	<i>Object</i> N/A	\$mpgTxn->setCustInfo(\$mpgCustInfo);
AVS Information	<i>Object</i> N/A	\$mpgTxn->setAvsInfo(\$mpgAvsInfo);
CVD Information NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent with cardholder-initiated transactions only—merchants must not store CVD information.	<i>Object</i> N/A	\$mpgTxn->setCvdInfo(\$mpgCvdInfo);

Sample Purchase with Customer Info and Convenience Fee

```

<?php
## Example php -q TestPurchase-CustInfo.php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00392';
$api_token='qYdISUHiOdfTr1CLNpN';
/***** Transactional Variables *****/
$type='purchase';
$order_id='ord-' . date("dmy-G:i:s");
$cust_id='my cust id';
$amount='114.28';
$pan='4242424242424242';
$expiry_date='0812'; //December 2008
$crypt='7';
/***** Customer Information Variables *****/
$first_name = 'Cedric';
$last_name = 'Benson';
$company_name = 'Chicago Bears';
$address = '334 Michigan Ave';
$city = 'Chicago';
$province = 'Illinois';
$postal_code = 'M1M1M1';
$country = 'United States';
$phone_number = '453-989-9876';
$fax = '453-989-9877';
$tax1 = '1.01';
$tax2 = '1.02';
$tax3 = '1.03';
$shipping_cost = '9.95';
$email = 'Joe@widgets.com';
$instructions = "Make it fast";
/***** Line Item Variables *****/
$item_name = array();
$item_quantity = array();
$item_product_code = array();
$item_extended_amount = array();
$item_name[0] = 'Guy Lafleur Retro Jersey';
$item_quantity[0] = '1';
$item_product_code[0] = 'JRSCDA344';
$item_extended_amount[0] = '129.99';
$item_name[1] = 'Patrick Roy Signed Koho Stick';
$item_quantity[1] = '1';
$item_product_code[1] = 'JPREEA344';
$item_extended_amount[1] = '59.99';
/***** Customer Information Object *****/
$mpgCustInfo = new mpgCustInfo();
/***** Set Customer Information *****/
$billing = array(
    'first_name' => $first_name,
    'last_name' => $last_name,
    'company_name' => $company_name,
    'address' => $address,
    'city' => $city,
    'province' => $province,
    'postal_code' => $postal_code,
    'country' => $country,
    'phone_number' => $phone_number,
    'fax' => $fax,
    'tax1' => $tax1,
    'tax2' => $tax2,

```


Sample Purchase with Customer Info and Convenience Fee

```

'tax3' => $tax3,
'shipping_cost' => $shipping_cost
);
$mpgCustInfo->setBilling($billing);
$shipping = array(
'first_name' => $first_name,
'last_name' => $last_name,
'company_name' => $company_name,
'address' => $address,
'city' => $city,
'province' => $province,
'postal_code' => $postal_code,
'country' => $country,
'phone_number' => $phone_number,
'fax' => $fax,
'tax1' => $tax1,
'tax2' => $tax2,
'tax3' => $tax3,
'shipping_cost' => $shipping_cost
);
$mpgCustInfo->setShipping($shipping);
$mpgCustInfo->setEmail($email);
$mpgCustInfo->setInstructions($instructions);
/***** Set Line Item Information *****/
$item[0] = array(
'name'=>$item_name[0],
'quantity'=>$item_quantity[0],
'product_code'=>$item_product_code[0],
'extended_amount'=>$item_extended_amount[0]
);
$item[1] = array(
'name'=>$item_name[1],
'quantity'=>$item_quantity[1],
'product_code'=>$item_product_code[1],
'extended_amount'=>$item_extended_amount[1]
);
$mpgCustInfo->setItems($item[0]);
$mpgCustInfo->setItems($item[1]);
/***** ConvFee Associative Array *****/
$convFeeTemplate = array(
'convenience_fee'=>'2.00'
);
/***** ConvFee Object *****/
$mpgConvFee = new mpgConvFeeInfo($convFeeTemplate);
/***** Transactional Associative Array *****/
$txnArray=array(
'type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'amount'=>$amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Set Customer Information *****/
$mpgTxn->setCustInfo($mpgCustInfo);
/***** Set ConvFee *****/
$mpgTxn->setConvFeeInfo($mpgConvFee);

```

Sample Purchase with Customer Info and Convenience Fee

```

/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
?>

```

12.5 Purchase with 3-D Secure and Convenience Fee

Information below describes a Purchase with 3-D Secure transaction request that also includes the Convenience Fee Info object.

Convenience Fee Purchase with 3-D Secure transaction object definition

```
$txnArray = array('type'=>'cavv_purchase', ...);
```

```
$mpgTxn = new mpgTransaction($txnArray);
```

HttpPostRequest object for Convenience Fee Purchase w/ 3-D Secure transaction

```
$mpgRequest = new mpgRequest($mpgTxn);
```

```
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
```

Convenience Fee Purchase with 3-D Secure transaction request fields – Required

For a full description of mandatory and optional values, see Appendix A Definition of Request Fields.

Variable Name	Type and Limits	Set Method
Convenience Fee Information	Object N/A	\$mpgConvFee = new mpgConvFeeInfo (\$convFeeTemplate);

Variable Name	Type and Limits	Set Method
order ID	<i>String</i> 50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	'order_id'=>\$order_id
amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	'amount'=>\$amount
credit card number	<i>String</i> max 20-character alphanumeric	'pan'=>\$pan
expiry date	<i>String</i> 4-character alphanumeric YYMM	'expiry_date'=>\$expiry_date
electronic commerce indicator	<i>String</i> 1-character alphanumeric	'crypt_type'=>\$crypt
Cardholder Authentication Verification Value (CAVV)	<i>String</i> 50-character alphanumeric	cavv=>\$cavv
convenience fee amount	<i>String</i> 9-character decimal	<pre>\$convFeeTemplate = array (convenience_fee=>\$convfee_ amount);</pre>

Purchase with 3-D Secure and Convenience Fee transaction request fields – Optional

Variable Name	Type and Limits	Set Method
status check	<i>Boolean</i>	\$mpgHttpPost =new

Variable Name	Type and Limits	Set Method
	true/false	<code>mpgHttpPostStatus(\$store_id, \$api_token, \$status, \$mpgRequest);</code>
customer ID	<i>String</i> 50-character alphanumeric NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	<code>'cust_id'=>\$cust_id</code>
dynamic descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	<code>'dynamic_descriptor'=>\$dynamic_descriptor</code>
electronic commerce indicator	<i>String</i> 1-character alphanumeric	<code>'crypt_type'=>\$crypt</code>
Customer Information	<i>Object</i> N/A	<code>\$mpgTxn->setCustInfo(\$mpgCustInfo);</code>
AVS Information	<i>Object</i> N/A	<code>\$mpgTxn->setAvsInfo(\$mpgAvsInfo);</code>
CVD Information	<i>Object</i> N/A NOTE: When storing credentials on the initial transaction, the CVD object must be sent; for subsequent transactions using stored credentials, CVD can be sent	<code>\$mpgTxn->setCvdInfo(\$mpgCvdInfo);</code>

Variable Name	Type and Limits	Set Method
with cardholder-initiated transactions only—merchants must not store CVD information.		

Sample Purchase with 3-D Secure and Convenience Fee

```

<?php
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='monca00392';
$api_token='qYdISUhHiOdfTr1CLNpN';
// $status = 'false';
/***** Transactional Variables *****/
$type='cavv_purchase';
$order_id="ord-".date("dmy-G:i:s");
$cust_id='customer1';
$amount='1.00';
$pan='4242424242424242';
$expiry_date='0912';
$cavv='AAABBJg0VhI0VniQEjRWAAAAAA';
// $cavv='AAABBJg0VhI0VniQEjRWAAAAAA=';
$commcard_invoice='Invoice 5757FRJ8';
$commcard_tax_amount='1.00';
$crypt_type = '7';
/***** Transaction Associative Array *****/
$txnArray=array(
    type=>$type,
    order_id=>$order_id,
    cust_id=>$cust_id,
    amount=>$amount,
    pan=>$pan,
    expdate=>$expiry_date,
    cavv=>$cavv,
    commcard_invoice=>$commcard_invoice,
    commcard_tax_amount=>$commcard_tax_amount,
    crypt_type=>$crypt_type, //mandatory for AMEX only
    dynamic_descriptor=>'test'
);
/***** ConvFee Associative Array *****/
$convFeeTemplate = array(
    convenience_fee=>'1.00'
);
/***** ConvFee Object *****/
$mpgConvFee = new mpgConvFeeInfo($convFeeTemplate);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Set ConvFee *****/
$mpgTxn->setConvFeeInfo($mpgConvFee);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "CA" for sending transaction to Canadian
environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/

```

Sample Purchase with 3-D Secure and Convenience Fee

```

$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
//Status check example
//$mpgHttpPost = new mpgHttpPostStatus($store_id,$api_token,$status,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTicket = " . $mpgResponse->getTicket());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nCardLevelResult = " . $mpgResponse->getCardLevelResult());
print("\nCavvResultCode = " . $mpgResponse->getCavvResultCode());
print("\nCfSuccess = " . $mpgResponse->getCfSuccess());
print("\nCfStatus = " . $mpgResponse->getCfStatus());
print("\nFeeAmount = " . $mpgResponse->getFeeAmount());
print("\nFeeRate = " . $mpgResponse->getFeeRate());
print("\nFeeType = " . $mpgResponse->getFeeType());
//print("\nStatusCode = " . $mpgResponse->getStatusCode());
//print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>

```

13 Recurring Billing

- 13.1 About Recurring Billing
- 13.2 Purchase with Recurring Billing
- 13.3 Recurring Billing Update
- 13.4 Recurring Billing Response Fields and Codes
- 13.5 Credential on File and Recurring Billing

13.1 About Recurring Billing

Recurring Billing allows you to set up payments whereby Moneris automatically processes the transactions and bills customers on your behalf based on the billing cycle information you provide.

Recurring Billing series are created by sending the Recurring Billing object in these transactions:

- Purchase
- Purchase with Vault
- Purchase with 3-D Secure (cavvPurchase)

You can modify a Recurring Billing series after it has been created by sending the Recurring Billing Update administrative transaction.

NOTE: Alternatively, if you prefer to manage recurring series on your own merchant system, you can send the periodic payments as basic Purchase transactions with the e-commerce indicator (`crypt_type`) value = 2 and with the Credential on File info object included.

13.2 Purchase with Recurring Billing

Recurring Billing Info Object Definition

```
$recurArray = array(  
    'recur_unit'=>$recurUnit, // (day | week | month)  
    'start_date'=>$startDate, //yyyy/mm/dd  
    'num_rekurs'=>$numRekurs,  
    'start_now'=>$startNow,  
    'period' => $recurInterval,
```

```
'recur_amount'=> $recurAmount
);

$mpgRecur = new mpgRecur($recurArray);
```

Transaction object set method

```
$mpgTxn->setRecur($mpgRecur);
```

Recurring Billing Info Object Request Fields

Variable Name	Type and Limits	Description
number of recurs	<i>String</i> numeric 1-999	The number of times that the transaction must recur
period	<i>String</i> numeric 1-999	Number of recur unit intervals that must pass between recurring billings
start date	<i>String</i> YYYYMMDD format	Date of the first future recurring billing transaction; this must be a date in the future If an additional charge will be made immediately, the start now variable must be set to true
start now	<i>String</i> true/false	Set to true if a charge will be made against the card immediately; otherwise set to false When set to false, use Card Verification prior to sending the Purchase with Recurring Billing and Credential on File objects
recurring amount	<i>String</i>	Dollar amount of the recurring transaction

NOTE: Amount to be billed immediately can differ from the subsequent recurring amounts

Variable Name	Type and Limits	Description
	10-character decimal, minimum three digits Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	This amount will be billed on the start date, and then billed repeatedly based on the interval defined by period and recur unit
recur unit	String day, week, month or eom	Unit to be used as a basis for the interval Works in conjunction with the period variable to define the billing frequency

Sample Purchase with Recurring Billing

```

<?php
##
## Example php -q TestPurchase-Recur.php store3 yesguy unique_order_id
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id = 'store5';
$api_token = 'yesguy';
/***** Recur Variables *****/
$recurUnit = 'eom';
$startDate = '2018/11/30';
$numRekurs = '4';
$recurInterval = '10';
$recurAmount = '31.00';
$startNow = 'true';
/***** Transactional Variables *****/
$orderId = 'ord-' . date("dmy-G:i:s");
$custId = 'student_number';
$creditCard = '5454545454545454';
$nowAmount = '10.00';
$expiryDate = '0912';
$scriptType = '7';
/***** Recur Associative Array *****/
$recurArray = array('recur_unit'=>$recurUnit, // (day | week | month)
'start_date'=>$startDate, // yyyy/mm/dd
'num_rekurs'=>$numRekurs,
'start_now'=>$startNow,
'period' => $recurInterval,
'recur_amount'=> $recurAmount
);
$mpgRecur = new mpgRecur($recurArray);
/***** Transactional Associative Array *****/
$txnArray=array('type'=>'purchase',
'order_id'=>$orderId,

```

Sample Purchase with Recurring Billing

```

'cust_id'=>$custId,
'amount'=>$nowAmount,
'pan'=>$creditCard,
'expdate'=>$expiryDate,
'crypt_type'=>$cryptType
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Recur Object *****/
$mpgTxn->setRecur($mpgRecur);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setPaymentIndicator("R");
$cof->setPaymentInformation("2");
$cof->setIssuerId("168451306048014");
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id, $api_token, $mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print ("\nCardType = " . $mpgResponse->getCardType());
print ("\nTransAmount = " . $mpgResponse->getTransAmount());
print ("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print ("\nReceiptId = " . $mpgResponse->getReceiptId());
print ("\nTransType = " . $mpgResponse->getTransType());
print ("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print ("\nResponseCode = " . $mpgResponse->getResponseCode());
print ("\nISO = " . $mpgResponse->getISO());
print ("\nMessage = " . $mpgResponse->getMessage());
print ("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print ("\nAuthCode = " . $mpgResponse->getAuthCode());
print ("\nComplete = " . $mpgResponse->getComplete());
print ("\nTransDate = " . $mpgResponse->getTransDate());
print ("\nTransTime = " . $mpgResponse->getTransTime());
print ("\nTicket = " . $mpgResponse->getTicket());
print ("\nTimedOut = " . $mpgResponse->getTimedOut());
print ("\nRecurSuccess = " . $mpgResponse->getRecurSuccess());
print ("\nIssuerId = " . $mpgResponse->getIssuerId());
?>

```

13.3 Recurring Billing Update

After you have set up a Recurring Billing transaction series, you can change some of the details of the series as long as it has not yet completed the preset recurring duration (i.e., it hasn't terminated yet).

Before sending a Recurring Billing Update transaction that updates the credit card number, you must send a Card Verification request. This requirement does not apply if you are only updating the schedule or amount.

Things to Consider:

- When completing the update recurring billing portion please keep in mind that the recur bill dates cannot be changed to have an end date greater than 10 years from today and cannot be changed to have an end date end today or earlier.

Recurring Billing Update transaction object definition

```
$txnArray=array('type'=>'recur_update',... );
```

HttpPostRequest object for Recurring Billing Update transaction

```
$mpgTxn = new mpgTransaction($txnArray);
```

```
$mpgRequest = new mpgRequest($mpgTxn);
```

Recurring Billing Update transaction values**Table 1 Recurring Billing Update – Basic Required Fields**

Variable Name	Type and Limits	Set Method
Order ID order_id	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id

Table 2 Recurring Billing Update – Basic Optional Fields

Variable Name	Type and Limits	Set Method
Customer ID cust_id	<i>String</i> 50-character alphanumeric	'cust_id'=>\$cust_id
Credit card number pan	<i>String</i> 20-character alphanumeric	'pan'=>\$pan
Expiry date expdate	<i>String</i> YYMM	'expdate'=>\$expiry_date

Table 3 Recurring Billing Update – Recurring Billing Required Fields

Variable Name	Type and Limits	Set Method	Description
Recurring amount recur_amount	<i>String</i>	'recur_amount'=>\$recur_amount	Changes the amount that is billed recurrently The change takes effect on the next charge
Add number of recurs add_num	<i>String</i> numeric, 1-999	'add_num_recur'=> \$add_num	Adds to the given number of recurring transactions to the current (remaining) number This can be used if a customer decides to extend a membership or subscription Cannot be used to decrease the current number of recurring transactions; use Change number of recurs instead
Change number of recurs total_num	<i>String</i> numeric, 1-999	'total_num_recur'=> \$total_num	Replaces the current (remaining) number of recurring transactions
Hold recurring billing hold	<i>String</i> true/false	'hold' => \$hold	Temporarily pauses recurring billing While a transaction is on hold, it is not billed for the recurring amount; however, the number of remaining recurs continues to be decremented during that time
Terminate recurring transaction terminate	<i>String</i> true/false	'terminate' => \$terminate	Terminates recurring billing NOTE: After it has been terminated, a recurring transaction cannot be reactivated; a new purchase transaction with recurring billing must be submitted.

Sample Recurring Billing Update

```

<?php
##
## Example php -q TestRecurUpdate.php store1
##
require "../mpgClasses.php";
/***** Request Variables *****/
$store_id='store5';
$api_token='yesguy';
/***** Transactional Variables *****/
$type='recur_update';
$cust_id='my cust id';
$order_id='ord-110515-10:45:21';
$recur_amount='1.00';
$pan='4242424242424242';
$expiry_date='1811';
$add_num='';
$total_num='7';
$hold = 'false';
$terminate = 'false';
/***** Transactional Associative Array *****/
$txnArray=array('type'=>$type,
'order_id'=>$order_id,
'cust_id'=>$cust_id,
'recur_amount'=>$recur_amount,
'pan'=>$pan,
'expdate'=>$expiry_date,
'add_num_rekurs' => $add_num,
'total_num_rekurs' => $total_num,
'hold' => $hold,
'terminate' => $terminate
);
/***** Credential on File *****/
$cof = new CofInfo();
$cof->setIssuerId("168451306048014");
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
$mpgTxn->setCofInfo($cof);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); // "US" for sending transaction to US environment
$mpgRequest->setTestMode(true); // false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost = new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());
print("\nTimedOut = " . $mpgResponse->getTimedOut());
print("\nRecurUpdateSuccess = " . $mpgResponse->getRecurUpdateSuccess());
print("\nNextRecurDate = " . $mpgResponse->getNextRecurDate());
print("\nRecurEndDate = " . $mpgResponse->getRecurEndDate());
?>

```

13.4 Recurring Billing Response Fields and Codes

Table 12 outlines the response fields that are part of recurring billing. Some are available when you set up recurring billing (such as with a Purchase transaction), and some are available when you update an existing transaction with the Recurring Billing transaction.

Receipt object definition

```
$mpgResponse=$mpgHttpPost->getMpgResponse();
```

Table 12: Recurring Billing response fields

Value	Type	Limits	Get method
	Description		
Transaction object with Recurring Billing response fields			
Response code	String	3-character numeric	\$mpgResponse->getResponseCode ()
	See Table 13: for a description of possible response codes.		
Recur success	String	TBD	\$mpgResponse->getRecurSuccess ()
	Indicates whether the transaction successfully registered		
Recur update object response fields			
Recur update success	String	true/false	\$mpgResponse->getRecurUpdateSuccess ()
	Indicates whether the transaction successfully updated.		
Next recur date	String	yyyy-mm-dd format	\$mpgResponse->getNextRecurDate ()
	Indicates when the transaction will be billed again.		
Recur end date	String	yyyy-mm-dd format	\$mpgResponse->getRecurEndDate ()
	Indicates when the Recurring Billing Transaction will end.		

The Recur Update response is a 3-digit numeric value. The following is a list of all possible responses after a Recur Update transaction has been sent.

Table 13: Recur update response codes

Request Value	Definition
001	Recurring transaction successfully updated (optional: terminated)
983	Cannot find the previous transaction
984	Data error: (optional: field name)
985	Invalid number of recurs
986	Incomplete: timed out

Table 13: Recur update response codes (continued)

Request Value	Definition
null	Error: Malformed XML

13.5 Credential on File and Recurring Billing

NOTE: The value of the **payment indicator** field must be **R** when sending Recurring Billing transactions.

For Recurring Billing transactions which are set to start **immediately**:

1. Send a Purchase transaction request with both the Recurring Billing and Credential on File info objects (with Recurring Billing object field **start now** = true)

For Recurring Billing transactions which are set to start on a **future** date:

1. Send Card Verification transaction request including the Credential on File info object to get the Issuer ID
2. Send Purchase transaction request with the Recur and Credential on File info objects included

For updating a Recurring Billing series where you are updating the card number (does not apply if you are only modifying the schedule or amount in a recurring series):

1. Send Card Verification request including the Credential on File info object to get the Issuer ID
2. Send a Recurring Billing Update transaction

For more information about the Recurring Billing object, see Definition of Request Fields – Recurring.

14 Customer Information

- 14.1 Using the Customer Information Object
- 14.2 Customer Information Sample Code

The Customer Information object offers a number of fields to be submitted as part of the financial transaction, and stored by Moneris. These details may be viewed in the future in the Merchant Resource Center.

The following transactions support the Customer Information object :

- Purchase (Basic, Interac Debit and Vault)
- Pre-Authorization (Basic and Vault)
- Re-Authorization (Basic)

The Customer Information object holds three types of information:

- Billing/Shipping information
- Miscellaneous customer information properties
- Item information

Things to Consider:

- If you send characters that are not included in the allowed list, these extra transaction details may not be stored.
- All fields are alphanumeric and allow the following characters: a-z A-Z 0-9 _ - : . @ \$ = /
- All French accents should be encoded as HTML entities, such as é.
- The data sent in Billing and Shipping Address fields will not be used for any address verification.

14.1 Using the Customer Information Object

- 14.1.1 Customer Info Object – Miscellaneous Properties
- 14.1.2 Customer Info Object – Billing/Shipping Information
- 14.1.3 Customer Info Object – Item Information

In addition to instantiating a transaction object and a connection object (as you would for a normal transaction), you must instantiate a CustInfo object.

Any transaction that supports CustInfo has a setCustInfo method. This is used to write the customer information to the transaction object before writing the transaction object to the connection object.

CustInfo object definition

```
$mpgCustInfo = new mpgCustInfo();
```

Transaction object set method

```
$mpgTxn->setCustInfo($mpgCustInfo);
```

14.1.1 Customer Info Object – Miscellaneous Properties

While most of the Customer Information data is organized into objects, there are some values that are properties of the CustInfo object itself. They are explained in the table below.

Table 14: CustInfo object miscellaneous properties

Value	Type	Limits	Set method
Email Address	String	60-character alphanumeric	<code>\$mpgCustInfo->setEmail(\$email);</code>
Instructions	String	100-character alphanumeric	<code>\$mpgCustInfo->setInstructions(\$note);</code>

14.1.2 Customer Info Object – Billing/Shipping Information

Billing and shipping information is stored as part of the Customer Information object. They can be written to the object in one of two ways:

- Using set methods
- Using hash tables

Whichever method you use, you will be writing the information found in the table below for both the billing information and the shipping information.

All values are alphanumeric strings. Their maximum lengths are given in the Limit column.

Table 15: Billing and shipping information values

Value	Limit	Hash table key
First name	30	"first_name"

Table 15: Billing and shipping information values (continued)

Value	Limit	Hash table key
Last name	30	"last_name"
Company name	50	"company_name"
Address	70	"address"
City	30	"city"
Province/State	30	"province"
Postal/Zip code	30	"postal_code"
Country	30	"country"
Phone number (voice)	30	"phone"
Fax number	30	"fax"
Federal tax	10	"tax1"
Provincial/State tax	10	"tax2"
County/Local/Specialty tax	10	"tax3"
Shipping cost	10	"shipping_cost"

14.1.2.1 Set Methods for Billing and Shipping Info

The billing information and the shipping information for a given CustInfo object are written by using the `$mpgCustInfo->setBilling($billing)` ; and `$mpgCustInfo->setShipping($shipping)` ; methods respectively:

```
$billing = array(
    'first_name' => $first_name,
    'last_name' => $last_name,
    'company_name' => $company_name,
    'address' => $address,
    'city' => $city,
    'province' => $province,
```

```
'postal_code' => $postal_code,
'country' => $country,
'phone_number' => $phone_number,
'fax' => $fax,
'tax1' => $tax1,
'tax2' => $tax2,
'tax3' => $tax3,
'shipping_cost' => $shipping_cost
);
$mpgCustInfo->setBilling($billing);
$shipping = array(
'first_name' => $first_name,
'last_name' => $last_name,
'company_name' => $company_name,
'address' => $address,
'city' => $city,
'province' => $province,
'postal_code' => $postal_code,
'country' => $country,
'phone_number' => $phone_number,
'fax' => $fax,
'tax1' => $tax1,
'tax2' => $tax2,
'tax3' => $tax3,
'shipping_cost' => $shipping_cost
);
$mpgCustInfo->setShipping($shipping);
```

Both of these methods have the same set of mandatory arguments. They are described in the Billing and shipping information values table in 14.1.2.1 Set Methods for Billing and Shipping Info.

For sample code, see 14.2 Customer Information Sample Code.

14.1.2.2 Using Hash Tables for Billing and Shipping Info

Writing billing or shipping information using hash tables is done as follows:

1. Instantiate a CustInfo object.
2. Instantiate a hash table object. (The sample code uses a different hash table for billing and shipping for clarity purposes. However, the skillful developer can re-use the same one.)
3. Build the hash table using put methods with the hash table keys found in the Billing and shipping information values table in 14.1.2 Customer Info Object – Billing/Shipping Information.
4. Call the CustInfo object's setBilling/setShipping method to pass the hash table information to the CustInfo object
5. Call the transaction object's setCustInfo method to write the CustInfo object (with the billing/-shipping information to the transaction object.

For sample code, see 14.2 Customer Information Sample Code.

14.1.3 Customer Info Object – Item Information

The Customer Information object can hold information about multiple items. For each item, the values in the table below can be written.

All values are strings, but note the guidelines in the Limits column.

Table 16: Item information values

Value	Limits	Hash table key
Item name	45-character alphanumeric	"name"
Item quantity	5-character numeric	"quantity"
Item product code	20-character alphanumeric	"product_code"
Item extended amount	9-character decimal with at least 3 digits and 2 penny values. 0.01-999999.99	"extended_amount"

One way of representing multiple items is with four arrays. This is the method used in the sample code. However, there are two ways to write the item information to the CustInfo object:

- Set methods
- Hash tables

14.1.3.1 Set Methods for Item Information

All the item information found in the Item information values table in 14.1.3 Customer Info Object – Item Information is written to the CustInfo object in one instruction for a given item. Such as:

```
customer.setItem(item_description, item_quantity, item_product_code, item_
extended_amount);
```

For sample code (showing how to use arrays to write information about two items), see 14.2 Customer Information Sample Code.

14.1.3.2 Using Hash Tables for Item Information

Writing item information using hash tables is done as follows:

1. Instantiate a CustInfo object.
2. Instantiate a hash table object. (The sample code uses a different hash table for each item for clarity purposes. However, the skillful developer can re-use the same one.)
3. Build the hash table using put methods with the hash table keys in the Item information values table in 14.1.3 Customer Info Object – Item Information.
4. Call the CustInfo object's setItem method to pass the hash table information to the CustInfo object
5. Call the transaction object's setCustInfo method to write the CustInfo object (with the item information to the transaction object.

For sample code that shows how to use arrays to write information about two items, see 14.2 Customer Information Sample Code.

14.2 Customer Information Sample Code

Below is an example of a Basic Purchase with Customer Information transaction.

Note that the two items ordered are represented by four arrays, and the billing and shipping details are the same.

Sample Purchase with Customer Information
<pre>## Example php -q TestPurchase-CustInfo.php require "../mpgClasses.php"; /***** Request Variables *****/ \$store_id='store5'; \$api_token='yesguy'; /***** Transactional Variables *****/ \$type='purchase'; \$order_id='ord-'.date("dmy-G:i:s"); \$cust_id='my cust id'; \$amount='1.00'; \$pan='4242424242424242';</pre>

Sample Purchase with Customer Information

```

$expiry_date='0812'; //December 2008
$crypt='7';
/***** Customer Information Variables *****/
$first_name = 'Cedric';
$last_name = 'Benson';
$company_name = 'Chicago Bears';
$address = '334 Michigan Ave';
$city = 'Chicago';
$province = 'Illinois';
$postal_code = 'M1M1M1';
$country = 'United States';
$phone_number = '453-989-9876';
$fax = '453-989-9877';
$tax1 = '1.01';
$tax2 = '1.02';
$tax3 = '1.03';
$shipping_cost = '9.95';
$email = 'Joe@widgets.com';
$instructions = "Make it fast";
/***** Line Item Variables *****/
$item_name[0] = 'Guy Lafleur Retro Jersey';
$item_quantity[0] = '1';
$item_product_code[0] = 'JRSCDA344';
$item_extended_amount[0] = '129.99';
$item_name[1] = 'Patrick Roy Signed Koho Stick';
$item_quantity[1] = '1';
$item_product_code[1] = 'JPREEA344';
$item_extended_amount[1] = '59.99';
/***** Customer Information Object *****/
$mpgCustInfo = new mpgCustInfo();
/***** Set Customer Information *****/
$billing = array(
    'first_name' => $first_name,
    'last_name' => $last_name,
    'company_name' => $company_name,
    'address' => $address,
    'city' => $city,
    'province' => $province,
    'postal_code' => $postal_code,
    'country' => $country,
    'phone_number' => $phone_number,
    'fax' => $fax,
    'tax1' => $tax1,
    'tax2' => $tax2,
    'tax3' => $tax3,
    'shipping_cost' => $shipping_cost
);
$mpgCustInfo->setBilling($billing);
$shipping = array(
    'first_name' => $first_name,
    'last_name' => $last_name,
    'company_name' => $company_name,
    'address' => $address,
    'city' => $city,
    'province' => $province,
    'postal_code' => $postal_code,
    'country' => $country,
    'phone_number' => $phone_number,
    'fax' => $fax,

```

Sample Purchase with Customer Information

```

'tax1' => $tax1,
'tax2' => $tax2,
'tax3' => $tax3,
'shipping_cost' => $shipping_cost
);
$mpgCustInfo->setShipping($shipping);
$mpgCustInfo->setEmail($email);
$mpgCustInfo->setInstructions($instructions);
/***** Set Line Item Information *****/
$item[0] = array(
    'name'=>$item_name[0],
    'quantity'=>$item_quantity[0],
    'product_code'=>$item_product_code[0],
    'extended_amount'=>$item_extended_amount[0]
);
$item[1] = array(
    'name'=>$item_name[1],
    'quantity'=>$item_quantity[1],
    'product_code'=>$item_product_code[1],
    'extended_amount'=>$item_extended_amount[1]
);
$mpgCustInfo->setItems($item[0]);
$mpgCustInfo->setItems($item[1]);
/***** Transactional Associative Array *****/
$txnArray=array(
    'type'=>$type,
    'order_id'=>$order_id,
    'cust_id'=>$cust_id,
    'amount'=>$amount,
    'pan'=>$pan,
    'expdate'=>$expiry_date,
    'crypt_type'=>$crypt
);
/***** Transaction Object *****/
$mpgTxn = new mpgTransaction($txnArray);
/***** Set Customer Information *****/
$mpgTxn->setCustInfo($mpgCustInfo);
/***** Request Object *****/
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment
$mpgRequest->setTestMode(true); //false for production transactions
/***** HTTPS Post Object *****/
$mpgHttpPost =new mpgHttpPost($store_id,$api_token,$mpgRequest);
/***** Response *****/
$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nCardType = " . $mpgResponse->getCardType());
print("\nTransAmount = " . $mpgResponse->getTransAmount());
print("\nTxnNumber = " . $mpgResponse->getTxnNumber());
print("\nReceiptId = " . $mpgResponse->getReceiptId());
print("\nTransType = " . $mpgResponse->getTransType());
print("\nReferenceNum = " . $mpgResponse->getReferenceNum());
print("\nResponseCode = " . $mpgResponse->getResponseCode());
print("\nISO = " . $mpgResponse->getISO());
print("\nMessage = " . $mpgResponse->getMessage());
print("\nIsVisaDebit = " . $mpgResponse->getIsVisaDebit());
print("\nAuthCode = " . $mpgResponse->getAuthCode());
print("\nComplete = " . $mpgResponse->getComplete());
print("\nTransDate = " . $mpgResponse->getTransDate());
print("\nTransTime = " . $mpgResponse->getTransTime());

```


Sample Purchase with Customer Information

```
print("\nTicket = " . $mpgResponse->getTicket());  
print("\nTimedOut = " . $mpgResponse->getTimedOut());  
?>
```


15 Status Check

- 15.1 About Status Check
- 15.2 Using Status Check Response Fields
- 15.3 Sample Purchase with Status Check

15.1 About Status Check

Status Check is a connection object value that allows merchants to verify whether a previously sent transaction was processed successfully.

To submit a Status Check request, resend the original transaction with all the same parameter values, but set the status check value to either `true` or `false`.

Once set to “true”, the gateway will check the status of a transaction that has an `order_id` that matches the one passed.

- If the transaction is found, the gateway will respond with the specifics of that transaction.
- If the transaction is not found, the gateway will respond with a not found message.

Once it is set to “false”, the transaction will process as a new transaction.

For example, if you send a Purchase transaction with Status Check, include the same values as the original Purchase such as the order ID and the amount.

The feature must be enabled in your merchant profile. To have it enabled, contact Moneris.

Things to Consider:

- The Status Check request should only be used once and immediately (within 2 minutes) after the last transaction that had failed.
- The Status Check request should not be used to check `openTotals` & `batchClose` requests.
- Do not resend the Status Check request if it has timed out. Additional investigation is required.

15.2 Using Status Check Response Fields

After you have used the connection object to send a Status Check request, you can use the Receipt object to obtain the information you want regarding the success of the original transaction.

The status response fields related to the status check are Status Code and Status Message.

Possible Status Code response values:

- 0-49: successful transaction
- 50-999: unsuccessful transaction.

Possible Status Message response values:

- Found: Status code is 0-49
- Not found or Null: Status code is 50-999)

If the Status Message is `Found`, all other response fields are the same as those from the original transaction.

If the Status Message is `Not found`, all other response fields will be `Null`.

15.3 Sample Purchase with Status Check

Sample Purchase transaction with Status Check

```
<?php
require "../mpgClasses.php";
$store_id='store5';
$api_token='yesguy';
$status_check = 'true';

$txnArray=array('type'=>'purchase',
    'order_id'=>'order',
    'cust_id'=>'cust',
    'amount'=>'1.00',
    'pan'=>'4242424242424242',
    'expdate'=>'2202',
    'crypt_type'=>'1',
    'dynamic_descriptor'=>'');

$mpgTxn = new mpgTransaction($txnArray);
$mpgRequest = new mpgRequest($mpgTxn);
$mpgRequest->setProcCountryCode("CA");
$mpgRequest->setTestMode(true); //false for production transactions
$mpgHttpPost =new mpgHttpPostStatus($store_id,$api_token,$status_check,$mpgRequest);

$mpgResponse=$mpgHttpPost->getMpgResponse();
print("\nStatusCode = " . $mpgResponse->getStatusCode());
print("\nStatusMessage = " . $mpgResponse->getStatusMessage());
?>
```

16 Testing a Solution

- 16.1 About the Merchant Resource Center
- 16.2 Logging In to the QA Merchant Resource Center
- 16.3 Test Credentials for Merchant Resource Center
- 16.4 Getting a Unique Test Store ID and API Token
- 16.5 Processing a Transaction
- 16.6 Testing MPI Solutions
- 1 Testing Visa Checkout
- 1 Test Cards
- 16.8 Simulator Host

16.1 About the Merchant Resource Center

The Merchant Resource Center is the user interface for Moneris Gateway services. There is also a QA version of the Merchant Resource Center site specifically allocated for you and other developers to use to test your API integrations with the gateway.

You can access the Merchant Resource Center in the test environment at:

<https://esqa.moneris.com/mpg> (Canada)

The test environment is generally available 24/7, but 100% availability is not guaranteed. Also, please be aware that other merchants are using the test environment in the Merchant Resource Center. Therefore, you may see transactions and user IDs that you did not create. As a courtesy to others who are testing, we ask that you use only the transactions/users that you created. This applies to processing Refund transactions, changing passwords or trying other functions.

16.2 Logging In to the QA Merchant Resource Center

To log in to the QA Merchant Resource Center for testing purposes:

1. Go to the Merchant Resource Center QA website at <https://esqa.moneris.com/mpg>
2. Enter your username and password, which are the same email address and password you use to log in to the Developer Portal
3. Enter your Store ID, which you obtained from the Developer Portal's My Testing Credentials as described in 16.3 Test Credentials for Merchant Resource Center

16.3 Test Credentials for Merchant Resource Center

For testing purposes, you can either use the pre-existing test stores in the Merchant Resource Center, or you can create your own unique test store where you will only see your own transactions. If you want to use the pre-existing stores, use the test credentials provided in the following tables with the corresponding lines of code, as in the examples below.

Example of Corresponding Code For Canada:

```
$store_id='monca00392';

$api_token='qYdISUhHiOdfTr1CLNpN';

$mpgRequest->setProcCountryCode("CA");

$mpgRequest->setTestMode(true);
```

Table 17: Test Server Credentials - Canada

store_id	api_token	Username	Password	Other Information
store1	yesguy	demouser	password	
store2	yesguy	demouser	password	
store3	yesguy	demouser	password	
store4	yesguy	demouser	password	
store5	yesguy	demouser	password	
monca00392	yesguy	demouser	password	Use this store to test Convenience Fee transactions
moncaqagt1	mgtokenguy1	demouser	password	Use this store to test Token Sharing
moncaqagt2	mgtokenguy2	demouser	password	Use this store to test Token Sharing
moncaqagt3	mgtokenguy3	demouser	password	Use this store to test Token Sharing

store_id	api_token	Username	Password	Other Information
monca01428	mcmpguy	demouser	password	Use this store to test MasterCard MasterPass

Alternatively, you can create and use a unique test store where you will only see your own transactions. For more on this, see [Getting a Unique Test Store ID and API Token](#) (page 495)

16.4 Getting a Unique Test Store ID and API Token

Transactions requests via the API will require you to have a Store ID and a corresponding API token. For testing purposes, you can either use the pre-existing test stores in the Merchant Resource Center, or you can create your own unique test store where you will only see your own transactions.

To get your unique Store ID and API token:

1. Log in to the Developer Portal at <https://developer.moneris.com>
2. In the My Profile dialog, click the Full Profile  button
3. Under My Testing Credentials, select Request Testing Credentials
4. Enter your Developer Portal password and select your country
5. Record the Store ID and API token that are given, as you will need them for logging in to the Merchant Resource Center (Store ID) and for API requests (API token).

Alternatively, you can use the pre-existing test stores already set up in the Merchant Resource Center as described in [Test Credentials for Merchant Resource Center](#) (page 494).

16.5 Processing a Transaction

- 1.1 Overview
- 1.2 HttpsPostRequest Object
- 1.3 Receipt Object

16.5.1 Overview

There are some common steps for every transaction that is processed.

1. Instantiate the transaction object (e.g., Purchase), and update it with object definitions that refer to the individual transaction.
2. Instantiate the HttpsPostRequest connection object and update it with connection information, host information and the transaction object that you created in step 16.5
Section 16.5 (page 497) provides the HttpsPostRequest connection object definition. This object and its variables apply to **every** transaction request.
3. Invoke the HttpsPostRequest object's `send()` method.
4. Instantiate the Receipt object, by invoking the HttpsPostRequest object's get Receipt method.
Use this object to retrieve the applicable response details.

Some transactions may require steps in addition to the ones listed here. Below is a sample Purchase transaction with each major step outlined. For extensive code samples of other transaction types, refer to the PHP API ZIP file.

NOTE: For illustrative purposes, the order in which lines of code appear below may differ slightly from the same sample code presented elsewhere in this document.

<pre><?php ## ## Example php -q TestPurchase.php store1 ## require "../mpgClasses.php"; \$type='purchase'; \$cust_id='cust id'; \$order_id='ord-' .date("dmy-G:i:s"); \$amount='1.00'; \$pan='4242424242424242'; \$expiry_date='1111'; \$script='7';</pre>	Include all necessary classes.
<pre>\$store_id='store5'; \$api_token='yesguy';</pre>	Define all mandatory values for the transaction object properties.
	Define all mandatory values for the connection object properties.

<pre> \$txnArray=array('type'=>\$type, 'order_id'=>\$order_id, 'cust_id'=>\$cust_id, 'amount'=>\$amount, 'pan'=>\$pan, 'expdate'=>\$expiry_date, 'crypt_type'=>\$crypt, 'dynamic_descriptor'=>\$dynamic_descriptor); \$mpgTxn = new mpgTransaction(\$txnArray); \$mpgRequest = new mpgRequest(\$mpgTxn); \$mpgRequest->setProcCountryCode("CA"); //"US" for sending transaction to US environment \$mpgRequest->setTestMode(true); //false for production transactions </pre>	Instantiate the transaction object and assign values to properties.
<pre> /* Status Check Example \$mpgHttpPost =new mpgHttpsPostStatus(\$store_id,\$api_token,\$status_ check,\$mpgRequest); */ \$mpgHttpPost =new mpgHttpsPost(\$store_id,\$api_token,\$mpgRequest); </pre>	Instantiate connection object and assign values to properties, including the transaction object you just created.
<pre> \$mpgResponse=\$mpgHttpPost->getMpgResponse(); print("\nCardType = " . \$mpgResponse->getCardType()); print("\nTransAmount = " . \$mpgResponse->getTransAmount()); print("\nTxnNumber = " . \$mpgResponse->getTxnNumber()); print("\nReceiptId = " . \$mpgResponse->getReceiptId()); print("\nTransType = " . \$mpgResponse->getTransType()); print("\nReferenceNum = " . \$mpgResponse->getReferenceNum()); print("\nResponseCode = " . \$mpgResponse->getResponseCode()); print("\nISO = " . \$mpgResponse->getISO()); print("\nMessage = " . \$mpgResponse->getMessage()); print("\nIsVisaDebit = " . \$mpgResponse->getIsVisaDebit()); print("\nAuthCode = " . \$mpgResponse->getAuthCode()); print("\nComplete = " . \$mpgResponse->getComplete()); print("\nTransDate = " . \$mpgResponse->getTransDate()); print("\nTransTime = " . \$mpgResponse->getTransTime()); print("\nTicket = " . \$mpgResponse->getTicket()); print("\nTimedOut = " . \$mpgResponse->getTimedOut()); print("\nStatusCode = " . \$mpgResponse->getStatusCode()); print("\nStatusMessage = " . \$mpgResponse->getStatusMessage()); ?> </pre>	Instantiate the Receipt object and use its get methods to retrieve the desired response data.

16.5.2 HttpsPostRequest Object

The transaction object that you instantiate becomes a property of this object when you call its set transaction method.

HttpsPostRequest Object Definition

```
HttpsPostRequest mpgReq = new HttpsPostRequest();
```

After instantiating the HttpsPostRequest object, update its mandatory and optional values as outlined in the following values tables.

Table 18: HttpsPostRequest object mandatory values

Value	Type	Limits	Set method
	Description		
Processing country code	String	2-character alphabetic	<code>\$mpgRequest->setProcCountryCode("CA");</code>
	CA for Canada, US for USA.		
Test mode	Boolean	true/false	<code>\$mpgRequest->setTestMode(true);</code>
	Set to <code>true</code> when in test mode. Set to <code>false</code> (or comment out entire line) when in production mode.		
Store ID	String	10-character alphanumeric	<code>\$mpgHttpPost = new mpgHttpPostStatus(\$store_id,\$api_token,\$status_check,\$mpgRequest);</code>
	Unique identifier provided by Moneris upon merchant account set up. See 16.1 About the Merchant Resource Center for test environment details.		
API Token	String	20-character alphanumeric	<code>\$mpgHttpPost = new mpgHttpPostStatus(\$store_id,\$api_token,\$status_check,\$mpgRequest);</code>
	Unique alphanumeric string assigned upon merchant account activation. To locate your production API token, refer to the Merchant Resource Center Admin Store Settings. See 16.3 Test Credentials for Merchant Resource Center for test environment details.		
Transaction	Object	Not applicable	<code>\$mpgRequest = new mpgRequest(\$mpgTxn);</code>
	This argument is one of the numerous transaction types discussed in the rest of this manual. (Such as Purchase, Refund and so on.) This object is instantiated in step 1 above.		

Table 1 HttpsPostRequest object optional values

Value	Type		Limits	Set method
	Description			
Status Check	Boolean	true/false	<code>\$mpgHttpPost = new mpgHttpPostStatus(\$store_id,\$api_token,\$status_check,\$mpgRequest);</code>	
	See Appendix A Definition of Request Fields.			
	NOTE: while this value belongs to the HttpsPostRequest object, it is only supported by some transactions. Check the individual transaction definition to find out whether Status Check can be used.			

16.5.3 Receipt Object

After you send a transaction using the HttpsPostRequest object's send method, you can instantiate a receipt object.

Receipt Object Definition

```
$mpgResponse=$mpgHttpPost->getMpgResponse();
```

For an in-depth explanation of Receipt object methods and properties, see Appendix B Definitions of Response Fields.

16.6 Testing MPI Solutions

When testing your implementation of the Moneris MPI, you can use the Visa/MasterCard/Amex PIT (production integration testing) environment. The testing process is slightly different than a production environment in that when the inline window is generated, it does not contain any input boxes. Instead, it contains a window of data and a **Submit** button. Clicking **Submit** loads the response in the testing window. The response will not be displayed in production.

NOTE: MasterCard SecureCode and Amex SafeKey may not be directly tested within our current test environment. However, the process and behavior tested with the Visa test cards will be the same for MCSC and SafeKey.

When testing you may use the following test card numbers with any future expiry date. Use the appropriate test card information from the tables below: Visa and MasterCard use the same test card information, while Amex uses unique information.

Table 19: MPI test card numbers (Visa and MasterCard only)

Card Number	VERes	PAREs	Action
4012001037141112 4242424242424242	Y	true	TXN – Call function to create inLine window. ACS – Send CAVV to Moneris Gateway using either the Cavv Purchase or the Cavv Pre-Authorization transaction.
4012001038488884	U	NA	Send transaction to Moneris Gateway using either the basic Purchase or the basic Pre-Authorization transaction. Set crypt_type = 7.
4012001038443335	N	NA	Send transaction to Moneris Gateway using either the basic Purchase or the basic Pre-Authorization transaction. Set crypt_type = 6.
4012001037461114	Y	false	Card failed to authenticate. Merchant may chose to send transaction or decline transaction. If transaction is sent, use crypt type = 7.

Table 20: MPI test card numbers (Amex only)

Card Number	Password			Action
	VERes	Required?	PAREs	
375987000000062	U	Not required	N/A	TXN – Call function to create inLine window. ACS – Send CAVV to Moneris Gateway using either the Cavv Purchase or the Cavv Pre-Authorization transaction. Set crypt_type = 7.
375987000000021	Y	Yes: test13fail	false	Card failed to authenticate. Merchant may chose to send transaction or decline transaction. If transaction is sent, use crypt type = 7.
375987000000013	N	Not required	N/A	Send transaction to Moneris Gateway using either the basic Purchase or the basic Pre-Authorization transaction. Set crypt_type = 6.
374500261001009	Y	Yes: test09	true	Card failed to authenticate. Merchant may choose to send transaction or decline transaction. Set crypt_type = 5.

VERes

The result U, Y or N is obtained by using getMessage().

PAREs

The result “true” or “false” is obtained by using getSuccess().

To access the Merchant Resource Center in the test environment go to <https://esqa.moneris.com/mpg>.

Transactions in the test environment should not exceed \$11.00.

16.7 Test Card Numbers

Because of security and compliance reasons, the use of live credit and debit card numbers for testing is strictly prohibited. Only test credit and debit card numbers are to be used.

To test general transactions, use the following test card numbers:

Card Plan	Test Card Number
Mastercard	5454545454545454
Visa	4242424242424242
Amex	373599005095005
JCB	3566007770015365
Diners	36462462742008
Track2	5258968987035454=06061015454001060101?
Discover	6011000992927602
UnionPay	6250944000000771

16.7.1 Test Card Numbers for Level 2/3

When testing Level 2/3 transactions, use the card numbers below.

Card Brand	Test Card Number
Mastercard	5454545442424242
Visa	4242424254545454
Amex	373269005095005

16.7.2 Test Cards for Visa Checkout

Card Plan	Test Card Number
Visa	4005520201264821 (without card art)
Visa	4242424242424242 (with card art)
MasterCard	5500005555555559
American Express	340353278080900
Discover	6011003179988686

16.8 Simulator Host

The test environment has been designed to replicate the production environment as closely as possible. One major difference is that Moneris is unable to send test transactions onto the production authorization network. Therefore, issuer responses are simulated. Additionally, the requirement to emulate approval, decline and error situations dictates that certain transaction variables initiate various response and error situations.

The test environment approves and declines transactions based on the penny value of the amount sent. For example, a transaction made for the amount of \$9.00 or \$1.00 is approved because of the .00 penny value.

Transactions in the test environment must not exceed \$11.00.

For a list of all current test environment responses for various penny values, please see the Test Environment Penny Response Table available at <https://developer.moneris.com>.

NOTE: These responses may change without notice. Check the Moneris Developer Portal (<https://developer.moneris.com>) regularly to access the latest documentation and downloads.

17 Moving to Production

- 17.1 Activating a Production Store Account
- 17.2 Configuring a Store for Production
- 17.3 Receipt Requirements
- 1 Getting Help

17.1 Activating a Production Store Account

The steps below outline how to activate your production account so that you can process production transactions.

1. Obtain your activation letter/fax from Moneris.
2. Go to <https://www.moneris.com/activate>.
3. Input your store ID and merchant ID from the letter/fax and click **Activate**.
4. Follow the on-screen instructions to create an administrator account. This account will grant you access to the Merchant Resource Center.
5. Log into the Merchant Resource Center at <https://www3.moneris.com/mpg> using the user credentials created in step 17.1.
6. Proceed to **ADMIN** and then **STORE SETTINGS**.
7. Locate the API token at the top of the page. You will use this API token along with the store ID that you received in your letter/fax and to send any production transactions through the API.

When your production store is activated, you need to configure your store so that it points to the production host. To learn how do to this, see Configuring a Store for Production (page 505)

NOTE: For more information about how to use the Merchant Resource Center, see the Moneris Gateway Merchant Resource Center User's Guide, which is available at <https://developer.moneris.com>.

17.2 Configuring a Store for Production

After you have completed your testing and have activated your production store, you are ready to point your store to the production host.

To configure a store for production:

1. Change the test mode set method from `true` to `false`.
2. Change the Store ID to reflect the production store ID that you received when you activated your production store. To review the steps for activating a production store, see Activating a Production Store Account (page 505).

3. Change the API token to the production token that you received during activation.
4. If you haven't done so already, change the code to reflect the correct processing country (Canada for most merchants). For more on this, see

The table below illustrates the steps above using the relevant code (and where **x** is an alphanumeric character).

Step	Code in Testing	Changes for Production
1	No string changes for this item, only set method is altered: <pre>\$mpgRequest->setTestMode(true);</pre>	Set method for production: <pre>\$mpgRequest->setTestMode(false);</pre>
2	String: <pre>\$store_id='store5';</pre> Associated Set Method: <pre>'store_id'=>\$store_id</pre>	String for Production: <pre>\$store_id='monXXXXXXXX';</pre>
3	String: <pre>\$api_token='yesguy';</pre> Associated Set Method: <pre>'api_token'=>\$api_token</pre>	String for Production: <pre>\$api_token='xxxx';</pre>

17.3 Receipt Requirements

Visa and MasterCard expect certain details to be provided to the cardholder and on the receipt when a transaction is approved.

Receipts must comply with the standards outlined within the Integration Receipts Requirements. For all the receipt requirements covering all transaction scenarios, visit the Moneris Developer Portal at <https://developer.moneris.com>.

Production of the receipt must begin when the appropriate response to the transaction request is received by the application. The transaction may be any of the following:

- **Sale** (Purchase)
- **Authorization** (PreAuth, Pre-Authorization)

- **Authorization Completion** (Completion, Capture)
- **Offline Sale** (Force Post)
- **Sale Void** (Purchase Correction, Void)
- **Refund**.

The boldface terms listed above are the names for transactions as they are to be displayed on receipts. Other terms used for the transaction are indicated in brackets.

17.3.1 Certification Requirements

Card-present transaction receipts are required to complete certification.

Card-not-present integration

Certification is optional but highly recommended.

Card-present integration

After you have completed the development and testing, your application must undergo a certification process where all the applicable transaction types must be demonstrated, and the corresponding receipts properly generated.

Contact a Client Integration Specialist for the Certification Test checklist that must be completed and returned for verification. (See "Getting Help" on page 1 for contact details.) Be sure to include the application version of your product. Any further changes to the product after certification requires re-certification.

After the certification requirements are met, Moneris will provide you with an official certification letter.

Appendix A Definition of Request Fields

This section defines transaction request variables. Not all fields are required — refer to individual transaction type topics for information on whether a field is required or optional.

- A.1 Definition of Request Fields – Connection Fields
- A.2 Definition of Request Fields – Core Fields
- A.3 Definition of Request Fields – Credential on File
- A.5 Definition of Request Fields – Vault
- A.6 Definition of Request Fields for Level 2/3 - Visa
- A.7 Definition of Request Fields for Level 2/3 - Mastercard
- A.8 Definition of Request Fields for Level 2/3 - Amex
- Appendix A Definition of Request Fields – MPI
- A.10 Definition of Request Fields – MCP
- A.11 Definition of Request Fields – Offlinx™
- A.12 Definition of Request Fields – Convenience Fee
- A.13 Definition of Request Fields – Recurring

A.1 Definition of Request Fields – Connection Fields

Core connection object fields (all API transactions)

Variable Name	Type and Limits	Description
store ID	String	Unique identifier provided by Moneris upon merchant account setup
store_id	N/A	
API token	String	Unique alphanumeric string assigned by Moneris upon merchant account activation
api_token	N/A	
To find your API token, refer to your test or production store's Admin settings in the Merchant Resource Center, at the following URLs:		

Variable Name	Type and Limits	Description
		Testing: https://esqa.moneris.com/mpg/ Production: https://www3.moneris.com/mpg/

A.2 Definition of Request Fields – Core Fields

Variable Name	Type and Limits	Description
amount	<i>String</i>	Transaction dollar amount
amount	10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	This must contain at least 3 digits, two of which are penny values Minimum allowable value = \$0.01, maximum allowable value = \$9999999.99
authorization code	<i>String</i>	An authorization code required to carry out a Force Post; provided in the transaction response from the issuing bank
auth_code	8-character alphanumeric	
completion amount	<i>String</i>	Dollar amount of a Pre-Authorization Completion transaction, which may differ from the original amount authorized in the Pre-Authorization
comp_amount	10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	
credit card number	<i>String</i>	Credit card number, usually 16 digits —field can be maximum 20 digits in support of future expansion of card number ranges.
pan	max 20-character alphanumeric	

Variable Name	Type and Limits	Description
dynamic descriptor dynamic_descriptor	<i>String</i> 20-character alphanumeric total of 22 characters including your merchant name and separator NOTE: Some special characters are not allowed: < > \$ % = ? ^ { } [] \	Carries the token for network tokenization transactions. Merchant-defined description sent on a per-transaction basis that will appear on the credit card statement appended to the merchant's business name Dependent on the card issuer, the statement will typically show the dynamic descriptor appended to the merchant's existing business name separated by the "/" character; additional characters will be truncated NOTE: The 22-character maximum limit must take the "/" into account as one of the characters
electronic commerce indicator crypt_type	<i>String</i> 1-character alphanumeric	Describes the category of e-commerce transaction being processed. Allowable values are: 1 – Mail Order / Telephone Order—Single 2 – Mail Order / Telephone Order—Recurring 3 – Mail Order / Telephone Order—Instalment 4 – Mail Order / Telephone Order—Unknown classification 5 – Authenticated e-commerce transaction (3-D Secure) 6 – Non-authenticated e-commerce transaction (3-D Secure) 7 – SSL-enabled merchant In Credential on File transactions where the request field e-commerce indicator is also being sent: the allowable values for e-commerce indicator are dependent on the value sent for payment indicator , as follows:

Variable Name	Type and Limits	Description
		if payment indicator = R, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = V, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = C, then allowable values for e-commerce indicator: 1, 5, 6 or 7 if payment indicator = U, then allowable values for e-commerce indicator: 1 or 7 if payment indicator = Z, then allowable values for e-commerce indicator: 1, 5, 6 or 7 For Apple Pay or Google Pay™ transactions where you are doing decryption: send the value of the eciIndicator or 3dsEciIndicator field returned in the payload If the value is not present in the payload, send the value as 5; if you get a 2-character value (e.g., 05 or 07), remove the initial 0 and just send us the 2nd character Allowable values for Apple Pay and Google Pay™ are: 5: Authenticated e-commerce transaction 7: SSL-enabled merchant
expiry date expdate	String 4-character alphanumeric YYMM	Expiry date of the credit card, in YYMM format. NOTE: This is the reverse of the MMYM date format that is presented on the card.
is_incremental	Boolean	'is_incremental'=>\$is_incremental
is_incremental	true/false	Indicates if this preauthorization is using an estimated amount. Estimations allow for incrementing the amount held via subsequent incre-

Variable Name	Type and Limits	Description
		mentalAuth requests. Defaults to false. NOTE: Please note that if this field is true, the preauthorization is only eligible for a single Preauthorization Completion. Any completion sent for partial completion is treated as a full completion (ship_indicator= P is treated as = F when is_incremental= true on the original preauth)
foreign indicator	<i>Boolean</i> true or false	'foreign_indicator'=>\$foreign_indicator Used to identify domestic transactions processed by a marketplace merchant that is in a different country.
order ID	<i>String</i>	Merchant-defined transaction identifier that must be unique for every Purchase, Pre-Authorization and Independent Refund transaction. No two transactions of these types may have the same order ID. For Refund, Completion and Purchase Correction transactions, the order ID must be the same as that of the original transaction.
order_id	50-character alphanumeric a-Z A-Z 0-9 _ - : . @ spaces	
original order ID	<i>String</i>	Order ID from the original Pre-Authorization transaction, used as a reference to retrieve the original payment details
orig_order_id		
shipping indicator	<i>String</i>	Used to identify completion transactions that require multiple shipments, also referred to as multiple completions By default, if shipping indicator is not sent, the Pre-Authorization Completion is listed as final
ship_indicator	1-character alphanumeric	

Variable Name	Type and Limits	Description
		To indicate that the Pre-Authorization Completion is to be left open by the issuer as supplemental shipments or completions are pending, submit shipping indicator with a value of P Possible values: P – Partial F – Final
transaction number txn_number	<i>String</i> 255-character, alpha-numeric, hyphens or underscores variable length	Used to reference the original transaction when performing a follow-on transaction (i.e., Pre-Authorization Completion, Purchase Correction or Refund) This value is returned in the response of the original transaction Pre-Authorization Completion: references a Pre-Authorization Refund/Purchase Correction: references a Purchase or Pre-Authorization Completion
wallet indicator wallet_indicator	<i>String</i> 3-character alphanumeric	Indicates when a card number has been collected via a digital wallet, such as in Apple Pay, Google Pay™, Visa Checkout and Mastercard MasterPass, or via network tokenization from the card brand. Required for Apple Pay, Google Pay™ transactions whereby you are using your own API to decrypt the payload Possible values: APP – Apple Pay In-App APW – Apple Pay on the Web

Variable Name	Type and Limits	Description
		GPP – Google Pay™ In-App
		GPW – Google Pay™ Web
		VCO – Visa Checkout
		MMP – Mastercard MasterPass
		NOTE: Please note that if this field is included to indicate Apple Pay or Google Pay™, then Convenience Fee is not supported. NOTE: Network tokenization wallet indicators are not in the API call but are in the merchant resource centre (MRC).

A.3 Definition of Request Fields – Credential on File

Variable Name	Type and Limits	Description
issuer ID NOTE: This variable is required for all merchant-initiated transactions following the first one; upon sending the first transaction, the issuer ID value is received in the transaction response and then used in subsequent transaction requests.	<i>String</i> 15-character alphanumeric variable length	Unique identifier for the cardholder's stored credentials Sent back in the response from the card brand when processing a Credential on File transaction If the cardholder's credentials are being stored for the first time, and the issuer ID was returned in the response, you must save the issuer ID on your system to use in subsequent Credential on File transactions (applies to merchant-initiated transactions only) The issuer ID must be saved to your systems when returned from Moneris Gateway in the response data, regardless if the value was received or not

Variable Name	Type and Limits	Description
		As a best practice, if the issuer ID is not returned and you received a value of NULL instead, store that value and send it in the subsequent transaction
payment indicator	String 1-character alphabetic	Indicates the current or intended use of the credentials Possible values for first transactions: C - unscheduled Credential on File (first transactions only) R - recurring V - recurring variable payment transaction Possible values for subsequent transactions: R - recurring V - recurring variable payment transaction U - unscheduled merchant-initiated transaction Z - unscheduled customer-initiated transaction In Credential on File transactions where the request field e-commerce indicator is also being sent, the acceptable values for e-commerce indicator are dependent on the value sent for payment indicator, as follows: if payment indicator = R, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = V, then allowable values for e-commerce indicator: 2, 5 or 6 if payment indicator = C, then allowable values for e-commerce indicator: 1, 5, 6 or 7 if payment indicator = U, then allowable values for e-commerce indicator: 1 or 7 if payment indicator = Z, then allowable values for e-commerce indicator: 1, 5, 6 or 7
payment information	String	Describes whether the transaction is

Variable Name	Type and Limits	Description
	1-character numeric	the first or subsequent in the series Possible values: 0 - first transaction in a series (storing payment details provided by the cardholder) 2 - subsequent transactions (using previously stored payment details)

A.4 Definition of Request Fields – GooglePay Token Temp Add

Variable Name	Type and Limits	Description
payment token	<i>Object</i> <i>N/A</i>	Payment details returned by Google in their <code>PaymentData</code> object for GooglePay transactions. See Definition of Request Fields – GooglePay Token Temp Add below for field details.
signature	<i>String</i>	Verifies that the message came from Google. It's base64-encoded, and created with ECDSA by the intermediate signing key. Returned by Google in their <code>PaymentData</code> object for GooglePay transactions
protocol version	<i>String</i>	Identifies the encryption or signing scheme under which the message is created. It allows the protocol to evolve over time, if needed. Returned by Google in their <code>PaymentData</code> object for GooglePay transactions
signed message	<i>String</i>	A JSON object serialized as an HTML-safe string that contains the encryptedMessage, ephemeralPublicKey, and tag. It's serialized to simplify the signature verification process. Returned by Google in their <code>PaymentData</code> object for GooglePay transactions

A.5 Definition of Request Fields – Vault

Variable Name	Type and Limits	Description
data key	<i>String</i> 25-character alphanumeric	Unique identifier for a Vault profile, and used in future Vault financial transactions to associate a transaction with that profile Data key is generated by Moneris and returned to you in the Receipt object when the profile is first registered
data key format	<i>String</i> 2-character alphanumeric	Specifies the data key format being returned If left blank, data key format will default to 25-character alphanumeric Possible values: 0 – 25 character alphanumeric data key 0U – unique 25-character alphanumeric data key
duration	<i>String</i> 3-character numeric maximum 900 seconds	Amount of time the temporary token should be available
email address	<i>String</i> 30-character alphanumeric	Customer's email address Can be sent in when creating or updating a Vault profile
note	<i>String</i> 30-character alphanumeric	Used for any supplementary information related to the customer Can be sent in when creating or updating a Vault profile
phone number	<i>String</i> 30-character alphanumeric	Customer's phone number Can be sent in when creating or updating a Vault profile
return issuer ID	<i>Boolean</i>	When true, Gateway returns the bank

Variable Name	Type and Limits	Description
	true/false	Issuer ID. Defaults to False.

A.6 Definition of Request Fields for Level 2/3 - Visa

Table 1 Visa - Corporate Card Common Data - Level 2 Request Fields

Req*	Field Name	Limits	Set Method	Description
Y	National Tax	12-character decimal	'national_tax'=>\$national_tax	Must reflect the amount of National Tax (GST or HST) appearing on the invoice. Minimum - 0.01 Maximum - 999999.99. Must have 2 decimal places.
Y	Merchant VAT Registration/Single Business Reference Number	20-character alphanumeric	'merchant_vat_no'=>\$merchant_vat_no	Merchant's Tax Registration Number must be provided if tax is included on the invoice NOTE: Must not be all spaces or all zeroes
C	Local Tax	12-character decimal	'local_tax'=>\$local_tax	Must reflect the amount of Local Tax (PST)

Req*	Field Name	Limits	Set Method	Description
				or QST) appearing on the invoice If Local Tax included then must not be all spaces or all zeroes; Must be provided if Local Tax (PST or QST) applies Minimum = 0.01 Maximum = 999999.99 Must have 2 decimal places
C	Local Tax (PST or QST) Registration Number	15-character alphanumeric	'local_tax_no'=>\$local_tax_no	Merchant's Local Tax (PST/QST) Registration Number Must be provided if tax is included on the invoice; If Local Tax included then must not be all spaces or all zeroes Must be provided if Local Tax (PST or QST) applies

Req*	Field Name	Limits	Set Method	Description
C	Customer VAT Registration Number	13-character alphanumeric	'customer_vat_no'=>\$customer_vat_no	If the Customer's Tax Registration Number appears on the invoice to support tax exempt transactions it must be provided here
C	Customer Code/Customer Reference Identifier (CRI)	16-character alphanumeric	'cri'=>\$cri	Value which the customer may choose to provide to the supplier at the point of sale – must be provided if given by the customer
N	Customer Code	17-character alphanumeric	'customer_code'=>\$customer_code	Optional customer code field that will not be passed along to Visa, but will be included on Moneris reporting
N	Invoice Number	17-character alphanumeric	'invoice_number'=>\$invoice_number	Optional invoice number field that will not be passed along to Visa, but will be included on Moneris reporting

*Y = Required, N = Optional, C = Conditional

Table 2 Visa - Corporate Card Common Data- Level 2 Request Fields (VSPurcha)

Req	Variable Name	Field Name	Size/Type	Description
C*	Buyer Name	buyer_name	30-character alpha-numeric	Buyer/Receipient Name *only required by CRA if transaction is >\$150
C*	Local tax rate	local_tax_rate	4-character decimal	Indicates the detailed tax rate applied in relationship to a local tax amount EXAMPLE: 8% PST should be 8.0. maximum 99.99 *Must be provided if Local Tax (PST or QST) applies.
N	Duty Amount	duty_amount	9-character decimal	Duty on total purchase amount A minus sign means 'amount is a credit', plus sign or no sign means 'amount is a debit' maximum without sign is 999999.99
N	Invoice Discount Treatment	discount_treatment	1-character numeric	Indicates how the merchant is managing discounts Must be one of the following values: 0 - if no invoice level discounts apply for this invoice

Req	Variable Name	Field Name	Size/Type	Description
				1 - if Tax was calculated on Post-Discout totals 2 - if Tax was calculated on Pre-Discout totals
N	Invoice Level Discount Amount	discount_amt	9-character decimal	Amount of discount (if provided at the invoice level according to the Invoice Discount Treatment) Must be non-zero if Invoice Discount Treatment is 1 or 2 Minimum amount is 0.00 and maximum is 999999.99
C*	Ship To Postal Code / Zip Code	ship_to_pos_code	10-character alphanumeric	The postal code or zip code for the destination where goods will be delivered *Required if shipment is involved Full alpha postal code - Valid ANA<space>NAN format required if shipping to an address within Canada
C	Ship From Postal Code / Zip Code	ship_from_pos_code	10-character alphanumeric	The postal code or zip code from which items were shipped For Canadian addresses, requires full alpha postal code for the merchant with Valid

Req	Variable Name	Field Name	Size/Type	Description
				ANA<space>NAN format
C*	Destination Country Code	des_cou_code	2-character alpha-numeric	Code of country where purchased goods will be delivered Use ISO 3166-1 alpha-2 format NOTE: Required if it appears on the invoice for an international transaction
Y	Unique VAT Invoice Reference Number	vat_ref_num	25-character alpha-numeric	Unique Value Added Tax Invoice Reference Number Must be populated with the invoice number and this cannot be all spaces or zeroes
Y	Tax Treatment	tax_treatment	1-character numeric	Must be one of the following values: 0 = Net Prices with tax calculated at line item level; 1 = Net Prices with tax calculated at invoice level; 2 = Gross prices given with tax information provided at line item level; 3 = Gross prices given with tax information provided at invoice level; 4 = No tax applies (small merchant) on the invoice for the transaction

Req	Variable Name	Field Name	Size/Type	Description
N	Freight/Shipping Amount (Ship Amount)	freight_amount	9-character decimal	Freight charges on total purchase If shipping is not provided as a line item it must be provided here, if applicable Signed monetary amount: minus sign means 'amount is a credit', plus sign or no sign means 'amount is a debit', maximum without sign is 999999.99
C	GST HST Freight Rate	gst_hst_freight_rate	4-character decimal	Rate of GST (excludes PST) or HST charged on the shipping amount (in accordance with the Tax Treatment) If Freight/Shipping Amount is provided then this (National GST or HST) tax rate must be provided. Monetary amount, maximum is 99.99. Such as 13% HST is 13.00
C	GST HST Freight Amount	gst_hst_freight_amount	9-character decimal	Amount of GST (excludes PST) or HST charged on the shipping amount If Freight/Shipping Amount is provided then this (National

Req	Variable Name	Field Name	Size/Type	Description
				GST or HST) tax amount must be provided if taxTreatment is 0 or 2 Signed monetary amount: maximum without sign is 999999.99.

Table 3 Visa - Line Item Details - Level 3 Request Fields (VSPurchI)

Req	Variable Name	Field Name	Size/Type	Description
C	Item Commodity Code	item_com_code	12-character alphanumeric	Line item Commodity Code (if this field is not sent, then productCode must be sent)
Y	Product Code	product_code	12-character alphanumeric	Product code for this line item – merchant’s product code, manufacturer’s product code or buyer’s product code Typically this will be the SKU or identifier by which the merchant tracks and prices the item or service This should always be provided for every line item

Req	Variable Name	Field Name	Size/Type	Description
Y	Item Description	item_description	35-character alpha-numeric	Line item description
Y	Item Quantity	item_quantity	12-character decimal	Quantity invoiced for this line item Up to 4 decimal places supported, whole numbers are accepted Minimum = 0.0001 Maximum = 999999999999
Y	Item Unit of Measure	item_uom	2-character alpha-numeric	Unit of Measure Use ANSI X-12 EDI Allowable Units of Measure and Codes
Y	Item Unit Cost	unit_cost	12-character decimal	Line item cost per unit 2-4 decimal places accepted Minimum = 0.0001 Maximum = 999999.9999
N	VAT Tax Amount	vat_tax_amt	12-character decimal	Any value-added tax or other sales tax amount Must have 2 decimal places Minimum = 0.01 Maximum =

Req	Variable Name	Field Name	Size/Type	Description
				999999.99
N	VAT Tax Rate	vat_tax_rate	4-character decimal	Sales tax rate EXAMPLE: 8% PST should be 8.0 maximum 99.99
Y	Discount Treatment	discount_treatmentL	1-character numeric	Must be one of the following values: 0 if no invoice level discounts apply for this invoice 1 if Tax was calculated on Post-Discout totals 2 if Tax was calculated on Pre-Discout totals.
C	Discount Amount	discount_amtL	12-character decimal	Amount of discount, if provided for this line item according to the Line Item Discount Treatment Must be non-zero if Line Item Discount Treatment is 1 or 2 Must have 2 decimal places Minimum = 0.01 Maximum = 999999.99

A.7 Definition of Request Fields for Level 2/3 - Mastercard

Table 1 Objects - Level 2/3 MasterCard

MCCorpais Objects	Description
MCCorpac	Corporate Card Common data
MCCorpal	Line Item Details

Table 2 MasterCard - Corporate Card Common Data (MCCorpac) - Level 2 Request Fields

Req	Variable Name	Field Name	Size/Type	Description
N	AustinTetraNumber	Austin-Tetra Number	15-character alphanumeric	Merchant's Austin-Tetra Number
N	NaicsCode	NAICS Code	15-character alphanumeric	North American Industry Classification System (NAICS) code assigned to the merchant
N	CustomerCode	Customer Code	25-character alphanumeric	A control number, such as purchase order number, project number, department allocation number or name that the purchaser supplied the merchant. Left-justified; may be spaces
N	UniqueInvoiceNumber	Unique Invoice Number	17-character alphanumeric	Unique number associated with the individual transaction provided by the merchant
N	CommodityCode	Commodity Code	15-character alphanumeric	Code assigned by the merchant that best categorizes the item(s) being purchased
N	OrderDate	Order Date	6-character numeric	The date the item was ordered. If present, must contain a valid date in the format YYMMDD.
N	CorporationVatNumber	Corporation VAT Number	20-character alphanumeric	Contains a corporation's value added tax (VAT) number

Req	Variable Name	Field Name	Size/Type	Description
N	CustomerVatNumber	Customer VAT Number	20-character alphanumeric	Contains the VAT number for the customer/cardholder used to identify the customer when purchasing goods and services from the merchant
N	FreightAmount	Freight Amount	12-character decimal	The freight on the total purchase. Must have 2 decimals
N	DutyAmount	Duty Amount	12-character decimal	The duty on the total purchase, Must have 2 decimals
N	DestinationProvinceCode	Destination State / Province Code	3-character alphanumeric	State or Province of the country where the goods will be delivered. Left justified with trailing spaces. e.g., ONT - Ontario
N	DestinationCountryCode	Destination Country Code	3-character alphanumeric	The country code where goods will be delivered. Left justified with trailing spaces. e.g., CAN - Canada
N	ShipFromPosCode	Ship From Postal Code	10-character alphanumeric	The postal code or zip code from which items were shipped
N	ShipToPosCode	Destination Postal Code	10-character alphanumeric	The postal code or zip code where goods will be delivered
N	AuthorizedContactName	Authorized Contact Name	36-character alphanumeric	Name of an individual or company contacted for company authorized purchases
N	AuthorizedContactPhone	Authorized Contact Phone	17-character alphanumeric	Phone number of an individual or company contacted for company authorized purchases

Req	Variable Name	Field Name	Size/Type	Description
N	AdditionalCardAcceptordata	Additional Card Acceptor Data	40-character alphanumeric	Information pertaining to the card acceptor
N	CardAcceptorType	Card Acceptor Type	8-character alphanumeric	Various classifications of business ownership characteristics This field takes 8 characters. Each character represents a different component, as follows: 1st character represents 'Business Type' and contains a code to identify the specific classification or type of business: - Corporation - Not known - Individual/Sole Proprietorship - Partnership - Association/Estate/Trust - Tax Exempt Organizations (501C) - International Organization - Limited Liability Company (LLC) - Government Agency 2nd character represents 'Business Owner Type'. Contains a code to identify specific characteristics about the business owner. 1 - No application classification

Req	Variable Name	Field Name	Size/Type	Description
				2 - Female business owner 3 - Physically handicapped female business owner 4 - Physically handicapped male business owner 0 - Unknown 3rd character represents 'Business Certification Type'. Contains a code to identify specific characteristics about the business certification type, such as small business, disadvantaged, or other certification type: 1 - Not certified 2 - Small Business Administration (SBA) certification small business 3 - SBA certification as small disadvantaged business 4 - Other government or agency-recognized certification (such as Minority Supplier Development Council) 5 - Self-certified small business 6 - SBA certification as small and other government or agency-recognized certification 7 - SBA certification as small disadvantaged business and other government or agency-recognized certification

Req	Variable Name	Field Name	Size/Type	Description
				8 - Other government or agency-recognized certification and self-certified small business A - SBA certification as 8(a) B - Self-certified small disadvantaged business (SDB) C - SBA certification as HUBZone 0 - Unknown 4th character represents 'Business Racial/Ethnic Type'. Contains a code identifying the racial or ethnic type of the majority owner of the business. 1 - African American 2 - Asian Pacific American 3 - Subcontinent Asian American 4 - Hispanic American 5 - Native American Indian 6 - Native Hawaiian 7 - Native Alaskan 8 - Caucasian 9 - Other 0 - Unknown 5th character represents 'Business Type Provided Code' Y - Business type is provided.

Req	Variable Name	Field Name	Size/Type	Description
				N - Business type was not provided. R - Card acceptor refused to provide business type 6th character represents 'Business Owner Type Provided Code' Y - Business owner type is provided. N - Business owner type was not provided. R - Card acceptor refused to provide business type 7th character represents 'Business Certification Type Provided Code' Y - Business certification type is provided. N - Business certification type was not provided. R - Card acceptor refused to provide business type 8th character represents 'Business Racial/Ethnic Type' Y - Business racial/ethnic type is provided. N - Business racial/ethnic type was not provided. R - Card acceptor refused to provide business type

Req	Variable Name	Field Name	Size/Type	Description
				ness racial/ethnic type
N	CardAcceptorTaxId	Card Acceptor Tax ID	20-character alphanumeric	US Federal tax ID number for value added tax (VAT) ID.
N	CardAcceptorReferenceNumber	Card Acceptor Reference Number	25-character alphanumeric	Code that facilitates card acceptor/corporation communication and record keeping
N	CardAcceptorVatNumber	Card Acceptor VAT Number	20-character alphanumeric	Value added tax (VAT) number for the card acceptor location used to identify the card acceptor when collecting and reporting taxes
C*	Tax	Tax	up to 6 arrays	Can have up to 6 arrays contains different tax details. See Tax Array below for each field description. *This field is conditionally mandatory — if you use this array, you must fill in all tax array fields as listed in the Tax Array Request Fields below.

Table 3 MasterCard - Line Item Details (MCCorpal) - Level 3 Request Fields

Req	Variable Name	Field Name	Size/Type	Description
N	CustomerCode	Customer Code	25-character alphanumeric	A control number, such as purchase order number, project number, department allocation number or name that

Req	Variable Name	Field Name	Size/Type	Description
				the purchaser supplied the merchant. Left-justified; may be spaces
N	LineItemDate	Line Item Date	6-character numeric	The purchase date of the line item referenced in the associated Corporate Card Line Item Detail. YYMMDD format
N	ShipDate	Ship Date	6-character numeric	The date the merchandise was shipped to the destination. YYMMDD format
N	OrderDate	Order Date	6-character numeric	The date the item was ordered YYMMDD format
Y	ProductCode	Product Code	12-character alphanumeric	Line item Product Code (if this field is not sent, then itemComCode) If the order has a Freight/Shipping line item, the productCode value has to be "Freight/Shipping" If the order has a Discount line item, the productCode value has to be "Discount"
Y	ItemDescription	Item Description	35-character alphanumeric	Line Item description

Req	Variable Name	Field Name	Size/Type	Description
Y	ItemQuantity	Item Quantity	12-character alpha-numeric	Quantity of line item
Y	UnitCost	Unit Cost	12-character decimal	Line item cost per unit. Must contain a minimum of 2 decimal places, up to 5 decimal places supported. Minimum amount is 0.00001 and maximum is 999999.99999
Y	ItemUnitMeasure	Item Unit Measure	12-character alpha-numeric	The line item unit of measurement code
Y	ExtItemAmount	Extended Item Amount	9-character decimal	Contains the individual item amount that is normally calculated as price multiplied by quantity Must contain 2 decimal places Minimum amount is 0.00 and maximum is 999999.99
N	DiscountAmount	Discount Amount	9-character decimal	Contains the item discount amount Must contain 2 decimal places Minimum amount is 0.00 and maximum is 999999.99
N	CommodityCode	Commodity Code	15-character alpha-numeric	Code assigned to the merchant that best

Req	Variable Name	Field Name	Size/Type	Description
				categorizes the item (s) being purchased
C*	Tax	Tax	Up to 6 arrays	Can have up to 6 arrays contains different tax details. See Tax Array below for each field description. *This field is conditionally mandatory — if you use this array, you must fill in all tax array fields as listed in the Tax Array Request Fields below.

Table 4 Tax Array Request Fields - MasterCard Level 2/3 Transactions

Req	Variable Name	Field Name	Size/Type	Description
M	tax_amount	Tax Amount	12-character decimal	Contains detail tax amount for purchase of goods or service Must be 2 decimal places Maximum 999999.99
M	tax_rate	Tax Rate	5-character decimal	Contains the detailed tax rate applied in relationship to a specific tax amount EXAMPLE: 5% GST should be '5.0' or or

Req	Variable Name	Field Name	Size/Type	Description
				9.975% QST should be '9.975' May contain up to 3 decimals, minimum 0.001, maximum up to 9999.9
M	tax_type	Tax Type	4-character alpha-numeric	Contains tax type such as GST,QST,PST,HST
M	tax_id	Tax ID	20-character alpha-numeric	Provides an identification number used by the card acceptor with the tax authority in relationship to a specific tax amount such as GST/HST number
M	tax_included_in_sales	Tax included in sales indicator	1-character alpha-numeric	This is the indicator used to reflect additional tax capture and reporting. Valid values are: Y = Tax included in total purchase amount N = Tax not included in total purchase amount

A.8 Definition of Request Fields for Level 2/3 - Amex

Table 1 Amex- Level 2/3 Request Fields - Table 1 - Heading Fields

Req	Variable Name	Field Name	Size/Type	Description
C	big04	Purchase Order Number	22-character alpha-numeric	The cardholder supplied Purchase Order Number,

Req	Variable Name	Field Name	Size/Type	Description								
				which is entered by the merchant at the point-of-sale This entry is used in the Statement/Reporting process and may include accounting information specific to the client Mandatory if the merchant's customer provides a Purchase Order Number								
N	big05	Release Number	30-character alphanumeric	A number that identifies a release against a Purchase Order previously placed by the parties involved in the transaction								
N	big10	Invoice Number	8-character alphanumeric	Contains the Amex invoice/reference number								
Y	n101	Entity Identifier Code	2-character alphanumeric	Supported values: 'R6' - Requester (required) 'BG' - Buying Group (optional) 'SF' - Ship From (optional) 'ST' - Ship To (optional) '40' - Receiver (optional)								
Y	n102	Name	40-character alphanumeric	<table><tr><th>n101 code</th><th>n102 meaning</th></tr><tr><td>R6</td><td>Requester Name</td></tr><tr><td>BG</td><td>Buying Group Name</td></tr><tr><td>SF</td><td>Ship From Name</td></tr></table>	n101 code	n102 meaning	R6	Requester Name	BG	Buying Group Name	SF	Ship From Name
n101 code	n102 meaning											
R6	Requester Name											
BG	Buying Group Name											
SF	Ship From Name											

Req	Variable Name	Field Name	Size/Type	Description												
				<table><tr><th>n101 code</th><th>n102 meaning</th></tr><tr><td>ST</td><td>Ship To Name</td></tr><tr><td>40</td><td>Receiver Name</td></tr></table>	n101 code	n102 meaning	ST	Ship To Name	40	Receiver Name						
n101 code	n102 meaning															
ST	Ship To Name															
40	Receiver Name															
N	n301	Address	40-character alpha-numeric	Address												
N	n401	City	30-character alpha-numeric	City												
N	n402	State or Province	2-character alpha-numeric	State or Province												
N	n403	Postal Code	15-character alpha-numeric	Postal Code												
Y	ref01	Reference Identification Qualifier	2-character alpha-numeric	This element may contain the following qualifiers for the corresponding occurrences of the N1Loop: <table><tr><th>n101 value</th><th>ref01 denotation</th></tr><tr><td>R6</td><td>Supported values: 4C - Shipment Destination Code (mandatory) CR - Customer Reference Number (conditional)</td></tr><tr><td>BG</td><td>n/a</td></tr><tr><td>SF</td><td>n/a</td></tr><tr><td>ST</td><td>n/a</td></tr><tr><td>40</td><td>n/a</td></tr></table>	n101 value	ref01 denotation	R6	Supported values: 4C - Shipment Destination Code (mandatory) CR - Customer Reference Number (conditional)	BG	n/a	SF	n/a	ST	n/a	40	n/a
n101 value	ref01 denotation															
R6	Supported values: 4C - Shipment Destination Code (mandatory) CR - Customer Reference Number (conditional)															
BG	n/a															
SF	n/a															
ST	n/a															
40	n/a															
Y	ref02	Reference Identification	15-character alpha-numeric	VR is the Vendor ID Number, other codes describe the following:												

Req	Variable Name	Field Name	Size/Type	Description						
				<table><tr><th>ref01 code</th><th>ref02 denotation</th></tr><tr><td>4C</td><td>Ship to Zip or Canadian Postal Code (required)</td></tr><tr><td>CR</td><td>Cardmember Reference Number (optional)</td></tr></table>	ref01 code	ref02 denotation	4C	Ship to Zip or Canadian Postal Code (required)	CR	Cardmember Reference Number (optional)
ref01 code	ref02 denotation									
4C	Ship to Zip or Canadian Postal Code (required)									
CR	Cardmember Reference Number (optional)									

Table 2 Amex - Level 2/3 Request Fields - Table 2 - Detail Fields

Req	Variable Name	Field Name	Size/Type	Description
Y	it102	Line Item Quantity Invoiced	10-character decimal	Quantity of line item. Up to 2 decimal places supported. Minimum amount is 0.0 and maximum is 9999999999.
Y	it103	Unit or Basis for Measurement Code	2-character alphanumeric	The line item unit of measurement code Must contain a code that specifies the units in which the value is expressed or the manner in which a measurement is taken EXAMPLE: EA = each, E5=inches See ANSI X-12 EDI Allowable Units of

Req	Variable Name	Field Name	Size/Type	Description
				Measure and Codes for the list of codes
Y	it104	Unit Price	15-character decimal	Line item cost per unit Must contain 2 decimal places Minimum amount is 0.00 and maximum is 999999.99
N	it105	Basis or Unit Price Code	2-character alphanumeric	Code identifying the type of unit price for an item EXAMPLE: DR = dealer, AP = advise price See ASC X12 004010 Element 639 for list of codes
N	it10618	Product/Service ID Qualifier	2-character alphanumeric	Supported values: 'MG' - Manufacturer's Part Number 'VC' - Supplier Catalog Number 'SK' - Supplier Stock Keeping Unit Number 'UP' - Universal Product Code 'VP' - Vendor Part Number 'PO' - Purchase Order Number 'AN' - Client Defined Asset Code

Req	Variable Name	Field Name	Size/Type	Description
N	it10719	Product/Service ID	it10618	Product/Service ID corresponds to the preceding qualifier defined in it10618 The maximum length depends on the qualifier defined in it10618
			VC	
			PO	
			Other	
C	txi01	Tax Type code	2-character alphanumeric	Supported values: 'CA' – City Tax (optional) 'CT' – County/Tax (optional) 'EV' – Environmental Tax (optional) 'GS' – Good and Services Tax (GST) (optional) 'LS' – State and Local Sales Tax (optional) 'LT' – Local Sales Tax (optional) 'PG' – Provincial Sales Tax (PST) (optional) 'SP' – State/Provincial Tax a.k.a. Quebec Sales Tax (QST) (optional) 'ST' – State Sales Tax (optional) 'TX' – All Taxes (required) 'VA' – Value-Added Tax a.k.a. Canadian Harmonized Sales Tax (HST) (optional)
C	txi02	Monetary Amount	6-character decimal	This element may contain the mon-

Req	Variable Name	Field Name	Size/Type	Description
				etary tax amount that corresponds to the Tax Type Code in txi01 NOTE: If txi02 is used in mandatory occurrence txi01=TX, txi02 must contain the total tax amount applicable to the entire invoice (transaction) If taxes are not applicable for the entire invoice (transaction), txi02 must be 0.00. The maximum value that can be entered in this field is "9999.99", which is \$9,999.99 (CAD) A debit is entered as: 9999.99 A credit is entered as: -9999.99
C	txi03	Percent	10-character decimal	Contains the tax percentage (in decimal format) that corresponds to the tax type code defined in txi01 Up to 2 decimal places supported
C	txi06	Tax Exempt Code	1-character alphanumeric	This element may contain the Tax Exempt Code that identifies the exemption status from sales and tax that corresponds to

Req	Variable Name	Field Name	Size/Type	Description
				the Tax Type Code in txi01 Supported values: 1 – Yes (Tax Exempt) 2 – No (Not Tax Exempt) 4 – Not Exempt/For Resale A – Labor Taxable, Material Exempt B – Material Taxable, Labor Exempt C – Not Taxable F – Exempt (Goods / Services Tax) G – Exempt (Provincial Sales Tax) L – Exempt Local Service R – Recurring Exempt U – Usage Exempt
Y	pam05	Line Item Extended Amount	8-character decimal	Contains the individual item amount that is normally calculated as price multiplied by quantity Must contain 2 decimal places Minimum amount is 0.00 and maximum is 99999.99
Y	pid05	Line Item Description	80-character alphanumeric	Line Item description

Req	Variable Name	Field Name	Size/Type	Description
				Contains the description of the individual item purchased This field pertain to each line item in the transaction

Table 3 Amex - Level 2/3 Request Fields - Table 3 - Summary Fields

Req	Variable Name	Field Name	Size/Type	Description
C	txi01	Tax Type code	2-character alpha-numeric	Supported values: 'CA' – City Tax (optional) 'CT' – County/Tax (optional) 'EV' – Environmental Tax (optional) 'GS' – Good and Services Tax (GST) (optional) 'LS' – State and Local Sales Tax (optional) 'LT' – Local Sales Tax (optional) 'PG' – Provincial Sales Tax (PST) (optional) 'SP' – State/Provincial Tax a.k.a. Quebec Sales Tax (QST) (optional) 'ST' – State Sales Tax (optional) 'TX' – All Taxes (required) 'VA' – Value-Added

Req	Variable Name	Field Name	Size/Type	Description
				Tax a.k.a. Canadian Harmonized Sales Tax (HST) (optional)
C	txi02	Monetary Amount	6-character decimal	This element may contain the monetary tax amount that corresponds to the Tax Type Code in txi01 NOTE: If txi02 is used in mandatory occurrence txi01=TX, txi02 must contain the total tax amount applicable to the entire invoice (transaction) If taxes are not applicable for the entire invoice (transaction), txi02 must be 0.00. The maximum value that can be entered in this field is “9999.99”, which is \$9,999.99 (CAD) A debit is entered as: 9999.99 A credit is entered as: -9999.99
C	txi03	Percent	10-character decimal	Contains the tax percentage (in decimal format) that corresponds to the tax type code defined in txi01 Up to 2 decimal places supported

Req	Variable Name	Field Name	Size/Type	Description
C	txi06	Tax Exempt Code	1-character alpha-numeric	Supported values: 1 – Yes (Tax Exempt) 2 – No (Not Tax Exempt) 4 – Not Exempt/For Resale A – Labor Taxable, Material Exempt B – Material Taxable, Labor Exempt C – Not Taxable F – Exempt (Goods / Services Tax) G – Exempt (Provincial Sales Tax) L – Exempt Local Service R – Recurring Exempt U – Usage Exempt

A.9 Definition of Request Fields – 3-D Secure 2.2

Variable Name	Type and Limits	Description
billing address BillAddress1	<i>String</i> 50-character alphanumeric	Cardholder billing address
billing city BillCity	<i>String</i> 50-character alphanumeric	Cardholder billing city
billing country BillCountry	<i>String</i> 3-character alphanumeric	Defined as 3 digit country code ISO 3166-1
billing postal code BillPostalCode	<i>String</i>	Cardholder billing postal code

Variable Name	Type and Limits	Description
	16-character alphanumeric	
billing province BillProvince	String 3-character alphanumeric	Cardholder province or state Defined in country subdivision ISO 3166-2
browser java enabled BrowserJavaEnabled	String 1-character alphabetic	Indicates whether Java is enabled in the browser Allowable values: T = True F = False
browser language BrowserLanguage	String 8-character alphanumeric	As defined in IETF BCP47
browser screen height BrowserScreenHeight	String 6-character numeric	Pixel height of cardholder screen
browser screen width BrowserScreenWidth	String 6-character numeric	Pixel width of cardholder screen
browser user agent BrowserUserAgent	String 2048-character alpha-numeric	Browser User Agent
cardholder name CardholderName	String 45-character alphanumeric NOTE: Accented characters are not allowable	Name of the cardholder
challenge window size ChallengeWindowSize	String 2-character alphanumeric	Relates to the rendering of the ACS challenge within the browser. Allowable values: 01 = 250 x 400

Variable Name	Type and Limits	Description
		02 = 390 x 400 03 = 500 x 600 04 = 600 x 400 05 = Full screen
cres CRes	<i>String</i> 200-character alpha-numeric	Response data from the challenge
currency currency	<i>String</i> 3-character numeric	ISO 4217 3 digit currency code (CAD = 124, USD = 840) NOTE: This field should not be sent unless Multi Currency Pricing is enabled on your merchant account
DS transaction ID DSTransId	<i>String</i> 36-character alphanumeric	Refers to the DSTransId in the response of the previous 3DS authentication. NOTE: Only used in financial transactions using 3rd Party 3-D Secure services.
decoupled request async URL DecoupledRequestAsyncUrl	<i>String</i> 256-character alpha-numeric	Your URL where Moneris will POST the response back from ACS. Moneris reattempts 3 times to POST the response.
decoupled request indicator DecoupledRequestIndicator	<i>String</i> 1-character alphabetic	Whether the request utilizes Decoupled Authentication or not, if the ACS confirms its use. Y = Decoupled Authentication is supported and preferred if challenge is necessary N = Do not use Decoupled Authentication (Default)
decoupled request max time	<i>String</i>	The maximum minutes that Moneris

Variable Name	Type and Limits	Description
DecoupledRequestMaxTime	5-character numeric	waits for an ACS to provide results. Numeric values between 1 and 10080. The max is equivalent to 7 days.
device channel DeviceChannel	String 2-character numeric	The interface used to initiate the authentication: 02 = Browser (BRW) 03 = 3DS Requestor Initiated (3RI)
email Email	String 254-character alpha-numeric	Cardholder email address NOTE: This field is not mandatory, but it is required. It is highly recommended to provide the cardholder's email address. Lack of providing the cardholder's address, might increase the risk of rejects.
message category MessageCategory	String 2-character numeric	Whether the authentication request is for a payment or non-payment use: 01 = payment authentication (PA) 02 = non-payment authentication (NPA)
notification URL NotificationURL	String 256-character alpha-numeric	Notification URL for receiving the 3DS Method POST response from the issuer ACS.
prior request ref prior_request_auth_ref	String 36-character alphanumeric	Refers to the 3DS ACS Transaction ID in the response of the previous 3DS authentication.
prior request auth method prior_request_auth_method	String 2-character numeric	Mechanism used by the cardholder to authenticate in the previous 3DS authentication: 01 = Frictionless authentication

Variable Name	Type and Limits	Description
		02 = Challenge authentication 03 = AVS verified 04 = Other issuer methods
prior request auth timestamp prior_request_auth_timestamp	<i>String</i> 12-character numeric	Date and time in UTC of the prior cardholder authentication. Found in the previous 3DS authentication response as 3DS Auth TimeStamp. Format is YYYYMMDDHHMM.
recurring expiry RecurringExpiry	<i>String</i> 8-character numeric	End date after which no further recurring transactions shall be performed. Format is YYYYMMDD.
recurring frequency RecurringFrequency	<i>String</i> 4-character numeric	The minimum number of days between recurring transactions. Numeric values between 1 and 9999, leading zeroes accepted.
request challenge RequestChallenge	<i>String</i> 2-character numeric	Indicates whether a browser-based challenge is requested for this transaction. Standard is "01" • 01 = No preference • 02 = No challenge requested • 03 = Challenge requested: 3DS Requestor Preference • 04 = Challenge requested: Mandate
request type RequestType	<i>String</i> 2-character alphanumeric	Indicates the type of browser-based authentication request: 01 = cardholder initiated payment 02 = recurring transaction

Variable Name	Type and Limits	Description
		03 = installment transaction 04 = add card 05 = maintain card 06 = cardholder verification as part of EMV token ID & V
shipping address	<i>String</i>	Shipping destination address
ShipAddress1	50-character alphanumeric	
ri indicator	<i>String</i>	The type of 3DS Requestor Initiated (3RI) request:
RiIndicator	2-character numeric	01 = Recurring 02 = Installment 03 = Add Card 04 = Maintain Card Information 05 = Account verification 06 = Split/Delayed Shipment 07 = Top-up 08 = Mail Order 09 = Telephone Order 10 = Whitelist 11 = Other Payment
NOTE: Visa Secure only support ri_Indicator = 01, 02, 06, 07, or 11 for Payment Transactions and ri Indicator = 03, 04, 05 and 10 for Non Payment Transactions		
shipping city	<i>String</i>	Shipping destination city
ShipCity	50-character alphanumeric	
shipping country	<i>String</i>	Shipping destination country
ShipCountry	3-character alphanumeric	Defined as 3-digit country code in ISO 3166-1
shipping postal code	<i>String</i>	Shipping destination postal or ZIP code

Variable Name	Type and Limits	Description
ShipPostalCode	16-character alphanumeric	
shipping province	<i>String</i>	Shipping destination province
ShipProvince	3-character alphanumeric	Defined in country subdivision ISO 3166-2
3DS completion indicator	<i>String</i>	indicates whether 3ds method
ThreeDSCompletionInd	1-character alphanumeric	MpiCardLookup was successfully completed
		Allowable values:
		Y = Successfully completed
		N = Did not successfully complete
		U = Unavailable
browser IP Address	<i>String</i>	IP address of the browser as
<BrowserIP>	Allows '.' and ':'	returned by the HTTP headers to
	45-character alphanumeric	the 3DS Requestor.
		NOTE: This field is not mandatory, but it is required. It is highly recommended to provide. Lack of providing this field, might increase the risk of rejects.
cardholder work phone number	<i>Object</i>	Cardholder work phone number
<WorkPhone>	N/A	NOTE: This field is not mandatory, but it is required. It is highly recommended to provide at least one of the Cardholder Phone Number. Lack of providing at least one of the Cardholder Phone Number, might increase the risk of rejects.
		NOTE: This is a nested object within the transaction. For information about fields in the Cardholder Phone Number Info object, see Cardholder Phone Number Info Object and Variables.
cardholder home phone number	<i>Object</i>	Cardholder home phone number

Variable Name	Type and Limits	Description
<home_phone>	N/A	NOTE: This field is not mandatory, but it is required. It is highly recommended to provide at least one of the Cardholder Phone Number. Lack of providing at least one of the Cardholder Phone Number, might increase the risk of rejects. NOTE: This is a nested object within the transaction. For information about fields in the Cardholder Phone Number Info object, see Cardholder Phone Number Info Object and Variables.
cardholder mobile phone number	<i>Object</i>	Cardholder mobile phone number
<mobile_phone>	N/A	NOTE: This field is not mandatory, but it is required. It is highly recommended to provide at least one of the Cardholder Phone Number. Lack of providing at least one of the Cardholder Phone Number, might increase the risk of rejects. NOTE: This is a nested object within the transaction. For information about fields in the Cardholder Phone Number Info object, see Cardholder Phone Number Info Object and Variables.

MPI 3DS Cardholder Phone Number

Variable Name	Type and Limits	Description
country code	<i>String</i>	Country Code of phone number provided by the Cardholder.
<country_code>	3-character numeric	
phone number	<i>String</i>	The phone number provided by the Cardholder.
<phone_number>	15-character numeric	

A.10 Definition of Request Fields – MCP

Variable Name	Type and Limits	Description
MCP version number	<i>String</i> numeric current version is 1.0	Release version number for MCP
cardholder amount	<i>String</i> 12-character numeric smallest discrete unit of foreign currency	Amount, in units of foreign currency, the cardholder will be charged on the transaction
cardholder currency code	<i>String</i> 3-character numeric	ISO code representing the foreign currency of the cardholder

Optional MCP fields

Variable Name	Type and Limits	Description
MCP rate token	<i>String</i> N/A	Token representing a temporarily locked-in foreign exchange rate, obtained in the response of the MCP Get Rate transaction and used in subsequent MCP financial transaction requests in order to redeem that rate

MCP Get Rate transaction request fields

Variable Name	Type and Limits	Description
MCP version number	<i>String</i> numeric current version is 1.0	Release version number for MCP
rate transaction type	<i>String</i> 1-character alphabetic	Value representing the type of subsequent transaction request that the rate token will be used for. Allowable values:

Variable Name	Type and Limits	Description
		P – Purchase R – Refund
MCP Rate Info	Object N/A	Nested object in the MCP Get Rate transaction containing the add cardholder amount and add merchant settlement fields

MCP Rate Info object request fields

At least one of the following variables must be sent:

Variable Name	Type and Limits	Description
add cardholder amount	String array 12-character numeric, 3-character numeric (smallest discrete unit of foreign currency, currency code)	A string array representing: - the amount, in units of foreign currency, the cardholder will be charged, and - the ISO currency code corresponding to the foreign currency of the cardholder
add merchant settlement amount	String array 12-character numeric, 3-character numeric (amount in CAD pennies, currency code)	A string array representing: - the amount the merchant will receive in the transaction, in Canadian dollars - the ISO currency code corresponding to the foreign currency of the cardholder

A.11 Definition of Request Fields – Offlinx™

Applies to Offlinx™ integration only

Variable Name	Type and Limits	Description
card match ID	<i>String</i> 50-character alphanumeric	Corresponds to the Transaction ID used for the Offlinx™ Card Match Pixel Tag, a unique identifier created by the merchant Must be unique value for each transaction

A.12 Definition of Request Fields – Convenience Fee

Variable Name	Type and Limits	Description
Convenience Fee Information	<i>Object</i> N/A	Contains fields related to the Convenience Fee feature
convenience fee amount	<i>String</i> 9-character decimal	Dollar amount charged to the customer as a convenience fee

A.13 Definition of Request Fields – Recurring

Recurring Billing Info Object Request Fields

Variable Name	Type and Limits	Description
number of recurs	<i>String</i> numeric 1-999	The number of times that the transaction must recur
period	<i>String</i> numeric 1-999	Number of recur unit intervals that must pass between recurring billings
start date	<i>String</i> YYYYMMDD format	Date of the first future recurring billing transaction; this must be a date in the future If an additional charge will be made

Variable Name	Type and Limits	Description
		immediately, the start now variable must be set to true
start now	String true/false	Set to true if a charge will be made against the card immediately; otherwise set to false When set to false, use Card Verification prior to sending the Purchase with Recurring Billing and Credential on File objects NOTE: Amount to be billed immediately can differ from the subsequent recurring amounts
recurring amount	String 10-character decimal, minimum three digits Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	Dollar amount of the recurring transaction This amount will be billed on the start date, and then billed repeatedly based on the interval defined by period and recur unit
recur unit	String day, week, month or eom	Unit to be used as a basis for the interval Works in conjunction with the period variable to define the billing frequency

A.14 Definition of Request Fields – Installments by Visa

Variable Name	Type and Limits	Description
Installment Info	Object N/A	Contains request fields related to installments

Variable Name	Type and Limits	Description
installment plan ID	<i>String</i> 36-character alphanumeric fixed length	Card brand-generated identifier for an installment plan
installment plan reference	<i>String</i> 10-character alphanumeric fixed length	Unique, human friendly name for the installment plan
terms and conditions version	<i>String</i> 10-character alphanumeric variable length (1-10 characters)	Version of the terms and conditions of the installment plan accepted by the cardholder The version is auto-incremented every time an update is made to the plan by the issuer

A.15 Definition of Request Fields – Account Name Verification Object

Request fields within the Account Name Verification object. The object can only be included in Card Verification transactions. Account name verification is only applicable to Visa credit cards.

Variable Name	Type and Limits	Description
First Name <first_name>	<i>String</i> 32-character alphanumeric	Cardholder first name
Middle Name <middle_name>	<i>String</i> 32-character alphanumeric	Cardholder middle name
Last Name <last_name>	<i>String</i> 32-character alphanumeric	Cardholder last name

Appendix B Definitions of Response Fields

Table 21: Receipt object response values

Value	Type	Limits	Get Method
	Description		
General response fields			
Card type	String	2-character alphabetic (min. 1)	\$mpgResponse->getCardType () ;
	Represents the type of card in the transaction, e.g., Visa, Mastercard. Possible values: • V = Visa • M = Mastercard • AX = American Express • DC = Diner's Card • NO = Novus/Discover • SE = Sears • D = Debit • C1 = JCB		
Transaction amount	String	10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	\$mpgResponse->getTransAmount () ;
	Transaction amount that was processed.		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Transaction number	String	255-character alphanumeric	<code>\$mpgResponse->getTxnNumber()</code> ;
	Gateway Transaction identifier often needed for follow-on transactions (such as Refund and Purchase Correction) to reference the originally processed transaction.		
Receipt ID	String	50-character alphanumeric	<code>\$mpgResponse->getReceiptId()</code> ;
	Order ID that was specified in the transaction request.		
Transaction type	String	2-character alphanumeric	<code>\$mpgResponse->getTransType()</code> ;
	• 0 = Purchase • 1 = Pre-Authorization • 2 = Completion • 4 = Refund • 11 = Void		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Reference number	String	18-character numeric	<code>\$mpgResponse->getReferenceNum()</code> ;
	Terminal used to process the transaction as well as the shift, batch and sequence number. This data is typically used to reference transactions on the host systems, and must be displayed on any receipt presented to the customer. This information is to be stored by the merchant. Example: 660123450010690030 • 66012345: Terminal ID • 001: Shift number • 069: Batch number • 003: Transaction number within the batch.		
Response code	String	3-character numeric	<code>\$mpgResponse->getResponseCode()</code> ;
	• < 50: Transaction approved • ≥ 50: Transaction declined • Null: Transaction incomplete. For further details on the response codes that are returned, see the Response Codes document at https://developer.moneris.com.		
ISO	String	2-character numeric	<code>\$mpgResponse->getISO()</code> ;
	ISO response code		
Bank totals	Object		code to come
	Response data returned in a Batch Close and Open Totals request. See "Definitions of Response Fields" on page 561.		

Table 21: Receipt object response values (continued)

Value	Type Limits		Get Method
	Description		
Message	String	100-character alphanumeric	<code>\$mpgResponse->getMessage();</code>
	Response description returned from issuer. The message returned from the issuer is intended for merchant information only, and is not intended for customer receipts.		
Authorization code	String	8-character alphanumeric	<code>\$mpgResponse->getAuthCode();</code>
	Authorization code returned from the issuing institution.		
Complete	String	true/false	<code>\$mpgResponse->getComplete();</code>
	Transaction was sent to authorization host and a response was received		
Transaction date	String	Format: yyyy-mm-dd	<code>\$mpgResponse->getTransDate();</code>
	Processing host date stamp		
Transaction time	String	Format: ##:##:##	<code>\$mpgResponse->getTransTime();</code>
	Processing host time stamp		
Ticket	String	N/A	<code>\$mpgResponse->getTicket();</code>
	Reserved field.		
Timed out	String	true/false	<code>\$mpgResponse->getTimedOut();</code>
	Transaction failed due to a process timing out.		
Is Visa Debit	String	true/false	<code>\$mpgResponse->getIsVisaDebit();</code>
	Indicates whether the card processed is a Visa Debit.		
PBBLifeCycleTraceID	String	15-alphanumeric	
	Unique transaction identifier from Interac Direct systems. Applies to Interac Direct transactions only and is used to link follow-on transactions.		

Table 21: Receipt object response values (continued)

Value	Type Limits		Get Method
	Description		
Account Name Verification Result	String	10-character alphanumeric	\$mpgResponse->getAccountNameResult();
	Code indicating the results of Visa Account Name Verification. Position 1 and 2: Overall inquiry status. Position 3 and 4: Full name match status Position 5 and 6: last name match status Position 7 and 8: middle name match status Position 9 to 10: first name match status Inquiry status values: 00 = Name match performed 01 = Name match no performed 02 = Name match not supported Values for full name match and last/middle/first name match: 01 = Match 50 = Partial Match 99 = No Match		
Batch Close/Open Totals response fields			
Processed card types	String Array	N/A	
	Returns all of the processed card types in the current batch for the terminal ID/ECR Number from the request.		
Terminal IDs	String	8-character alphanumeric	code to come
	Returns the terminal ID/ECR Number from the request.		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Purchase count	String	4-character numeric	<code>\$mpgResponse->getPurchaseCount(\$secr_number,\$creditCards[\$i]);</code>
	Indicates the # of Purchase, Pre-Authorization Completion and Force Post transactions processed. If none were processed in the batch, then the value returned will be 0000.		
Purchase amount	String	11-character alphanumeric	<code>\$mpgResponse->getPurchaseAmount(\$secr_number,\$creditCards[\$i]);</code>
	Indicates the dollar amount processed for Purchase, Pre-Authorization Completion or Force Post transactions. This field begins with a + and is followed by 10 numbers, the first 8 indicate the amount and the last 2 indicate the penny value. EXAMPLE: +0000000000 = 0.00 and +0000041625 = 416.25		
Refund count	String	4-character numeric	<code>\$mpgResponse->getRefundAmount(\$secr_number,\$creditCards[\$i]);</code>
	Indicates the # of Refund or Independent Refund transactions processed. If none were processed in the batch, then the value returned will be 0000.		
Refund amount	String	11-character alphanumeric	<code>\$mpgResponse->getRefundAmount(\$secr_number,\$creditCards[\$i]);</code>
	Indicates the dollar amount processed for Refund, Independent Refund or ACH Credit transactions. This field begins with a + and is followed by 10 numbers, the first 8 indicate the amount and the last 2 indicate the penny value. Example, +0000000000 = 0.00 and +0000041625 = 416.25		
Correction count	String	4-character numeric	<code>\$mpgResponse->getCorrectionCount(\$secr_number,\$creditCards[\$i]);</code>
	Indicates the # of Purchase Correction transactions processed. If none were processed in the batch, then the value returned will be 0000.		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Correction amount	String	11-character alphanumeric	\$mpgResponse->getCorrectionAmount(\$secr_number,\$creditCards[\$i]);
	Indicates the dollar amount processed for Purchase Correction transactions. This field begins with a + and is followed by 10 numbers, the first 8 indicate the amount and the last 2 indicate the penny value.		
	EXAMPLE: +0000000000 = 0.00 and +0000041625 = 416.25		
Recurring Billing Response Fields (see Appendix A, page 1)			
Recurring billing success	String	true/false	\$mpgResponse->getRecurSuccess();
	Indicates whether the recurring billing transaction has been successfully set up for future billing.		
Recur update success	String	true/false	\$mpgResponse->getRecurUpdateSuccess();
	Indicates recur update success.		
Next recur date	String	yyyy-mm-dd	\$mpgResponse->getNextRecurDate();
	Indicates next recur billing date.		
Recur end date	String	yyyy-mm-dd	\$mpgResponse->getRecurEndDate();
	Indicates final recur billing date.		
Status Check response fields (see)			
Status code	String	3-character alphanumeric	\$mpgResponse->getStatusCode();
	< 50: Transaction found and successful ≥ 50: Transaction not found and not successful		
	NOTE: the status code is only populated if the connection object's Status Check property is set to true.		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Status message	String	found/not found	\$mpgResponse->getStatusMessage() ();
	- Found: $0 \leq \text{Status Code} \leq 49$ - Not Found or null: $50 \leq \text{Status Code} \leq 999$.		
	NOTE: The status message is only populated if the connection object's Status Check property is set to true.		
AVS response fields (see 9.1, page 382)			
AVS result code	String	1-character alpha-numeric	\$mpgResponse->getAvsResultCode() ();
	Indicates the address verification result. For a full list of possible response codes refer to Section Appendix B.		
CVD response fields (see)			
CVD result code	String	2-character alpha-numeric	\$mpgResponse->getCvdResultCode() ();
	Indicates the CVD validation result. The first byte is the numeric CVD indicator sent in the request; the second byte is the response code. Possible response codes are shown in Appendix B		
GooglePay Token response fields			
GooglePay Payment Method	String	4-character alpha-numeric	\$mpgResponse->GetGooglepayPaymentMethod() ();
	Indicates if the underlying card used in the GooglePay digital wallet is the funding card number ("FPAN") or a tokenized card number ("DPAN"). If a GoogleTokenTempAdd returns an FPAN, you may perform 3DS authentication with it; if it returns a DPAN, 3DS is not required.		
MPI response fields (see "MPI" on page 1)			
Type	String	99-character alphanumeric	
	VERes, PARes or error defines what type of response you are receiving .		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Success	Boolean	true/false	<code>\$mpgResponse->getMpiSuccess()</code> ;
	True if attempt was successful, false if attempt was unsuccessful.		
Message	String	100-character alphabetic	<code>\$mpgResponse->getMpiMessage()</code> ;
	MPI TXN transactions can produce the following values: - Y: Create VBV verification form popup window. - N: Send purchase or preauth with crypt type 6 - U: Send purchase or preauth with crypt type 7. MPI ACS transactions can produce the following values: - Y or A: (Also <code>receipt.getMpiSuccess()=true</code>) Proceed with cavv purchase or cavv preauth. - N: Authentication failed or high-risk transaction. It is recommended that you do not to proceed with the transaction. Depending on a merchant's risk tolerance and results from other methods of fraud detection, transaction may proceed with crypt type 7. - U or time out: Send purchase or preauth as crypt type 7.		
Term URL	String	255-character alphanumeric	
	URL to which the PAREs is returned		
MD	String	1024-character alphanumeric	
	Merchant-defined data that was echoed back		
ACS URL	String	255-character alphanumeric	
	URL that will be for the generated pop-up		
MPI CAVV	String	28-character alphanumeric	<code>\$mpgResponse->getMpiCavv()</code>
	VbV/MCSC/American Express SafeKey authentication data		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
MPI E-Commerce Indicator	String	1-character alpha-numeric	
CAVV result code	String	1-character alpha-numeric	<code>\$mpgResponse->getCavvResultCode()</code> ;
	Indicates the Visa CAVV result. For more information, see 1 Cavv Result Codes for Verified by Visa. • 0 = CAVV authentication results invalid • 1 = CAVV failed validation; authentication • 2 = CAVV passed validation; authentication • 3 = CAVV passed validation; attempt • 4 = CAVV failed validation; attempt • 7 = CAVV failed validation; attempt (US issued cards only) • 8 = CAVV passed validation; attempt (US issued cards only) • The CAVV result code indicates the result of the CAVV validation.		
MPI inline form			<code>\$mpgResponse->getMpiInLineForm()</code> ;
Vault response fields (see 4.1, page 66)			
Data key	String	28-character alphanumeric	<code>\$mpgResponse->getDataKey()</code> ;
	The data key response field is populated when you send a Vault Add Credit Card – ResAddCC (page 69), Vault Encrypted Add Credit Card – EncResAddCC (page 73), Vault Tokenize Credit Card – ResTokenizeCC (page 97), Vault Temporary Token Add – ResTempAdd (page 76) or Vault Add Token – ResAddToken (page 94) transaction. It is the profile identifier that all future financial Vault transactions will use to associate with the saved information.		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Vault payment type	String	cc	\$mpgResponse->getPaymentType();
	Indicates the payment type associated with a Vault profile		
Expiring card's Payment type	String	cc	\$mpgResponse->getExpPaymentType();
	Indicates the payment type associated with a Vault profile. Applicable to Vault Get Expiring transaction type.		
Vault masked PAN	String	20-character numeric	\$mpgResponse->getResDataMaskedPan();
	Returns the first 4 and/or last 4 of the card number saved in the profile.		
Expiring card's Masked PAN	String	20-character numeric	\$mpgResponse->getResDataMaskedPan();
	Returns the first 4 and/or last 4 of the card number saved in the profile. Applicable to Vault Get Expiring transaction type.		
Vault success	String	true/false	\$mpgResponse->getResSuccess();
	Indicates whether Vault transaction was successful.		
Vault customer ID	String	30-character alphanumeric	\$mpgResponse->getResDataCustId();
	Returns the customer ID saved in the profile.		
Expiring card's customer ID	String	30-character alphanumeric	\$mpgResponse->getResDataCustId();
	Returns the customer ID saved in the profile. Applicable to Vault Get Expiring transaction type.		
Vault phone number	String	30-character alphanumeric	\$mpgResponse->getResDataPhone();
	Returns the phone number saved in the profile.		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Expiring card's phone number	String	30-character alphanumeric	<code>\$mpgResponse->getResDataPhone()</code> ;
	Returns the phone number saved in the profile. Applicable to Vault Get Expiring transaction type.		
Vault email address	String	30-character alphanumeric	<code>\$mpgResponse->getResDataEmail()</code> ;
	Returns the email address saved in the profile.		
Expiring card's email address	String	30-character alphanumeric	<code>\$mpgResponse->getResDataEmail()</code> ;
	Returns the email address saved in the profile. Applicable to Vault Get Expiring transaction type.		
Vault note	String	30-character alphanumeric	<code>\$mpgResponse->getResDataNote()</code> ;
	Returns the note saved in the profile.		
Expiring card's note	String	30-character alphanumeric	<code>\$mpgResponse->getResDataNote()</code> ;
	Returns the note saved in the profile. Applicable to Vault Get Expiring transaction type.		
Vault expiry date	String	4-character numeric	<code>\$mpgResponse->getResDataExpDate()</code> ;
	Returns the expiry date of the card number saved in the profile. YYYYMM format.		
Expiring card's expiry date	String	4-character numeric	<code>\$mpgResponse->getResDataExpDate()</code> ;
	Returns the expiry date of the card number saved in the profile. YYYYMM format. Applicable to Vault Get Expiring transaction type.		
Vault E-commerce indicator	String	1-character numeric	<code>\$mpgResponse->getResDataCryptType()</code> ;
	Returns the e-commerce indicator saved in the profile.		

Table 21: Receipt object response values (continued)

Value	Type Limits		Get Method
	Description		
Expiring card's E-commerce indicator	String	1-character numeric	\$mpgResponse->getResDataCryptType();
	Returns the e-commerce indicator saved in the profile. Applicable to Vault Get Expiring transaction type.		
Vault AVS street number	String	19-character alphanumeric	\$mpgResponse->getResDataAvsStreetNumber();
	Returns the AVS street number saved in the profile. If no other AVS street number is passed in the transaction request, this value will be submitted along with the financial transaction to the issuer.		
Expiring card's AVS street number	String	19-character alphanumeric	\$mpgResponse->getResDataAvsStreetNumber();
	Returns the AVS street number saved in the profile. If no other AVS street number is passed in the transaction request, this value will be submitted along with the financial transaction to the issuer. Applicable to Vault Get Expiring transaction type.		
Vault AVS street name	String	19-character alphanumeric	\$mpgResponse->getResDataAvsStreetName();
	Returns the AVS street name saved in the profile. If no other AVS street number is passed in the transaction request, this value will be submitted along with the financial transaction to the issuer.		
Expiring card's AVS street name	String	19-character alphanumeric	\$mpgResponse->getResDataAvsStreetName();
	Returns the AVS street name saved in the profile. If no other AVS street number is passed in the transaction request, this value will be submitted along with the financial transaction to the issuer. Applicable to Vault Get Expiring transaction type.		
Vault AVS ZIP code	String	9-character alphanumeric	\$mpgResponse->getResDataAvsZipcode();
	Returns the AVS zip/postal code saved in the profile. If no other AVS street number is passed in the transaction request, this value will be submitted along with the financial transaction to the issuer.		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Expiring card's AVS ZIP code	String	9-character alpha-numeric	\$mpgResponse->getResDataAvsZipcode();
	Returns the AVS zip/postal code saved in the profile. If no other AVS street number is passed in the transaction request, this value will be submitted along with the financial transaction to the issuer. Applicable to Vault Get Expiring transaction type.		
Vault credit card number	String	20-character numeric	\$mpgResponse->getResDataPan();
	Returns the full credit card number saved in the Vault profile. Applicable to Vault Lookup Full transaction only.		
Corporate card	String	true/false	\$mpgResponse->getCorporateCard();
	Indicates whether the card associated with the Vault profile is a corporate card.		
Encrypted Mag Swipe response fields (see Section 1, page 1)			
Masked credit card number	String	20-character alphanumeric	\$mpgResponse->getMaskedPan();
Convenience Fee response fields (see Appendix A, page 1)			
Convenience fee success	String	true/false	\$mpgResponse->getCfSuccess();
	Indicates whether the Convenience Fee transaction processed successfully.		

Table 21: Receipt object response values (continued)

Value	Type Limits Get Method		
	Description		
Convenience fee status	String	3-character alpha-numeric	<code>\$mpgResponse->getCfStatus();</code>
	Indicates the status of the merchant and convenience fee transactions. The CfStatus field provides details about the transaction behavior and should be referenced when contacting Moneris Customer Support. Possible values are: • 1 or 1F – Completed 1st purchase transaction • 2 or 2F – Completed 2nd purchase transaction • 3 – Completed void transaction • 4A or 4D – Completed refund transaction • 7 or 7F – Completed merchant independent refund transaction • 8 or 8F – Completed merchant refund transaction • 9 or 9F – Completed 1st void transaction • 10 or 10F – Completed 2nd void transaction • 11A or 11D – Completed refund transaction		
Convenience fee amount	String	9-character decimal	<code>\$mpgResponse->getFeeAmount();</code>
	The expected Convenience Fee amount. This field will return the amount submitted by the merchant for a successful transaction. For an unsuccessful transaction, it will return the expected convenience fee amount		

Table 21: Receipt object response values (continued)

Value	Type	Limits	Get Method
	Description		
Convenience fee rate	String	9-character decimal	\$mpgResponse->getFeeRate () ;
	The convenience fee rate that has been defined on the merchant's profile. For example: 1.00 – a fixed amount or 10.0 - a percentage amount		
Convenience fee type	String	AMT/PCT	\$mpgResponse->getFeeType () ;
	The type of convenience fee that has been defined on the merchant's profile. Available options are: AMT – fixed amount PCT – percentage		
Merchant Advice Code response field			
Advice Code	String	2-character alpha-numeric	\$mpgResponse->getAdviceCode () ;
	The message returned from the issuer is intended for merchant information only, and is not intended for customer receipts. For further details on the response codes that are returned, see the Advice Code document at https://developer.moneris.com.		

Table 22: Financial transaction response codes

Code	Description
< 50	Transaction approved
≥ 50	Transaction declined
NULL	Transaction was not sent for authorization

For more details on the response codes that are returned, see the Response Codes document available at <https://developer.moneris.com>

Table 23: Vault Admin Responses

Code	Description
001	Successfully registered CC details. Successfully updated CC details. Successfully deleted CC details. Successfully located CC details. Successfully located # expiring cards. (NOTE: # = the number of cards located)
983	Cannot find previous
986	Incomplete: timed out
987	Invalid transaction
988	Cannot find expiring cards
Null	Error: Malformed XML

B.1 Definition of Response Fields – 3-D Secure

The following response fields are specific to 3-D Secure transactions

Variable Name	Type and Limits	Description
Cardholder Authentication Value (CAVV)	<i>String</i> 50-character alphanumeric	<code>\$mpgResponse->getMpiCavv()</code> Cardholder Authentication Value, to be provided in the financial request
challenge completion indicator	<i>String</i> 1-character alphabetic	<code>\$mpgResponse->getMpiChallengeCompletionIndicator()</code> Indicates the result of the challenge request Y = challenge complete

Variable Name	Type and Limits	Description
		- N = challenge not complete - null = field not present
challenge URL	<i>String</i> 2048-character alphanumeric	<code>\$mpgResponse->getMpiChallengeURL()</code> If the transaction status is "C" this field will be populated with the URL to POST the challenge data to create the cardholder challenge screen.
Message Type	<i>String</i> 4-character alpha-betic	<code>\$mpgResponse->getMpiMessageType()</code> Denotes the request message EMV nomenclature "ARES"
3DS method URL	<i>String</i> alphanumeric	<code>\$mpgResponse->getMpiThreeDSMethodURL()</code> Device fingerprinting endpoint
3DS method data	<i>String</i> alphanumeric	<code>\$mpgResponse->getMpiThreeDSMethodData()</code> Data that must be posted to 3DS Method URL
3DS server transaction ID	<i>String</i> 36-character alphanumeric	<code>\$mpgResponse->getMpiThreeDSServerTransId()</code> 3-D Secure unique transaction identifier
3DS version	<i>String</i> 10-character numeric	<code>\$mpgResponse->getThreeDSVersion();</code> The 3DS version number.
transaction status	<i>String</i> 1-character alpha-betic	<code>\$mpgResponse->getMpiTransStatus()</code> Indicates the result of the authentication. See 3DS TransStatus in the appendix for a full list: Y or A = successful authentication

Variable Name	Type and Limits	Description
		• C = proceed with challenge • D = wait for decoupled authentication
challenge data	alphanumeric	If the transStatus is “C” this field will be populated with the data to post the challengeURL. The data must be posted in a field with the name “creq”.
ECI	1-character alphabetic	Crypt Type to be provided in the financial request. If transaction status is “Y” or “A” this value will return in the response. The ECI is used by subsequent financial transactions.
DS trans ID	36-character alphanumeric	Universally unique transaction identifier assigned by the 3DS Directory Server (DS) to identify a single transaction.
transaction status reason	2-character numeric	Provides additional information on why the TransStatus field has the specified value. Possible values: • 01 = Card authentication failed • 02 = Unknown Device • 03 = Unsupported Device • 04 = Exceeds authentication frequency limit • 05 = Expired card • 06 = Invalid card number • 07 = Invalid transaction

Variable Name	Type and Limits	Description
		• 08 = No Card record • 09 = Security failure • 10 = Stolen card • 11 = Suspected fraud • 12 = Transaction not permitted to cardholder • 13 = Cardholder not enrolled in service • 14 = Transaction timed out at the ACS • 15 = Low confidence • 16 = Medium confidence
cardholder information	128-character alphanumeric	Text provided by the ACS/Issuer to Cardholder during a Frictionless or Decoupled transaction. This provides information to the cardholder. For example, "Additional authentication is needed for this transaction, please contact (Issuer Name) at xxx-xxx-xxxx."
Authentication-Type	2-character numeric	Authentication method the issuer will use to challenge the cardholder. • 01 = Static • 02 = Dynamic • 03 = OOB • 04 = Decoupled
3DS ACS Transaction ID	36-character alphanumeric	Universally Unique transaction identifier assigned by the issuer Access Control Server (ACS) to

Variable Name	Type and Limits	Description
		identify a single transaction.
3DS Auth TimeStamp	12-character numeric	Date and time in UTC of the cardholder authentication. Date format = YYYYMMDDHHMM

B.2 Definition of Response Fields – MCP

MCP response fields

Variable Name	Type and Limits	Get Method and Description
MCP rate	<i>String</i> 9-character decimal variable length	<code>\$mpgResponse->getMCPRate();</code> The foreign exchange rate (foreign currency to CAD) that will be used for the transaction If a MCP rate token was used, it will reflect the rate secured by the MCP Get Rate transaction; if no token was used, the rate is the current exchange rate retrieved by the Moneris Gateway
merchant settlement currency	<i>String</i> 3-character numeric	<code>\$mpgResponse->getMerchantSettlementCurrency();</code> Currency that the merchant is settling in
merchant settlement amount	<i>String</i> 10-character decimal Up to 7 digits (dollars) + decimal point (.) + 2 digits (cents) after the decimal point EXAMPLE: 1234567.89	<code>\$mpgResponse->getMerchantSettlementAmount();</code> Amount that will be paid to the merchant, in Canadian dollars

Variable Name	Type and Limits	Get Method and Description
cardholder currency code	<i>String</i> 3-character numeric	<code>\$mpgResponse->getCardholderCurrencyCode();</code> ISO code for the foreign currency the cardholder is using to pay
cardholder amount	<i>String</i> 12-character numeric variable length	<code>\$mpgResponse->getCardholderAmount();</code> Amount, in units of foreign currency, the cardholder will pay on the transaction
MCP error status code	<i>String</i> 4-character numeric variable length	<code>\$mpgResponse->getMCPErrorStatusCode();</code> A number representing a MCP error code response
MCP error message	<i>String</i> 250-character alpha-numeric variable length	<code>\$mpgResponse->getMCPErrorMessage();</code> Message corresponding with an MCP error code
host ID	<i>String</i> 15-character alpha-numeric	<code>\$mpgResponse->getHostId();</code> Unique identifier used across the Moneris platform

Response fields specific to MCP Get Rate

Variable Name	Type and Limits	Get Method and Description
rate transaction type	<i>String</i> max 8-character alphabetic PURCHASE or REFUND	<code>\$mpgResponse->getRateTxnType();</code> Reflects the transaction type being sent in the request
MCP rate token	<i>String</i> 17-character alphanumeric	<code>\$mpgResponse->getMCPRateToken();</code> Time-limited token representing a temporarily locked in foreign exchange rate for use in financial transactions

Variable Name	Type and Limits	Get Method and Description
		This field is returned in the response to a MCP Get Rate request
rate inquiry start time	<i>String</i> 24-character alphanumeric	<code>\$mpgResponse->getRateInqStartTime();</code> The local time (ISO 8601) when the rate is requested
rate inquiry end time	<i>String</i> 24-character alphanumeric	<code>\$mpgResponse->getRateInqEndTime();</code> The local time (ISO 8601) when the rate is returned
rate validity start time	<i>String</i> 10-character numeric	<code>\$mpgResponse->getRateValidityStartTime();</code> The time (unix UTC) of when the rate is valid from
rate validity end time	<i>String</i> 10-character numeric	<code>\$mpgResponse->getRateValidityEndTime();</code> The time (unix UTC) of when the rate is valid until
rate validity period	<i>String</i> 3-character numeric variable length	<code>\$mpgResponse->getRateValidityPeriod();</code> The time in minutes this rate is valid for

B.3 Definition of Response Fields –Installments by Visa

Response fields appearing in the Installment Plan Lookup transaction

Variable Name	Type and Limits	Description
Eligible Installment Plans	<i>Object</i> N/A	Contains fields related to the installment plan
plan count	<i>String</i>	Total number of installment plans available for offer to the cardholder

Variable Name	Type and Limits	Description
	numeric	
Plan Details	Array object N/A	Contains fields related to the particular installment plan Each installment plan on offer to the cardholder is represented by a distinct Plan Details object
annual percentage rate (APR)	String numeric	Annual percentage rate (APR) attached to the installment plan payments; for display purposes only and not used for calculations Allowable values: 0-10000 Percentage rate is represented with two implicit decimals EXAMPLE: 320 is 3.2%
installment frequency	String max 10-character alphabetic	Frequency of installments for the plan Potential values: WEEKLY BIWEEKLY MONTHLY BIMONTHLY
installment plan ID	String 36-character alphanumeric fixed length	Card brand-generated identifier for an installment plan Used as a request field in the Installment Info object
installment plan name	String max 255-character alphanumeric	Name of the installment plan; may not be unique
installment plan reference	String 10-character alphanumeric	Unique, human friendly name for the installment plan

Variable Name	Type and Limits	Description
	fixed length	Used as a request field in the Installment Info object
installment plan type	<i>String</i> max 20 character alpha-numeric	Type of installment plans Potential values: ISSUER_PROMOTION BI_LATERAL ISSUER_DEFAULT MARKET
number of installments	<i>String</i> 4-character numeric min 1, max 1000	Maximum number of installments in the plan
First Installment	<i>Object</i> N/A	Contains cost details for the first installment
first installment amount	<i>String</i> max 9-character numeric	Amount of the first installment payment Final two digits represent penny values EXAMPLE: 123112 = \$1231.12
first installment fee	<i>String</i> max 9-character numeric	Fee charged on the first installment Final two digits represent penny values EXAMPLE: 123112 = \$1231.12
upfront fee	<i>String</i> numeric	The up-front fee charged to the cardholder for the installment plan; only charged on the first installment

Variable Name	Type and Limits	Description
total amount	<i>String</i> numeric	Sum of upfront fee and installment fee
Last Installment	<i>Object</i> N/A	Contains cost details for the last installment
last installment amount	<i>String</i> max 9-character numeric	Amount of the final installment payment Final two digits represent penny values EXAMPLE: 123112 = \$1231.12
last installment fee	<i>String</i> max 9-character numeric	Fee charged on the last installment Final two digits represent penny values EXAMPLE: 123112 = \$1231.12
total amount	<i>String</i> numeric	Sum of upfront fee and installment fee
Promotion Info	<i>Object</i> N/A	Contains promotion information shared between the issuer and the merchant
promotion code	<i>String</i> 2-character alphanumeric	An external identifier for the plan provided by the issuer
promotion ID	<i>String</i> max 8-character alphanumeric	An external identifier provided by the issuer that identifies a program or promotion
Terms and Conditions	<i>Array object</i>	Contains fields related to terms and

Variable Name	Type and Limits	Description
	N/A	conditions presented to the cardholder
terms and conditions count	<i>String</i> numeric	Number of instances of the set of terms and conditions attached to a particular installment plan, representing the number of languages they are offered in
Terms and Conditions Details	<i>Object</i> N/A	Contains details related to a particular language set (English, French, etc.) of terms and conditions being offered Each language set has its own object
language code	<i>String</i> 3-character alphanumeric	Language code for the terms and conditions text
text	<i>String</i> max 2000-character alphanumeric	Text of the terms and conditions for the installment plan
terms and conditions URL	<i>String</i> max 1000 character-alphanumeric	A terms and conditions HTTPS URL hosted by the issuer for displaying to the cardholder
terms and conditions version	<i>String</i> 10-character alphanumeric variable length (1-10 characters)	Version of the terms and conditions of the installment plan accepted by the cardholder The version is auto-incremented every time an update is made to the plan by the issuer
total fees	<i>String</i> max 9-character numeric	Total fees charged by the plan Final two digits represent penny values

EXAMPLE: 123112 = \$1231.12

Variable Name	Type and Limits	Description
total plan cost	String numeric	Represents the total amount the selected installment plan will cost The right-most digits represent minor units (e.g., cents in CAD); no fractional minor units EXAMPLE: 123112 in CAD represents CAD \$1231.12

Response fields appearing in financial transactions

Variable Name	Type and Limits	Description
Installment Results	Object N/A	Contains fields related to the installment plan in financial transactions
installment plan ID	String 36-character alphanumeric fixed length	Card brand-generated identifier for an installment plan
installment plan reference	String 10-character alphanumeric fixed length	Unique, human friendly name for the installment plan
terms and conditions version	String 10-character alphanumeric variable length (1-10 characters)	Version of the terms and conditions of the installment plan accepted by the cardholder. The version is auto-incremented every time an update is made to the plan by the issuer.
plan acceptance ID	String 36-character alphanumeric fixed length	Visa-generated, alphanumeric, unique and short human-readable name for the installment plan
installment plan status	String	Potential values:

Variable Name	Type and Limits	Description
	1-character alphabetic fixed length	N – new plan, not accepted yet A – accepted plan C – cancelled plan
plan response	<i>String</i> max 50-character numeric	Response code for the installment plan Potential values: 00 – processed and approved If not 00, indicates installment plan processing failure; a verbose error response message as received from from Visa is returned

Appendix C Response Codes

Approved Response Codes

Response Code	Messages
000	Approved, Account Balances Included (Balance Inquiry), No Reason to Decline Approved (Balances) File Processed/Successful transaction with fault
001	Approved, Account Balances Not Included Approved – No Balances/Approved or completed successfully VIP Approved (No Balances)/Advice Acknowledged – Financial Liability Accepted
002	Approved, country club
003	Approved, maybe more ID
004	Approved, pending ID (sign paper draft)
005	Approved, blind
006	Approved, VIP
007	Approved, administrative transaction
008	Approved, national NEG file hit OK
009	Approved, commercial
010	Approved for partial amount
023	Amex - credit approval
024	Amex 77 - credit approval

Response Code	Messages
027	Transaction already reversed
028	VIP Credit Approved
029	Credit Response Acknowledgement
900	Global Error
901	Invalid URL
902	Malformed XML

Declined Response Codes

Response Code	Messages
050	Do Not Honor Decline Refer to card issuer ID certification fails Deny – Do not Honour Card not initialized Declined: Deny – Unacceptable Fee Unable to locate original transaction Suspected Fraud Deny – Card Acceptor Call Acquirer's Security Dep Amount Not Reconciled – Totals Provided ATM/POS terminal number cannot be located MAC failed Declined: MAC failed

Response Code	Messages
	Reserved Security processing failure No arrears (transaction receipt not printed) Invalid File Type No such File File Locked Unsuccessful Incorrect File Length File Decompression Error File Name Error File cannot be received Deny – Do Not Honour
051	Expired Card
052	PIN retries exceeded PIN try limit exceeded Allowable number of PIN tries exceeded
053	No sharing
054	No security module
055	Invalid transaction
056	No Support/Transaction Not Permitted to Acquirer Tran Not Supported by FI/Not Supported by Receiver
057	Lost or stolen card
058	Invalid status
059	Deny (Keep Card) – Restricted Card

Response Code	Messages
	Restricted Card
060	No Chequing account No Savings Account
061	No PBF
062	PBF update error
063	Invalid authorization type
064	Bad Track 2
065	Adjustment not allowed
066	Invalid credit card advance increment
067	Invalid transaction date
068	PTLF error
069	Bad Message Error/No CVM Results Bad message – edit error/Format error
070	No IDF Invalid Issuer Invalid Issuer/Deny – Issuer/Bank Not Found
071	Invalid route authorization Unable to route/Financial institution or intermediate network facility cannot be found for routing Invalid Rout to Auth /Incorrect IIN
072	Card on National NEG file
073	Invalid route service (destination)
074	Unable to authorize

Response Code	Messages
	Re-enter Transaction Transaction Cannot be Completed Deny – Security Violation Deny – Violation of Law System problem - ask cardholder to insert card in chip card reader Merchant Link not logged on (Network Management Logon required)
075	Invalid PAN length
076	Low funds
077	Pre-auth full
078	Duplicate transaction Duplicate transaction/Request in progress
079	Maximum online refund reached
080	Maximum offline refund reached
081	Maximum credit per refund reached
082	Number of times used exceeded
083	Maximum refund credit reached
084	Duplicate transaction - authorization number has already been corrected by host
085	Inquiry not allowed
086	Over floor limit
087	Maximum number of refund credit by retailer
088	Place call
089	CAF status inactive or closed
090	Referral file full

Response Code	Messages
091	NEG file problem
092	Advance less than minimum
093	Delinquent
094	Over table limit
095	Amount over maximum Amt Over Max/Transaction amount limit exceeded
096	PIN required
097	Mod 10 check failure
098	Force Post
099	Bad PBF

Referral Response Codes

Response Code	Messages
100	Unable to process transaction Invalid Request. Contact Moneris Client POS Certification for repeat declines. Network Unavailable System Malfunction
101	Place call
102	Refer – Call Expired Card Card Acceptor Contact Call Card Accpt Acq Secur
103	NEG file problem

Response Code	Messages
104	CAF problem
105	Card not supported
106	Amount over maximum
107	Over daily limit
108	CAF Problem
109	Advance less than minimum
110	Number of times used exceeded
111	Delinquent
112	Over table limit
113	Timeout
115	PTLF error
121	Administration file problem
122	Unable to validate PIN: security module down

System Error Response Codes

Response Code	Messages
150	Invalid Service Code/Merchant Merchant Not On File Merchant Not on File/Invalid Merchant
200	Invalid account Invalid Card Number Invalid Account/Deny – No Account Type Requested
201	Incorrect PIN

Response Code	Messages
	Invalid PIN/Incorrect personal identification number PIN Block Error
202	Advance less than minimum
203	Administrative card needed
204	Amount over maximum
205	Invalid Advance Amount Original Amnt Incorrect Bad message/Invalid Amount Original transaction amount error
206	CAF not found Invalid “to” account Invalid “from” account Invalid account
207	Invalid transaction date
208	Invalid expiration date
209	Invalid transaction code
210	PIN key sync error
212	Destination not available
251	Error on cash amount
252	Debit not supported

American Express Response Codes (Declines)

Response Code	Messages
426	AMEX - Denial 12
427	AMEX - Invalid merchant
429	AMEX - Account error
430	AMEX - Expired card
431	AMEX - Call Amex
434	AMEX - Call 03 Note: Invalid CVD (CID)
435	AMEX - System down
436	AMEX - Call 05
437	AMEX - Declined
438	AMEX - Declined
439	AMEX - Service error
440	AMEX - Call Amex
441	AMEX - Amount error

Credit Card Response Codes (Declines)

Response Code	Messages
408	CREDIT CARD - Card use limited - Refer to branch
475	CREDIT CARD - Invalid expiration date
476	CREDIT CARD - Invalid transaction, rejected No Credit Account Invalid transaction/Invalid related transactions

Response Code	Messages
	Unable to process/Suspected malfunction; related transaction error Unable to Authorize: Cut off is in process Issuer not capable to process Switch system malfunction Issuer response not received by CUPS Unable to Authorize/Illegal Status of Acquirer
477	CREDIT CARD - Refer Call/Invalid Card Number Invalid card number (no such account) Deny – Card Not Found Items not on Bankbook beyond limit, declined/Invalid card number
478	CREDIT CARD - Decline, Pick up card, Call
479	CREDIT CARD - Decline, Pick up card
480	CREDIT CARD - Decline, Pick up card
481	CREDIT CARD - Decline Transaction not allowed to be processed by cardholder Low funds/Insufficient Balance Invalid Transaction Transaction not allowed to be processed by merchant
482	CREDIT CARD - Expired Card
483	CREDIT CARD – Refer/Refer to Issuer Deny – Card Acceptor Contact Acquirer
484	CREDIT CARD - Expired card - refer
485	CREDIT CARD - Not authorized

Response Code	Messages
486	CREDIT CARD - CVV Cryptographic error
487	CREDIT CARD - Invalid CVV
489	CREDIT CARD - Invalid CVV
490	CREDIT CARD - Invalid CVV
492	System problem - ask cardholder to insert card in chip card reader Withdrawal count exceeded

System Decline Response Codes

Response Code	Messages
800	Bad format
801	Bad data
802	Invalid Clerk ID
809	Bad close
810	System timeout
811	System error
821	Bad response length
842	Installment plan lookup failure
877	Invalid PIN block
878	PIN length error
880	Final packet of a multi-packet transaction
881	Intermediate packet of a multi-packet transaction
889	MAC key sync error

Response Code	Messages
898	Bad MAC value
899	Bad sequence number - resend transaction
900	Capture - PIN Tries Exceeded
901	Capture - Expired Card
902	Capture - NEG Capture
903	Capture - CAF Status 3
904	Capture - Advance < Minimum
905	Capture - Num Times Used
906	Capture - Delinquent
907	Capture - Over Limit Table
908	Capture - Amount Over Maximum Capture - Capture Pick up Card Suspected Fraud Hard Capture Deny – Keep Card: Special Conditions Expired Card Fraud Card Acceptor Call Acquirer's Do Not Honour
950	Admin card is not enabled on Merchant profile

Other Response Codes

Response Code	Message
599	Decline

Admin Response Codes

Response Code	Messages
960	Initialization Failure - No Match on Merchant ID
961	Initialization Failure - No Match on PED ID
962	Initialization Failure - No match on Printer ID
963	No match on Poll code
964	Initialization Failure - No match on Concentrator ID
965	Invalid software version number
966	Duplicate terminal name
970	Terminal/Clerk table full
983	Clerk Totals Unavailable: selected Clerk IDs do not exist or have zero totals
989	MAC Error on Transaction 95 (Initialization and Handshake), most often, this indicates that the wrong keys have been injected into a device/KMAC Sync Error

EMV Reversal Request Codes

Response Code	Messages
990	Chip card declines a host approved transaction
991	Chip card removed before ICC communications are completed

Surcharge Response Codes

Response Code	Messages
976	BIN not found - DO NOT RETRY
977	Prepaid Card BIN
978	Debit Card BIN
979	Internal System Error - RETRY
980	Surcharge Limit Exceeded

Appendix D Error Messages

Error messages that are returned if the gateway is unreachable

Global Error Receipt

You are not connecting to our servers. This can be caused by a firewall or your internet connection.

Response Code = NULL

The response code can be returned as null for a variety of reasons. The majority of the time, the explanation is contained within the Message field.

When a 'NULL' response is returned, it can indicate that the issuer, the credit card host, or the gateway is unavailable. This may be because they are offline or because you are unable to connect to the internet.

A 'NULL' can also be returned when a transaction message is improperly formatted.

Error messages that are returned in the Message field of the response

XML Parse Error in Request: <System specific detail>

An improper XML document was sent from the API to the servlet.

XML Parse Error in Response: <System specific detail>

An improper XML document was sent back from the servlet.

Transaction Not Completed Timed Out

Transaction timed out before the host responds to the gateway.

Request was not allowed at this time

The host is disconnected.

Could not establish connection with the gateway: <System specific detail>

Gateway is not accepting transactions or server does not have proper access to internet.

Input/Output Error: <System specific detail>

Servlet is not running.

The transaction was not sent to the host because of a duplicate order id

Tried to use an order id which was already in use.

The transaction was not sent to the host because of a duplicate order id

Expiry Date was sent in the wrong format.

Vault error messages

Can not find previous

Data key provided was not found in our records or profile is no longer active.

Invalid Transaction

Transaction cannot be performed because improper data was sent.

or

Mandatory field is missing or an invalid SEC code was sent.

Malformed XML

Parse error.

Incomplete

Timed out.

or

Cannot find expiring cards.

Copyright Notice

Copyright © September 2025 Moneris Solutions, 3300 Bloor Street West, Toronto, Ontario, M8X 2X2

All Rights Reserved. This manual shall not wholly or in part, in any form or by any means, electronic, mechanical, including photocopying, be reproduced or transmitted without the authorized, written consent of Moneris Solutions.

This document has been produced as a reference guide to assist Moneris client's hereafter referred to as merchants. Every effort has been made to the make the information in this reference guide as accurate as possible. The authors of Moneris Solutions shall have neither liability nor responsibility to any person or entity with respect to any loss or damage in connection with or arising from the information contained in this reference guide.

Trademarks

Moneris and the Moneris Solutions logo are registered trademarks of Moneris Solutions Corporation.

Any software, hardware and or technology products named in this document are claimed as trademarks or registered trademarks of their respective companies.

Printed in Canada.

10 9 8 7 6 5 4 3 2 1

